

Legal Metrology Research

Project Name: DrishtiTathya

Problem Statement: SIH26034

Domain: Legal Metrology / Packaged Commodities

Research Version: 01

Research Date: September 2026

TABLE OF CONTENTS

1. Research Methodology
2. Legal & Regulatory Research
3. Applicability & Amendments
4. Existing Inspection Process & Research Gap
5. Legal Requirements → Software Rules
6. DrishtiTathya System Architecture
7. AI / OCR & Information Extraction
8. Version-Aware Legal Rule Engine
9. Compliance Detection & Violation Analysis
10. Evidence Management & Human Review
11. Database & Application Design
12. Consumer–Manufacturer–Government Ecosystem
13. Unique Innovation & USP
14. Advanced & Future Features
15. Security, Privacy & Data Governance
16. Testing & Validation
17. Implementation Roadmap & Feasibility
18. Cost, Resources & Infrastructure
19. Impact, KPIs & Success Metrics
20. Future Scope
21. Conclusion
22. References

1. Research Methodology

The research follows the process:

Read → Understand → Verify → Record → Analyse → Design

For each important requirement, the research distinguishes between the legal requirement and the proposed software implementation.

Classification	Meaning
LEGAL FACT	Information directly stated in an official law, rule or government source
RESEARCH FINDING	A conclusion obtained by studying one or more sources
SOFTWARE INTERPRETATION	Understanding how a legal requirement could be checked by software
ASSUMPTION	A condition assumed for development or testing
PROPOSED DESIGN	A feature or architecture proposed for DrishtiTathya

This distinction helps prevent the software design from being incorrectly presented as a legal requirement.

2. LEGAL METROLOGY ACT, 2009

2.1 Purpose of the Act

The Legal Metrology Act, 2009 provides the legal framework for establishing and enforcing standards of weights and measures and regulating trade and commerce involving goods sold or distributed by weight, measure or number.

The Act also provides the broader framework for matters such as verification, enforcement, offences and penalties, and requirements relating to pre-packaged commodities.

Research Finding — RF001

Legal Foundation

The Legal Metrology Act, 2009 provides the broader legal foundation for Legal Metrology in India. The detailed requirements applicable to packaged commodities are provided through the relevant rules made under the Act.

Software Relevance

For DrishtiTathya, the Act represents the **higher-level legal framework**, while the Packaged Commodities Rules provide the detailed requirements that the software needs to analyse.

2.2 Pre-Packaged Commodity

A pre-packaged commodity is a commodity placed in a package, without the purchaser being present, in such a way that the product has a predetermined quantity.

Research Finding — RF002

The concept of a pre-packaged commodity is important because DrishtiTathya is intended to assist with the verification of information appearing on packaged commodities.

Software Relevance

The system should not assume that every object or product image is automatically subject to the same packaged-commodity requirements.

A future version of the system may therefore need to identify the package/product context before selecting the applicable compliance checks.

2.3 Section 18 — Declarations on Pre-Packaged Commodities

Section 18 of the Legal Metrology Act, 2009 establishes requirements relating to declarations on pre-packaged commodities. It provides that prescribed declarations and particulars must be provided on packages in accordance with the applicable requirements.

Research Finding — RF003

The Act establishes the legal requirement for prescribed declarations on pre-packaged commodities, while the detailed requirements are provided through the applicable rules.

Software Relevance

This creates the fundamental relationship for DrishtiTathya:

Legal Requirement → Package Declaration → Information Extraction → Compliance Check

The software should therefore combine:

- image processing,
- OCR,
- declaration extraction, and
- a rule-based compliance engine.

3. LEGAL METROLOGY (PACKAGED COMMODITIES) RULES, 2011

3.1 Origin of the Rules

The **Legal Metrology (Packaged Commodities) Rules, 2011** were made under the Legal Metrology Act, 2009.

The Rules were notified on **7 March 2011** through G.S.R. 202(E) and came into force on **1 April 2011**. This is stated at the beginning of the Rules document.

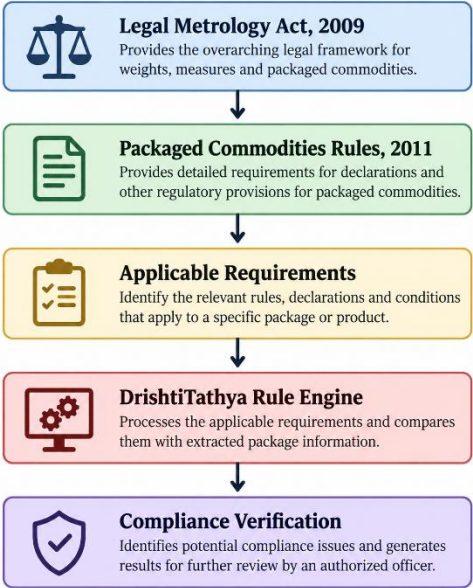
- **Research Finding — RF004**

The Packaged Commodities Rules, 2011 establish detailed requirements applicable to packaged commodities within the Legal Metrology framework.

- **Software Relevance**

These Rules form the **primary legal rule source** for the compliance-verification component of DrishtiTathya.

The proposed system can therefore be represented as:



4. DEFINITIONS

4.1 Importance of Definitions

Rule 2 of the Packaged Commodities Rules, 2011 provides definitions for terms used throughout the Rules.

The document defines terms including:

- Manufacturer
- Maximum permissible error
- Net quantity
- Packer
- Principal display panel
- Quantity
- Retail dealer
- Retail package
- Retail sale
- Retail sale price
- Standard package
- Wholesale dealer
- Wholesale package

These definitions are provided in Rule 2 of the uploaded Rules document.

4.2 Definitions Relevant to DrishtiTathya

Legal Term	Relevance to DrishtiTathya
Manufacturer	May need to be identified from package declarations
Packer	May need to be identified from package declarations
Net Quantity	Important declaration to extract and verify
Principal Display Panel	Relevant to locating package declarations
Retail Package	Helps determine the relevant package category

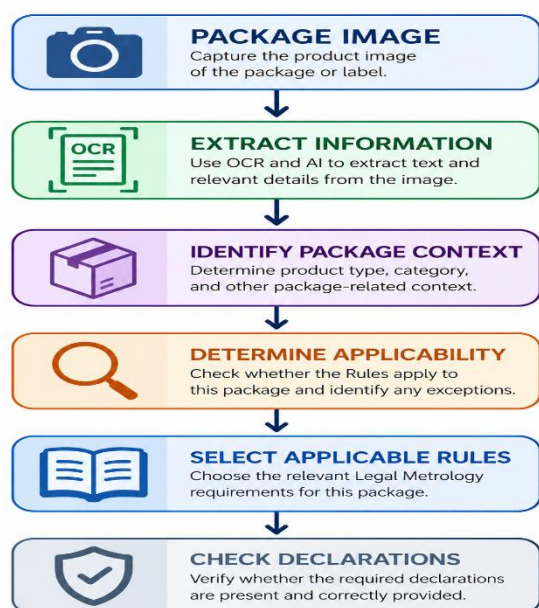
Retail Sale	Relevant to applicability
Retail Sale Price	Important for MRP-related verification
Wholesale Package	Important to avoid applying retail requirements incorrectly

➤ Research Finding — RF005

The legal definitions in Rule 2 are important for software design because the compliance requirements cannot be applied correctly without understanding the type and context of the package.

- **Software Interpretation**

This suggests that DrishtiTathya should eventually follow a process such as :



5. APPLICABILITY

5.1 Rule 1 — Short Title and Commencement

Rule 1 of the Legal Metrology (Packaged Commodities) Rules, 2011 establishes the name of the Rules and specifies their commencement.

The Rules are titled the **Legal Metrology (Packaged Commodities) Rules, 2011**. They were notified by the Government of India on **7 March 2011** through Notification No. G.S.R. 202(E) and came into force on **1 April 2011**.

The purpose of Rule 1 is primarily administrative and temporal. It establishes the regulatory framework under which the subsequent provisions of the Rules operate. It does not by itself determine whether an individual package is compliant.

Research Finding — RF009

The original 2011 Rules provide the base regulatory framework, but the framework has subsequently been modified through several amendment notifications. Therefore, the 2011 Rules should be treated as the **base version** rather than automatically being treated as the complete current version.

- **Software Relevance**

For DrishtiTathya, Rule 1 introduces the importance of **regulatory version management**.

The software should be capable of storing:

- Rule version
- Notification number
- Notification date
- Effective date
- Amendment status
- Applicable period

This prevents the software from treating the original 2011 provisions as permanently unchanged.

5.2 Rule 2 — Definitions

Rule 2 provides definitions of important terms used throughout the Legal Metrology (Packaged Commodities) Rules, 2011.

Important terms include:

- Manufacturer
- Packer
- Importer
- Net quantity
- Quantity
- Principal Display Panel
- Retail package
- Retail sale
- Retail sale price
- Retail dealer
- Wholesale package
- Wholesale dealer
- Institutional consumer
- Industrial consumer
- Other terms introduced or modified through subsequent amendments

These definitions are important because the applicability of other provisions may depend on the meaning of these terms.

Rule 2 should not be treated as a simple dictionary section. Its definitions influence how other provisions of the Rules are interpreted and applied.

For example, determining whether a package is intended for retail sale or whether a particular buyer falls within an institutional or industrial category can affect whether the retail-package provisions apply.

Amendment Research

Some definitions have been modified or additional concepts have been introduced through subsequent amendments.

The **2017 amendment** is particularly important because it modified definitions and introduced concepts related to e-commerce, including e-commerce entities and marketplace-based models. It also modified the definition relating to retail sale price. The amendment came into force on **1 January 2018**.

The **2021 amendment** subsequently modified the definition of consumer by replacing the reference to the Consumer Protection Act, 1986 with the Consumer Protection Act, 2019.

- **Software Relevance**

DrishtiTathya can use definitions as inputs to an **Applicability Engine**.

This means that the software should not immediately perform all compliance checks on every package. It should first determine **which provisions are applicable**.

5.3 Rule 3 — Applicability of Chapter II

Rule 3 determines the applicability of Chapter II of the Packaged Commodities Rules.

The original 2011 version contained exclusions for certain packages, including packages containing quantities above specified limits and packaged commodities intended for industrial or institutional consumers.

However, Rule 3 was subsequently substituted through the **Legal Metrology (Packaged Commodities) Amendment Rules, 2017**.

The amended Rule 3 provides exclusions relating to:

1. Packages containing commodities of more than 25 kilogram or 25 litre;
2. Cement, fertilizer and agricultural farm produce sold in bags above 50 kilogram; and
3. Packaged commodities meant for industrial consumers or institutional consumers.

The 2017 amendment came into force on **1 January 2018**.

Research Finding — RF011

The comparison between the original Rule 3 and the amended Rule 3 demonstrates that the applicability of the Rules cannot always be determined using a simple fixed checklist.

Applicability may depend on:

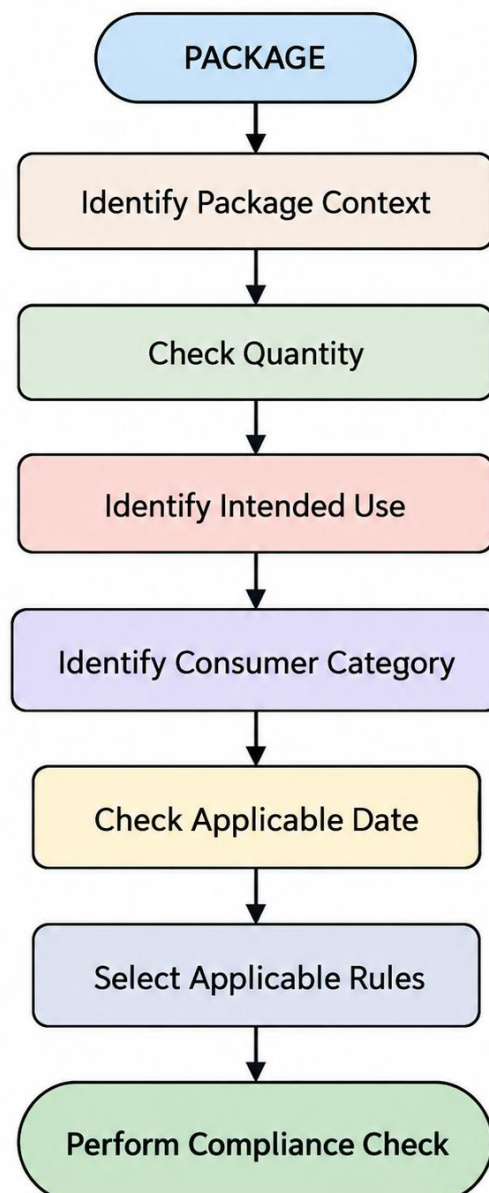
- Quantity of the commodity
- Type of commodity

- Intended use
- Type of consumer
- Package characteristics
- Effective date of the applicable rule
- Relevant amendments and exceptions

Software Relevance

DrishtiTathya should therefore implement an **Applicability Engine** before the detailed compliance engine.

The basic logic can be represented as:

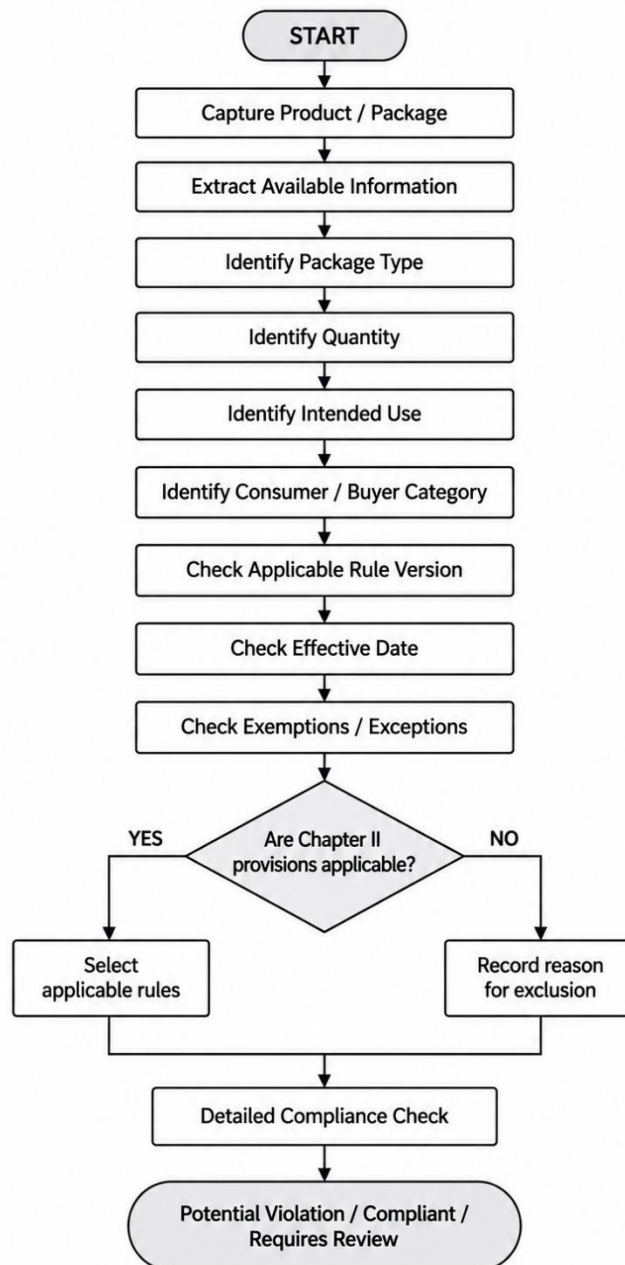


Important Research Observation

The Department of Consumer Affairs also considered changes to Rule 3 through a proposal in 2024. Such proposals must not automatically be treated as enacted law.

This demonstrates why a compliance system should distinguish between:

- Enacted rules
- Amended rules
- Future-effective provisions
- Proposed amendments



Research Finding — RF012

The research of Rules 1–3 indicates that **applicability should be determined before compliance verification**.

Therefore, the proposed DrishtiTathya architecture should follow:

Identify → Determine Applicability → Select Rules → Verify → Review

rather than:

6. OLD RULES VS UPDATED RULES

6.1 Why Amendment Research is Necessary

The Legal Metrology (Packaged Commodities) Rules, 2011 were originally notified in 2011. However, the Rules have been amended several times over the years.

The Department of Consumer Affairs currently provides the original Rules and multiple amendment notifications. Its official repository includes amendments through **2026**.

Therefore, using only the original 2011 Rules for developing a compliance system could result in outdated or incorrect compliance logic.

Research Finding — RF013

A software system designed for regulatory compliance must consider not only the original rule text but also:

- Amendments
- Effective dates
- Modified definitions
- New requirements
- Exceptions
- Future-effective provisions
- Proposed changes

DrishtiTathya Principle:

The system should verify the applicable version of a rule rather than assuming that the 2011 version is permanently valid without modification.

6.2 Amendment Research Methodology

For every important rule, DrishtiTathya research follows the following process:

Research Step	Question
1. Identify original rule	What did the 2011 Rule state?
2. Find amendments	Was the Rule subsequently changed?
3. Identify modification	What exactly was changed?
4. Identify notification	Which amendment notification made the change?
5. Identify effective date	When did the change become applicable?
6. Determine current status	Is it enacted, future-effective or proposed?

7. Identify applicability	To which packages/entities does it apply?
8. Determine software relevance	Can the requirement be checked automatically?
9. Identify required data	What information does the software need?
10. Define human review	Which parts require officer verification?

Research Classification

Each research statement in this document should be classified as one of the following:

LEGAL FACT

Information directly supported by legislation, Rules or official notifications.

RESEARCH FINDING

A conclusion obtained by comparing legal sources.

SOFTWARE INTERPRETATION

How the legal requirement could be represented in software.

ASSUMPTION

An assumption made where sufficient official information is not available.

PROPOSED DESIGN

A feature or architecture proposed for DrishtiTathya.

This classification helps prevent software assumptions from being presented as legal requirements.

6.3 Rule 1 Comparison

Aspect	Original 2011 Framework	Updated Understanding
Name	Legal Metrology (Packaged Commodities) Rules, 2011	Same base Rules with subsequent amendments
Notification	7 March 2011	Subsequent amendment notifications
Original effective date	1 April 2011	Individual amendments may have different effective dates
Purpose	Establishes the Rules	Provides base framework together with amendments
Software importance	Establishes original rule framework	Requires version and effective-date management

Research Finding — RF014

Rule 1 demonstrates that the regulatory framework has a **temporal dimension**. A software system should therefore associate rules with their applicable dates.

6.4 Rule 2 Comparison

Rule 2 has evolved through amendments.

Concept	Original Framework	Later Development	Software Relevance
Consumer	Defined in original framework	Modified through later amendment	Consumer classification
Institutional Consumer	Original definition	Modified through 2017 amendment	Applicability determination
Retail Sale Price	Original definition	Modified through 2017 amendment	MRP validation
E-commerce	Not present in the original 2011 framework	Added through later amendment	Online compliance
E-commerce Entity	Not present in original framework	Added through later amendment	Identify online platform/entity
Marketplace-based Model	Not present in original framework	Added through later amendment	E-commerce classification

The 2017 amendment introduced important e-commerce-related terminology and modified certain definitions.

The 2021 amendment also updated the consumer definition to reference the Consumer Protection Act, 2019.

Research Finding — RF015

Definitions themselves can change over time. Therefore, the software should not hard-code definitions permanently without maintaining their regulatory version.

6.5 Rule 3 Comparison

Rule 3 provides a strong example of why amendment research is necessary.

Aspect	Original 2011 Rule	2017 Amended Rule
Purpose	Determines applicability of Chapter II	Determines applicability of Chapter II
Quantity exclusion	Packages above specified quantity limits	Packages above 25 kg or 25 litre
Special commodities	Original wording	Cement, fertilizer and agricultural farm produce specifically addressed
Industrial consumers	Excluded	Excluded
Institutional consumers	Excluded	Excluded
Effective framework	2011	Effective from 1 January 2018
Software impact	Basic applicability check	More detailed applicability classification

The 2017 amendment substituted Rule 3 and came into force on 1 January 2018.

Research Finding — RF016

Rule 3 demonstrates that a package should not automatically be subjected to every retail-package compliance check. The system must first determine whether the relevant provisions apply.

6.6 What Changed and Why It Matters

The comparison of the original and amended Rules shows several important patterns.

1. Definitions can change

Changes to definitions can affect the interpretation of other Rules.

2. Applicability can change

An amendment can modify which packages or entities are covered by a provision.

3. New digital requirements can be introduced

The evolution of the Rules has included provisions relevant to e-commerce.

4. Effective dates matter

An amendment may be notified on one date but become effective on another date.

5. Proposed changes are different from enacted changes

A draft or proposal should not automatically be treated as a legal requirement.

Research Finding — RF017

Regulatory compliance is not a static checklist. It is a version-dependent decision process.

This is an important foundation for the DrishtiTathya system.

6.7 Amendment Timeline

The following timeline provides a high-level view of the evolution of the Packaged Commodities Rules.

Period	Regulatory Development	Importance
2011	Original Packaged Commodities Rules notified	Base framework
2012	Amendments	Modification of requirements
2013	Amendments	Modification of requirements
2014	Amendments	Modification of requirements
2015	Amendments	Modification of requirements
2016	Amendments	Modification of requirements
2017	Major amendment	Important changes to definitions and applicability

2021	Amendment	Changes including consumer definition and retail-package provisions
2022	Multiple amendments	Further updates
2023	Multiple amendments	Further updates
2025	Amendments	New/special provisions
2026	Multiple amendments	Current and future-effective digital/regulatory requirements

The Department of Consumer Affairs' official repository lists multiple amendments to the Packaged Commodities Rules, including amendments in 2022, 2023, 2025 and 2026.

Research Note

The timeline is a **research index**, not a substitute for reading each amendment notification. For implementation, each relevant amendment should be examined from its official notification.

6.8 Recent Amendments

Recent amendments demonstrate that the Rules continue to evolve and that digital compliance can extend beyond physical package labels.

2025 Amendment — Medical Devices

The 2025 amendment introduced special provisions for packages containing medical devices. It provides that, for such packages, provisions of the Medical Devices Rules, 2017 apply for making declarations and for certain requirements concerning the height and width of numerals and letters.

DrishtiTathya Relevance

This creates a requirement for **special-case classification**.

The software should be able to identify when a package belongs to a special regulatory category and avoid blindly applying the general rule.

2025 Second Amendment — Pan Masala

The Department also notified the Legal Metrology (Packaged Commodities) Second Amendment Rules, 2025. The amendment modified Rule 26(a) by adding a further proviso concerning pan masala.

DrishtiTathya Relevance

This demonstrates that certain commodities may require **commodity-specific compliance logic**.

2026 Amendment — E-Commerce Country-of-Origin Filter

The Legal Metrology (Packaged Commodities) Amendment Rules, 2026, notified on **13 February 2026**, inserted Rule 6(10A).

The provision requires every e-commerce entity selling imported products to provide product listings with a searchable and sortable filter specifying the country of origin. The amendment was specified to come into force on **1 July 2026**.

DrishtiTathya Relevance

This is important because it expands the possible scope of DrishtiTathya beyond physical package scanning.

A future **E-Commerce Compliance Module** could inspect:

- Product listing
- Country of origin
- Import status
- Required online information
- Availability of required filters

6.9 Future-Effective Provisions

Not every notified amendment becomes effective immediately.

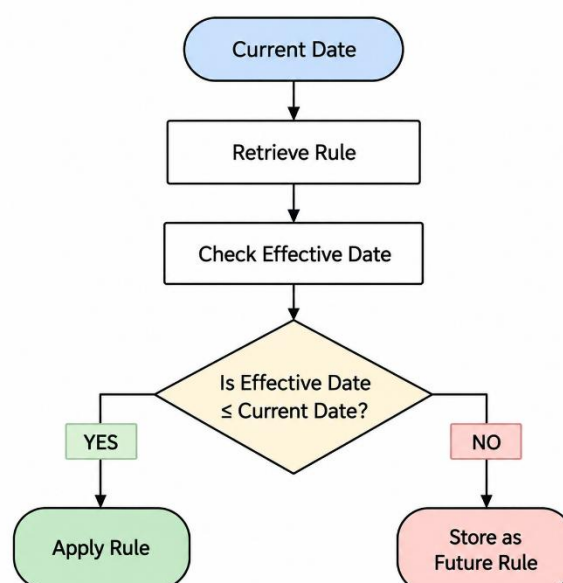
A good example is the **Second Amendment Rules, 2026**.

The April 2026 amendment states that it will come into force on **1 July 2027**. It substitutes Rule 6(10A) and requires e-commerce entities offering imported products to ensure that product listings contain a searchable and sortable country-of-origin filter from that date.

Research Finding — RF018

A compliance system must distinguish between: Currently Applicable and Not Yet Effective

- **Proposed Software Logic**



This prevents future requirements from being incorrectly reported as current violations.

6.10 Proposed vs Enacted Provisions

During legal research, the Department may publish draft amendments or proposals for comments.

A proposed amendment is not automatically equivalent to an enacted amendment.

Therefore, DrishtiTathya research should classify regulatory documents according to their status.

Status	Meaning	Treatment by Software
ENACTED	Legally notified provision	Can be used according to effective date
AMENDED	Existing provision modified through notification	Replace/update applicable rule version
FUTURE-EFFECTIVE	Notified but effective from a future date	Store but do not apply before effective date
PROPOSED / DRAFT	Proposal open for comments or consideration	Do not treat as current legal requirement
ADVISORY / GUIDANCE	Official guidance or clarification	Store separately from mandatory rule logic

- **Research Finding — RF019**

Regulatory source status is an important data field for a compliance system.

The software should not simply store the text of a rule. It should also store the **legal status of that text**.

6.11 Version-Aware Compliance

The research of Rules 1–3 and subsequent amendments leads to an important architectural requirement.

Version-Aware Compliance

Instead of storing only:

Rule 6 → MRP must be declared the system should conceptually store:

Rule ID



Requirement



Source Notification



Notification Date



Effective Date



Applicability Conditions



Exceptions



Legal Status



Validation Logic

Example

Rule: 6(10A)

Source:

Legal Metrology (Packaged Commodities)

Amendment Rules, 2026

Notification Date:

13 February 2026

Effective Date:

1 July 2026

Requirement:

Country-of-origin filter for imported
products on e-commerce listings

Status:

Enacted

Software Module:

E-Commerce Compliance

A later amendment can then update the relevant rule record without destroying the historical information.

- **Research Finding — RF020**

The proposed DrishtiTathya rule engine should be version-aware, date-aware and applicability-aware.

This is one of the key research-derived design requirements of the project.

6.12 Software Requirements Derived from the Research

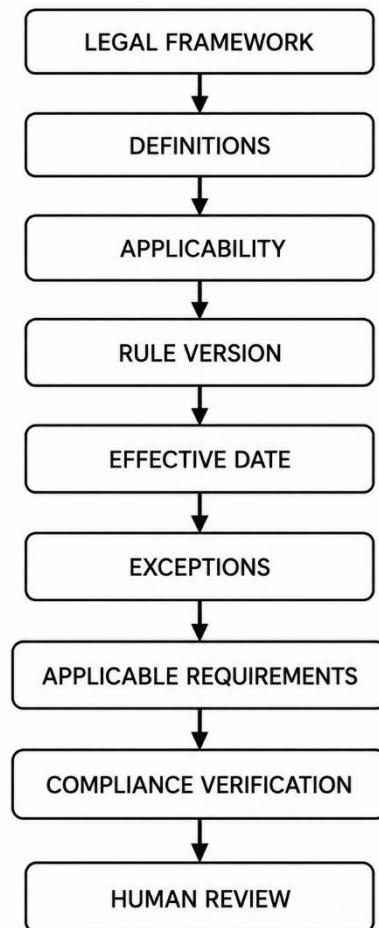
The legal research conducted in this section leads to the following software requirements.

No.	Research Requirement	Proposed Software Feature
1	Rules change over time	Version-controlled rule database
2	Amendments have effective dates	Effective-date engine
3	Rules have different applicability	Applicability engine
4	Definitions influence applicability	Context/classification module
5	Exceptions exist	Conditional rule logic
6	Some requirements are future-effective	Future-rule storage
7	Proposed rules are not current law	Legal-status classification
8	Some requirements are commodity-specific	Product-category rules
9	Some information is visible on packages	OCR and image analysis
10	Some decisions require context	Human review workflow
11	E-commerce requirements exist	Online listing compliance module
12	Compliance decisions should be explainable	Explainable violation report

Research Conclusion

The comparison of the original 2011 Rules with subsequent amendments demonstrates that Legal Metrology compliance is a **dynamic regulatory problem** rather than a fixed checklist.

The research of Rules 1, 2 and 3 establishes the following sequence:



7. RULE 4 — CONDITIONS FOR PRE-PACKING

7.1 Rule 4 — Basic Requirement

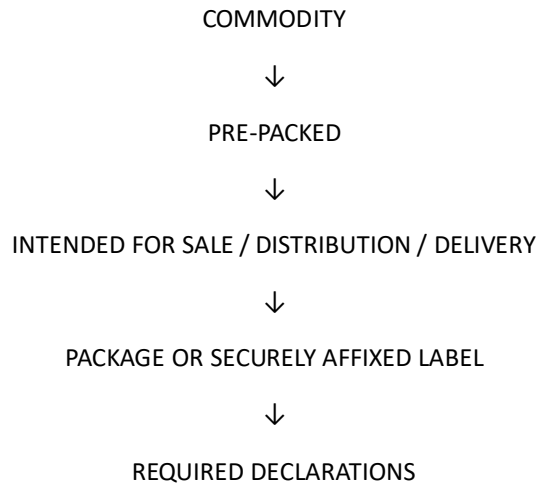
Rule 4 of the Legal Metrology (Packaged Commodities) Rules, 2011 establishes a basic requirement for commodities that are pre-packed for sale, distribution or delivery.

The Rule provides that no person shall pre-pack or cause or permit a commodity to be pre-packed for sale, distribution or delivery unless the package or a label securely affixed to the package bears the required declarations.

Therefore, Rule 4 establishes a basic connection between the **pre-packaged commodity** and the **declarations required on its package or label**.

Legal Requirement

The requirement can be represented as:



7.2 Purpose of Rule 4

The purpose of Rule 4 is to ensure that a pre-packaged commodity does not enter the market without carrying the declarations required under the applicable provisions of the Rules.

Rule 4 therefore acts as a basic declaration requirement, while the specific declarations and their conditions are provided through the relevant subsequent provisions of the Rules.

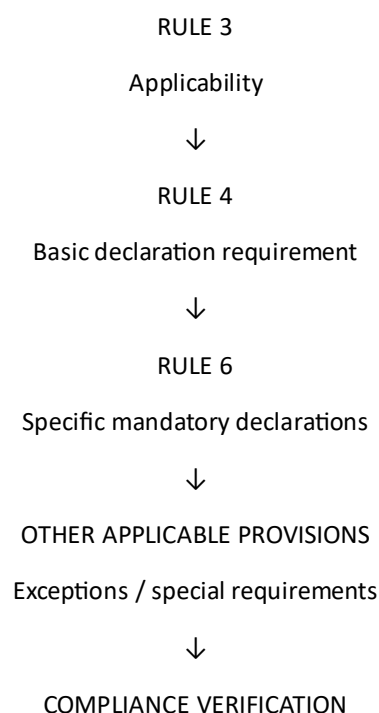
Research Finding — RF021

Rule 4 establishes the basic requirement that a pre-packaged commodity intended for sale, distribution or delivery must carry the declarations required under the applicable provisions of the Rules.

7.3 Relationship Between Rule 4 and Other Rules

Rule 4 should not be interpreted independently of the other provisions.

The relationship can be represented as:



This means that the system should first determine whether the relevant provisions apply to the package and then determine which declarations are required.

7.4 Rule 4 and Software Compliance

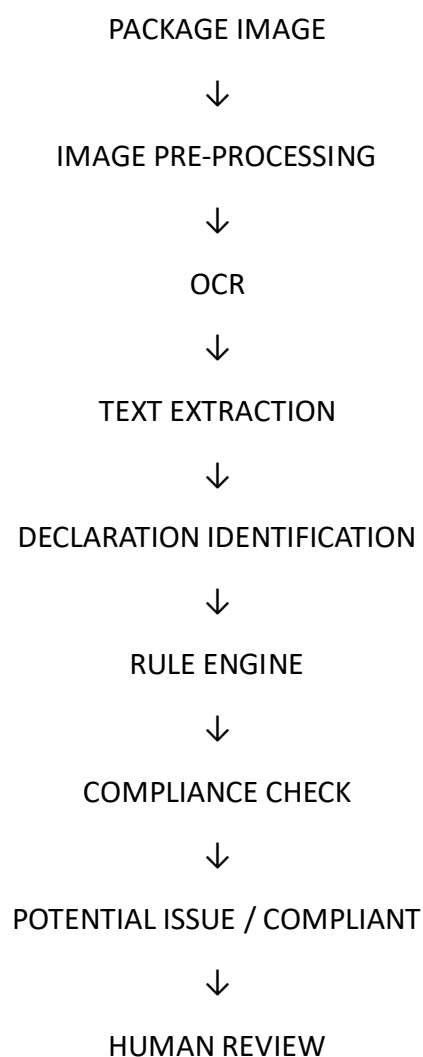
Rule 4 provides an important foundation for the proposed DrishtiTathya system.

The system can use a product image to identify and extract visible information from the package using:

- Optical Character Recognition (OCR)
- Image processing
- Text classification
- Pattern recognition
- Field extraction

The extracted information can then be compared with the requirements selected by the Legal Metrology Rule Engine.

Proposed Processing Flow



7.5 Information That Can Be Extracted From an Image

A package image may contain several types of information that can potentially be detected by DrishtiTathya.

Information	Possible Technology
Printed declarations	OCR
MRP	OCR + pattern recognition
Net quantity	OCR + unit recognition
Manufacturer details	OCR + entity extraction
Packer details	OCR + entity extraction
Importer details	OCR + entity extraction
Country of origin	OCR + keyword detection
Consumer-care information	OCR + pattern detection
Barcode / QR code	Barcode or QR decoder
Location of declarations	Image processing

However, extracting information from an image does not automatically prove that the information is legally correct.

7.6 Limitation of Image-Based Verification

An important distinction must be maintained between **information extraction** and **legal verification**.

For example, OCR may successfully extract:

“Manufactured by ABC Ltd.”

However, the OCR system alone cannot establish whether ABC Ltd. is legally the correct manufacturer or whether the declaration is otherwise legally valid.

Similarly, detecting an MRP value does not by itself prove that the declared MRP satisfies every applicable legal requirement.

Research Finding — RF022

OCR can extract information from a package, but information extraction should not be treated as equivalent to final legal verification.

Therefore, DrishtiTathya should use AI and OCR to **assist** compliance verification rather than independently making every final legal determination.

7.7 Evidence-Based Verification

When the system identifies a potential issue, it should retain the evidence used to reach that result.

For example:

Potential Issue: Net quantity declaration not detected

Evidence: Package image analysed

OCR text extracted: No recognised net-quantity declaration detected

Confidence: 91%

Status: Requires Human Review

This allows an officer to understand why the system generated the alert.

Proposed Design

Each potential violation should ideally contain:

- Applicable Rule
- Detected information
- Missing or suspicious information
- Image evidence
- OCR evidence
- Confidence score
- Reason for flagging
- Human verification status

7.8 Rule 4 — Software Interpretation

Based on the research, Rule 4 can be represented conceptually as:

IF commodity is pre-packed AND commodity is intended for sale /distribution / delivery THEN
required package declarations must be identified and verified according to applicable rules.

This is a **software interpretation**, not a replacement for the legal text.

The actual requirements applied by the software must be derived from the relevant provisions and their applicable amendments.

7.9 Rule 4 — Research Questions

The following questions should be investigated before finalising the software implementation:

1. What was the original wording of Rule 4?
2. Has Rule 4 been amended?
3. Which amendment notifications affected Rule 4?
4. What was changed?
5. What is the effective date of each change?
6. Which packages are covered?

7. What exceptions affect the requirement?
 8. Which declarations are connected with Rule 4?
 9. Which declarations can be detected using OCR?
 10. Which requirements require information outside the package image?
 11. Which checks can be automated?
 12. Which checks require human verification?
-

7.10 Research Finding

The study of Rule 4 leads to the following conclusion:

Rule 4 establishes the basic declaration requirement for pre-packaged commodities, while the detailed compliance checklist must be derived from the applicable provisions of the Rules.

Therefore, DrishtiTathya should not implement Rule 4 as a simple keyword search.

7.11 DrishtiTathya Relevance

Rule 4 provides the foundation for the **Declaration Verification Module** of DrishtiTathya.

The module should be designed to:

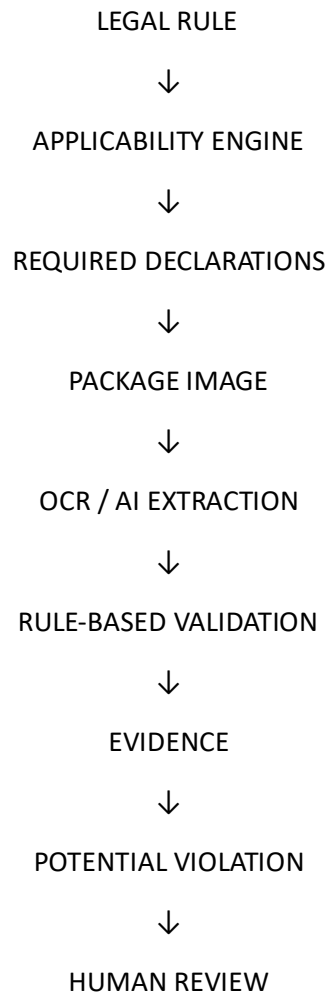
- Scan package images
- Extract visible declarations
- Identify declaration fields
- Compare them with applicable requirements
- Detect potentially missing declarations
- Highlight suspicious information
- Provide evidence for each flag
- Assign an AI/OCR confidence level
- Send uncertain cases for human review
- **Key Design Principle**

DrishtiTathya should assist an inspector in identifying potential compliance issues; the final legal determination should remain subject to human verification.

7.12 Rule 4 Conclusion

Rule 4 establishes an important link between the legal requirement for declarations and the technical process of package inspection.

The research of Rule 4 therefore supports the following DrishtiTathya architecture:



Research Finding — RF023

Rule 4 provides the legal foundation for automated declaration detection, but automated extraction must be combined with rule-based validation, evidence and human review to support reliable compliance assessment.

8. MANDATORY DECLARATIONS ON PACKAGES

8.1 Rule 6 — Declarations to be Made on Every Package

Rule 6 of the Legal Metrology (Packaged Commodities) Rules, 2011 deals with the declarations required to be made on packages intended for retail sale.

This Rule is particularly important for DrishtiTathya because many of the declarations required on a package are visible in the product's packaging and can potentially be extracted using image processing and OCR.

The original 2011 Rules therefore provide an important starting point for identifying the information that a compliance system needs to detect.

Research Finding — RF027

Rule 6 provides the foundation for identifying and validating mandatory declarations appearing on retail packages.

The uploaded 2011 Rules should be treated as the historical/base source for this research. Subsequent amendments must then be checked to determine the applicable updated requirement.

8.2 Why Rule 6 is Important for DrishtiTathya

The objective of DrishtiTathya is not simply to read text from a package.

The system should determine:

1. What information is present?
2. What information is required?
3. Which requirements apply to this package?
4. Is the required information detectable?
5. Does the extracted information satisfy the applicable requirement?
6. Is the result sufficiently reliable for automated reporting?
7. Does the case require human review?

8.3 Categories of Information to be Researched

For the purpose of software design, the declarations should be divided into individual information fields.

The main categories to investigate include:

Declaration / Information	Possible Detection Method
Name and address of manufacturer	OCR + entity extraction
Name and address of packer	OCR + entity extraction
Name and address of importer	OCR + entity extraction
Country of origin for imported goods	OCR + keyword detection
Common / generic name of commodity	OCR + text classification
Net quantity	OCR + unit recognition
MRP / retail sale price	OCR + currency/number pattern
Month and year of manufacture/packing/import	OCR + date recognition
Consumer-care details	OCR + phone/email/keyword detection
Other applicable declarations	Rule-specific OCR validation

Important: The exact applicability of each declaration must be checked against the relevant rule, amendment and exception. The table above is a **software research classification**, not a statement that every field applies identically to every package.

8.4 Manufacturer, Packer and Importer Information

The Rules contain provisions concerning the name and address of the manufacturer, packer and importer.

The original 2011 Rules provide requirements concerning the name and complete address of the manufacturer and, where applicable, the packer and importer. They also contain special provisions for very small packages and for commodities manufactured outside India but packed in India.

The Rules further specify that the name should correspond to the actual corporate name or, where the business is not incorporated, the name under which the business is conducted.

Software Relevance

DrishtiTathya can attempt to identify:

- MANUFACTURED BY
- PACKED BY
- IMPORTED BY
- MANUFACTURER
- PACKER
- IMPORTER

The OCR system can extract the associated text.

However, the software cannot establish authenticity merely from the image.

Example

If the package contains:

- Manufactured by ABC Foods Pvt. Ltd.
- The system may extract:
- Entity Type: Manufacturer
- Name: ABC Foods Pvt. Ltd.

But verification that the entity is genuine or legally registered would require an appropriate external source or database.

Research Finding — RF028

Text extraction and identity verification are two different compliance tasks.

8.5 Net Quantity

Net quantity is one of the most important fields for packaged commodity compliance.

The Rules distinguish the quantity of the commodity from the packaging material and contain provisions regarding how quantity is to be declared. The original Rules state that the weight of wrappers and materials other than the commodity is excluded when declaring net quantity.

The Rules also require the declaration of quantity to be expressed using appropriate units of weight, measure or number.

Software Relevance

DrishtiTathya can identify patterns such as:

1. 500 g
2. 1 kg
3. 250 ml

4. 1 L
5. 10 tablets

The system can extract:

Quantity = 500

Unit = g

and then pass this information to the rule engine.

Important Limitation

A camera image can determine what quantity is **declared** on the package.

It cannot independently determine whether the package actually contains that quantity.

Therefore:

DECLARED QUANTITY

≠

ACTUAL QUANTITY

Actual quantity verification may require physical measurement or an inspection process.

Research Finding — RF029

Image-based verification can validate the presence and format of a quantity declaration, but actual quantity verification requires physical measurement or another appropriate verification method.

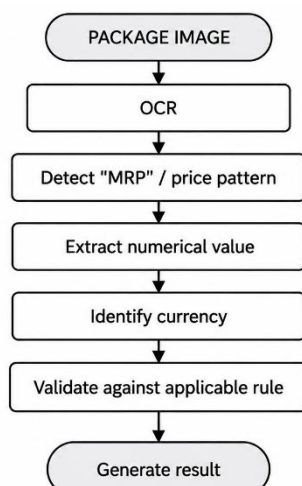
8.6 Maximum Retail Price / Retail Sale Price

The retail sale price is another important package declaration for DrishtiTathya.

The system should identify common patterns such as:

1. MRP ₹100
2. MRP Rs. 100
3. Maximum Retail Price ₹100

Possible Software Pipeline :



➤ Research Finding — RF030

MRP detection is suitable for automated information extraction, but the legal validity of the detected price must be evaluated using the applicable rule and package context.

8.7 Country of Origin

Country-of-origin information is particularly relevant for imported products.

The system can search extracted text for expressions such as:

- Country of Origin
- Made in India
- Made in China
- Product of India
- Imported from

Software Relevance

The system could classify:

Domestic Product

OR

Imported Product

and use that information to select additional applicable checks.

Important Development

Recent amendments have also introduced requirements relevant to **country-of-origin information in e-commerce listings**. This creates a potential future module for checking online product listings in addition to physical packages.

8.8 Consumer-Care Information

Consumer-care information may appear on packaging in forms such as:

- Consumer Care
- Customer Care
- For complaints contact
- Phone number
- Email
- Address

OCR and pattern recognition can be used to identify these fields.

Software Relevance

DrishtiTathya could extract:

- Phone Number
- Email Address
- Postal Address

- Consumer-care heading

and determine whether the relevant declaration is present.

However, the software should distinguish between:

Presence Check

and

Correctness Check

For example, detecting a phone number does not prove that the number actually belongs to the manufacturer.

8.9 Month and Year Information

Packages may contain date-related information associated with manufacture, packing or import, depending on the applicable requirement.

OCR can detect patterns such as:

- MFG: 06/2026
- PKD: 06/2026
- MFD: JUN 2026

Software Relevance

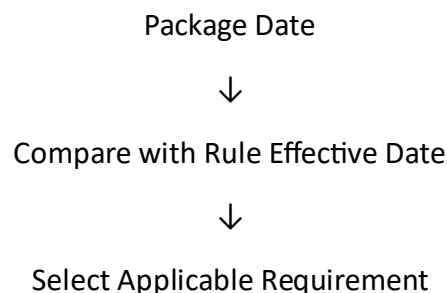
The system can use:

- OCR
- Date pattern recognition
- Regular expressions
- Context classification

to identify date information.

The extracted date can also become an input to the **version-aware rule engine**.

For example:



8.10 Declaration Presence vs Declaration Validity

This distinction is extremely important.

DrishtiTathya should differentiate between:

Level 1 — Presence

Is the declaration present?

Level 2 — Extraction

What does the declaration say?

Level 3 — Format

Does it follow the required format?

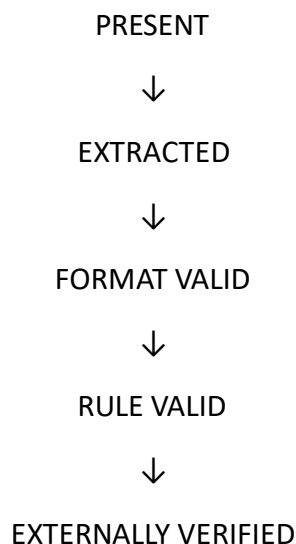
Level 4 — Legal Validation

Does it satisfy the applicable legal requirement?

Level 5 — External Verification

Can the information be verified against an external authoritative source?

Therefore:



Not every level can necessarily be automated from a photograph.

➤ Research Finding — RF031

A declaration being visible on a package does not automatically establish that the declaration is legally valid or factually correct.

8.11 OCR Research for Rule 6

Rule 6 provides a useful test case for evaluating OCR.

For each declaration, the research should record:

Field	OCR Possible?	Pattern Detection?	External Data Needed?	Human Review?
Manufacturer	Yes	Yes	Sometimes	Sometimes
Packer	Yes	Yes	Sometimes	Sometimes

Importer	Yes	Yes	Sometimes	Sometimes
Net quantity	Yes	Yes	Usually not for presence	Sometimes
MRP	Yes	Yes	Context may be needed	Sometimes
Country of origin	Yes	Yes	Sometimes	Sometimes
Consumer care	Yes	Yes	Sometimes	Sometimes
Date information	Yes	Yes	Context may be needed	Sometimes

This table should later be converted into your **technical requirements document**.

8.12 Rule 6 Exceptions and Special Cases

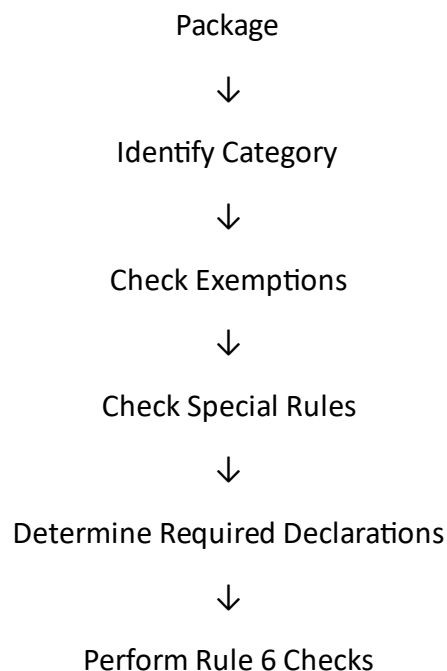
Rule 6 should not be implemented as an unconditional checklist.

The Rules contain exemptions and special provisions for certain categories of packages.

For example, the uploaded 2011 Rules contain provisions under Rule 26 dealing with exemptions for certain packages and quantities.

Therefore, the system must determine whether an exemption or special provision applies before declaring a missing declaration to be a violation.

Software Logic



8.13 Automated Checks vs Human Checks

A major research outcome is that not every requirement can be completely automated.

Suitable for Automation

- Text detection
- MRP extraction
- Quantity extraction
- Date extraction
- Presence/absence detection
- Keyword detection
- Basic formatting checks
- Barcode/QR detection

Partially Automatable

- Manufacturer identification
- Importer identification
- Country-of-origin classification
- Declaration location
- Font/readability analysis
- Special-category classification

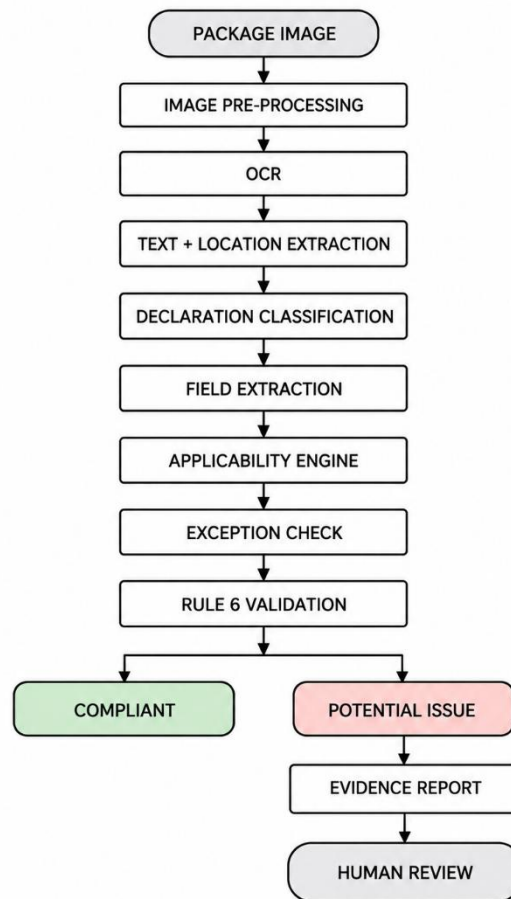
Requires Human or External Verification

- Authenticity of manufacturer
- Actual quantity inside package
- Legal applicability in ambiguous cases
- Authenticity of declarations
- Complex exemptions
- Final enforcement decision

➤ Research Finding — RF032

The most appropriate role of AI in DrishtiTathya is assisted compliance verification rather than autonomous legal enforcement.

8.14 Proposed Rule 6 Compliance Flow



8.15 Proposed Rule Database Structure

The research from Rule 6 can be converted into a structured rule record.

- Rule ID: LM-PCR-R6
- Rule Number: Rule 6
- Requirement: Mandatory declaration
- Field: Net Quantity
- Applicability: Applicable package categories
- Detection: OCR + Pattern Recognition
- Validation: Rule Engine
- Exception: Check applicable exemptions
- Source: Official Legal Metrology Rules / Applicable Amendment
- Effective Date: Version dependent
- Output: Compliant / Potential Issue / Requires Review

This structure can later be implemented in your backend database.

8.16 Explainable Violation Detection

If the system identifies a possible problem, it should explain the result.

For example: POTENTIAL ISSUE

- Field: Net Quantity
- Detected: No recognised net quantity declaration
- Expected: Applicable quantity declaration
- Evidence: Package image + OCR result
- Confidence: 92%
- Status: Requires Human Review

This is more useful than simply displaying:

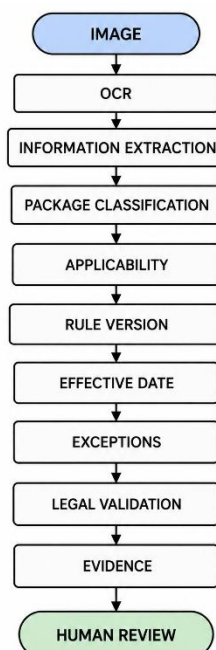
✗ NON-COMPLIANT

because an inspector needs to understand **why** the system generated the result.

8.17 Research Gap Identified

Existing package-scanning systems can identify text or product information, but a regulatory compliance system requires an additional layer that connects extracted information to the applicable legal requirements.

The research therefore identifies the following gap:



➤ Research Finding — RF033

The research gap is not simply the absence of OCR-based package scanning; it is the absence of a reliable connection between extracted package information and a version-aware, applicability-aware regulatory rule engine.

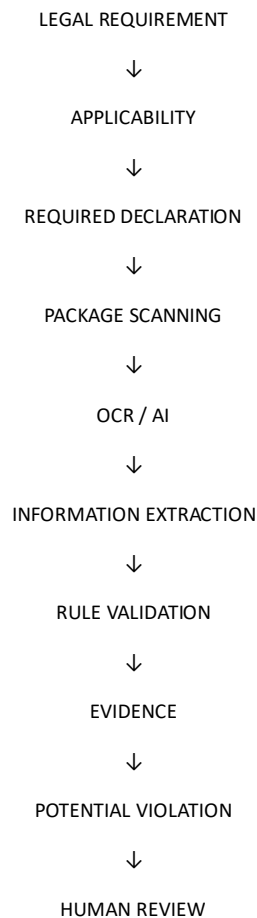
8.18 Rule 6 — Overall Research Conclusion

Rule 6 is central to DrishtiTathya because it provides the basis for identifying the declarations that need to be checked on applicable packages.

However, the research also shows that:

- Not every declaration applies identically to every package.
- Amendments can change requirements.
- Exemptions must be considered.
- OCR provides extracted evidence rather than final legal judgment.
- Some requirements need external information.
- Some checks require physical inspection.
- Ambiguous cases require human review.

Therefore, the proposed compliance model is:



Research Finding — RF034

Rule 6 can provide the foundation for automated declaration verification, but reliable compliance requires the combination of legal rules, applicability conditions, amendments, OCR/AI extraction, evidence and human review.

8.3 Rule 6 — Clause-by-Clause Research

Rule 6 is one of the most important provisions for DrishtiTathya because it specifies declarations that must appear on applicable packages.

The Department of Consumer Affairs identifies mandatory package declarations including manufacturer/packer/importer details, country of origin for imported products, common or generic name, net quantity, manufacture information, best-before/use-by information where applicable, retail sale price, consumer-care details, dimensions where relevant, and unit sale price.

The original 2011 Rule 6 begins by requiring every applicable package to bear a definite, plain and conspicuous declaration on the package or on a securely affixed label.

For DrishtiTathya, each declaration should be treated as an individual **compliance field**.

8.4 Rule 6(1)(a) — Manufacturer, Packer and Importer

The first major declaration category concerns the identity and address of the manufacturer, packer and importer.

The original Rule 6 requires the name and address of the manufacturer and, where the manufacturer is not the packer, the name and address of both manufacturer and packer. For imported packages, the name and address of the importer are required.

Information to Extract

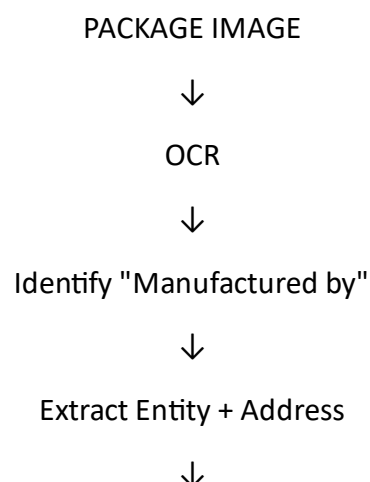
DrishtiTathya should attempt to identify:

- Manufacturer name
- Manufacturer address
- Packer name
- Packer address
- Importer name
- Importer address

Possible OCR Patterns

- Manufactured by:
- Manufactured & Marketed by:
- Packed by:
- Packed & Marketed by:
- Imported by:
- Importer:
- Mfd. by:
- Pkd. by:

Software Flow



Classify as Manufacturer



Rule Validation

Research Finding — RF035

Manufacturer, packer and importer information can be treated as structured entities rather than unstructured OCR text.

This means the AI system should not merely return the entire OCR paragraph. It should convert it into structured fields.

Example:

- Role: Manufacturer
- Name: ABC Foods Pvt. Ltd.
- Address: Pune, Maharashtra
- Confidence: 94%

8.5 Rule 6(1)(b) — Country of Origin

For imported products, country-of-origin information is an important declaration category.

The Department's Legal Metrology overview specifically lists **country of origin for imported products** among the mandatory declarations.

Possible Text Patterns

- Country of Origin: China
- Made in China
- Product of China
- Country of Origin: India

Software Logic :

Is package imported?



YES



Search country-of-origin information



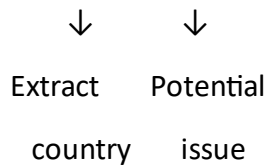
Found?



YES



NO



Research Finding — RF036

Country of origin should be treated as a **conditional declaration**, because its relevance depends on whether the package is an imported product.

Therefore:

The rule engine must determine applicability before declaring a missing country-of-origin field as a violation.

8.6 Rule 6(1)(c) — Common or Generic Name

The package should identify the commodity using its common or generic name, as applicable.

Examples may include:

- Wheat Flour
- Bath Soap
- Toothpaste
- Rice
- Cooking Oil

The software should distinguish the **brand name** from the **commodity name**.

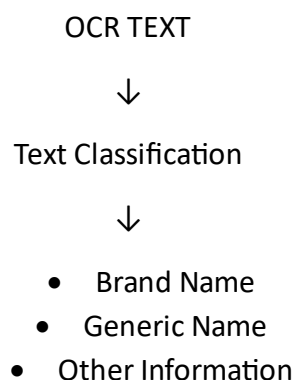
For example:

- Brand:ABC
- Generic / Commodity Name:Wheat Flour

Software Relevance

This requires more than OCR.

The system needs **semantic classification**.



➤ Research Finding — RF037

Rule-based compliance may require semantic understanding of extracted text, not merely character recognition.

This is one reason DrishtiTathya requires an information-extraction layer between OCR and the legal rule engine.

8.7 Rule 6(1)(d) — Net Quantity

Net quantity is one of the most important declarations for packaged commodities.

The Department identifies net quantity, expressed in terms of standard units of weight, measure or number, as a mandatory package declaration.

The Rules also contain separate provisions concerning the manner in which quantity is declared and how quantity variations are treated.

Examples

1. 500 g
2. 1 kg
3. 250 ml
4. 1 L
5. 10 tablets

Information Extraction

DrishtiTathya should extract:

1. Value = 500
2. Unit = g
3. Field = Net Quantity

Validation

The software can check:

- Whether a quantity declaration is detected
- Whether a recognised unit is present
- Whether the value can be parsed
- Whether the declaration format appears valid
- Whether the applicable rule permits the particular form

Important Limitation

The camera can determine the **declared quantity**.

It cannot determine the **actual physical quantity** inside the package.

Therefore:

Declared Quantity



OCR Validation

is different from:

Actual Quantity



Physical Measurement

➤ **Research Finding — RF038**

DrishtiTathya can automate declaration-level quantity checks, but actual quantity verification remains a physical inspection task.

8.8 Rule 6(1)(e) — Date Information

Date-related information is another important package field.

The Department's overview identifies **month and year of manufacture** and, where applicable, **best before/use by information** among mandatory declarations.

Possible package patterns include:

1. MFG: 06/2026
2. MFD: JUN 2026
3. PKD: 06/2026
4. Best Before: 12 Months
5. Use By: 06/2027

Software Process

OCR



Date Pattern Detection



Date Classification



Manufacture / Packing / Use-by /

Best-before



Rule Validation

➤ **Research Finding — RF039**

Date extraction is technically feasible using OCR and pattern recognition, but the meaning of the date must be determined from its surrounding label context.

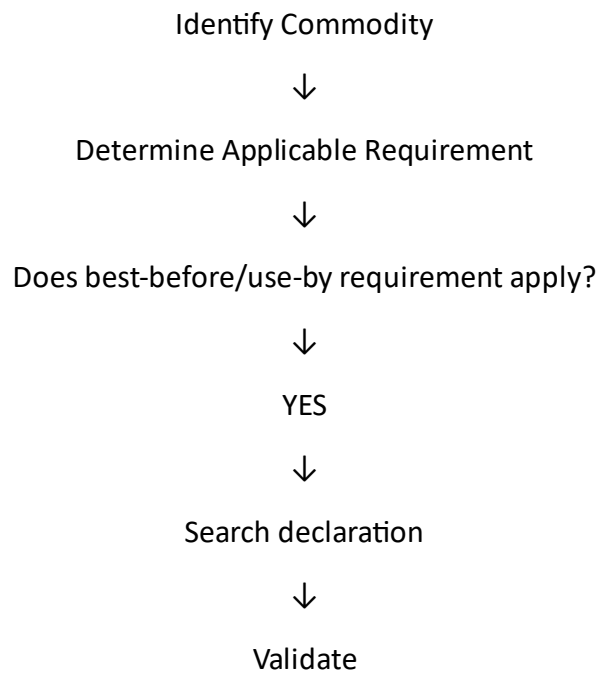
8.9 Best Before / Use By

For commodities that may become unfit for human consumption, the Department identifies best-before/use-by information as an applicable declaration.

This creates an important **conditional rule**.

The system should not simply search every package for the same date field without considering the commodity category and applicable provisions.

Proposed Logic



8.10 Rule 6(1)(f) — Retail Sale Price / MRP

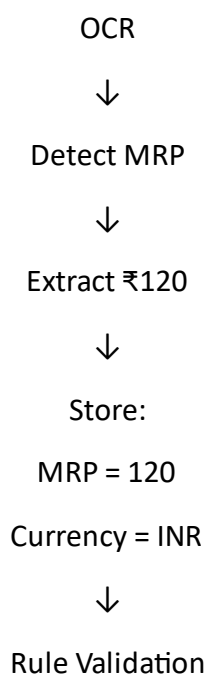
Retail sale price is one of the most important fields for consumer-facing package compliance.

The Department identifies the retail sale price in the form of **Maximum Retail Price (MRP)**, including taxes, among the mandatory declarations.

Example

MRP ₹120 Inclusive of all taxes

OCR Extraction :



Important Distinction

The system should distinguish: **MRP detection** from **MRP legal validation** because detecting a number after the word "MRP" does not by itself establish complete legal compliance.

➤ **Research Finding — RF040**

MRP extraction is highly suitable for automation, while complete legal validation may require additional package context and applicable rules.

8.11 Rule 6(1)(g) — Consumer Care Details

Consumer-care information is another important declaration category identified by the Department.

Possible patterns include:

- Consumer Care:
- Customer Care:
- For complaints contact:
- Toll Free:
- Email:
- Phone:

OCR + Pattern Detection

The system can identify:

- Telephone numbers
- Mobile numbers
- Email addresses
- Postal addresses
- Consumer-care keywords

Example

- Customer Care
- 1800-XXX-XXXX
- care@example.com

The system can extract:

- Field: Consumer Care
- Phone: 1800-XXX-XXXX
- Email: care@example.com

Limitation

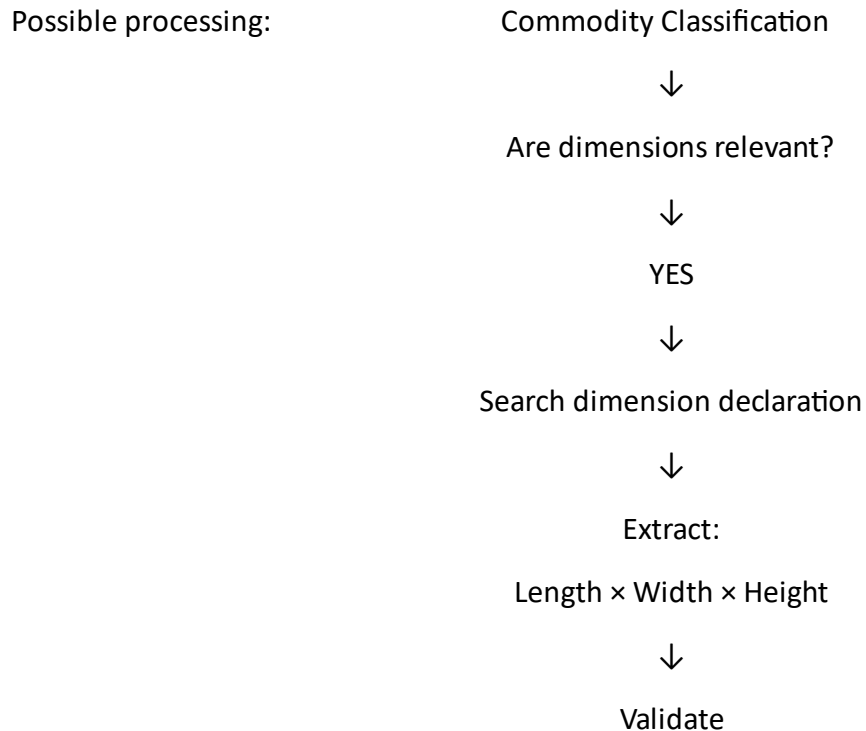
The system cannot establish from the image alone whether the telephone number or email address is genuine.

8.12 Dimensions of Commodity

The Department also lists dimensions where the sizes of the commodity contained in the package are relevant. Where different pieces have different dimensions, the dimensions of each such different piece may need to be mentioned.

Software Relevance

This is a more difficult declaration to automate because the system first needs to determine whether dimensions are relevant to the particular commodity.



➤ Research Finding — RF041

Some legal requirements depend on the physical characteristics or category of the commodity and therefore cannot be implemented as universal text searches.

8.13 Unit Sale Price

The Department's current Legal Metrology overview also lists **unit sale price** among the mandatory declaration categories.

This is particularly relevant to automated compliance because it introduces another numerical field that the system may need to distinguish from:

- MRP
- Net quantity
- Discounted price
- Offer price

Example Concept

- MRP = ₹100
- Net Quantity = 500 g

- Unit Sale Price = ₹200/kg

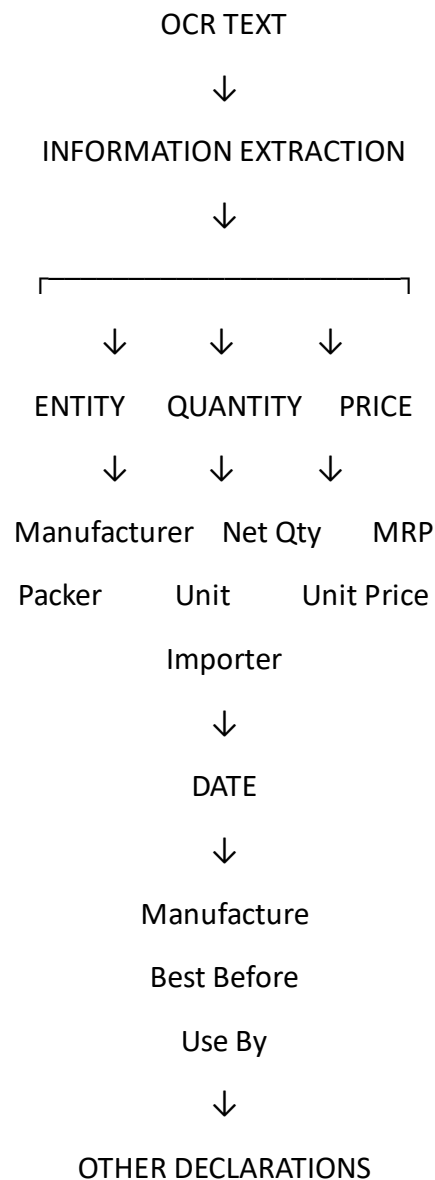
The software should not confuse these values.

➤ **Research Finding — RF042**

The information-extraction layer must classify numerical values according to their semantic meaning rather than treating every detected number as a price.

8.14 Declaration Classification Model

Based on the Rule 6 research, DrishtiTathya can classify extracted text into structured fields.



This is a major difference between a simple OCR application and DrishtiTathya.

8.15 Rule 6 — Machine-Detectable vs Human-Dependent

Requirement	Machine Detection	Legal Validation

Manufacturer name	High	May require external verification
Manufacturer address	High	May require verification
Packer	High	May require verification
Importer	High	May require verification
Country of origin	High	Context-dependent
Generic name	Medium–High	Context-dependent
Net quantity	High	Declaration can be checked; actual quantity requires measurement
Manufacture date	High	Context required
Best before / use by	High	Commodity/context dependent
MRP	High	Context/rule validation required
Consumer care	High	Authenticity may require external verification
Dimensions	Medium	Commodity-dependent
Unit sale price	Medium–High	Requires contextual interpretation

➤ **Research Finding — RF043**

The strongest automation opportunities are declaration presence and information extraction. The weakest areas are authenticity, physical measurement and ambiguous legal applicability.

8.16 Rule 6 — Old Rule vs Updated Framework

The original 2011 Rule 6 provides the base declaration framework. However, the Department's current Legal Metrology overview demonstrates that the mandatory-declaration framework must be read together with subsequent amendments and current provisions.

Therefore, DrishtiTathya should maintain:

2011 BASE RULE



AMENDMENT



UPDATED REQUIREMENT



EFFECTIVE DATE



CURRENT RULE VERSION

Research Finding — RF044

Rule 6 should be represented in the software as a collection of versioned declaration requirements rather than as one fixed checklist.

8.17 Rule 6 Compliance Matrix

This is a table you should definitely keep in your Word document.

Declaration	Extract	Validate	Exception Check	Human Review
Manufacturer	✓	✓	✓	Sometimes
Packer	✓	✓	✓	Sometimes
Importer	✓	✓	✓	Sometimes
Country of origin	✓	✓	✓	Sometimes
Generic name	✓	✓	✓	Sometimes
Net quantity	✓	✓	✓	Yes, for physical quantity
Manufacture date	✓	✓	✓	Sometimes
Best before / Use by	✓	✓	✓	Sometimes
MRP	✓	✓	✓	Sometimes
Consumer care	✓	✓	✓	Sometimes
Dimensions	✓	✓	✓	Sometimes
Unit sale price	✓	✓	✓	Sometimes

8.18 Rule 6 — Proposed Software Rule

The legal requirement can be translated into the following **software concept**:

FOR EACH APPLICABLE PACKAGE:

1. Determine applicable Rule 6 version.
2. Determine required declarations.
3. Scan package.
4. Extract text and visual information.
5. Classify extracted information.
6. Match extracted fields with required declarations.
7. Check applicable exceptions.
8. Validate detectable requirements.
9. Generate evidence.
10. Assign confidence.
11. Flag uncertain cases for human review.

This is a **proposed software interpretation**, not a replacement for the legal text.

8.19 Key Research Insight

The research of Rule 6 leads to an important distinction:

OCR



"What text is present?"

INFORMATION EXTRACTION



"What does the text represent?"

RULE ENGINE



"What information is legally required?"

COMPLIANCE ENGINE



"Does the extracted information satisfy the applicable requirement?"

HUMAN REVIEW



"Should this result be accepted as an enforcement finding?"

➤ **Research Finding — RF045**

A legally useful compliance system requires at least four distinct layers: OCR, information extraction, rule selection and compliance validation.

8.20 Rule 6 Conclusion

Rule 6 is a critical component of the DrishtiTathya research because it transforms the legal requirement for package declarations into identifiable information fields.

The research demonstrates that a robust system should not simply search an image for keywords.

Instead, it should:

PACKAGE



OCR



INFORMATION EXTRACTION



PACKAGE CLASSIFICATION



APPLICABILITY



RULE VERSION



REQUIRED DECLARATIONS



EXCEPTION CHECK



VALIDATION



EVIDENCE



HUMAN REVIEW

➤ **Final Research Finding — RF046**

Rule 6 provides the foundation for the declaration-verification component of DrishtiTathya. Its effective implementation requires combining legal research, amendment tracking, applicability analysis, OCR, semantic information extraction, rule-based validation and human review.

9. MANNER OF MAKING DECLARATIONS

9.1 Rule 9 — Manner in Which Declaration Shall Be Made

Rule 9 of the Legal Metrology (Packaged Commodities) Rules, 2011 specifies requirements concerning the manner in which declarations on packages are to be presented.

This Rule is important for DrishtiTathya because compliance is not limited to the **presence of a declaration**. The manner in which the declaration is presented can also be relevant.

The original Rules require declarations to be **legible and prominent**. They also contain specific requirements concerning the presentation of retail sale price and net quantity declarations, including contrast with the background in specified circumstances.

9.2 Legibility and Prominence

The Rules require declarations on packages to be legible and prominent.

This creates an additional dimension of compliance.

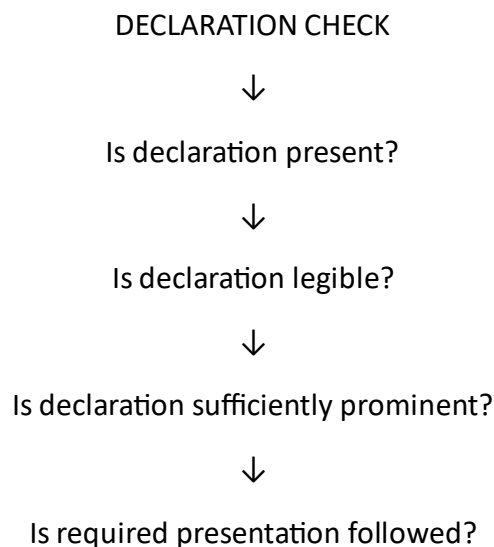
Previously, the system was asking:

Is the declaration present?

Rule 9 introduces another question:

Can the declaration be clearly read and identified?

Therefore:



➤ **Research Finding — RF047**

Declaration presence and declaration readability are separate compliance checks.

9.3 Contrast of MRP and Net Quantity

The original Rule 9 requires numerals relating to retail sale price and net quantity to be printed, painted or inscribed in a colour that contrasts conspicuously with the background of the label. The Rule also contains exceptions for certain information blown, formed or moulded on glass or plastic surfaces.

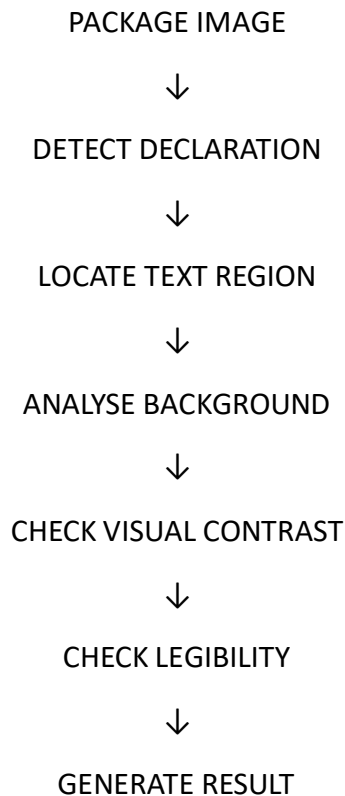
Software Relevance

This creates an opportunity for computer vision.

The system could analyse:

- Text region
- Background region
- Relative contrast
- Character visibility
- Text size
- Position of declaration

Proposed Computer-Vision Flow



Important Limitation

A computer-vision model should not automatically declare a legal violation solely because the calculated visual contrast is low.

Lighting, camera angle, image quality, reflections and package curvature can affect the analysis.

Therefore: **Low-confidence visual findings should be sent for human review.**

9.4 Handwritten or Hand-Script Declarations

The original Rule 9 also provides that where a declaration is presented in handwriting or hand-script, it should be clear, unambiguous and legible.

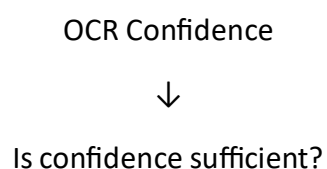
Software Relevance

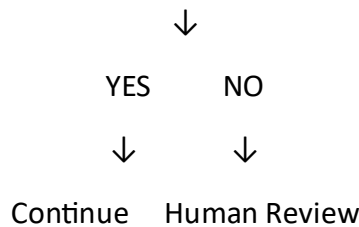
This creates an interesting OCR challenge.

Standard OCR may perform poorly on:

- Handwriting
- Stylised fonts
- Curved surfaces
- Low-resolution images

Therefore, DrishtiTathya could use:





➤ **Research Finding — RF048**

OCR confidence should be treated as an input to the review process rather than as proof of legal compliance.

9.5 Declaration Through Liquid

The original Rule 9 provides that a declaration should not be made in a manner that requires it to be read through the liquid commodity contained in the package.

Software Relevance

This demonstrates that some requirements concern the **physical presentation of information**, not just the text itself.

A package image could potentially help identify:

- Text location
- Whether text appears behind liquid
- Whether text is visible
- Whether the package uses transparent material

However, this is a difficult computer-vision problem and should initially be treated as an **assisted inspection feature**, not a fully automated legal check.

9.6 Outside Container or Wrapper

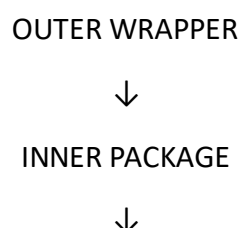
Rule 9 also addresses packages that are provided with an outside container or wrapper.

The Rule requires the outside container or wrapper to contain the required declarations in specified circumstances. An exception exists where the outside wrapper is transparent and the declarations on the package can be easily read through it.

Software Relevance

This introduces the concept of **multi-layer packaging**.

For example:

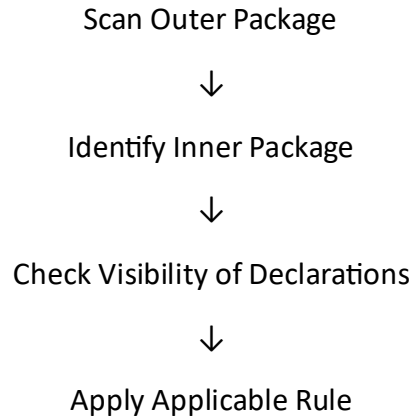


DECLARATIONS

DrishtiTathya should not assume that scanning only one visible surface is always sufficient.

Proposed Feature

Multi-Layer Package Inspection :



9.7 Language Requirement

The original Rule 9 provides that the particulars of declarations required under the Rules shall be in **Hindi in Devanagari script or English**, while allowing additional languages as well.

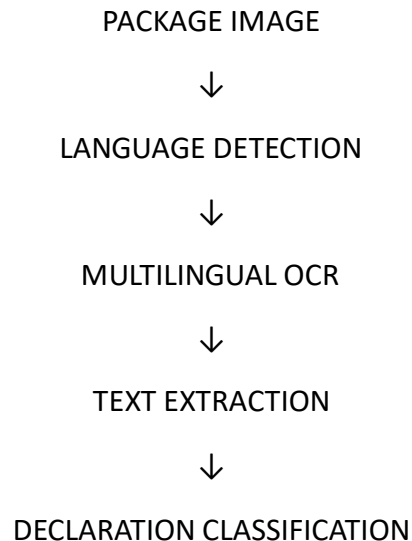
Software Relevance

This creates a multilingual OCR requirement.

DrishtiTathya may encounter:

- English
- Hindi
- Hindi + English
- Regional language + English
- Multiple-language packaging

Proposed Architecture





RULE VALIDATION

➤ Research Finding — RF049

A multilingual package-compliance system requires language-aware OCR rather than relying only on English text recognition.

9.8 Rule 9 — Machine-Detectable Requirements

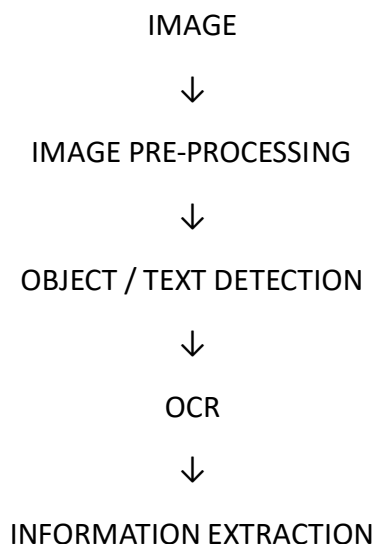
The requirements can be divided according to their technical detectability.

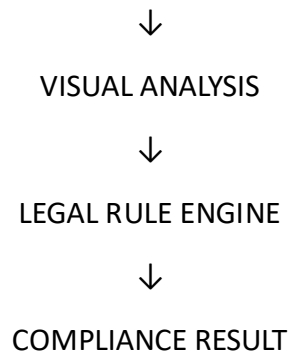
Requirement	Computer Vision / OCR	Human Review
Declaration present	High	Sometimes
Declaration legibility	Medium	Yes for borderline cases
Text prominence	Medium	Yes
MRP visibility	High	Sometimes
Net quantity visibility	High	Sometimes
Text/background contrast	Medium–High	Borderline cases
Handwritten declaration	Medium	Often
Declaration behind liquid	Low–Medium	Often
Outer-wrapper visibility	Medium	Often
Language detection	High	Sometimes

9.9 Rule 9 — From OCR to Computer Vision

Rule 9 demonstrates why DrishtiTathya should not be designed as an OCR-only application.

The technical architecture should include:





OCR answers:

What does the package say?

Computer vision additionally helps answer:

How is the information visually presented?

The legal rule engine answers:

What presentation is required?

9.10 Confidence-Based Verification

Because image quality can affect visual analysis, DrishtiTathya should maintain confidence values.

Example:

- Declaration: MRP
- OCR Confidence: 96
- Visual Legibility: 88%
- Contrast Analysis: 82%
- Overall AI Confidence: 89%
- Result: Potential Issue / Requires Review

The exact method of combining these confidence values should be determined during technical research and testing rather than assumed at this stage.

➤ **Research Finding — RF050**

AI confidence should support prioritisation and human review; it should not replace the legal determination of compliance.

9.11 Rule 9 — Evidence Capture

For every visual compliance flag, the system should retain evidence.

Example:

- Issue: Possible low-contrast MRP declaration
- Evidence: Original package image + Highlighted MRP region
- Analysis: Text detected, Background detected, Contrast analysis performed
- Confidence: 84%
- Status: Requires Human Review

This provides an inspector with a visual basis for reviewing the AI result.

9.12 Rule 9 — Proposed Software Requirements

Based on the research, DrishtiTathya may require the following capabilities:

1. Text detection
 2. OCR
 3. Multilingual OCR
 4. Text-region localisation
 5. Font/readability analysis
 6. Background analysis
 7. Contrast analysis
 8. Package-layer detection
 9. Declaration-location detection
 10. Confidence estimation
 11. Evidence highlighting
 12. Human-review workflow
-

9.13 Rule 9 Research Questions

Before finalising this module, the following questions should be researched:

1. Has Rule 9 been amended after 2011?
 2. Which amendment notifications affect Rule 9?
 3. What exact changes were introduced?
 4. What are the applicable effective dates?
 5. Are there product-specific exceptions?
 6. What constitutes a measurable readability issue?
 7. What font/character-height requirements are specified elsewhere in the Rules?
 8. Which declarations have special presentation requirements?
 9. Which visual requirements can be reliably measured from photographs?
 10. What image quality is necessary for reliable analysis?
-

9.14 Research Finding

The study of Rule 9 produces an important insight for DrishtiTathya:

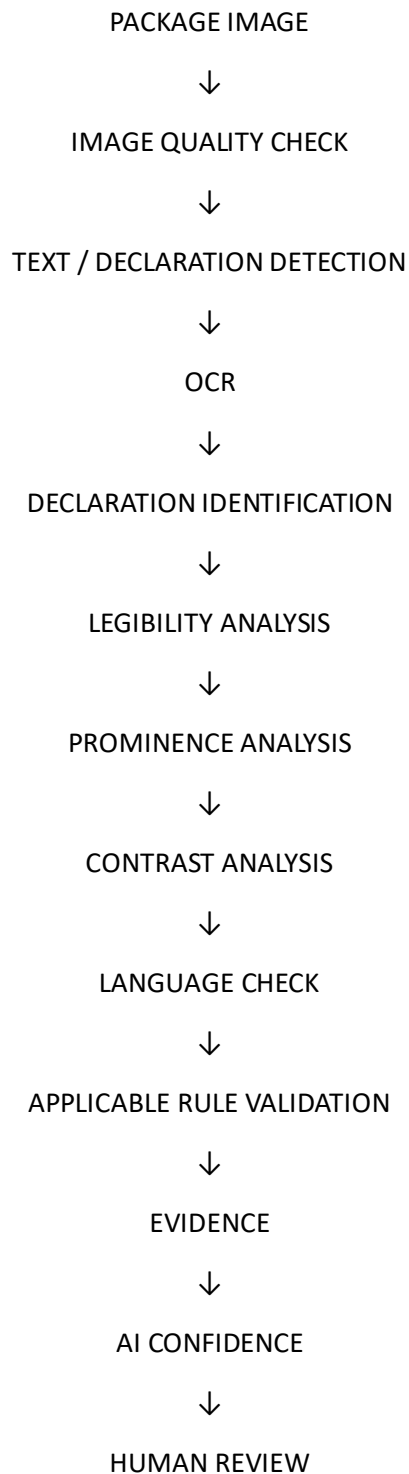
Compliance is not limited to whether required information exists. The presentation, visibility, legibility and context of the declaration can also be relevant.

Therefore, the system should combine:

OCR + Computer Vision + Legal Rule Engine + Human Review

rather than relying exclusively on OCR.

9.15 Rule 9 — Proposed Compliance Flow



9.16 Rule 9 Conclusion

Rule 9 expands the scope of DrishtiTathya from **text extraction** to **visual compliance analysis**.

The research demonstrates that a package may need to be evaluated not only for the presence of declarations but also for their presentation and readability.

Therefore:

OCR + COMPUTER VISION + RULE ENGINE + EVIDENCE + HUMAN REVIEW

forms a stronger technical approach for packaged-commodity compliance.

Research Finding — RF051

Rule 9 provides a technical bridge between legal declaration requirements and computer-vision-based inspection because several requirements concern how declarations are presented and perceived on the package.

10. RULE 10 — DECLARATION OF NAME AND ADDRESS OF MANUFACTURER, PACKER AND IMPORTER

10.1 Basic Requirement

Rule 10 specifies requirements regarding the name and address of the manufacturer, packer and importer appearing on a package.

The Rule states that, subject to Rule 6, every package kept, offered or exposed for sale or sold shall conspicuously bear the name and complete address of the manufacturer. Where the manufacturer is different from the packer, the name and address of both are required. For imported packages, the name and address of the importer are required.

Software Interpretation

DrishtiTathya should therefore identify the role associated with each detected business name/address, rather than simply searching for any company name.

10.2 Manufacturer, Packer and Importer

The package may contain different entities:

Entity	Information to Detect
Manufacturer	Name + address
Packer	Name + address where applicable
Importer	Name + address where applicable
Indian packer/importer	Required in specified imported-product situation

The distinction is important because:

Manufacturer ≠ Packer ≠ Importer

The software should not automatically treat every detected company name as the manufacturer.

10.3 Imported Products

Where a commodity is manufactured outside India but packed in India, Rule 10 requires the package to contain, on the principal display panel, the name and complete address of the packer or importer in India.

Proposed Detection Flow



Research Finding — RF052

Country-of-origin detection and importer/packer identification can be connected into a single compliance workflow.

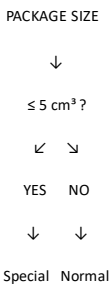
10.4 Small Packages

The Rule provides a specific provision for packages having a capacity of 5 cubic centimetres or less. For such packages, a mark or inscription that enables the consumer to identify the manufacturer, packer or importer can be sufficient for compliance with this sub-rule.

Software Relevance

This is an example of why the system should not use a single universal rule such as: “Full address must always be visible.”

Instead:



provision requirement

10.5 Meaning of Complete Address

The Rule explains the meaning of “complete address.”

It includes the postal address of the factory and, in other cases, the relevant street and premises details together with the city and State or PIN Code, so that the consumer can identify and locate the manufacturer, packer or importer.

Software Relevance

This means address verification should involve more than checking whether the word “Address” appears.

The system may extract:

ENTITY NAME + STREET / PREMISES + CITY / STATE + PIN CODE

and construct a structured address record.

10.6 Actual Corporate or Business Name

Rule 10 specifies that the name of the manufacturer, packer or importer should be the actual corporate name, or, where the business is not incorporated, the name under which the business is conducted in India.

Software Relevance

DrishtiTathya can classify detected names into:

Brand Name



Brand Owner



Manufacturer



Packer



Importer

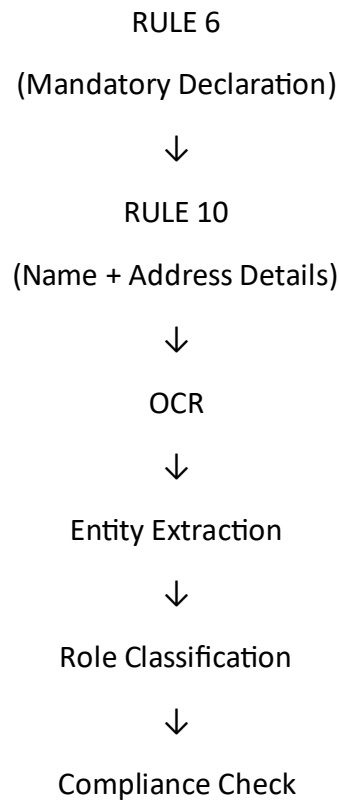
This prevents the system from incorrectly treating a brand name as the legal manufacturer.

10.7 Rule 10 + Rule 6 Relationship

Rule 10 should not be treated as an isolated rule.

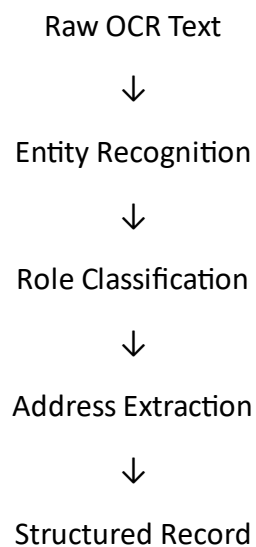
Rule 6 establishes the declaration requirement, while Rule 10 provides additional detail regarding the manufacturer, packer and importer declarations.

Therefore:



10.8 Information Extraction Model

DrishtiTathya can convert package text into structured information:



Example:

- ABC Foods Pvt. Ltd.
- Plot 25, Industrial Area
- Pune, Maharashtra - 411019

Role → Manufacturer

Name → ABC Foods Pvt. Ltd.

City → Pune

State → Maharashtra

PIN → 411019

This structured representation can then be passed to the rule engine.

10.9 Proposed Software Rule

A simplified software representation could be: IF package is applicable AND manufacturer/packer/importer declaration is required THEN identify applicable entity roles.

IF manufacturer ≠ packer , check manufacturer details , check packer details

IF imported , check importer details

IF foreign manufactured + packed in India, check Indian packer/importer details , check principal display panel requirement

IF package capacity $\leq 5 \text{ cm}^3$, apply small-package provision , generate compliance status

This is a software interpretation, not a replacement for the legal text.

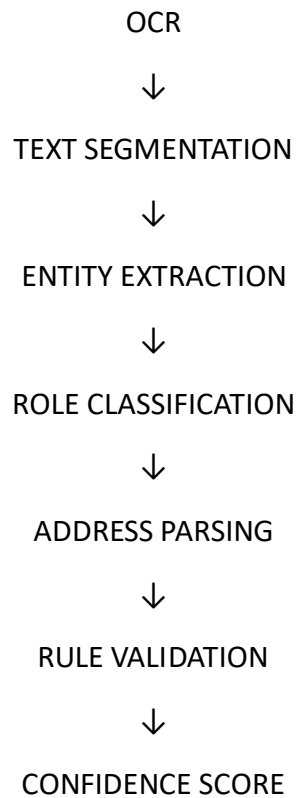
10.10 AI Detection Challenges

The system may encounter:

- Brand name confused with manufacturer
- Multiple company names
- “Marketed by”
- “Manufactured by”
- “Packed by”
- “Imported by”
- Very small text
- Curved packaging
- Decorative fonts
- Multilingual addresses
- Incomplete OCR
- Address split across multiple lines

Therefore, simple keyword matching is insufficient.

Proposed AI Pipeline



10.11 Evidence-Based Verification

For every detected manufacturer/packer/importer declaration, the system should preserve:

- Original image
- OCR text
- Highlighted declaration region
- Detected entity
- Detected role
- Extracted address
- Rule applied
- Confidence score

Example:

Detected: Imported by XYZ Imports Pvt. Ltd.

Role: Importer

Address: Mumbai, Maharashtra

Evidence: [Highlighted package region]

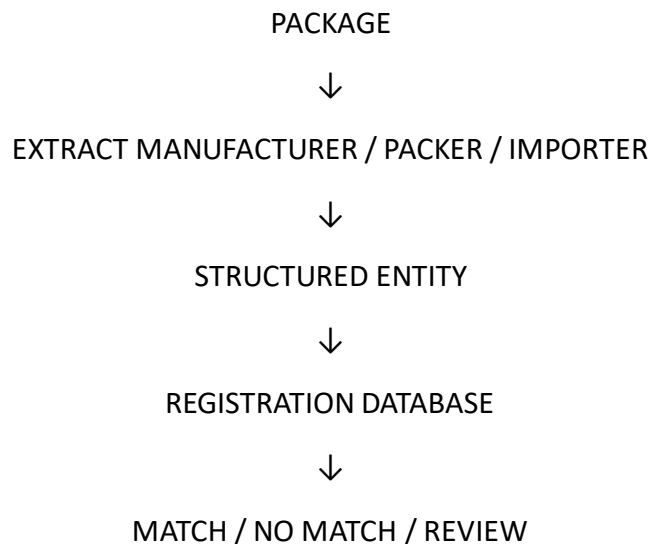
AI Confidence: 94%

Status: Detected — Requires Rule Validation

10.12 Connection with Registration Data

The Rules also contain provisions relating to registration of manufacturers, packers and importers under Rule 27. The uploaded Rules state that applicants provide their name, complete address and commodities pre-packed or imported, and registration certificates are issued where the application is complete.

This creates a potential future integration:



Important Limitation

A database mismatch should initially be treated as a verification flag, not automatically as proof of a legal violation.

10.13 Rule 10 — Automation Classification

Requirement	Automation Potential
Detect company name	High
Detect address	High
Identify “manufactured by”	High
Identify “packed by”	High
Identify “imported by”	High
Distinguish entity roles	Medium–High
Parse address	High
Verify complete address	Medium
Verify actual legal entity	Medium
Verify registration	Requires authorised data source
Final legal determination	Human review

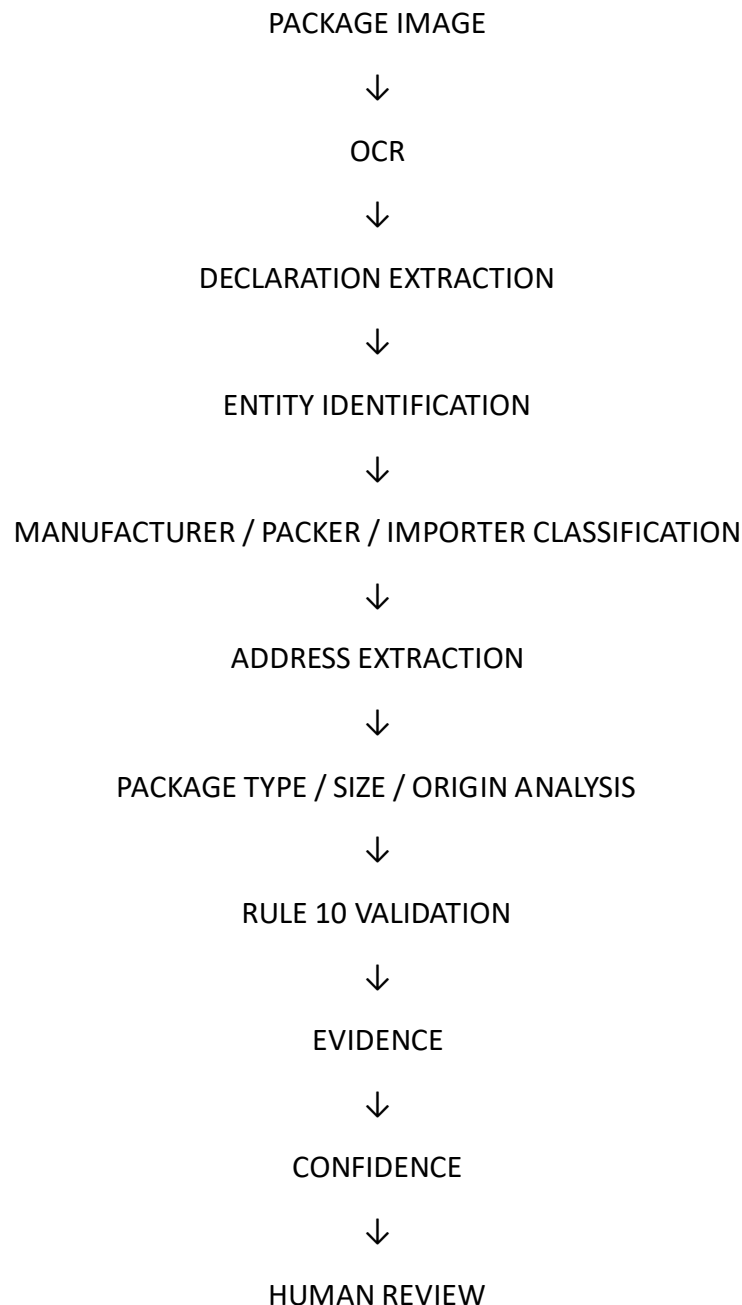
10.14 Research Finding

RF053

Manufacturer, packer and importer information should be represented as separate structured entities in the compliance system rather than as a single “company details” field.

This will make the rule engine more accurate and explainable.

10.15 Rule 10 — Proposed Compliance Flow



10.16 Rule 10 Conclusion

Rule 10 is particularly important for DrishtiTathya because it converts unstructured package text into legally meaningful entities.

The system should therefore move from:

“Find a company name.”

to:

“Identify who the entity is, what role it performs, where it is located, and which legal requirement applies.”

Key Research Finding — RF054

Rule 10 demonstrates the need for entity-aware information extraction, contextual rule application and evidence-based verification in an automated packaged-commodity compliance system.

11. RULE 11 — GENERAL PROVISIONS RELATING TO DECLARATION OF QUANTITY

11.1 Basic Requirement

Rule 11 deals with the **declaration of the net quantity of the commodity contained in a package**.

The Rule specifies that, while declaring the net quantity, the weight of wrappers and materials other than the commodity is to be excluded.

Software Interpretation

DrishtiTathya should therefore distinguish between:

TOTAL PACKAGE WEIGHT



PACKAGING MATERIAL



NET COMMODITY QUANTITY

The value declared on the package represents the **commodity quantity**, not the combined weight of the commodity and its packaging materials.

11.2 Net Quantity

Net quantity is a core declaration because consumers need to know the actual quantity of commodity being purchased.

For software purposes, the system should identify:

- Numerical value
- Unit
- Quantity type
- Related declaration
- Location on package

Example: 500 g

- Value → 500
- Unit → g
- Quantity Type → Mass

11.3 Exclusion of Wrappers and Packaging Material

The Rule specifically provides that wrappers and materials other than the commodity are excluded while declaring net quantity.

Therefore: Net Quantity= Commodity Quantity NOT Commodity + Wrapper + Container

Research Finding — RF055

Net quantity is a commodity-level measurement and should not be confused with the gross weight of the complete package.

11.4 Quantity Must Match the Commodity

Rule 11 contains provisions dealing with variation in weight or measure and circumstances in which the declared quantity corresponds to the quantity at the time of packing.

This is important because some commodities may naturally undergo changes due to environmental conditions.

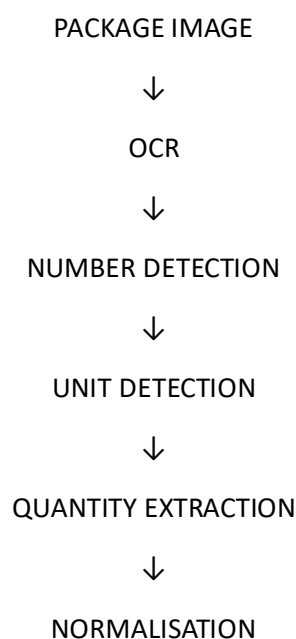
Therefore, DrishtiTathya should not assume:

Declared quantity = physically measured quantity at every point in time without considering the applicable legal provision.

11.5 Software Detection of Quantity

The quantity declaration can be extracted using OCR.

Proposed Pipeline :





RULE VALIDATION

Example:

OCR: NET WT. 500 g

Extracted:

- Quantity = 500
- Unit = g
- Type = Mass

11.6 Quantity Classification

The system should classify quantity according to how the commodity is sold.

The Rules contain provisions for quantity expressed by:

- Mass
- Length
- Area
- Volume
- Number

The uploaded Rules specify, for example, length where the commodity is sold by linear measure, area where sold by area measure, volume for liquids/cubic measure, and number where sold by number.

Proposed Data Structure

Quantity

└— Value

└— Unit

└— Measurement Type

└— Mass

└— Length

└— Area

└— Volume

└— Number

11.7 Quantity and Unit Recognition

OCR should not extract only the numerical value.

For example:500 g

should not become: Quantity = 500

It should become:

- Quantity = 500
- Unit = gram
- Type = mass

Similarly: 1 L

should become:

- Quantity = 1
 - Unit = litre
 - Type = volume
- **Research Finding — RF056**

Quantity extraction must be unit-aware; a numerical value without its measurement unit is incomplete information.

11.8 Additional Quantity Information

The Rules also provide circumstances where an additional declaration regarding the **number of commodities contained in the package** may be made when quantity is declared by mass.

This suggests that the system should distinguish between:

PRIMARY QUANTITY

+

ADDITIONAL QUANTITY INFORMATION

For example:

Net Weight: 500 g

Contents: 10 pieces

The software should not incorrectly interpret “10 pieces” as the primary net quantity when the legally relevant quantity declaration is based on mass.

11.9 Dimensions and Quantity

The Rules also address situations where declaring quantity by weight, measure or number may not provide the consumer with full information regarding the dimensions or number of commodities contained in the package.

This is relevant to the system because **quantity compliance may depend on product characteristics**.

Therefore, the rule engine should be capable of handling:

Quantity +Product Category +Measurement Type +Additional Information

rather than checking every package using one universal condition.

11.10 Unit Normalisation

For reliable software processing, extracted units should be normalised.

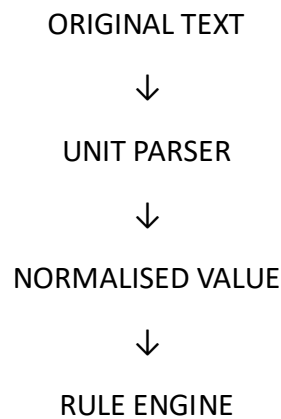
Example:

- 500 g
- 500 gm
- 0.5 kg

may be converted internally into a common representation for comparison.

However, the system should preserve the **original declaration exactly as observed** for evidence purposes.

Proposed Architecture

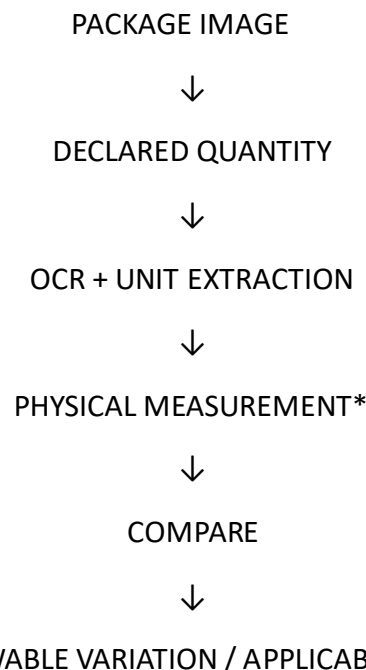


while retaining: ORIGINAL OCR TEXT + ORIGINAL IMAGE for audit purposes.

11.11 Quantity Validation

A future version of DrishtiTathya could compare the declared quantity against an independently measured quantity.

Possible workflow:





FLAG FOR REVIEW

*Physical measurement would require appropriate calibrated equipment or an authorised measurement process.

Important Limitation

A camera image alone cannot reliably establish the actual mass or volume of a commodity.

Therefore:

Image-based quantity extraction is different from physical quantity verification.

11.12 AI + Measurement Distinction

This distinction is important for the project:

Function	AI/Image System	Physical Instrument
Read declared quantity	✓	—
Identify unit	✓	—
Identify quantity type	✓	—
Verify printed declaration	✓	—
Measure actual mass	✗	✓
Measure actual volume	✗	✓
Compare actual vs declared	Partially	✓
Final legal determination	Human authority	Human authority

11.13 Evidence Capture

For every quantity-related finding, the system should retain:

- Original package photograph
- Cropped quantity declaration
- OCR result
- Extracted numerical value
- Extracted unit
- Normalised quantity
- Applicable rule
- Confidence score
- Review status

Example:DECLARATION

Net Qty: 500 g

OCR Confidence: 97%

Extracted:

- Value = 500
- Unit = g
- Type = Mass

Status:Declaration Detected

11.14 Quantity Compliance Rule Representation

A simplified software representation could be:

- IF package is subject to Chapter II THEN detect declared quantity
- IF quantity declaration is missing FLAG

ELSE extract numerical value , extract unit , classify measurement type , validate against applicable quantity rules , retain evidence

This is a **proposed software interpretation**, not the legal text itself.

11.15 Key Technical Challenge

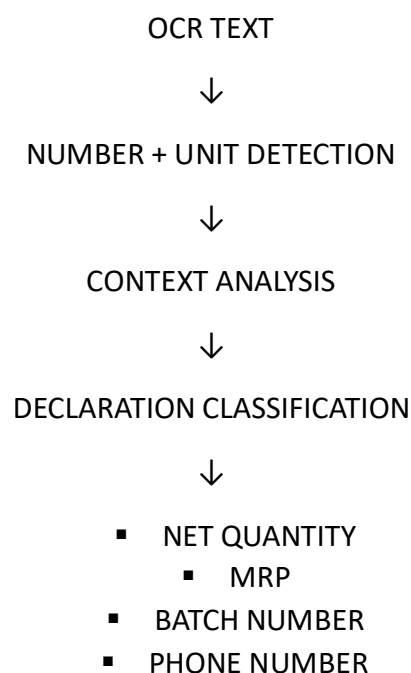
One of the major challenges is distinguishing a genuine quantity declaration from unrelated numbers appearing on packaging.

A package may contain:

- 500 g
- ₹250
- Batch No. 50023
- Customer Care: 1800...

A basic OCR system may detect all of these as numbers.

Therefore, DrishtiTathya needs **declaration classification**.



11.16 Research Finding

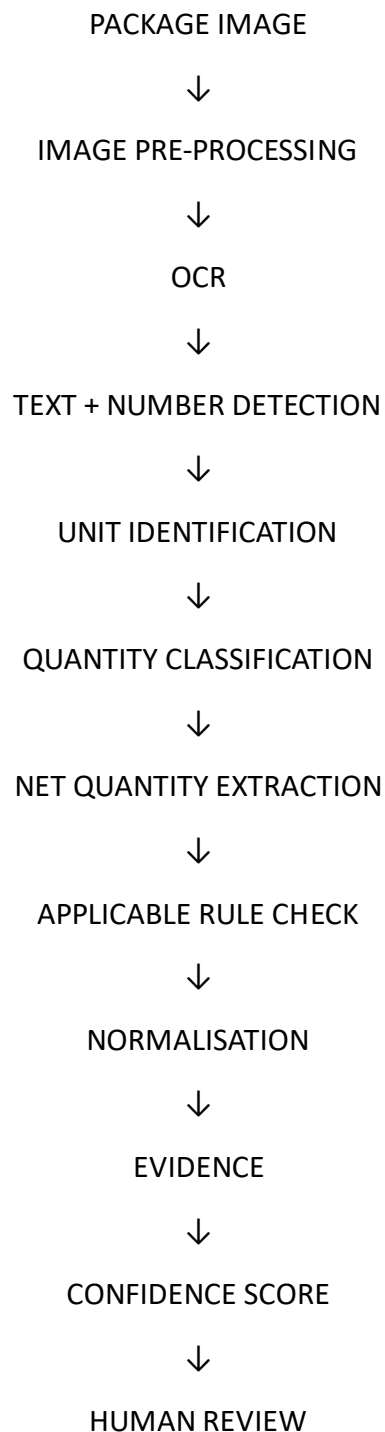
RF057

Quantity extraction is a semantic task, not merely a numerical OCR task.

The system must understand the relationship between:

number + unit + surrounding words + package context.

11.17 Rule 11 — Proposed Compliance Flow



11.18 Rule 11 Conclusion

Rule 11 demonstrates that **net quantity compliance requires more than finding a number on a package**.

DrishtiTathya must determine:

1. Which number represents quantity
2. Which unit is being used
3. What type of measurement it represents
4. Whether additional quantity information is relevant
5. Which legal provision applies
6. Whether the result requires human verification

➤ **Key Research Finding — RF058**

A reliable packaged-commodity compliance system should use unit-aware, context-aware quantity extraction while keeping image evidence and human verification in the compliance workflow.

2. RULE 12 — MANNER IN WHICH DECLARATION OF QUANTITY SHALL BE

12.1 Basic Requirement

Rule 12 specifies **how the quantity of a commodity must be expressed** on a package.

The declaration must use a unit of **weight, measure, number, or an appropriate combination** that gives the consumer accurate and adequate information about the quantity contained in the package.

Software Interpretation

For DrishtiTathya, this means the system should not only detect:

“500”

It should understand:

“500 g” → 500 grams → Mass

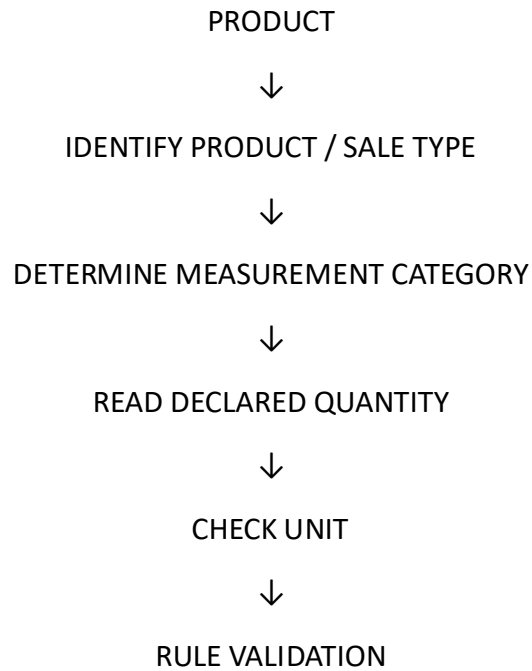
12.2 Classification of Quantity

Except for commodities covered by the Fourth Schedule, Rule 12 specifies the general measurement category according to the nature of the commodity:

Commodity / Sale Method	Quantity Unit
Solid	Mass
Semi-solid	Mass
Viscous	Mass
Mixture of solid and liquid	Mass
Sold by linear measure	Length
Sold by area measure	Area
Liquid / cubic measure	Volume

Sold by number	Number
----------------	--------

Proposed Software Logic:



12.3 Quantity as Mass

For solid, semi-solid, viscous commodities and mixtures of solid and liquid, the general rule uses **mass**.

Examples of extracted declarations could be:

- 100 g
- 500 g
- 1 kg

The software should represent them internally as:

- Value → 500
- Unit → gram
- Measurement Type → Mass

12.4 Quantity as Length

Where a commodity is sold by **linear measure**, the quantity should be expressed in terms of length.

Example:

- m
- 10 m
- 50 cm

The system should classify:

Measurement Type → Length

rather than incorrectly treating the number as mass or volume.

12.5 Quantity as Area

Where the commodity is sold by area measure, the quantity should be represented using an appropriate **area measurement**.

For example:

2 m²

The software should identify:

- Value → 2
- Unit → m²
- Type → Area

12.6 Quantity as Volume

Where the commodity is liquid or sold by cubic measure, the general rule provides for **volume**.

Examples:

- 500 mL
- 1 L
- 2 L

The system should recognise:

Measurement Type → Volume

12.7 Quantity as Number

Where a commodity is sold by number, the quantity should be expressed as **number**.

Examples:

- 10 pieces
- 12 units
- 50 pieces

The system should distinguish: Number = 10

from:

- 10 g
- 10 mL
- 10 cm

Research Finding — RF059

The same numerical value can have completely different legal meanings depending on its unit and commodity context.

12.8 Combination of Quantity Units

Rule 12 allows a quantity declaration to use a **combination of weight, measure or number** where necessary to provide accurate and adequate information to the consumer.

Therefore, the information-extraction system should support multiple quantity fields.

Example:

- Net Weight: 500 g
- Contents: 10 pieces

Possible structured representation:

Primary Quantity→ 500 g

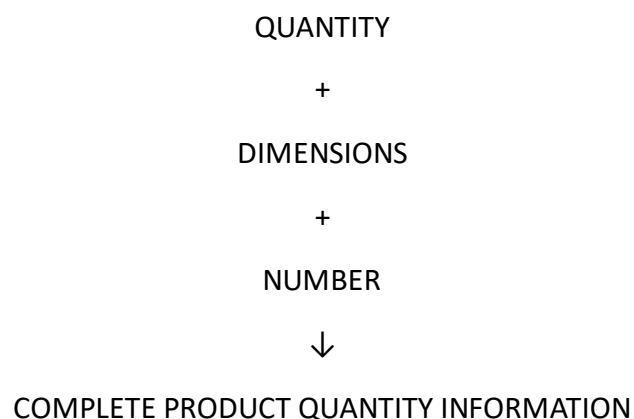
Additional Quantity→ 10 pieces

12.9 Additional Information

Where weight, measure or number alone does not provide sufficient information regarding the **dimensions or number of commodities**, Rule 12 requires the relevant additional declaration where necessary.

This is particularly important for products where size or dimensions affect consumer understanding.

Proposed Software Model



12.10 Same-Panel Information

Where additional information about the commodity is required under the Rule, it is to appear on the **same panel** where the other required information is indicated.

Computer-Vision Relevance

This introduces a spatial relationship:

DECLARATION A + DECLARATION B



SAME PANEL

Therefore, the system can potentially use **text-location detection** to determine whether related declarations appear together.

This should initially be treated as an assisted check rather than a fully automated legal conclusion.

12.11 Misleading Quantity Expressions

Rule 12 also prohibits quantity declarations containing words or expressions that tend to create an **exaggerated, misleading or inadequate impression** regarding quantity.

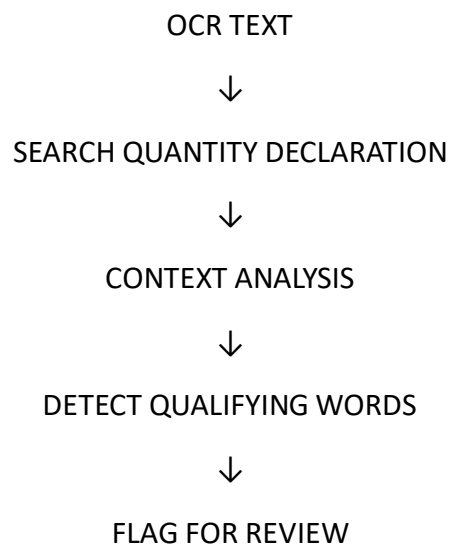
Examples specifically given include:

- “minimum”
- “not less than”
- “average”
- “about”
- “approximately”

and similar expressions.

Software Opportunity

This is highly suitable for NLP-based detection.



Example

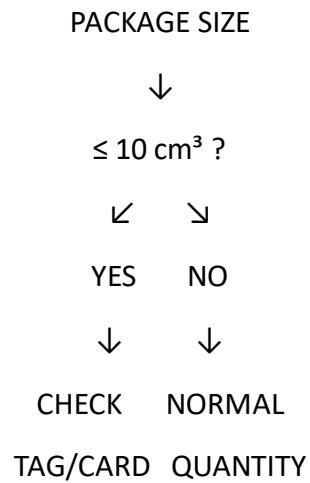
“Approximately 500 g”

The system could generate: **Potential Rule 12 issue — review required** rather than directly declaring a violation.

12.12 Small Packages

Rule 12 contains a special provision for packages having capacity of **10 cubic centimetres or less**. For such packages, the quantity declaration may be made on a tag, card, tape or similar device affixed to the container so that it cannot be removed without opening the container. The device must also contain a mark or inscription enabling identification of the manufacturer or packer.

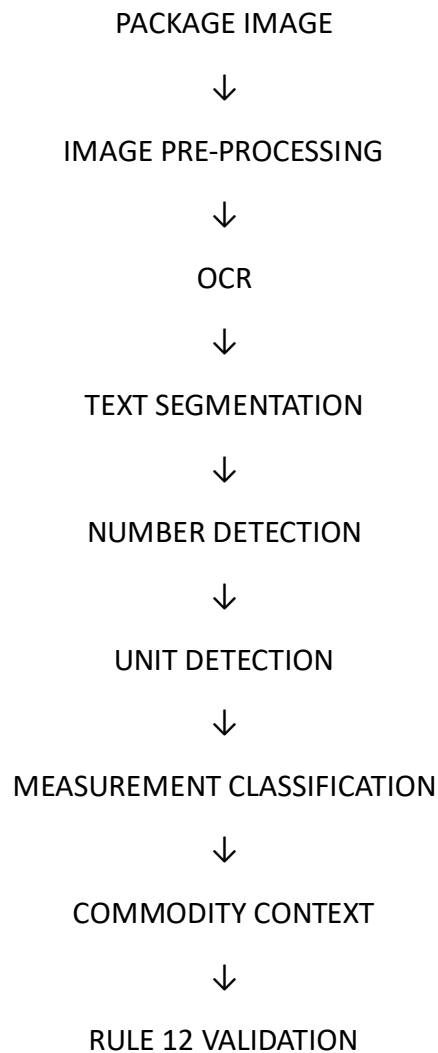
Software Flow



This is another example showing why **package-size context must be available to the rule engine.**

12.13 Quantity Extraction Architecture

Rule 12 suggests that the OCR module should be combined with a **unit-aware parser**.



12.14 Example of Structured Extraction

Suppose OCR detects:

NET QTY: 750 ML

DrishtiTathya could convert this into:

- Declaration Type → Net Quantity
- Value → 750
- Unit → mL
- Measurement Type → Volume
- OCR Confidence → 97%

For: 10 PIECES

the representation becomes:

- Declaration Type → Quantity
- Value → 10
- Unit → Pieces
- Measurement Type → Number

12.15 Semantic Quantity Detection

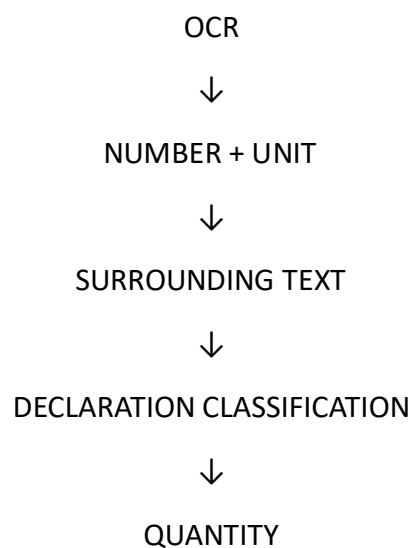
A major challenge is that packages contain many numbers.

Example:

- MRP ₹250
- 500 g
- Batch No. 250823
- 12 pieces
- 1800-123-456

OCR alone cannot determine which number represents quantity.

Therefore:



Research Finding — RF060

Quantity compliance requires semantic interpretation of numbers, units and surrounding text, not simple OCR extraction.

12.16 Unit Normalisation

For internal processing, the system can normalise equivalent units.

For example:

- 1000 g → 1 kg
- 1000 mL → 1 L
- 100 cm → 1 m

However, the original printed declaration should always be preserved for evidence.

Original: 1000 mL

Normalised: 1 L

Evidence: Original image + OCR text

This separation is important for auditability.

12.17 Rule 12 Compliance Matrix

Check	AI Capability	Human Review
Quantity detected	High	Sometimes
Unit detected	High	Sometimes
Mass/length/area/volume classification	High	Sometimes
Number classification	High	Sometimes
Combination quantity	Medium–High	Yes
Additional dimensions	Medium	Yes
Misleading expressions	High	Yes
Same-panel detection	Medium	Yes
Small-package special case	Medium	Yes
Fourth Schedule exception	Requires rule database	Yes

12.18 Fourth Schedule Exception

Rule 12 itself states that its general unit classification is subject to exceptions specified in the **Fourth Schedule**.

This is extremely important for your architecture.

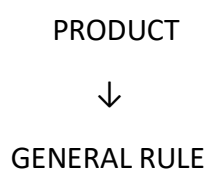
The system should **not hard-code**:

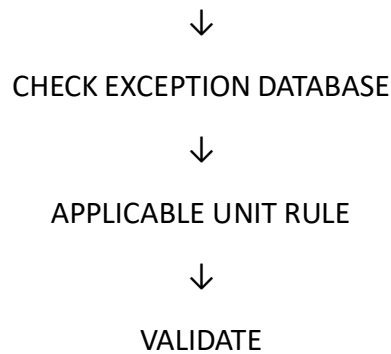
Liquid → Volume

Solid → Mass

as an absolute rule.

Instead:





Research Finding — RF061

Exception handling must be part of the rule engine because the general unit rule is expressly subject to scheduled exceptions.

12.19 Version-Aware Rule 12

Because the Rules have been amended over time, the software should store:

Rule Number + Clause + Version + Effective Date + Exception

Example conceptual representation:

- Rule: 12
- Clause: 2
- Requirement: Unit classification
- Exception: Fourth Schedule
- Status: Applicable
- Effective Date: [rule-version date]
- Source: Official notification

This supports the project's **version-aware Legal Metrology engine**.

12.20 Proposed Software Rule

IF package is applicable

THEN

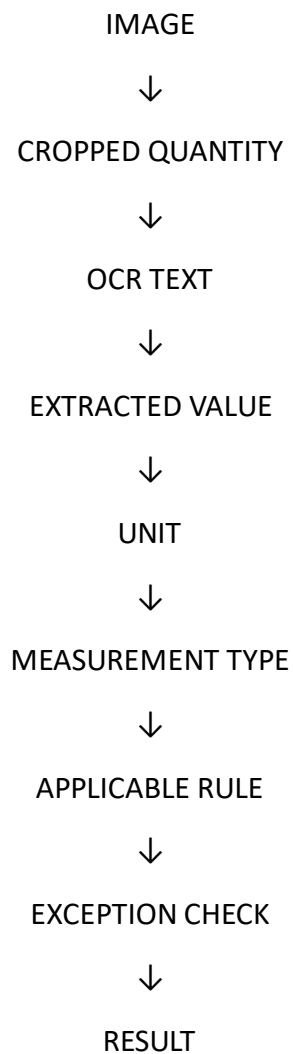
- extract quantity declaration
- identify value and unit
- classify measurement type
- check applicable product category
- check Fourth Schedule exceptions

- check additional quantity information
- check misleading quantity expressions
- check small-package provision

generate: COMPLIANT , POTENTIAL ISSUE OR HUMAN REVIEW

12.21 Evidence Model

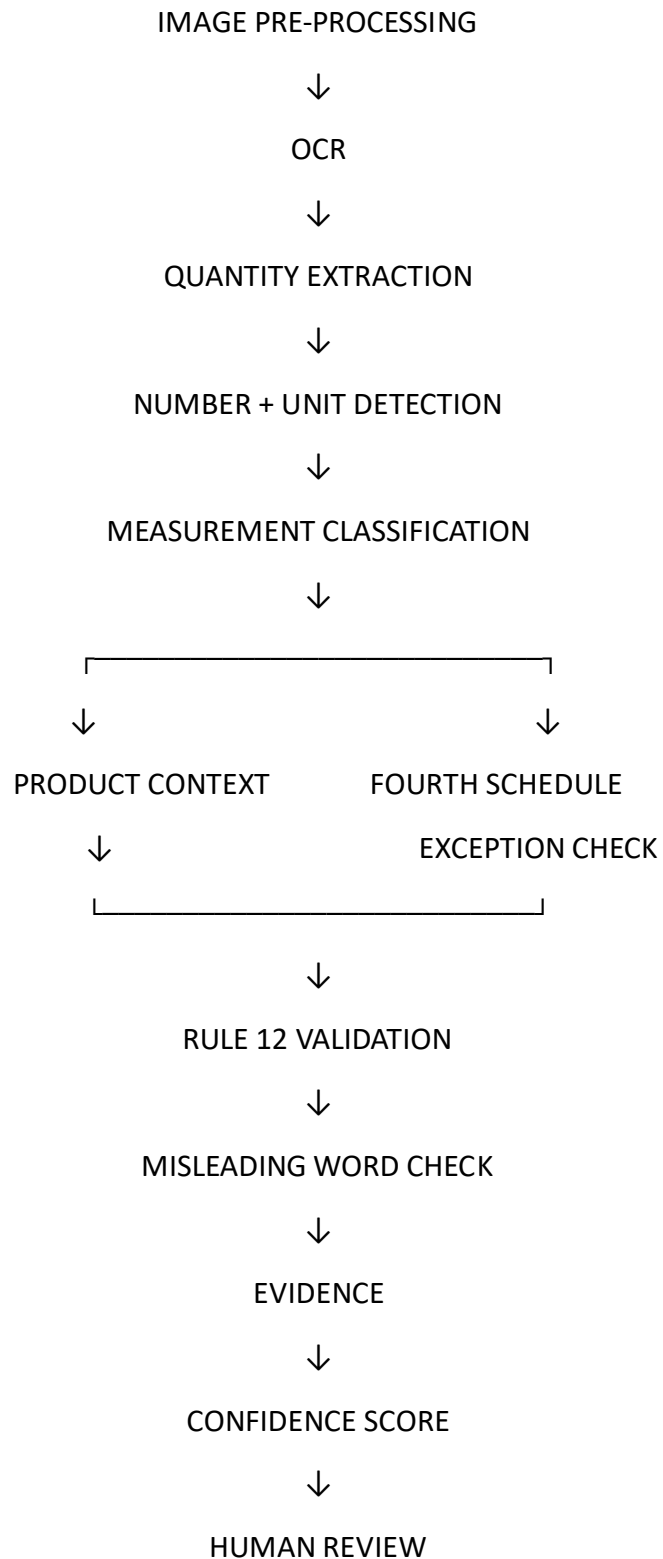
For every Rule 12 decision:



The inspector should be able to see **why** the system produced the result.

12.22 Rule 12 — Proposed Compliance Flow





12.23 Key Research Insight

Rule 12 provides a strong technical justification for making **unit-aware information extraction** a core component of DrishtiTathya.

The system should understand:

- **What is the number?**
- **What is its unit?**
- **What does that unit represent?**
- **Is that unit appropriate for the commodity?**

- **Is there an applicable exception?**

This moves the project beyond ordinary OCR into **legal-semantic information extraction**.

➤ **Research Finding — RF062**

Rule 12 demonstrates that the compliance engine must combine OCR, unit recognition, commodity context, exception handling and rule-version management to evaluate quantity declarations reliably.

13. RULE 13 — STATEMENT OF UNITS OF WEIGHT, MEASURE OR NUMBER

13.1 Basic Requirement

Rule 13 specifies the **units in which the quantity of a packaged commodity should be expressed**.

The Rule provides specific unit conventions depending on whether the quantity is below, equal to, or above specified thresholds. It also requires the use of the **International System of Units (SI)** for net quantity declarations.

Software Interpretation

For DrishtiTathya, this means that quantity validation must check not only:

Is quantity present?

but also:

Is the quantity expressed using the appropriate unit and format?

13.2 Quantity Below the Specified Threshold

Rule 13(2) specifies the units to be used when expressing quantities below certain thresholds:

Quantity	Unit
Less than 1 kilogram	Gram
Less than 1 metre	Centimetre
Less than 1 square metre	Square decimetre
Less than 1 cubic metre	Cubic centimetre
Less than 1 cubic decimetre	Cubic centimetre
Less than 1 litre	Millilitre

Software Representation

VALUE

+

UNIT

↓

THRESHOLD CHECK

↓

EXPECTED UNIT



COMPLIANCE RESULT

Example:

500 g

The system can identify:

$500 < 1000$ g

→ gram-based expression

13.3 Quantity Equal to or Greater Than the Threshold

Rule 13(3) specifies the corresponding larger units when quantities are equal to or greater than the specified thresholds.

Examples include:

- Kilogram for quantities of at least one kilogram
- Metre for quantities of at least one metre
- Square metre for quantities of at least one square metre
- Cubic metre for quantities of at least one cubic metre
- Litre for quantities of at least one litre.

Proposed Software Logic

IF quantity < threshold



use smaller specified unit

IF quantity $\geq$ threshold



use larger specified unit

13.4 Decimal Sub-Multiples

Where the quantity is equal to or greater than the relevant threshold but includes a fraction, Rule 13 permits the fraction to be expressed using decimal sub-multiples of the relevant larger unit or, where specified, in terms of the smaller unit.

For software purposes, this means the system should recognise both:

1.5 kg

and the relevant permitted equivalent representation.

Research Finding — RF063

Unit validation must consider both the numerical value and the permitted representation of fractional quantities.

13.5 Special Option at Exactly One Unit

Rule 13 contains a proviso allowing certain quantities exactly equal to one kilogram, one metre, one square metre, one cubic decimetre, one cubic metre or one litre to be expressed using the corresponding smaller unit at the option of the manufacturer, packer or importer.

Therefore, the software should **not flag every alternative expression automatically**.

Example of the Rule-Engine Concept

QUANTITY = EXACTLY 1 UNIT



CHECK SPECIAL PROVISIO



PERMITTED ALTERNATIVE?



YES → ACCEPT

NO → REVIEW

This is a good example of why the system needs **exception-aware rules**.

13.6 Prohibition on Certain Counting Expressions

Rule 13(4) provides that expressions such as:

- dozen
- score
- gross
- great gross
- similar expressions

shall not be specified or indicated on packages for expressing quantity.

Software Opportunity

This requirement is highly suitable for NLP.

OCR TEXT



QUANTITY DECLARATION

↓

PROHIBITED-TERM CHECK

↓

MATCH?

↓ ↓

YES NO

↓ ↓

FLAG CONTINUE

Example

“2 dozen”

could produce:

Potential Rule 13 issue — prohibited quantity expression detected.

The final legal determination should remain subject to review.

13.7 International System of Units

Rule 13(5)(i) provides that **no system of units other than the International System of Units** shall be used in furnishing the net quantity of packages.

The Department of Consumer Affairs also describes the metric system based on the International System of Units as the basis for legal metrology requirements.

Software Relevance

The rule engine should contain a unit dictionary.

UNIT DATABASE

Mass

→ g

→ kg

Length

→ cm

→ m

Volume

→ mL

→ L

Area

→ dm²

→ m²

Volume/Cubic

→ cm³

→ dm³

→ m³

The exact accepted representation should be linked to the applicable rule rather than assumed from a generic unit library.

13.8 Items Sold by Number

The Rules contain special provisions for items sold by number.

The consolidated official material records that, for items sold by number, the symbol **N or U** may be used. Later amendment text also addresses the words used to represent quantity, including number, unit, piece, pair and set.

Software Interpretation

The system should therefore recognise:

10 N

10 U

10 pieces

10 units

10 pairs

10 sets

according to the applicable rule version and product context.

Important Design Principle

Do not create one permanent rule such as:

"N is mandatory."

Instead:

RULE VERSION

↓

PRODUCT CONTEXT



APPLICABLE UNIT FORMAT

13.9 Rule Version Awareness

This is particularly important for Rule 13 because the consolidated source contains amendment history.

The Department's official repository lists amendments to the Packaged Commodities Rules across multiple years, including 2017, 2021, 2023, 2025 and 2026.

Therefore, DrishtiTathya should store:

Rule Number

+

Sub-rule

+

Requirement

+

Amendment

+

Effective Date

+

Source

This prevents the system from applying an outdated unit rule to a current package.

13.10 OCR + Unit Recognition

A normal OCR system might produce:

NET WT 500 G

DrishtiTathya should transform this into:

Declaration Type → Net Quantity

Value → 500

Unit → g

Measurement → Mass

Then the Rule 13 engine evaluates:

500 g

↓

Value threshold

↓

Expected unit

↓

Applicable version

↓

Result

13.11 Unit Normalisation

The software should maintain two representations.

Original Declaration

500 g

Internal Representation

Value = 500

Base Measurement = Mass

Base Unit = gram

This allows calculations without losing the original evidence.

PACKAGE IMAGE

↓

OCR

↓

ORIGINAL DECLARATION

↓

NORMALISATION

↓

RULE ENGINE

13.12 Quantity Threshold Engine

Rule 13 can be represented computationally as a threshold-based engine.

Conceptually:

INPUT:

Quantity + Unit + Measurement Type



THRESHOLD ENGINE



Applicable Unit Rule



Exception / Proviso Check



Version Check



Compliance Result

This is one of the strongest examples of converting legislation into a **machine-readable rule structure**.

13.13 Prohibited-Expression Detection

The system can maintain a vocabulary of potentially relevant prohibited or restricted expressions.

Example:

"dozen"

"score"

"gross"

"great gross"

The NLP module can search quantity-related text for these terms.

OCR

↓

Sentence / phrase extraction

↓

Quantity context detection

↓

Term matching

↓

Rule 13 check

13.14 Example

Suppose the package contains:

NET QUANTITY: 2 DOZEN

The system extracts:

Quantity = 2

Unit = dozen

Type = Number

The rule engine then detects:

Unit category → Number

Expression → "dozen"

Rule 13 → Relevant restriction

Output:

Potential Rule 13 issue — quantity expression requires review.

This is preferable to simply displaying:

NON-COMPLIANT

because AI detection is not itself the final legal determination.

13.15 Rule 13 Compliance Matrix

Check	Automation Potential
Quantity value extraction	High
Unit extraction	High
Mass/length/area/volume classification	High
Threshold comparison	High
SI-unit validation	High
Decimal representation check	High
Prohibited expression detection	High

Number/piece/pair/set classification	Medium–High
Exception/proviso application	High if rule database is accurate
Final legal determination	Human review

13.16 Rule 13 — Proposed Software Rule

IF net quantity is detected

identify:

- value
- unit
- measurement type

- determine applicable threshold

- determine permitted unit

- check SI-unit requirement

- check prohibited quantity expressions

- check number/unit/piece/pair/set provisions

- check exceptions and provisos

- check rule version

- preserve evidence

generate: COMPLIANT , POTENTIAL ISSUE OR HUMAN REVIEW

13.17 Evidence Model

Every Rule 13 decision should retain:

Original Package Image



Quantity Crop



OCR Text



Extracted Value



Detected Unit



Measurement Type



Threshold Applied



Rule Version



Exception / Proviso



Result

This creates an **auditable compliance chain**.

13.18 Key Research Insight

Rule 13 demonstrates that a legal-compliance system cannot simply search for numbers and units.

It must understand the relationship:

QUANTITY

+

UNIT

+

THRESHOLD

+

PRODUCT CONTEXT

+

RULE VERSION

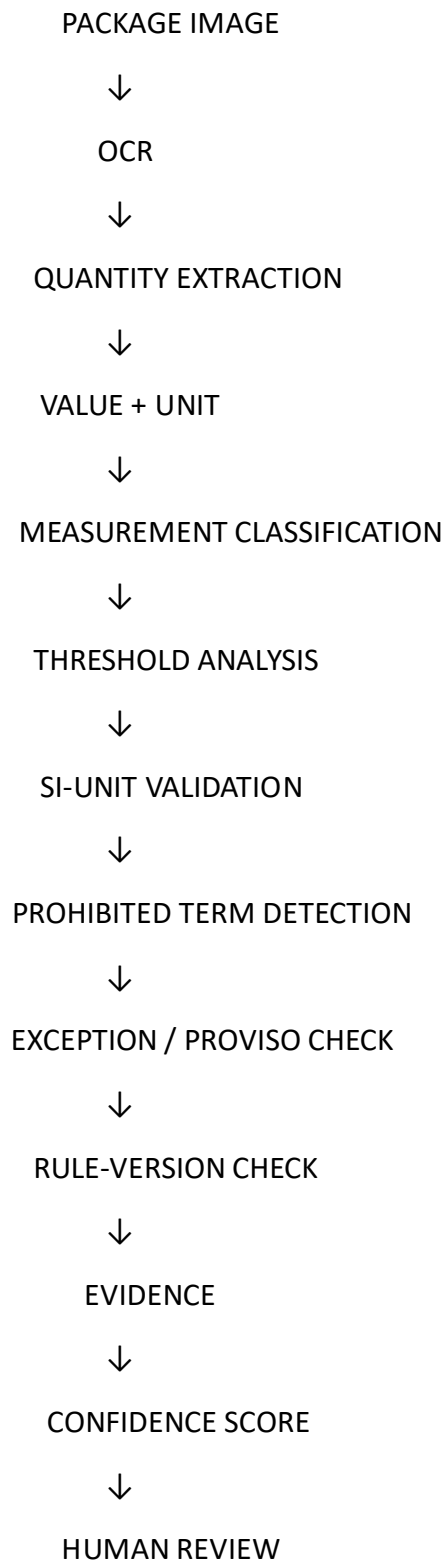
+

EXCEPTION

Research Finding — RF064

Rule 13 provides a strong basis for implementing a machine-readable unit-validation and threshold-validation layer within the DrishtiTathya Legal Metrology Rule Engine.

13.19 Rule 13 — Proposed Compliance Flow



13.20 Rule 13 Conclusion

Rule 13 converts quantity declaration into a **structured, measurable and machine-checkable requirement**.

For DrishtiTathya, it provides the foundation for:

- Unit recognition
- Quantity normalisation
- Threshold checking
- SI-unit validation
- NLP-based prohibited-expression detection
- Exception handling
- Version-aware validation
- Evidence generation

➤ **Key Research Finding — RF065**

Rule 13 is highly suitable for automation because many of its requirements can be represented as structured rules involving units, numerical thresholds, prohibited expressions, exceptions and effective rule versions.

14. RULE 14 — DECLARATIONS WITH REGARD TO DIMENSIONS OF CERTAIN COMMODITIES

14.1 Basic Requirement

Rule 14 deals with commodities for which **finished dimensions are important information for the consumer**.

It specifically covers commodities such as:

- Bed-sheets
- Hemmed fabric materials
- Dhoties
- Sarees
- Napkins
- Pillow-covers
- Towels
- Table cloths
- Similar commodities

For such packages, the **number of pieces and the dimensions of the finished size** must be declared on the package or on the label affixed to it.

Software Interpretation

This means that DrishtiTathya cannot treat every product as a simple:

Quantity = Value + Unit

For certain products:

Quantity

+

Number of Pieces

+

Finished Dimensions

must be considered.

14.2 Why Rule 14 Is Important

Consider a package containing:

1 Bedsheet

220 cm × 240 cm

Simply detecting:

1 piece

does not provide the complete information required for this type of commodity.

The system therefore needs to detect both:

NUMBER

+

DIMENSIONS

14.3 Multiple Pieces of Different Dimensions

Rule 14 contains a special provision where a package contains **more than one piece having different dimensions**.

In such a case, the package must contain a declaration of:

- Dimensions of each piece
- Retail sale price of each piece

The dimensions and retail sale price must also be marked on each individual piece.

Example

Package contains:

Piece 1 → 100 cm × 150 cm → ₹200

Piece 2 → 150 cm × 200 cm → ₹350

The system should not simply extract:

2 pieces

It should attempt to construct:

Piece 1 → Dimensions + Price

Piece 2 → Dimensions + Price

14.4 Software Challenge — Dimension Extraction

Dimensions can appear in different formats:

220 cm × 240 cm

220 × 240 cm

2.2 m × 2.4 m

220cm x 240cm

OCR must therefore be combined with pattern recognition.

Proposed Extraction

OCR TEXT



NUMBER DETECTION



DIMENSION SYMBOL DETECTION



UNIT DETECTION



DIMENSION PARSING



STRUCTURED DATA

Example:

220 cm × 240 cm

becomes:

Length = 220 cm

Width = 240 cm

Unit = cm

14.5 Computer Vision + OCR

Rule 14 provides a strong reason to combine OCR with computer vision.

OCR identifies:

What dimensions are printed?

Computer vision can help identify:

Where are those dimensions located?

The legal rule engine determines:

Whether those declarations are applicable and sufficient.

Therefore:

IMAGE



OCR



DIMENSION EXTRACTION



SPATIAL ANALYSIS



RULE 14



COMPLIANCE RESULT

14.6 Number of Pieces

The software should separately detect the number of commodities.

Examples:

1 Piece

2 Pieces

6 Towels

3 Napkins

The structured representation could be:

Quantity Type → Number

Number → 6

Commodity → Towels

This should remain separate from dimensions.

14.7 Dimension Normalisation

Different units may be encountered.

For example:

2.2 m × 2.4 m

can internally be normalised to:

220 cm × 240 cm

However, the original printed declaration must remain available as evidence.

Proposed Data Model

Original:

2.2 m × 2.4 m

Normalised:

220 cm × 240 cm

Evidence:

Original package image

+

OCR text

14.8 Retail Sale Price of Individual Pieces

For packages containing multiple pieces with different dimensions, Rule 14 also connects the dimensions with the **retail sale price of each piece**.

This creates a cross-field relationship:

PIECE

↓

DIMENSION

+

RETAIL SALE PRICE

The rule engine should therefore be capable of representing:

Piece ID

Dimensions

Individual RSP

14.9 Proposed Structured Representation

Example:

Package:

Piece 1

└─ Dimensions: 100 × 150 cm

└─ RSP: ₹200

Piece 2

└─ Dimensions: 150 × 200 cm

└─ RSP: ₹350

This is more powerful than simply storing all detected text in one field.

14.10 Rule 14 — Declaration Classification

The system may classify extracted information as:

Information Classification

2 pieces	Number
220 cm	Dimension
240 cm	Dimension
₹350	Retail Sale Price
“Bedsheet”	Commodity
“Cotton”	Product description

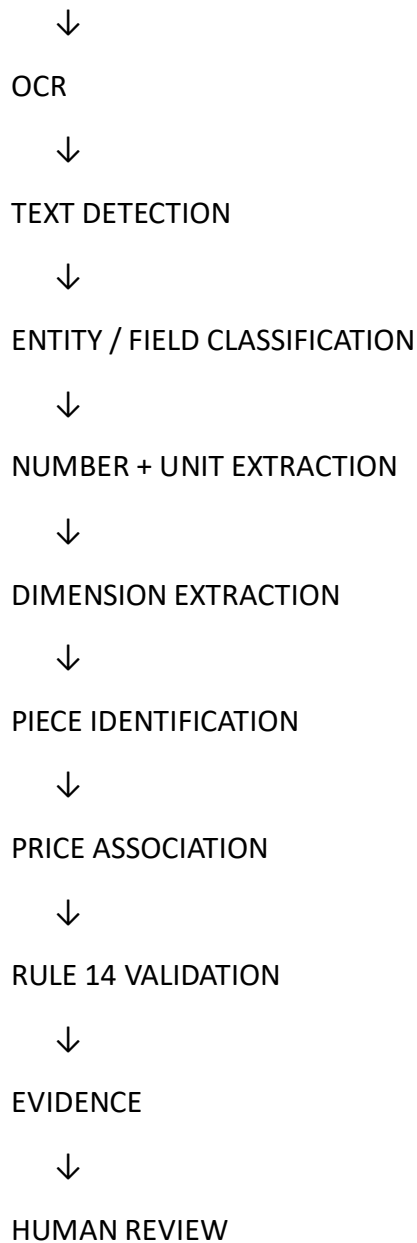
This demonstrates why **declaration extraction** is more useful than raw OCR.

14.11 Potential AI Pipeline

PACKAGE IMAGE



IMAGE PRE-PROCESSING



14.12 Spatial Relationship

One advanced feature for DrishtiTathya can be **spatial association**.

Suppose OCR finds:

220 × 240 cm

₹999

The system can record their approximate positions on the package.

Text A → Dimension

Text B → Price

For a multi-piece package, spatial analysis may help associate declarations with the relevant product representation.

Important Limitation

Image-based association should be treated as an **AI-assisted finding**, especially when packaging contains many pieces or repeated information.

14.13 Research Finding

RF066

Rule 14 demonstrates that some packaged-commodity requirements are relationship-based: number of pieces, dimensions and retail sale price may need to be interpreted together.

This is an important design principle for the DrishtiTathya rule engine.

14.14 Rule 14 Compliance Matrix

Requirement	Automation Potential
Detect number of pieces	High
Detect dimensions	High
Identify measurement units	High
Normalise dimensions	High
Identify individual prices	High
Associate price with piece	Medium
Handle different piece dimensions	Medium
Verify declaration on individual piece	Medium
Final legal determination	Human review

14.15 Proposed Software Rule

IF product belongs to Rule 14 category

THEN

- detect number of pieces
- detect finished dimensions

IF multiple pieces have different dimensions

- detect dimensions of each piece
 - detect retail sale price of each piece
 - preserve original evidence
 - calculate AI confidence
 - send uncertain cases for human review
-

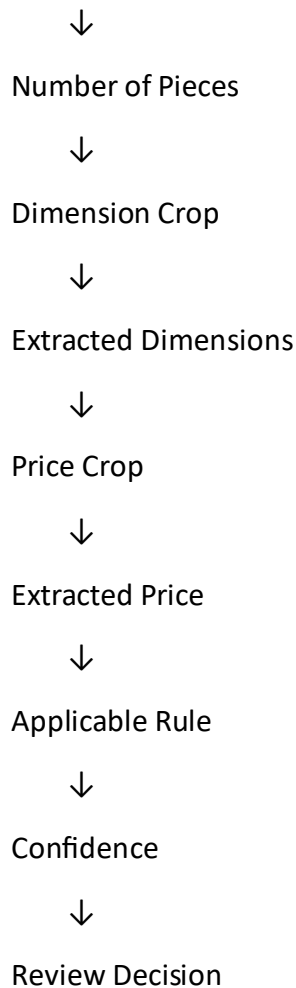
14.16 Evidence Model

For a Rule 14 inspection, the evidence package could contain:

Original Package Image



Detected Commodity



14.17 Example

Suppose the system scans:

- BED SHEET
- 1 PIECE
- 220 × 240 CM
- MRP ₹999

The system produces:

- Commodity → Bed Sheet
- Pieces → 1
- Dimensions → 220 × 240 cm
- MRP → ₹999
- Rule 14 → Applicable
- Dimension declaration → Detected
- Status → No issue detected from this check

This should **not** be interpreted as a complete product compliance result because other mandatory declarations still need to be checked.

14.18 Rule 14 and Product Classification

This rule also demonstrates why product classification is an important part of DrishtiTathya.

PACKAGE



PRODUCT IDENTIFICATION



CATEGORY CLASSIFICATION



APPLICABLE RULES



RULE 14?



YES → DIMENSION CHECK

NO → CONTINUE OTHER RULES

Therefore, the system should have an **applicability layer before detailed rule validation**.

14.19 Research Finding

RF067

Product classification is a prerequisite for applying several specialised declaration rules.

This supports the proposed DrishtiTathya architecture:

IMAGE



OCR



INFORMATION EXTRACTION



PACKAGE CLASSIFICATION



APPLICABILITY

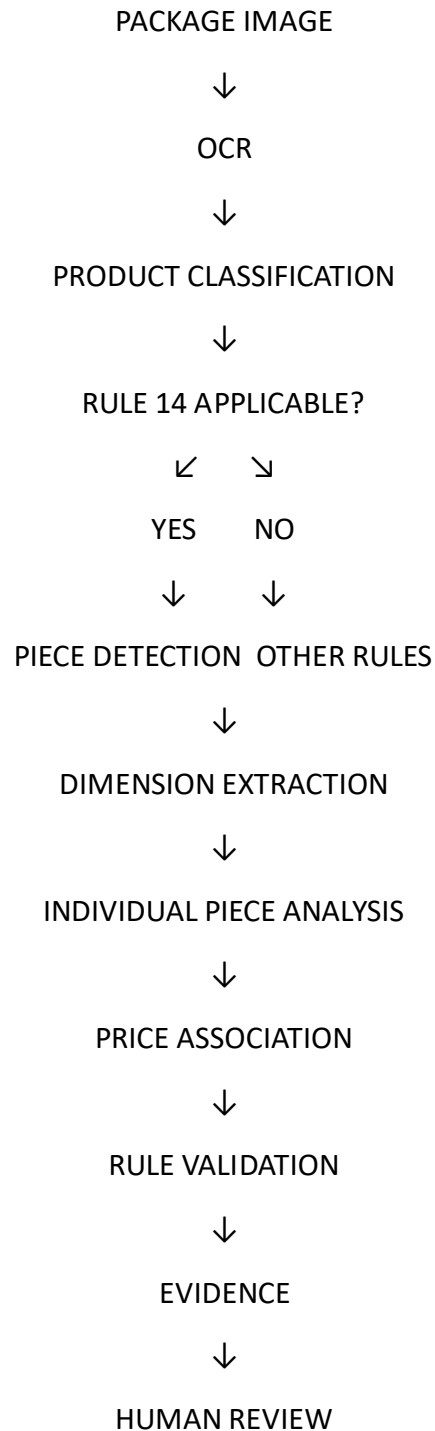


RULE VERSION



SPECIFIC RULE CHECK

14.20 Rule 14 — Proposed Compliance Flow



14.21 Rule 14 Conclusion

Rule 14 introduces **specialised product-specific declarations**.

It demonstrates that DrishtiTathya cannot rely only on a universal checklist of declarations.

Instead, the system needs:

Product Classification → Applicable Rule Selection → Specialised Declaration Check

➤ **Key Research Finding — RF068**

Rule 14 demonstrates the need for product-specific rule activation, dimension extraction and relationship-based validation within the Legal Metrology compliance engine.

15. RULE 15 — DECLARATION OF DIMENSIONS AND WEIGHT IN CERTAIN CASES

15.1 Basic Requirement

Rule 15 deals with commodities where the **dimensions, weight, or a combination of both has a relationship with the price of the commodity.**

The Rule provides that, in such cases, the quantity declaration on the package must also include the relevant **dimensions, weight, or combination of dimensions and weight.**

Software Interpretation

This means that for certain products, the system must establish a relationship between:

QUANTITY

+

DIMENSION / WEIGHT

+

PRICE

This is different from a simple declaration-presence check.

15.2 Why Rule 15 Is Important

Suppose the price of a commodity depends on its:

- Weight
- Length
- Area
- Dimensions
- Combination of measurements

Then detecting only the MRP and net quantity may not be sufficient.

The system may need to identify the measurement that has a relationship with the price.

Proposed Logic

PACKAGE

↓

PRODUCT CLASSIFICATION

↓

IDENTIFY PRICE



IDENTIFY QUANTITY



IDENTIFY RELEVANT DIMENSION / WEIGHT



RULE 15 CHECK

15.3 Relationship Between Price and Measurement

The important concept in Rule 15 is the **relationship between measurement and price**.

Therefore, DrishtiTathya should not assume that every product requires a dimension declaration under Rule 15.

Instead:

Does measurement have a relationship
with the price?



YES



Apply Rule 15

This makes Rule 15 a **conditional rule**.

Research Finding — RF069

Rule 15 is activated by product characteristics and the relationship between measurement and price, rather than by a universal requirement applicable to every package.

15.4 Dimensions

Where dimensions have a relationship with price, the relevant dimensions should be declared.

Possible dimension structures include:

Length

Width

Height

Area

Other applicable dimensions

The exact dimension depends on the commodity.

Software Representation

Dimension

└— Type

└— Value

└— Unit

└— Relationship to Price

Example:

Length = 10 m

Price = ₹500

15.5 Weight

Where weight has a relationship with price, the package should include the relevant weight declaration.

Example:

Weight = 500 g

Price = ₹250

The software should therefore associate:

WEIGHT → PRICE

rather than treating them as unrelated pieces of OCR text.

15.6 Combination of Dimensions and Weight

Rule 15 also covers situations where a **combination** of dimensions and weight has a relationship with price.

Therefore, the data model should support:

PRODUCT

- Weight
- Length
- Width
- Height
- Price

rather than restricting the system to a single measurement.

15.7 Rule 15 and Information Extraction

A basic OCR engine may extract:

- 10 m

- 500 g
- ₹250

But DrishtiTathya must determine what these values represent.

Proposed Pipeline

OCR



NUMBER + UNIT DETECTION



SEMANTIC CLASSIFICATION



DIMENSION / WEIGHT / PRICE



RELATIONSHIP ANALYSIS



RULE 15

15.8 Price Association

One technical challenge is associating the correct price with the correct measurement.

For example:

- 10 m
- ₹500
- MRP ₹550

The system must distinguish:

- Relevant price
- MRP
- Other numerical values

Therefore, the software should use **context and declaration labels**.

"MRP"



Retail Sale Price

"Net Weight"



Weight

"Length"



Dimension

Research Finding — RF070

Rule 15 requires semantic association between different declarations rather than independent OCR extraction.

15.9 Rule 15 vs Rule 14

These two rules should not be treated as identical.

Rule	Main Focus
-------------	-------------------

Rule 14	Dimensions and number of pieces for specified commodities
---------	---

Rule 15	Dimensions/weight where they have a relationship with price
---------	---

Therefore:

Rule 14→ Product-specific dimension declaration

Rule 15→ Measurement–price relationship

This distinction should be represented in the rule engine.

15.10 Product Classification

Rule 15 reinforces the importance of product classification.

IMAGE



OCR



PRODUCT IDENTIFICATION



MEASUREMENT EXTRACTION



PRICE EXTRACTION



CHECK RELATIONSHIP



RULE 15

The classification layer determines whether Rule 15 should be considered.

15.11 Proposed Data Structure

A structured record could be:

- Product:[Commodity Name]
- Price:₹500
- Measurement:10 m
- Measurement Type:Length
- Relationship:Measurement-related pricing
- Applicable Rule:Rule 15
- Evidence:[Package image]
- Confidence:92%

This makes the result explainable.

15.12 Rule 15 — Computer Vision Role

Computer vision can assist with:

- Locating price
- Locating quantity
- Locating dimensions
- Detecting text groups
- Associating nearby declarations
- Highlighting evidence

The system can highlight each relevant region for the inspector.

15.13 Rule 15 — NLP Role

NLP can identify contextual phrases such as:

- weight
- net weight
- length
- width
- height
- size
- dimension
- MRP
- price
- per metre

- per kg

This allows the system to determine what each numerical value represents.

Proposed NLP Flow

OCR TEXT



TOKENISATION



KEYWORD / ENTITY DETECTION



DECLARATION CLASSIFICATION



RELATIONSHIP EXTRACTION

15.14 Evidence-Based Verification

A Rule 15 finding should contain:

Original Image



Relevant Measurement Crop



Price Crop



OCR Text



Extracted Values



Relationship Identified



Applicable Rule



Confidence

This enables the officer to verify the AI interpretation.

15.15 Automated vs Human Verification

Check	Automation Potential
Detect dimensions	High
Detect weight	High
Detect price	High
Identify units	High
Associate nearby declarations	Medium
Determine price relationship	Medium
Determine applicability in complex cases	Medium
Final legal determination	Human

15.16 Proposed Software Rule

IF package is subject to the Rules

- identify product category
- identify declared price
- identify quantity
- identify dimensions / weight
- determine whether dimensions or weight have a relationship with price

IF relationship exists

- check required declaration
- preserve evidence
- generate confidence score
- send uncertain cases for human review

This is a **proposed software interpretation**, not the legal text itself.

15.17 Example

Suppose OCR detects:

10 METRES

₹500

The system may produce:

Measurement → 10 metres

Price → ₹500

Potential Rule 15 applicability → Detected

It should then consult the applicable rule configuration before producing a compliance conclusion.

15.18 Important Limitation

The relationship between price and measurement may not always be inferable from an image alone.

Therefore, the system should avoid making unsupported assumptions such as:

“Every product showing metres and price is automatically governed by Rule 15.”

Instead:

AI Detection



Potential Applicability



Rule Engine



Human Review if Required

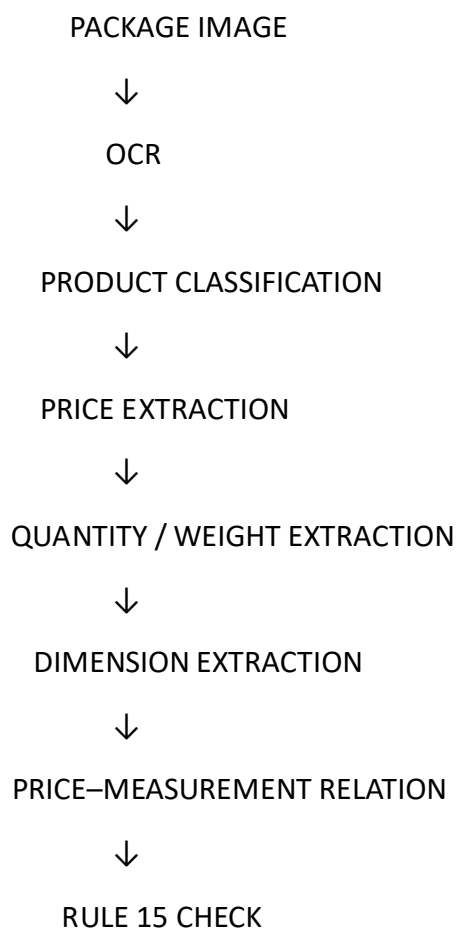
15.19 Research Finding

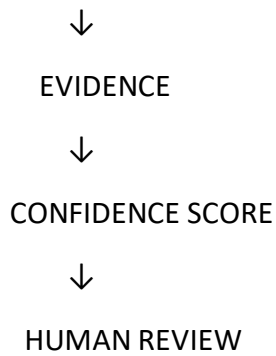
RF071

Rule 15 demonstrates the need for relationship-based rule evaluation, where compliance depends on the connection between multiple extracted declarations rather than on the presence of an individual field.

This is an important differentiator for DrishtiTathya.

15.20 Rule 15 — Proposed Compliance Flow





15.21 Rule 15 Conclusion

Rule 15 extends the system from **field-level validation** to **relationship-level validation**.

Instead of asking:

“Is price present?”

or:

“Is weight present?”

DrishtiTathya can ask:

“Where dimensions or weight have a relationship with price, has the required measurement information also been declared?”

➤ **Research Finding — RF072**

Rule 15 supports the development of a relationship-aware Legal Metrology rule engine capable of connecting price, quantity, dimensions and weight rather than evaluating each declaration independently.

16. RULE 16 — DECLARATION OF NUMBER OF USABLE SHEETS

16.1 Basic Requirement

Rule 16 applies to packages containing **sheets**, such as:

- Aluminium foil
- Facial tissues
- Waxed paper
- Toilet paper
- Other similar sheet-type commodities

For these products, the quantity declaration must include **two important pieces of information**:

1. **Number of usable sheets**
2. **Dimensions of each sheet**

This requirement is directly stated in the Rules.

16.2 Simple Example

Suppose a tissue package contains:

100 Sheets

20 cm × 20 cm

The system should understand:

Number of sheets → 100

Length → 20 cm

Width → 20 cm

So simply detecting:

100 sheets

is not enough for this particular requirement.

16.3 Why This Rule Matters

This Rule shows that **quantity can have multiple components**.

For ordinary products:

Net Quantity → 500 g

But for sheet products:

Number of Sheets

+

Dimensions of Each Sheet

Therefore, DrishtiTathya needs product-specific rules.

Research Finding — RF073

For specified sheet-based commodities, quantity information includes both the number of usable sheets and the dimensions of each sheet.

16.4 OCR Detection

This rule is highly suitable for OCR.

Example package text:

100 Facial Tissues

20 cm × 20 cm

DrishtiTathya can extract:

Product → Facial Tissues

Number → 100 sheets

Dimensions → 20 cm × 20 cm

Proposed Flow

PACKAGE IMAGE



OCR



TEXT EXTRACTION



NUMBER + UNIT DETECTION



SHEET INFORMATION



RULE 16 CHECK

16.5 Dimension Extraction

Dimensions may appear in different formats:

20 cm × 20 cm

20 × 20 cm

20cm x 20cm

The system should recognise these patterns and convert them into structured information.

Example:

Original:

20 cm × 20 cm

Structured:

Length = 20 cm

Width = 20 cm

16.6 Number of Usable Sheets

The system should specifically identify the **number of usable sheets**, rather than treating every number on the package as quantity.

For example:

100 Sheets

Batch No. 2506

MRP ₹120

OCR may detect all three numbers.

The AI must determine:

100 → Number of sheets

2506 → Batch information

120 → MRP

Research Finding — RF074

Context-based declaration classification is necessary because package labels contain many unrelated numerical values.

16.7 Product Classification

Rule 16 should only be activated when the product falls within the relevant sheet-based category.

Therefore:

IMAGE

↓

OCR

↓

PRODUCT CLASSIFICATION

↓

Is it a sheet-type commodity?

↓

YES

↓

APPLY RULE 16

This follows the broader DrishtiTathya principle:

First determine applicability, then perform the detailed compliance check.

16.8 Proposed Software Rule

IF product is a sheet-type commodity

- check number of usable sheets
- check dimensions of each sheet

IF number is missing

FLAG

IF dimensions are missing

- FLAG
- preserve image evidence
- calculate AI confidence
- send uncertain cases for human review

This is a **software interpretation**, not a replacement for the legal rule.

16.9 Evidence Capture

For every Rule 16 check, the system should save:

Original Package Image



Sheet Quantity Crop



Dimension Crop



OCR Text



Extracted Information



Applicable Rule



Confidence



Review Result

Example:

Detected:

Sheets → 100

Dimensions → 20 cm × 20 cm

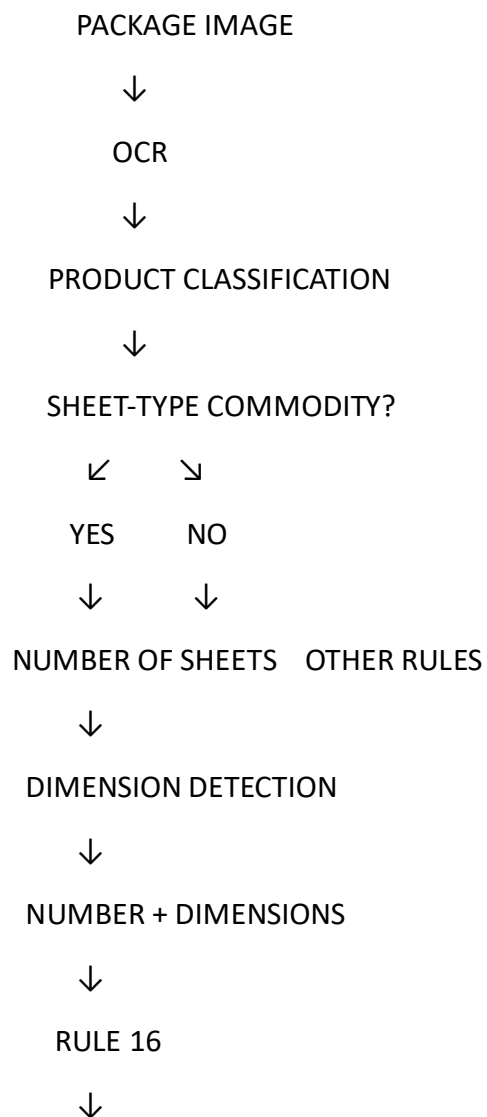
Rule 16: Required information detected

Confidence: 94%

16.10 Automation Potential

Requirement	Automation
Identify sheet-type product	High
Detect number of sheets	High
Detect dimensions	High
Detect units	High
Parse dimensions	High
Verify that dimensions refer to each sheet	Medium
Handle unclear packaging	Human review
Final legal determination	Human

16.11 Rule 16 — Proposed Compliance Flow



EVIDENCE



CONFIDENCE SCORE



HUMAN REVIEW

16.12 Key Research Insight

Rule 16 is a good example of why DrishtiTathya should use **product-specific compliance logic**.

The system should not use one generic rule such as:

“Every package needs only net quantity.”

Instead:

PRODUCT TYPE



APPLICABLE RULE



REQUIRED INFORMATION

For sheet commodities:

NUMBER OF USABLE SHEETS + DIMENSIONS OF EACH SHEET

➤ Research Finding — RF075

Rule 16 demonstrates that the Legal Metrology rule engine must activate specialised declaration requirements based on the type of commodity being inspected.

16.13 Rule 16 Conclusion

Rule 16 is relatively simple from a legal perspective but technically valuable for DrishtiTathya.

It requires the system to combine:

Product Classification + OCR + Number Extraction + Dimension Extraction + Rule Validation

This makes Rule 16 a good example for demonstrating how your project goes beyond simple OCR.

17. RULE 17 — DIMENSIONS OF CONTAINER-TYPE COMMODITIES

Rule 17 deals with **pre-packaged commodities that are themselves containers or are sold as containers**, such as bags, boxes, cups, pans and similar items. The rule ensures that consumers can understand the **number of containers and their relevant dimensions/capacity** from the package declaration.

17.1 Basic Requirement

For container-type commodities, the declaration should provide the **number of containers** along with the dimensions required for that particular type of container.

The exact information depends on the shape and design of the container.

17.2 Types of Containers Covered

Container Type	Required Declaration
Bags	Number of bags + linear dimensions
Square/Rectangular/Oblong Containers	Number + length + width + depth, where required
Circular/Round Containers	Number + diameter + depth, where necessary
Other Similar Containers	Number and relevant dimensions/capacity as applicable

For bags, the dimensions are required whether the bags are supplied individually or in a perforated roll.

17.3 Simple Examples

Example 1 — Bags

A package may contain:

“20 Bags — 30 cm × 40 cm”

The system should identify:

- Number of bags = 20
- Length = 30 cm
- Width = 40 cm

Example 2 — Rectangular Containers

“10 Boxes — 20 cm × 15 cm × 5 cm”

The system can extract:

- Number = 10
- Length = 20 cm
- Width = 15 cm
- Depth = 5 cm

Example 3 — Circular Containers

“25 Containers — Diameter 10 cm × Depth 5 cm”

The system should recognize the circular-container format and verify whether the required dimensions are present.

17.4 OCR Detection

DrishtiTathya can first use OCR to extract declarations from the package.

For example:

Image → OCR → “50 Bags 30 cm × 40 cm”

The AI extraction module can convert this into structured information:

- Product Type: Bag
- Quantity: 50
- Length: 30 cm
- Width: 40 cm

This structured information can then be passed to the Legal Metrology Rule Engine.

17.5 Product Classification

A major requirement for Rule 17 is identifying **what type of container the product represents**.

The proposed system can classify the package into categories such as:

- Bag
- Box
- Circular container
- Other container
- Not applicable

This classification can use both **OCR text and image analysis**.

For example, words such as *bags*, *boxes*, *containers* can provide textual clues, while the package image can provide additional visual information.

17.6 Dimension Extraction

The system should identify common dimensional patterns such as:

- 30 cm × 40 cm
- 20 × 15 × 5 cm
- Diameter 10 cm
- 10 cm × 5 cm

It should also normalize units where necessary so that different representations can be compared consistently.

17.7 Proposed Software Rule

A simplified software representation can be:

IF product_category = "container-type commodity"

THEN

- identify container type
- identify number of containers
- identify required dimensions
- check whether required declarations are present
- flag missing information

The system should **not automatically conclude a legal violation solely from an OCR failure**. If the text is unclear or the product classification has low confidence, it should send the case for human review.

17.8 Evidence Capture

When the system identifies a possible Rule 17 issue, it should preserve evidence such as:

- Original product photograph
- Cropped declaration area
- OCR text
- Extracted number
- Extracted dimensions
- Detected container type
- Applicable rule
- Confidence score
- Reason for the flag

Example:

Potential Rule 17 Issue: Number of containers detected, but required dimensions could not be identified.

This makes the result more useful for an inspecting officer.

17.9 Rule 17 Compliance Flow

PRODUCT IMAGE



OCR + IMAGE ANALYSIS



IDENTIFY CONTAINER TYPE



EXTRACT NUMBER



EXTRACT DIMENSIONS



CHECK RULE 17 REQUIREMENTS



COMPLIANT / POTENTIAL VIOLATION



EVIDENCE + HUMAN REVIEW

17.10 Key Research Insight

Rule 17 demonstrates why a simple **OCR-only system is insufficient** for packaged commodity compliance.

The system must understand the **type of commodity** before deciding which declaration is required. Therefore, DrishtiTathya should combine:

OCR + Product Classification + Rule Engine + Human Review

rather than treating every package using the same checklist.

17.11 Rule 17 Conclusion

Rule 17 provides commodity-specific declaration requirements for container-type products. For DrishtiTathya, this rule can be converted into a structured decision system that identifies the container type, extracts its number and dimensions, checks the applicable requirement and produces evidence for officer verification.

18. RULE 18 — PROHIBITION OF SALE, DISPLAY AND STORAGE OF NON-COMPLIANT PACKAGES

Rule 18 is important from an **enforcement perspective** because it goes beyond checking whether declarations are present. It places responsibility on dealers and other persons involved in the sale or distribution of packaged commodities.

18.1 Basic Requirement

A wholesale dealer, retail dealer or importer must not **sell, distribute, deliver, display or store** a packaged commodity if the package does not comply with the Legal Metrology Act and the applicable Rules.

In simple terms:

A package that does not satisfy the prescribed legal requirements should not be offered in the market.

Rule 18 also provides that a retail dealer or other concerned person, including the manufacturer, packer, importer or wholesale dealer, must not sell a packaged commodity at a price **higher than the declared retail sale price (MRP)**.

18.2 Two Major Compliance Checks

For DrishtiTathya, Rule 18 can be divided into two major checks:

A. Package Compliance

The system checks whether the package satisfies applicable requirements such as:

- Mandatory declarations
- Correct net quantity
- Manufacturer/packer/importer details
- MRP
- Consumer-care information
- Country of origin, where applicable
- Other commodity-specific declarations

B. Price Compliance

The system can compare the **declared MRP** with the **actual selling price**, where actual transaction/selling-price information is available.

Example:

Declared MRP = ₹100

Selling Price = ₹120



Potential Price Violation



Human Verification

The system should treat this as a **potential violation** until the relevant evidence and circumstances are verified by an authorised officer.

18.3 Why Rule 18 Is Important

Rules such as Rule 6 help determine **what information must appear on a package**.

Rule 18 is different because it connects package compliance with **actual market activity**.

Therefore, DrishtiTathya can use Rule 18 to move from:

“Is the label compliant?”

to:

“Is the compliant package actually being sold according to the declared requirements?”

This makes the proposed system more useful for field inspections.

18.4 Proposed Software Implementation

The system can maintain a compliance status for each scanned product:

PRODUCT SCAN



CHECK APPLICABLE RULES



PACKAGE COMPLIANT?



NO YES



FLAG CHECK PRICE



ACTUAL PRICE AVAILABLE?



YES NO

↓ ↓

COMPARE MRP RECORD MRP

↓

POTENTIAL ISSUE

18.5 MRP Detection

OCR can identify common price patterns such as:

- MRP ₹100
- M.R.P. Rs. 50
- Maximum Retail Price ₹250
- Inclusive of all taxes

The system should extract the price into structured data:

MRP:

₹100

Currency:

INR

Tax Status:

Inclusive of applicable taxes

The extracted information can then be passed to the rule engine.

18.6 Actual Selling Price

A more advanced version of DrishtiTathya could obtain actual selling-price evidence from:

- Officer-entered inspection data
- Digital invoice/receipt
- POS information where integrated
- Consumer-uploaded bill
- E-commerce listing
- Marketplace information

This enables a comparison between the **declared MRP and observed selling price**.

18.7 Evidence Management

If a possible Rule 18 issue is detected, the application should preserve supporting evidence.

For example:

Package Evidence

- Product photograph
- MRP photograph
- OCR result

Price Evidence

- Receipt/invoice
- E-commerce screenshot
- Officer-entered observed price

System Evidence

- Applicable rule
- Extracted values
- Comparison result
- Date and time
- AI confidence
- Officer review status

This creates an **evidence-based inspection record** instead of only displaying an AI-generated warning.

18.8 Important Design Principle

DrishtiTathya should **not declare a person legally guilty based only on AI output**.

The application should use terminology such as:

- **Potential violation**
- **Non-compliance detected**
- **Requires verification**
- **Evidence captured**
- **Officer confirmed**

The final legal decision should remain with the authorised enforcement authority.

18.9 Key Research Insight

Rule 18 shows that packaged commodity compliance is not limited to **label scanning**.

A complete system should connect:

Package → Declaration → Price → Market Activity → Evidence → Enforcement

This provides a strong basis for extending DrishtiTathya from an OCR application into a **complete compliance and inspection-support platform**.

18.10 Rule 18 Conclusion

Rule 18 provides an important enforcement layer by restricting the sale, distribution, delivery, display and storage of non-compliant packaged commodities and by prohibiting sale above the declared retail sale price.

For DrishtiTathya, this rule can be implemented through **package compliance verification, MRP extraction, price comparison, evidence capture and human officer review**.

19. RULES 19–22 — INSPECTION, VERIFICATION AND ENFORCEMENT

Rules 19 to 22 are highly relevant to **DrishtiTathya's officer and inspection module**. They explain how packaged commodities may be inspected, what happens after non-compliance is detected, how checks may occur at dealer premises, and how quantity errors are evaluated.

These rules help convert DrishtiTathya from only a **label-scanning application** into an **inspection-support and evidence-management system**.

19.1 Rule 19 — Inspection at Manufacturer or Packer Premises

Rule 19 allows the Director, Controller or authorised Legal Metrology Officer to examine packages and conduct tests at the premises of the manufacturer or packer to determine whether a package or lot complies with the Rules. Samples are to be drawn according to the **Fifth Schedule**, and testing follows the **Sixth Schedule**.

Inspection Process

The basic process can be understood as:

MANUFACTURER / PACKER



SELECT PACKAGE LOT



DRAW SAMPLES



CHECK DECLARATIONS



VERIFY NET QUANTITY



CHECK PERMISSIBLE ERROR



PREPARE INSPECTION REPORT



COMPLIANT / NON-COMPLIANT

The detailed test results are recorded in the prescribed form, and the manufacturer, packer or authorised agent is normally asked to sign it. A copy of the data sheet is provided to the manufacturer or packer.

19.2 What Is Checked During Inspection?

Rule 19 makes three areas particularly important:

Inspection Check	Meaning for DrishtiTathya
Required declarations	AI/OCR label compliance
Actual net quantity	Quantity verification
Error in quantity	Permissible-error calculation

A lot is approved for sale only when the prescribed test conditions are satisfied, including that the statistical average is not below the declared quantity, deficiency does not exceed the maximum permissible error, and required declarations are present.

19.3 DrishtiTathya Implementation

DrishtiTathya can create a digital inspection record containing:

- Inspection ID
- Product name
- Manufacturer/packer
- Batch or lot information
- Product photographs
- Sample details
- Declared quantity
- Observed quantity
- OCR-extracted declarations
- Missing declarations
- Rule checks
- Officer remarks
- Evidence photographs
- Final review status

This would digitize a large part of the **inspection documentation workflow** while leaving legally required physical measurements and officer decisions to the authorised person.

20. RULE 20 — ACTION AFTER INSPECTION

Rule 20 explains what action follows an inspection under Rule 19.

If the inspection report shows that the statistical average is below the declared quantity, a package has deficiency beyond the maximum permissible error, or required declarations are missing, the Rule provides for action by the Director, Controller or Legal Metrology Officer.

20.1 Evidence and Enforcement

The Rule provides for seizure of sample packages and their safe custody for production as evidence, along with initiation of action based on the evidence.

This is particularly useful for designing DrishtiTathya's **Evidence Management Module**.

NON-COMPLIANCE DETECTED



CAPTURE EVIDENCE



STORE SAMPLE DETAILS



LINK APPLICABLE RULE



OFFICER VERIFICATION



CASE / ENFORCEMENT RECORD

DrishtiTathya should not itself perform legal seizure or impose penalties. Instead, it can help the authorised officer **record, organize and preserve the digital evidence associated with the inspection**.

21. RULE 21 — INSPECTION AT WHOLESALE OR RETAIL DEALER PREMISES

Rule 21 deals with quantity and error checks at the premises of a **wholesale or retail dealer**.

According to the uploaded Rules, quantity testing at such premises is ordinarily not carried out unless circumstances such as the following exist:

- A complaint that the package contains less than the declared quantity.
- Suspicion of tampering, pilferage or leakage.
- Required package declarations appear to be missing.

21.1 Consumer Complaint Integration

This creates a strong opportunity for DrishtiTathya.

For example:

CONSUMER SCANS PRODUCT



POTENTIAL ISSUE DETECTED



CONSUMER SUBMITS COMPLAINT



PRODUCT + LOCATION + EVIDENCE RECORDED



OFFICER DASHBOARD



OFFICER DECIDES WHETHER INSPECTION IS REQUIRED

This creates a connection between the **consumer application and government inspection system**.

21.2 Quantity Verification

When a test is carried out, the officer verifies whether the actual quantity corresponds to the declared quantity. If it is lower, the officer determines whether the deficiency exceeds the applicable maximum permissible error.

If the deficiency exceeds the permitted limit, the Rule provides for seizure and appropriate action in accordance with the Act.

Proposed DrishtiTathya Feature

The officer could enter:

- Declared Quantity: 500 g
- Measured Quantity: 485 g
- Difference: 15 g
- Applicable Permissible Error:[Rule Engine Lookup]
- Result:Potential Quantity Non-Compliance
- Status:Requires Officer Confirmation

The important point is that the software performs the **calculation and rule lookup**, while the authorised officer remains responsible for the legal determination.

22. RULE 22 — MAXIMUM PERMISSIBLE ERROR

Rule 22 establishes the concept of **Maximum Permissible Error (MPE)**.

The maximum permissible error for commodities is determined according to the **First Schedule**.

This is important because a small difference between the declared and measured quantity does **not automatically mean a violation**.

The Rules recognize that variations may occur because of factors such as:

- Unavoidable variations during weighing, measuring or counting
- Normal climate, transportation or storage conditions
- Nature of the packaging material or container

22.1 Why This Matters for AI

A poorly designed system might use:

Declared = 500 g

Measured = 498 g

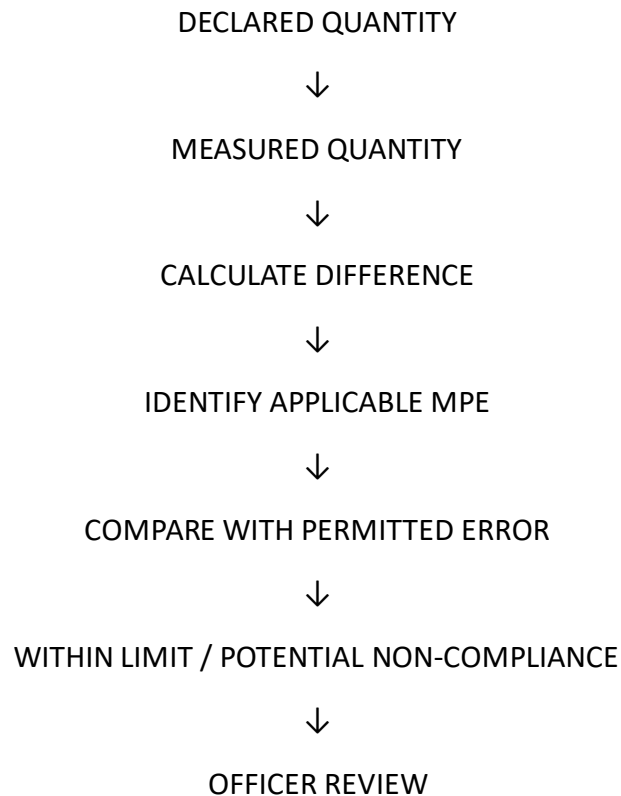
$498 < 500$



VIOLATION

This would be an incorrect software approach.

DrishtiTathya should instead use:



This is an important example of why **OCR alone is insufficient**.

22.2 Proposed Rule Engine

A simplified software logic can be:

- `declared_quantity = OCR / Officer Input`
- `measured_quantity = Officer Input`
- `difference = declared_quantity - measured_quantity`
- `permissible_error = RuleEngine.getApplicableMPE( commodity, declared_quantity )`

IF `difference > permissible_error` → Potential Quantity Violation

ELSE → Within Permissible Error

The real implementation must apply the relevant Schedule and any applicable conditions rather than using one fixed tolerance for every product.

22.3 Key Research Insight

Rules 19–22 show that Legal Metrology compliance is **not simply an image-classification problem**.

A useful system needs to understand:

Product → Applicable Rule → Declaration → Physical Measurement → Permissible Error → Evidence → Officer Decision

Therefore, DrishtiTathya should have two connected layers:

AI Layer: OCR, declaration extraction, product classification and potential violation detection.

Legal/Inspection Layer: rule engine, quantity verification, permissible-error calculation, evidence management and human officer validation.

22.4 Conclusion

Rules 19–22 provide a strong legal basis for designing DrishtiTathya's **Inspection and Enforcement Support Module**. They demonstrate that the system should not merely say “**compliant**” or “**non-compliant**.”

Instead, it should generate a traceable result:

What was detected → Which rule applies → What evidence supports it → What calculation was performed → What requires officer verification.

This approach makes DrishtiTathya more **explainable, evidence-oriented and suitable for government inspection workflows**.

23. RULE 23 — DECEPTIVE PACKAGES

Rule 23 deals with **deceptive packages**, meaning packages whose design or presentation may give consumers an **exaggerated or misleading impression about the quantity of the commodity**.

This rule is important for DrishtiTathya because it introduces a requirement that cannot always be checked through OCR alone. The system may need to combine **package-image analysis with the declared quantity and product information**.

23.1 Basic Requirement

A package may be considered deceptive when its design creates an exaggerated or misleading impression regarding the quantity of the commodity inside it.

However, the Rules recognize that some packaging space may be necessary for legitimate reasons, such as **protection of the commodity or requirements of the packaging machine**.

Therefore, the software should not automatically classify unusual packaging as a legal violation. It should generate a **potential deceptive-package flag for human verification**.

23.2 Why Rule 23 Is Different

Most earlier rules can be checked by identifying specific declarations.

For example:

- MRP → ₹100
- Net Quantity → 500 g
- Manufacturer → Present

Rule 23 is more visual.

A package might contain:

Large Package



Small Amount of Product



Potentially Misleading Presentation

Therefore, image analysis becomes important.

23.3 Proposed AI Detection

DrishtiTathya could analyse:

- Overall package dimensions
- Visible product-filled area
- Empty space inside transparent packaging
- Shape and design of the package
- Declared net quantity
- Product type
- Images from different sides of the package

The system could calculate an **estimated packaging-to-product proportion** where the package structure makes such analysis technically possible.

However, this should be treated as an **AI-assisted screening indicator**, not a legal conclusion.

23.4 Example

Suppose an image shows a large transparent container with a relatively small quantity of product.

The system could produce:

PACKAGE IMAGE



IMAGE ANALYSIS



ESTIMATE PRODUCT-FILLED REGION



COMPARE WITH PACKAGE STRUCTURE



POTENTIAL DECEPTIVE PACKAGING



OFFICER REVIEW

The result could be displayed as:

Potential Issue: Package design may create an exaggerated impression of quantity.

Reason: Large apparent package volume compared with visible product quantity.

Status: Requires human verification.

23.5 Evidence Capture

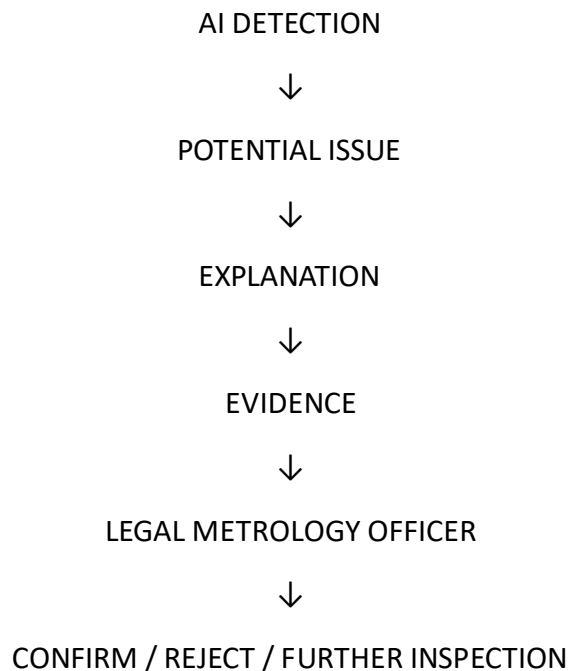
For this type of case, the application should preserve:

- Original package photograph
- Multiple-angle photographs, where available
- Highlighted region of concern
- OCR-extracted quantity
- Image-analysis observations
- Product classification
- Applicable Rule 23 reference
- AI confidence
- Officer remarks
- Final decision

This is important because a visual AI prediction by itself would not be sufficient evidence for a legal determination.

23.6 Human-in-the-Loop Design

Rule 23 is a good example of where **human review must remain part of the system**.



This prevents the system from treating every large or unusual package as deceptive.

23.7 Proposed Software Rule

A simplified software approach could be:

IF package_design_indicates_possible_exaggerated_quantity

AND supporting_visual_evidence_exists

THEN

- create Rule 23 potential-issue flag
- capture evidence
- request human review

ELSE

no Rule 23 flag

The actual legal assessment should remain with the authorised authority.

23.8 Key Research Insight

Rule 23 demonstrates an important limitation of a conventional OCR-based compliance system:

Some compliance requirements depend on how the package is designed and presented, not only on the words printed on it.

Therefore, DrishtiTathya can combine:

OCR + Computer Vision + Product Classification + Legal Rule Engine + Human Review

This is one of the areas that can make the project more technically distinctive.

23.9 Rule 23 Conclusion

Rule 23 extends packaged-commodity compliance from **textual declarations to visual presentation**. DrishtiTathya can use computer vision to identify packages that may require further examination and provide the officer with visual evidence and an explanation of why the package was flagged.

The system should use “**potentially deceptive**” or “**requires review**”, rather than automatically declaring a legal violation.

24. RULES 24–26 — WHOLESALE PACKAGES, EXEMPTIONS AND SPECIAL CATEGORIES

Rules 24–26 are important because not every packaged commodity is treated in exactly the same way. The applicable requirements can change depending on whether the package is a **wholesale package, retail package, small package or a specially exempted category**.

For DrishtiTathya, this means the system should first determine **which category of package it is dealing with** before applying the compliance checklist.

24. RULE 24 — WHOLESALE PACKAGES

Rule 24 deals with declarations required on **wholesale packages**.

A wholesale package is different from an ordinary retail package because it is generally intended for wholesale distribution rather than direct sale to an individual consumer.

The package therefore needs to be evaluated using the requirements applicable to wholesale packages.

24.1 Why Classification Is Important

A common mistake in an automated system would be:

PRODUCT IMAGE



APPLY RETAIL CHECKLIST

This can produce incorrect results.

DrishtiTathya should instead use:

PRODUCT IMAGE



PACKAGE CLASSIFICATION



RETAIL / WHOLESALE / SPECIAL CATEGORY



APPLICABLE RULES



COMPLIANCE CHECK

This **package classification stage** is therefore a key component of the proposed architecture.

24.2 Proposed Software Implementation

The application can collect information such as:

- Package type
- Intended market
- Quantity
- Product category
- Packaging format
- Wholesale/retail indicators

The rule engine can then select the appropriate compliance requirements.

For example:

IF package_type = WHOLESALE



APPLY WHOLESALE PACKAGE RULES

IF package_type = RETAIL



APPLY RETAIL PACKAGE RULES

If the system is uncertain, it should show:

Package category uncertain — manual classification required.

25. RULE 25 — PACKAGES INTENDED FOR EXPORT

Packages intended for export may require different treatment from packages intended for sale within India.

This is important for software design because the **intended market** can affect which legal requirements are applicable.

DrishtiTathya can therefore include an initial question or detection field:

Market:

[Domestic] [Export] [Unknown]

If the system detects an export-oriented package, the appropriate legal conditions should be selected rather than blindly applying every domestic retail-package check.

Proposed Workflow

PACKAGE SCAN



IDENTIFY MARKET



DOMESTIC / EXPORT



SELECT APPLICABLE RULE SET



COMPLIANCE ANALYSIS

This reinforces an important principle for the project:

The legal rule engine should be context-aware, not just keyword-based.

26. RULE 26 — EXEMPTIONS AND SPECIAL CATEGORIES

Rule 26 is particularly important for DrishtiTathya because it demonstrates that **some packages may be exempt from particular provisions of Chapter II**, subject to the conditions specified in the Rule.

The uploaded Rules contain exemptions for certain categories and package sizes, along with specific conditions and provisos.

Therefore, the software should never use a simple logic such as:

IF declaration missing



VIOLATION

Instead:

DECLARATION MISSING



CHECK PRODUCT CATEGORY



CHECK PACKAGE SIZE



CHECK EXEMPTIONS



CHECK AMENDMENT / EFFECTIVE DATE



APPLY FINAL RULE

26.1 Why Exemptions Matter

Consider a simplified example:

OCR detects:

"Net Quantity — Missing"

A basic application might immediately generate:

✗ Violation

But DrishtiTathya should first ask:

- Is this type of package covered?
- Is the package within an exempted size/category?
- Does a special rule apply?
- Is there an amendment affecting this requirement?
- What was the applicable rule on the relevant date?

Only after these checks should the system produce a compliance result.

26.2 Special Category Detection

The application can maintain a **special-category classifier**:

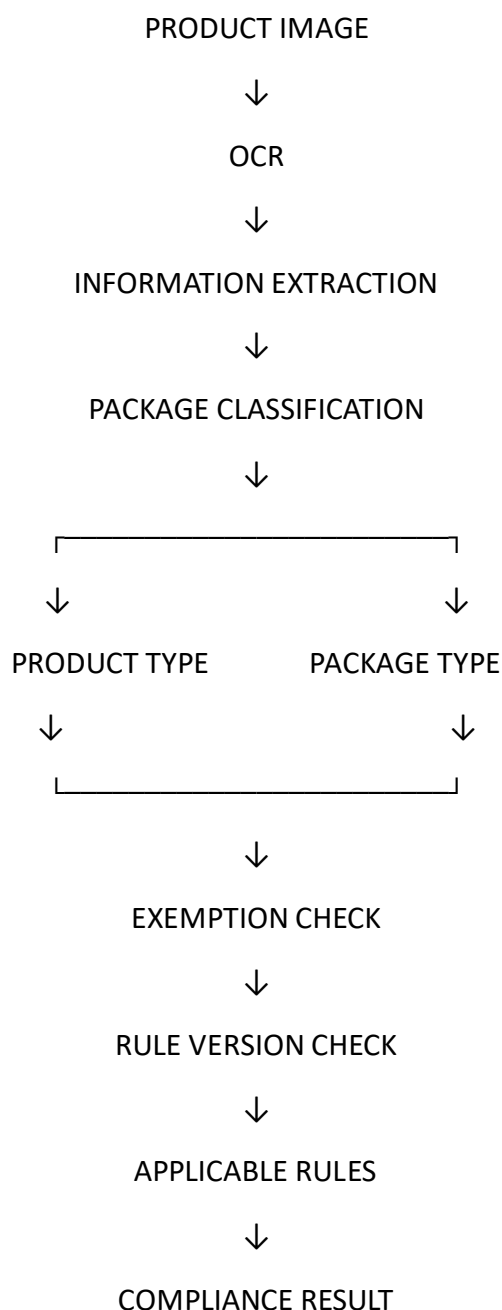
Category	Software Action
Normal retail package	Apply standard rules
Wholesale package	Apply wholesale requirements

Small package	Check applicable exemption
Special commodity	Apply relevant special provision
Medical device	Check applicable special regulatory framework
Export package	Apply relevant conditions
Uncertain	Human review

This makes the system significantly more reliable.

26.3 Rule Engine Architecture

A useful architecture for DrishtiTathya is:



This is much stronger than applying a fixed checklist to every product.

26.4 Amendment-Aware Exemption Checking

One of the most important features of DrishtiTathya should be **version-aware legal validation**.

An exemption or requirement may change through an amendment.

Therefore, the system should store:

- Rule Number
- Requirement
- Product Category
- Package Condition
- Amendment
- Effective Date
- Status
- Source

Then the system can determine:

Which version of the rule was applicable on the relevant inspection/packaging date?

This is especially important because the uploaded document contains amendments and later legal changes need to be tracked separately.

26.5 Evidence-Based Result

Instead of simply showing:

 Non-Compliant

the application should display something like: POTENTIAL NON-COMPLIANCE

- Rule: Rule 26
- Requirement: [specific requirement]
- Package Category: Small Package
- Exemption Check: Completed
- Rule Version: [applicable version]
- Evidence: Product Image + OCR
- Confidence: 91%
- Status: Requires Officer Verification

This creates an **explainable compliance result**.

26.6 Key Research Insight

Rules 24–26 demonstrate a fundamental requirement for DrishtiTathya:

Before checking whether a declaration is missing, the system must first determine whether that declaration is actually applicable to the particular package.

Therefore, the correct sequence is:

Classify → Determine Applicability → Check Exemption → Select Rule Version → Validate → Generate Evidence

This can become one of the strongest differentiating features of the project.

26.7 Conclusion

Rules 24–26 introduce **package-specific requirements, special categories and exemptions**. For DrishtiTathya, these provisions support the design of a context-aware legal rule engine rather than a simple OCR checklist.

The system should therefore answer two questions in order:

1. What rules apply to this package?

2. Does the package comply with those applicable rules?

Only then should it generate a potential violation report.

27. RULES 27–30 — REGISTRATION OF MANUFACTURERS, PACKERS AND IMPORTERS

Rules 27–30 introduce an important **registration and traceability layer**. These provisions connect a packaged commodity with the manufacturer, packer or importer responsible for it.

For DrishtiTathya, this can be used to build a **Manufacturer/Importer Verification Module** and improve traceability during inspections.

27. RULE 27 — REGISTRATION OF MANUFACTURERS, PACKERS AND IMPORTERS

Rule 27 requires persons or entities who **pre-pack or import commodities for sale, distribution or delivery** to apply for registration with the Director or Controller. The application includes the applicant's name, complete address of the premises and the commodities being pre-packed or imported.

The Rule also provides for a registration certificate when the application is complete.

27.1 Information Required

The registration information can be represented as:

MANUFACTURER / PACKER / IMPORTER



NAME



COMPLETE ADDRESS



COMMODITY DETAILS



REGISTRATION



REGISTRATION CERTIFICATE

27.2 DrishtiTathya Implementation

When a product is scanned, the system can extract:

- Manufacturer name

- Packer name
- Importer name
- Address
- Product/commodity name

The extracted information can then be compared against an authorised registration database, where such data is available to the system.

Example:

OCR:ABC Foods Pvt. Ltd.

Pune, Maharashtra



REGISTRATION DATABASE



MATCH FOUND / NOT FOUND / UNCERTAIN

The system should distinguish between “**not found in available database**” and “**legally unregistered**”. The former is a software finding; the latter requires authoritative verification.

27.3 Registration Data as a Traceability Layer

Registration information can help answer:

Who is responsible for this packaged commodity?

This is particularly useful when an officer identifies a potential violation.

Instead of storing only:

Product: XYZ

Violation: Missing declaration

DrishtiTathya can maintain:

Product



Manufacturer / Packer / Importer



Registration Information



Inspection History



Potential Violations



Evidence

This creates a much stronger enforcement record.

28. RULE 28 — REGISTRATION OF SHORTER ADDRESS

Rule 28 permits a manufacturer or packer to apply for registration of a **shorter address** in addition to the complete address. The shorter address may be used where the authority is satisfied that it is sufficient to identify the manufacturer or packer.

28.1 Importance for OCR

This is important because an OCR system might see a package containing an address that appears incomplete.

A basic system could incorrectly flag:

✗ Complete address missing.

But DrishtiTathya should consider whether the displayed shorter address has been **authorised/registered**.

Therefore:

ADDRESS EXTRACTED



COMPLETE / SHORTER ADDRESS?



CHECK REGISTRATION DATA



VALID / UNCERTAIN

This is another example of why the AI layer should not make a legal decision independently.

29. RULE 29 — REGISTRATION RECORD

Rule 29 provides for maintaining a register containing the **name and complete address of registered manufacturers or packers**. The uploaded Rules state that this register is open for public inspection without payment of a fee.

29.1 Proposed Digital Registry

DrishtiTathya can represent this concept digitally:

- MANUFACTURER REGISTRY
- Name
- Address
- Commodity
- Registration Status

- Registration Number
- Last Verified

Where authorised data access exists, the application could allow an officer to search the registry directly from an inspection.

30. RULE 30 — STATE-WISE LIST OF MANUFACTURERS AND PACKERS

Rule 30 provides for compilation and circulation of **State-wise lists of registered manufacturers and packers**. These lists can support State-level sampling and inspection activities.

This creates an important connection between **registration data and inspection planning**.

30.1 Proposed DrishtiTathya Workflow

REGISTERED MANUFACTURERS



STATE-WISE DATA



INSPECTION DATABASE



PRODUCT / VIOLATION HISTORY



RISK PRIORITIZATION



TARGETED INSPECTION

For example, if a particular manufacturer repeatedly receives potential non-compliance flags, the system could present the history to an authorised officer for consideration during inspection planning.

The software should **support prioritization, not independently decide that an inspection must occur**.

30.2 Manufacturer Verification Feature

A dedicated screen could show:

MANUFACTURER VERIFICATION	
Name: ABC Foods Pvt. Ltd.	
Address: Pune, Maharashtra	
Commodity: Packaged Snacks	

Registration: Found	
Status: Verified / Review	
Previous Inspections: 4	
Potential Issues: 2	

This gives the officer context without replacing official records.

30.3 AI + Registry Integration

This feature can make DrishtiTathya more advanced than a conventional OCR application.

Basic OCR System :

IMAGE



OCR



TEXT

DrishtiTathya :

IMAGE



OCR



INFORMATION EXTRACTION



MANUFACTURER / PACKER / IMPORTER IDENTIFICATION



REGISTRATION VERIFICATION



RULE ENGINE



COMPLIANCE RESULT



EVIDENCE

The second approach provides **traceability in addition to label recognition**.

30.4 Key Research Insight

Rules 27–30 demonstrate that packaged commodity compliance involves not only **what is printed on the package**, but also **who is responsible for the package**.

Therefore, DrishtiTathya should maintain two connected forms of information:

Label Information

- Product
- Quantity
- MRP
- Manufacturer/packer/importer
- Other declarations

Entity Information

- Registration
- Address
- Commodity category
- Inspection history
- Previous compliance findings

This can support a future **manufacturer risk and inspection intelligence system**.

30.5 Conclusion

Rules 27–30 provide the foundation for **registration, identity and traceability** of manufacturers and packers.

For DrishtiTathya, these rules can be transformed into:

- Manufacturer/packer/importer identification
 - Registration verification
 - Address validation
 - Digital registry
 - State-wise manufacturer database
 - Inspection history
 - Traceability
 - Risk-based inspection support
-

31. RULES 31–34 — ADVERTISEMENTS, PENALTIES, RELAXATION AND REPEAL/SAVINGS

Rules 31–34 complete the main provisions of the **Legal Metrology (Packaged Commodities) Rules, 2011** covered in the uploaded document. These rules are useful for DrishtiTathya because they extend compliance beyond the physical package to **advertisements, enforcement consequences and historical legal records**.

31. RULE 31 — DECLARATIONS IN ADVERTISEMENTS

Rule 31 applies when an advertisement mentions the **retail sale price** of a pre-packaged commodity.

The advertisement must also contain a declaration of the **net quantity or number of the commodity**. The uploaded Rules further state that the font size of the net quantity in the advertisement should be the same as that of the retail sale price.

31.1 Example

Suppose an advertisement says:

“Special Price ₹199”

If it is an advertisement for a pre-packaged commodity, the system should check whether the required quantity/number declaration is also present.

Example:

ADVERTISEMENT



MRP / RETAIL PRICE DETECTED



SEARCH FOR NET QUANTITY / NUMBER



CHECK PRESENTATION



COMPLIANT / POTENTIAL ISSUE

31.2 DrishtiTathya Feature

This creates an opportunity to extend the system from **package scanning to advertisement compliance checking**.

Possible inputs include:

- Advertisement image
- Poster
- Promotional banner
- Product advertisement screenshot
- E-commerce promotional image

OCR can extract the price and quantity information.

Example

OCR TEXT:"₹99

Pack of 500 g"



PRICE FOUND



QUANTITY FOUND



RULE 31 CHECK



PASS

If the price is detected but the required quantity/number declaration cannot be identified, the system can flag it for review.

32. RULE 32 — PENALTY FOR CONTRAVENTION

Rule 32 specifies penalties for contravention of the Rules.

The uploaded document states that contravention of Rules **27 to 31** attracts a fine of ₹4,000, while certain other contraventions for which no specific punishment is provided attract a fine of ₹2,000 under the stated provision.

32.1 Software Design Principle

DrishtiTathya should **not automatically impose a penalty**.

Instead, it should provide:

POTENTIAL VIOLATION



APPLICABLE RULE



LEGAL PROVISION



EVIDENCE



OFFICER REVIEW



AUTHORIZED ACTION

The application can therefore help the officer understand the relevant provision while keeping the final enforcement decision with the competent authority.

32.2 Penalty Information in the Report

A compliance report could contain:

- Violation:Missing required declaration
- Applicable Rule:Rule 31
- Evidence:Advertisement Image
- Status:Potential Non-Compliance
- Enforcement:Refer to applicable legal provision
- Final Decision:Pending Officer Review

This is safer and more legally appropriate than allowing AI to automatically declare a penalty.

33. RULE 33 — POWER TO RELAX

Rule 33 provides a mechanism through which the Central Government may, subject to the conditions specified in the Rule, permit a manufacturer or packer to pack for sale for a reasonable period by **relaxing one or more provisions** with corrective measures.

33.1 Importance for the Software

This demonstrates another reason why a simple system should not permanently store:

Rule X = Always Required

Instead, the legal rule engine should be capable of representing:

Normal Rule

+

Exception / Relaxation

+

Conditions

+

Validity Period

↓

Applicable Requirement

33.2 Proposed Rule Database

DrishtiTathya can maintain fields such as:

Field	Purpose
Rule Number	Identify legal provision
Requirement	What normally applies
Exception	Special condition

Relaxation	Approved deviation
Conditions	Restrictions attached
Effective From	Start date
Effective Until	End date, if applicable
Source	Gazette/official document
Status	Active/Expired

This supports the project's proposed **version-aware legal rule engine**.

34. RULE 34 — REPEAL AND SAVINGS

Rule 34 deals with the repeal of the earlier **Standards of Weights and Measures (Packaged Commodities) Rules, 1977** and provides savings for earlier operations, rights, obligations, penalties, investigations and legal proceedings.

34.1 Why This Matters for DrishtiTathya

This is particularly relevant to the project's **legal-version management**.

A compliance system may encounter an inspection, package or legal case associated with an earlier period. Therefore, the software should not assume that the latest rule version is automatically the correct one for every historical record.

The system should store:

Inspection Date



Relevant Rule Version



Applicable Requirement



Evidence



Compliance Result

34.2 Version-Aware Legal Engine

This supports one of the strongest features proposed for DrishtiTathya:

The system should determine the applicable rule based on the relevant date instead of blindly applying today's rules to every package.

For example:

PRODUCT / CASE



RELEVANT DATE



RULE VERSION DATABASE



AMENDMENTS



EFFECTIVE DATE



APPLICABLE RULE



COMPLIANCE CHECK

34.3 Complete Rule 31–34 Software Flow

PACKAGE / ADVERTISEMENT



OCR / AI



INFORMATION EXTRACTION



IDENTIFY APPLICABLE RULE



CHECK EXCEPTIONS / RELAXATION



COMPLIANCE VALIDATION



POTENTIAL VIOLATION



EVIDENCE



LEGAL PROVISION



HUMAN REVIEW



AUTHORIZED ACTION

34.4 Key Research Insight

Rules 31–34 show that a complete compliance platform must go beyond **scanning a product label**.

It should also consider:

- Advertisements
- Legal provisions
- Penalty references
- Approved relaxations
- Exceptions
- Historical rule versions
- Effective dates
- Evidence
- Human authorization

This strongly supports the proposed architecture of DrishtiTathya as a **Legal Compliance Decision-Support System**, rather than merely an OCR application.

34.5 Conclusion

Rules 31–34 provide the final layer of the Rules covered in the uploaded document. Rule 31 extends compliance to advertisements, Rule 32 provides penalty provisions, Rule 33 provides for relaxation under specified circumstances, and Rule 34 addresses repeal and savings.

For DrishtiTathya, these provisions support the development of:

Advertisement Scanner + Legal Rule Engine + Exception Manager + Version Control + Evidence Management + Officer Review

35. SCHEDULES — STANDARD PACK SIZES, MAXIMUM PERMISSIBLE ERROR AND SAMPLE SELECTION

The Schedules attached to the Legal Metrology (Packaged Commodities) Rules, 2011 provide additional technical details that support the main Rules. For DrishtiTathya, these Schedules are particularly important because they can be converted into **structured data for automated rule checking and inspection calculations**.

The three areas most relevant to the software are:

1. **Standard pack sizes**
 2. **Maximum Permissible Error (MPE)**
 3. **Sample selection for inspection**
-

35.1 SECOND SCHEDULE — STANDARD PACK SIZES

The Rules specify certain commodities that are required to be packed in the **standard quantities** given in the Second Schedule. The uploaded document also contains a provision that, where a specified commodity is packed in a size other than the prescribed standard size, the package is subject to the declaration requirement described in the Rules.

Proposed Software Implementation

DrishtiTathya can maintain a database like:

Product Category	Declared Quantity	Standard Quantity	Result
Product A	X	Y	Check applicable rule
Product B	X	Y	Check applicable rule
Product C	X	Y	Check applicable rule

The system should **not assume that every product has a standard pack size**. It should first identify whether the commodity appears in the relevant Schedule.

Software Flow

PRODUCT IDENTIFICATION



CHECK SECOND SCHEDULE



STANDARD PACK REQUIREMENT?



YES NO



COMPARE CONTINUE

QUANTITY NORMAL CHECKS



COMPLIANCE RESULT

35.2 FIRST SCHEDULE — MAXIMUM PERMISSIBLE ERROR

The First Schedule is important for quantity verification because Rule 22 states that the **Maximum Permissible Error** for commodities is determined according to the First Schedule.

MPE is important because the measured quantity does not always have to be mathematically identical to the declared quantity.

The software should therefore calculate:

Declared Quantity



Measured Quantity



Quantity Difference



Applicable MPE



Compare



Within Limit / Potential Non-Compliance

Example

Suppose:

Declared Quantity = X

Measured Quantity = Y

Difference = X – Y

Applicable MPE = Z

The rule engine can determine whether:

Difference ≤ Z

or

Difference > Z

The actual value of **Z** **must be retrieved from the applicable Schedule and commodity conditions**, rather than being hard-coded as one universal tolerance.

35.3 SIXTH SCHEDULE — METHOD OF TESTING

Rule 19 states that inspections at manufacturer or packer premises use samples selected according to the **Fifth Schedule**, while the tests are conducted according to the method specified in the **Sixth Schedule**.

This means the software should distinguish between:

Sample Selection

and

Actual Testing

DrishtiTathya can assist with recording and calculating results, but it should not claim that a photograph alone can replace the prescribed physical testing procedure.

35.4 FIFTH SCHEDULE — SAMPLE SELECTION

The Fifth Schedule specifies sample sizes according to the **lot size**. The uploaded document states that the sample size for determining net quantity is selected according to the corresponding lot-size entry in the Schedule.

The document contains a table mapping lot sizes to sample sizes.

For example, the uploaded version shows entries including:

Lot Size	Sample Size
-----------------	--------------------

Less than 4000	32
----------------	----

More than 4000	80
----------------	----

For the final implementation, the complete applicable Schedule should be entered into the rule database rather than relying on only these example rows.

35.5 Digital Sample Management

This provides a useful feature for the officer application.

The officer can enter:

Lot Size: _____



Rule Engine



Required Sample Size



Sample Collection



Sample IDs



Individual Test Results

For example:

Lot ID: LOT-2026-001

Lot Size: 3500

Required Samples: 32

Sample 01 → Quantity → Result

Sample 02 → Quantity → Result

Sample 03 → Quantity → Result

...

Sample 32 → Quantity → Result

This makes the inspection process more organized and reduces manual calculation errors.

35.6 Statistical Average

Rule 19 also uses the **statistical average of the net quantity** of the sample packages as one of the criteria for approval of a lot. The uploaded Rules state that a lot is approved when the statistical average of the sample net quantity is equal to or greater than the declared quantity, subject to the other requirements of the Rule.

DrishtiTathya can therefore calculate:

Sample Quantities

↓

Q1, Q2, Q3 ... Qn

↓

Calculate Statistical Average

↓

Compare With Declared Quantity

↓

Check Individual MPE

↓

Generate Inspection Result

This is a strong example of how the project combines **legal rules + data processing + mathematical validation**.

35.7 Proposed Inspection Dashboard

The officer could see:

LOT INSPECTION		
Lot Size:	3,500	
Required Samples:	32	
Samples Tested:	32	
Declared Quantity:	500 g	
Average Quantity:	XXX g	

Applicable MPE:	XXX g	
Declaration Check:	PASS	
Quantity Check:	REVIEW	
Final Status:	Officer	
	Verification	

35.8 Important Design Limitation

DrishtiTathya should clearly separate:

AI-based checks

- OCR
- Declaration extraction
- Product classification
- Label analysis
- Potential missing declarations
- Image-based observations

Physical inspection checks

- Actual quantity measurement
- Sampling
- Testing according to prescribed methods
- Verification of physical packages

The application can **record and calculate physical inspection results**, but the phone camera should not be presented as a substitute for legally prescribed measurement instruments or testing procedures.

35.9 Complete Schedule-Based Flow

PRODUCT / LOT



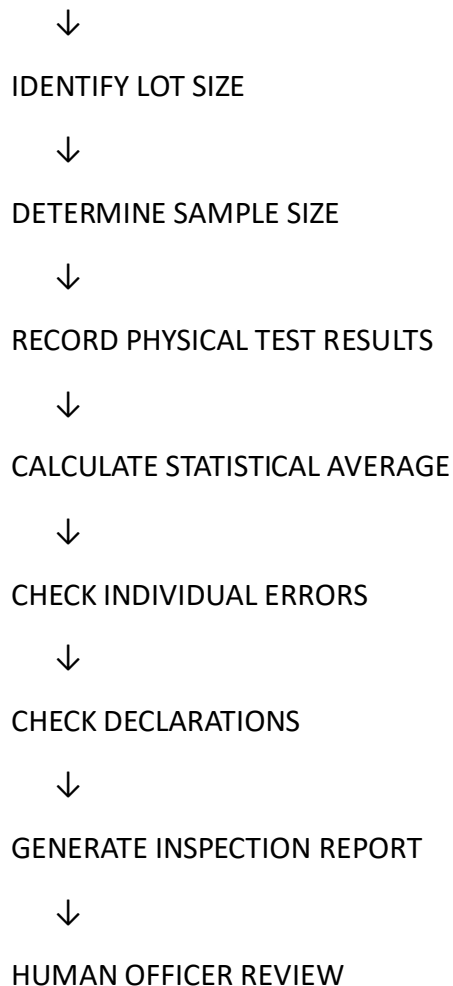
IDENTIFY COMMODITY



CHECK STANDARD PACK SIZE



CHECK APPLICABLE MPE



35.10 Key Research Insight

The Schedules demonstrate that Legal Metrology compliance contains a significant **data and calculation component**.

Therefore, DrishtiTathya is not only:

Camera + OCR

It can become:

Camera + OCR + Classification + Rule Database + Schedule Database + Mathematical Validation + Evidence + Human Review

This is an important technical distinction for the SIH project.

35.11 Conclusion

The Schedules provide the technical information required to apply several Rules correctly. Standard pack-size requirements help determine whether the declared quantity is appropriate, the First Schedule provides the basis for Maximum Permissible Error, and the Fifth/Sixth Schedules support sampling and testing during inspection.

For DrishtiTathya, these can be implemented as a **structured Schedule Database and Inspection Calculation Engine**.

36. AMENDMENTS TO THE PACKAGED COMMODITIES RULES — NEED FOR A VERSION-AWARE LEGAL RULE ENGINE

The Legal Metrology (Packaged Commodities) Rules, 2011 have been amended several times after their original notification. Therefore, a compliance system cannot simply store one static copy of the Rules and apply it to every product.

The uploaded Rules themselves show that the original framework contains provisions for applicability, declarations, standard quantities, inspection, registration and other requirements.

For DrishtiTathya, this creates an important software requirement:

The system must determine which version of a legal requirement is applicable before declaring a package compliant or non-compliant.

36.1 Why Amendments Matter

A legal requirement can change because of an amendment.

For example, a rule may have:

OLD REQUIREMENT



AMENDMENT



NEW REQUIREMENT



EFFECTIVE DATE



CURRENT REQUIREMENT

If the software uses an outdated rule, it may generate an incorrect compliance result.

Therefore, **legal version control** should be a core component of DrishtiTathya.

36.2 Important Amendment Timeline

The project research identified the following important amendments and changes:

Year	Major Change	Software Importance
2017	Changes to applicability, definitions and package requirements	Applicability engine
2021	Changes to declarations and unit sale price provisions	Declaration engine
2022	Change in commencement date of an amendment	Effective-date handling
2025	Changes concerning medical devices	Product-category rules
2025	Change concerning pan masala exemption	Exception handling
2026	Country-of-origin filter requirement for e-commerce	E-commerce compliance

2026	Further amendment to e-commerce country-of-origin requirement	Future-dated rule activation
2026	Changes concerning imports and registration information	Registration/import checks

Important: proposed amendments must not be treated as enacted law. The 2024 proposal concerning Rule 3 should therefore be stored separately from active legal requirements.

36.3 Rule Version Database

DrishtiTathya can store each legal requirement with metadata rather than storing only the rule text.

Suggested structure

Field	Example
Rule ID	PCR-R6-01
Rule Number	Rule 6
Requirement	Manufacturer/packer/importer declaration
Category	Mandatory Declaration
Effective From	Date
Effective Until	Date / NULL
Amendment	Amendment notification
Applicability	Retail package
Exception	Applicable exceptions
Source	Official notification
Status	Active / Superseded / Proposed

This allows the system to answer:

“Which version of this requirement was applicable on the inspection date?”

36.4 Effective-Date Logic

The software should not simply ask:

“What is the current Rule?”

It should ask:

“What Rule was applicable on the relevant date?”

For example:

INSPECTION DATE



IDENTIFY RULE



CHECK AMENDMENT HISTORY



CHECK EFFECTIVE DATE



SELECT APPLICABLE VERSION



RUN COMPLIANCE CHECK

This is one of the most important features that can make the project different from a simple OCR application.

36.5 Example: Applicability Changes

The original uploaded document contains Rule 3, which describes packages to which Chapter II does not apply.

Later amendments changed the applicability framework.

Therefore, the software should not implement Rule 3 simply as:

IF quantity > X

→ EXEMPT

Instead:

PRODUCT



PACKAGE TYPE



QUANTITY



CONSUMER TYPE



PRODUCT CATEGORY



INSPECTION DATE



APPLICABLE RULE VERSION

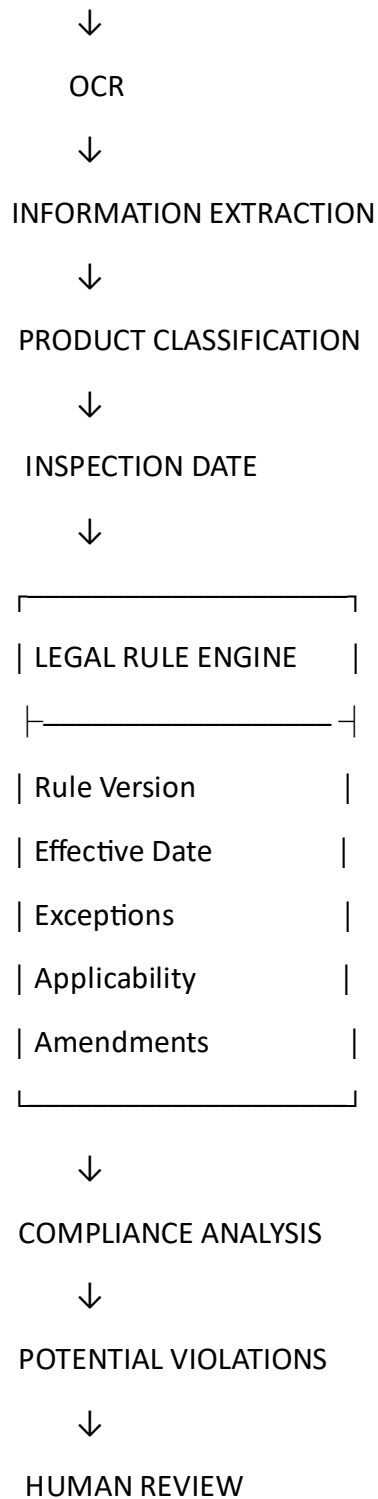


APPLICABILITY RESULT

36.6 Amendment-Aware Rule Engine

The proposed architecture is:

PRODUCT IMAGE



36.7 Future-Dated Rules

An especially useful feature is the ability to store a rule before it becomes effective.

For example:

Rule



Notification Date



Effective Date



ACTIVE?



NO YES



Store Apply

as Future Rule

This prevents the system from accidentally applying a future requirement before its commencement date.

36.8 Proposed, Active and Superseded Rules

Every legal rule in the database can have one of three statuses:

1. ACTIVE

Currently applicable.

2. SUPERSEDED

Previously applicable but replaced by a later amendment.

3. PROPOSED

Published for consultation but not yet applicable.

This distinction is critical for legal accuracy.

36.9 Legal Traceability

When DrishtiTathya flags a package, the officer should be able to see **why**.

Example: ⚠ POTENTIAL NON-COMPLIANCE

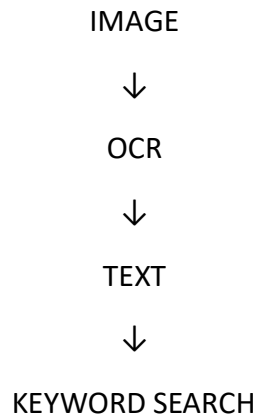
- Issue: Required declaration may be missing.
- Rule: Rule 6
- Rule Version: Applicable version on inspection date
- Effective From: DD/MM/YYYY
- Reason: Required declaration could not be detected.
- Evidence: Image → OCR text → Detected fields
- Confidence: 87%
- Status: Pending Human Review

The system should therefore provide a **legal trace** from:

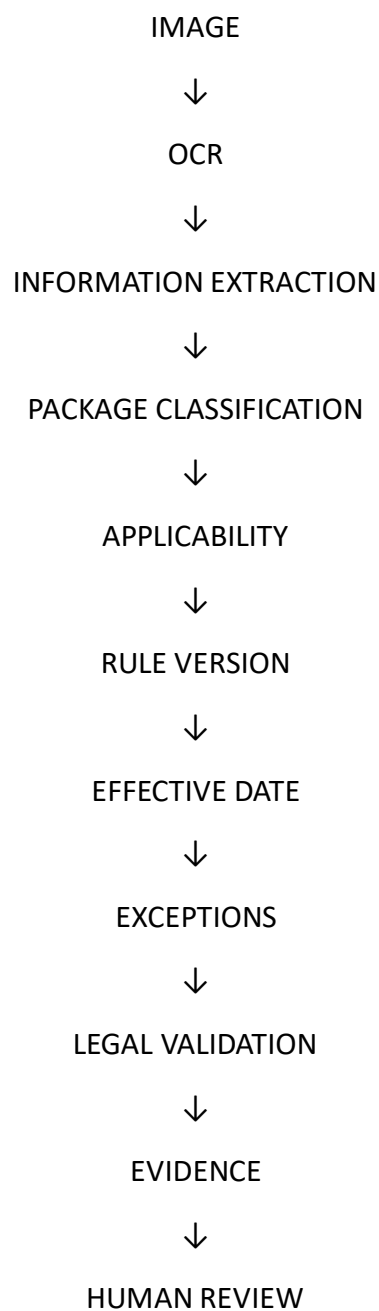
Image → Extracted Fact → Rule → Amendment → Result

36.10 Why This Is Unique for DrishtiTathya

A normal label-scanning application may work like:



DrishtiTathya can work like:



This transforms the project from an **OCR-based label reader** into an **AI-assisted legal compliance system**.

36.11 Important Legal-Safety Principle

The system should not state:

“The manufacturer has violated the law.”

Instead, the AI should state:

“Potential non-compliance detected — officer verification required.”

The final legal decision should remain with the authorized human officer.

This makes the system more suitable for real-world government enforcement because AI is being used for **assistance, prioritization and evidence preparation**, rather than replacing statutory authority.

36.12 Research Finding

The amendment history leads to an important research conclusion:

Legal compliance software must be version-aware, date-aware and exception-aware.

For DrishtiTathya, this can become one of the project's major differentiators.

Proposed Feature Name

“Version-Aware Legal Metrology Rule Engine”

Core capabilities

- Rule version management
 - Amendment tracking
 - Effective-date checking
 - Future-rule storage
 - Exception handling
 - Product-specific applicability
 - Legal source traceability
 - Audit trail
 - Human verification
-

37. EXISTING INSPECTION PROCESS AND ITS DIGITIZATION

Legal Metrology inspection is not limited to reading the label. It involves **examination of packages, sampling, testing, recording results and taking action when non-compliance is found**.

Rule 19 of the uploaded Rules specifically provides for examination and testing of packages at the premises of the manufacturer or packer. Samples are to be drawn according to the Fifth Schedule and tests are to be conducted according to the Sixth Schedule.

37.1 Existing Inspection Workflow

A simplified inspection workflow can be represented as:

INSPECTION LOCATION



IDENTIFY MANUFACTURER / PACKER



IDENTIFY PRODUCT / LOT



EXAMINE PACKAGES



SELECT SAMPLES



CHECK DECLARATIONS



CONDUCT REQUIRED TESTS



RECORD RESULTS



PREPARE INSPECTION REPORT



DETERMINE COMPLIANCE



ACTION / FOLLOW-UP

Rule 19 also requires the detailed test results to be entered in the form specified in the Seventh Schedule.

37.2 Step 1 — Identification of Package

The first stage is to identify the package being inspected.

Important information can include:

- Product name
- Manufacturer
- Packer

- Importer, where applicable
- Declared quantity
- Retail sale price
- Batch/lot information
- Date-related declarations
- Country of origin, where applicable

The software can reduce manual entry by extracting visible information from package images.

37.3 Step 2 — Package Examination

The officer examines whether the required declarations are present on the package.

Rule 4 establishes that a commodity should not be pre-packed for sale, distribution or delivery unless the package or securely affixed label bears the declarations required by the Rules.

DrishtiTathya can assist by automatically checking for expected declarations.

Example

DECLARATION CHECK		
Manufacturer	✓	
Address	✓	
Net Quantity	✓	
MRP	✓	
Consumer Care	X	
Country of Origin	✓	

The missing field should be shown as a **potential issue**, not automatically treated as a final legal violation.

37.4 Step 3 — Sample Selection

Where inspection requires sampling, the officer selects packages according to the applicable procedure.

Rule 19 specifies that samples are drawn in the manner and number prescribed in the Fifth Schedule.

DrishtiTathya can provide:

LOT SIZE



FIFTH SCHEDULE



REQUIRED SAMPLE SIZE



GENERATE SAMPLE IDs



RECORD EACH SAMPLE

For example:

LOT: LOT-001

Required Samples: 32

S-001 , S-002 , S-003 , ... , S-032

37.5 Step 4 — Quantity Verification

The actual quantity may need to be physically measured.

The application can record:

Declared Quantity



Actual Measured Quantity



Difference



Applicable MPE



Result

Rule 19 requires the inspection report to record the net quantity actually contained in sample packages and the extent of error, if any.

Important

The smartphone camera can assist with **documentation and data entry**, but it should not be presented as replacing prescribed physical measurement or testing equipment.

37.6 Step 5 — Statistical Analysis

The inspection process can involve analysis of the sample results.

Rule 19 provides conditions involving the statistical average of the net quantity and the permissible error of individual packages.

DrishtiTathya can automatically calculate:

Sample Results



Average Quantity



Compare With Declared Quantity



Check Individual Errors



Generate Result

This reduces manual calculations during field inspection.

37.7 Step 6 — Digital Evidence Collection

A major improvement can be the creation of a structured evidence record.

Each inspection can contain:

INSPECTION ID



PRODUCT IMAGES



OCR TEXT



EXTRACTED DECLARATIONS



RULES CHECKED



PHYSICAL TEST RESULTS



OFFICER OBSERVATIONS



FINAL REPORT

The system should preserve the original photograph along with the extracted information so that an officer can verify what the AI detected.

37.8 Step 7 — Inspection Report

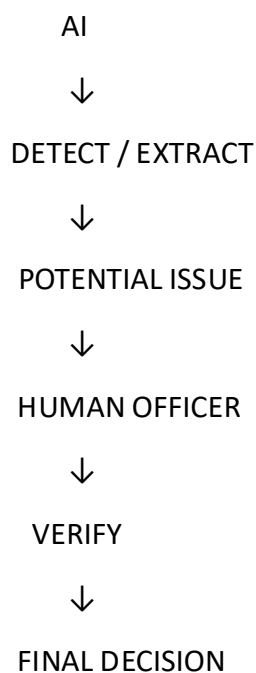
Rule 19 requires the authorized person to prepare a report containing findings regarding the required declarations, actual net quantity and extent of error.

DrishtiTathya can automatically prepare a structured report containing:

Section	Information
Inspection ID	Unique identifier
Date & Location	Inspection details
Product	Product identification
Manufacturer/Packer	Extracted/entered details
Declaration Check	Pass / Potential Issue
Quantity	Declared and measured
MPE	Applicable tolerance
Samples	Sample-wise results
Evidence	Images and observations
AI Findings	Potential issues
Officer Decision	Final verification
Action	Follow-up/action recorded

37.9 AI + Human Officer Model

The proposed system should follow:



This is safer than allowing the AI to directly make legal enforcement decisions.

Example

AI:

“Consumer-care declaration not detected.”

Officer:

Reviews the package image.

Officer:

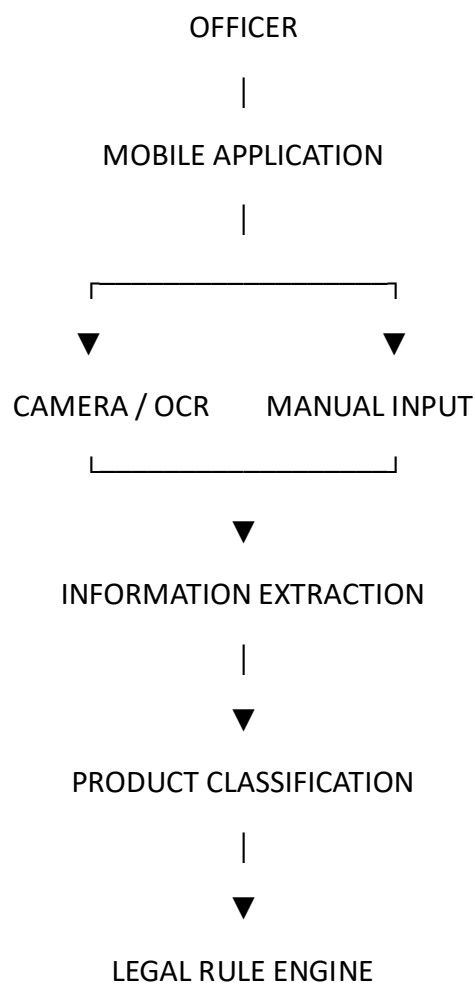
Confirms whether the declaration is actually absent.

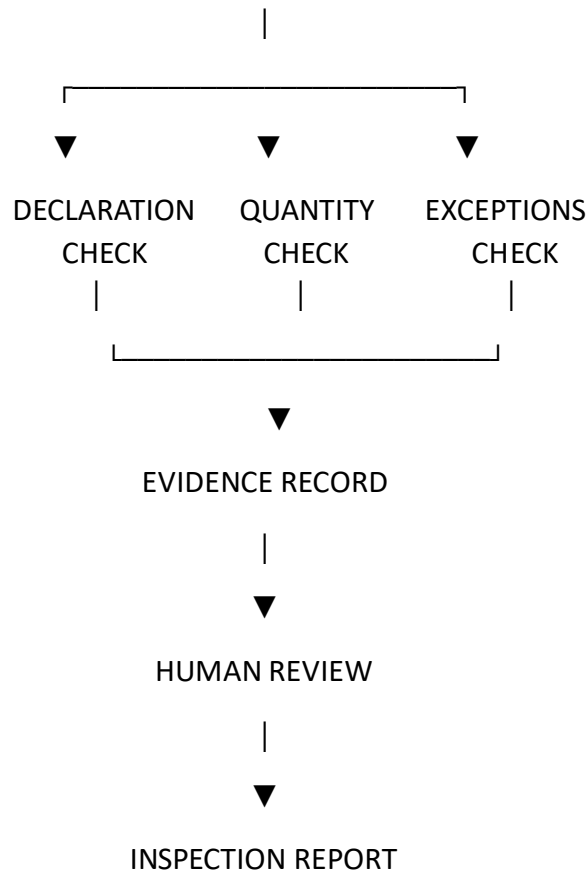
System:

Records the officer's final determination.

37.10 Digitization of the Existing Process

Existing Activity	DrishtiTathya Enhancement
Manual package examination	AI-assisted image analysis
Manual reading of label	OCR
Manual declaration checking	Declaration extraction
Manual rule lookup	Rule engine
Manual amendment checking	Version-aware rules
Manual sample calculation	Automatic sample-size calculation
Manual calculations	Quantity/statistical calculator
Paper evidence	Digital evidence
Manual report preparation	Auto-generated report
Separate records	Central inspection history

37.11 Proposed Digital Inspection Architecture



37.12 Key Research Gap

The inspection process demonstrates that the problem is larger than simply extracting text from product packaging.

A useful compliance system needs to connect:

Visual Information → Structured Facts → Legal Rules → Inspection Procedure → Evidence → Human Decision

This is the core research gap that DrishtiTathya attempts to address.

37.13 Conclusion

The Legal Metrology inspection workflow already contains defined stages for examination, sampling, testing and reporting. Rule 19 explicitly connects these activities with the Fifth, Sixth and Seventh Schedules.

DrishtiTathya can digitize this workflow while keeping the **authorized officer as the final decision-maker**.

Proposed Transformation

- Manual Inspection → Digital Inspection
- Paper/visual checking → AI-assisted checking
- Manual rule lookup → Version-aware rule engine
- Manual calculations → Automated calculations
- Paper evidence → Digital evidence trail

- Manual report → Auto-generated inspection report

38. EXISTING SYSTEMS AND RESEARCH GAP

Before developing DrishtiTathya, it is important to understand what existing approaches can already do and where they are insufficient for **Legal Metrology packaged-commodity compliance**.

The research shows that simply using OCR or a barcode scanner is not enough. The proposed system needs to connect **package information with legal applicability, rules, exceptions, evidence and human verification**.

38.1 Existing Technology Approaches

Current technology-based approaches can broadly be divided into the following categories:

Existing Approach	Main Capability	Limitation
OCR applications	Extract text from images	Do not determine legal compliance
Barcode/QR scanners	Identify products	Code does not prove label compliance
Product-information apps	Display product information	Usually not legal-rule based
Manual inspection	Physical/legal verification	Time-consuming and dependent on manual work
Generic AI vision systems	Detect objects/text	Not specifically trained for Legal Metrology
Static rule/document systems	Store legal information	Usually do not connect rules with package images
E-commerce checking	Inspect online listings	Limited legal-rule reasoning

38.2 OCR-Based Systems

OCR can convert a package image into machine-readable text.

Example:

PACKAGE IMAGE

↓

OCR

↓

"MRP ₹120"

"Net Qty 500 g"

"Manufactured by XYZ"

This is useful, but OCR alone cannot answer:

- Is the declaration applicable?
- Is the quantity expressed in the correct unit?
- Is an exception applicable?
- Which version of the Rule applies?

- Was the requirement effective on the inspection date?
- Is the declaration sufficient under the relevant Rule?

Therefore:

OCR extracts information; it does not establish legal compliance.

38.3 Barcode / QR-Based Systems

Barcode or QR scanning can help identify a product or retrieve information.

However, identification and compliance are different problems.

BARCODE



PRODUCT IDENTIFICATION



PRODUCT DATA

does not automatically mean:

PRODUCT DATA



LEGAL COMPLIANCE

A product can have a valid barcode while still having a potentially non-compliant declaration on its package.

Therefore, barcode scanning should be an **additional identity layer**, not the primary compliance mechanism.

38.4 Manual Inspection

Traditional inspection provides an important advantage: an authorized officer can interpret the package, physically inspect it and apply the relevant legal provisions.

However, several activities may require significant manual effort:

- Reading small package text
- Checking multiple declarations
- Finding applicable Rules
- Checking amendments
- Recording evidence
- Calculating quantity differences
- Preparing reports

- Maintaining inspection history

DrishtiTathya is intended to **assist rather than replace** this process.

38.5 Generic AI Vision Systems

Modern computer-vision systems can detect:

- Text
- Objects
- Logos
- Product categories
- Visual features

But a general vision model does not automatically understand the complete Legal Metrology framework.

For example:

AI sees:

"500 g"

The compliance system needs to determine:

500 g

↓

Extract quantity

↓

Identify commodity

↓

Determine applicable requirement

↓

Check unit format

↓

Check relevant exceptions

↓

Check applicable Rule version

↓

Generate evidence-backed result

This requires a **domain-specific legal rule engine**.

38.6 Research Gap

The major research gap can therefore be expressed as:

There is a need for a system that connects visual package information with structured Legal Metrology requirements and produces explainable, evidence-backed compliance observations for human verification.

The gap is not simply:

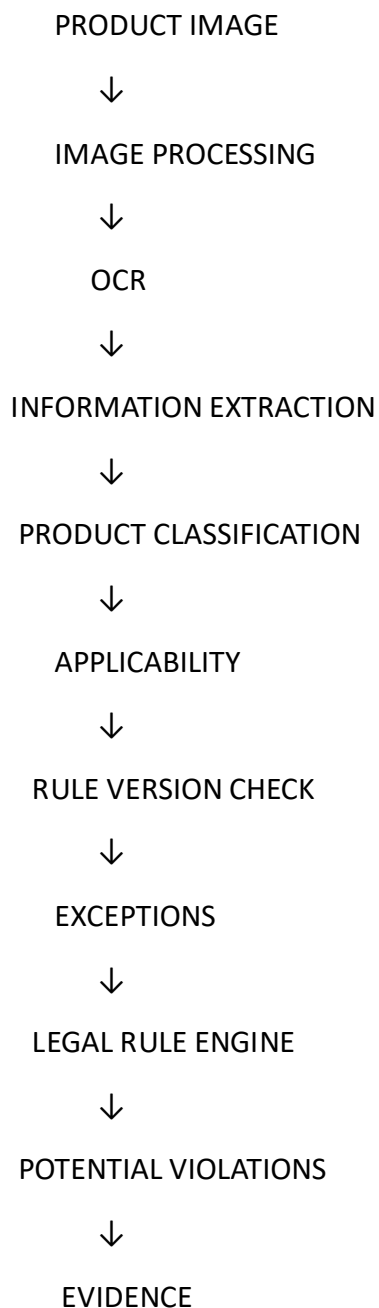
“OCR is missing.”

The deeper gap is:

“The connection between package facts and applicable legal requirements is missing.”

38.7 DrishtiTathya's Proposed Solution

DrishtiTathya addresses this gap through a layered architecture:





38.8 Comparison: Existing Approach vs DrishtiTathya

Capability	Traditional / Generic Approach	DrishtiTathya
Product image	✓	✓
OCR	✓	✓
Product information extraction	Limited	✓
Declaration identification	Limited	✓
Legal rule mapping	Usually absent	✓
Applicability checking	Limited	✓
Amendment awareness	Usually absent	✓
Effective-date checking	Usually absent	✓
Exception handling	Limited	✓
Evidence linking	Limited	✓
Quantity calculation	Manual / separate	Integrated
Human review	✓	✓
Inspection history	Limited	✓
Government-oriented workflow	Limited	✓

38.9 Explainable Compliance

One of the major differences should be **explainability**.

Instead of:

✗ Non-Compliant

the system should provide:

⚠ **Potential Non-Compliance**

Detected fact:

Consumer-care declaration could not be detected.

Expected requirement:

Applicable declaration requirement under the relevant Rule.

Evidence:

Package image + OCR region.

AI confidence:

87%

Status:

Human officer verification required.

This makes the result understandable and auditable.

38.10 Research-to-Software Mapping

The research gap can directly translate into software modules:

Research Problem	Proposed Module
Difficult package reading	OCR
Unstructured label information	Information Extraction
Different commodity requirements	Product Classification
Different applicability conditions	Applicability Engine
Changing law	Version-Aware Rule Engine
Special cases	Exception Engine
Manual verification	Officer Review
Evidence management	Evidence Repository
Repeated inspections	Inspection History
Reporting workload	Report Generator

38.11 Key Innovation

The most important innovation is not any individual technology.

It is the **combination of technologies and legal reasoning**:

Computer Vision

+

OCR

+

Information Extraction

+

Legal Knowledge Base

+

Rule Engine

+

Version Control

+

Evidence Management

+

Human Review

This makes DrishtiTathya a **Legal Compliance Decision-Support System**, rather than just a product-scanning application.

38.12 Final Research Gap Statement

For the SIH documentation, the research gap can be summarized as follows:

Existing approaches primarily focus on text extraction, product identification or manual inspection. They do not adequately integrate package-image analysis with applicability determination, amendment-aware legal rules, exceptions, evidence generation and officer verification. DrishtiTathya proposes an integrated AI-assisted platform to bridge this gap.

38.13 Conclusion

The research indicates that the strongest opportunity is to build a system that moves through the complete chain:

See → Extract → Understand → Apply Law → Explain → Record → Verify

This becomes the foundation for designing the actual DrishtiTathya system architecture.

39. CONVERTING LEGAL RULES INTO SOFTWARE RULES

The main technical challenge in DrishtiTathya is converting legal provisions written in natural language into **structured rules that a computer can evaluate**.

A legal rule cannot simply be stored as a paragraph and searched using keywords. The software needs to understand:

- What requirement exists?
 - To which package does it apply?
 - What information must be present?
 - Are there exceptions?
 - Which version of the Rule is applicable?
 - What evidence is required?
 - What should happen when information is uncertain?
-

39.1 Legal Rule → Software Rule

The transformation can be represented as:

LEGAL TEXT



RULE INTERPRETATION



REQUIREMENT



CONDITIONS



EXCEPTIONS



EFFECTIVE DATE



MACHINE-READABLE RULE



COMPLIANCE RESULT

For example, Rule 4 states that a package intended for sale, distribution or delivery must bear the declarations required by the Rules.

The software can represent the concept as:

IF package_is_applicable

AND package_is_prepacked_for_sale

THEN

required_declarations = applicable_declarations

39.2 Rule Components

Each software rule should contain structured fields.

Component	Purpose
Rule ID	Unique identification
Rule Number	Legal reference
Requirement	What must be checked
Product Category	Products to which it applies
Conditions	Conditions for application
Exceptions	Cases where requirement changes
Effective From	Start date
Effective Until	End date, if applicable
Evidence	Information required for verification
Severity	Priority of potential issue
Source	Legal notification/reference
Review Required	Whether officer confirmation is needed

39.3 Example: Declaration Rule

Rule 6 requires specified declarations to be made on packages. The uploaded Rules begin Rule 6 by requiring every package to bear definite, plain and conspicuous declarations, including manufacturer/packer/importer information.

DrishtiTathya can convert this into: RULE_ID = PCR_R6_MANUFACTURER

IF package_is_applicable

THEN

manufacturer_information = REQUIRED

IF imported_package

THEN

importer_information = REQUIRED

The system then checks the extracted package information.

39.4 Rule Engine Decision Process

The compliance engine can follow:

PACKAGE DATA



IS RULE APPLICABLE?



YES



WHAT DECLARATIONS ARE REQUIRED?



EXTRACT DECLARATIONS



COMPARE



COMPLIANT / POTENTIAL ISSUE / UNCERTAIN

Three outcomes are preferable to a simple binary result.

✓ COMPLIANT

Required information was detected and appears to satisfy the programmed check.

⚠ POTENTIAL ISSUE

A possible non-compliance was detected.

? UNCERTAIN

The image, OCR or available information is insufficient for a reliable determination.

39.5 Exception Handling

Legal rules frequently contain exceptions.

Therefore, the engine should not operate as:

IF X

THEN VIOLATION

It should operate as:

IF X

AND NOT EXCEPTION_A

AND NOT EXCEPTION_B

AND RULE_VERSION_APPLIES

THEN

FLAG

This is particularly important for DrishtiTathya because the Rules contain provisions dealing with different package types, consumer categories and exemptions.

39.6 Applicability Engine

Before checking individual declarations, the software should determine whether the relevant Chapter/Rule applies.

PACKAGE



PRODUCT CATEGORY



PACKAGE SIZE



PURPOSE OF PACKAGE



CONSUMER TYPE



IMPORT / DOMESTIC



APPLICABILITY

This prevents the system from incorrectly applying a requirement to a package that falls outside its scope.

39.7 Version-Aware Logic

The rule engine should also consider the inspection date.

RULE

↓

AMENDMENT HISTORY

↓

INSPECTION DATE

↓

EFFECTIVE VERSION

↓

APPLY RULE

For example:

IF inspection_date >= effective_date

THEN

 use amended_rule

ELSE

 use previous_rule

This makes the system capable of handling changing legislation.

39.8 Evidence-Based Rule Evaluation

Every automated result should be connected to evidence.

For example:

- Rule:Rule 6
- Required Field:Net Quantity
- Detected Value:500 g
- Evidence:Image #003 , OCR Region #12
- Confidence:94%
- Result:Requirement detected
- Review: Officer verification

This creates a traceable chain:**Rule → Requirement → Extracted Fact → Evidence → Result**

39.9 Confidence-Aware Compliance

AI systems can make OCR or classification mistakes.

Therefore, DrishtiTathya should maintain confidence values.

Example:

Detection	Confidence	Action
MRP	98%	Automatically show
Net quantity	95%	Automatically show
Manufacturer	89%	Show + review
Country of origin	62%	Manual verification
Small/unclear text	35%	Request better image

The confidence score should indicate **AI certainty**, not legal certainty.

39.10 Rule Severity

Potential issues can also be prioritized.

HIGH



Important potential compliance issue

MEDIUM



Requires officer verification

LOW



Minor / informational observation

However, severity should be configurable and should not be presented as a legal penalty unless legally established.

39.11 Example Machine-Readable Rule

A conceptual structure could be:

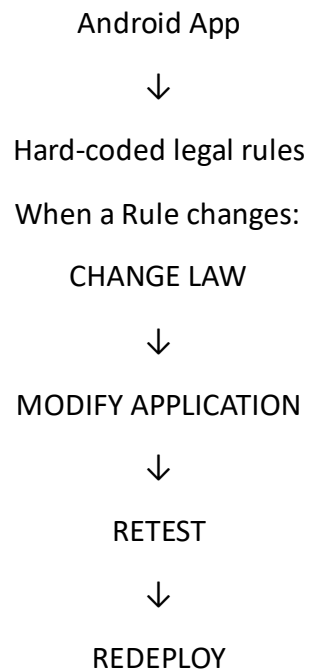
- RULE_ID:PCR-R6-MRP
- RULE_NUMBER:6
- REQUIREMENT:Retail Sale Price declaration
- APPLICABILITY:Applicable package categories
- CONDITION:Package is within scope of Chapter II
- CHECK:MRP detected on package
- EXCEPTION:Check applicable exemptions
- EFFECTIVE_FROM:Applicable amendment date
- OUTPUT:PASS / POTENTIAL ISSUE / UNCERTAIN
- EVIDENCE:Package image + OCR region
- HUMAN_REVIEW:YES

This structure allows developers to store legal requirements in a database rather than embedding them permanently inside application code.

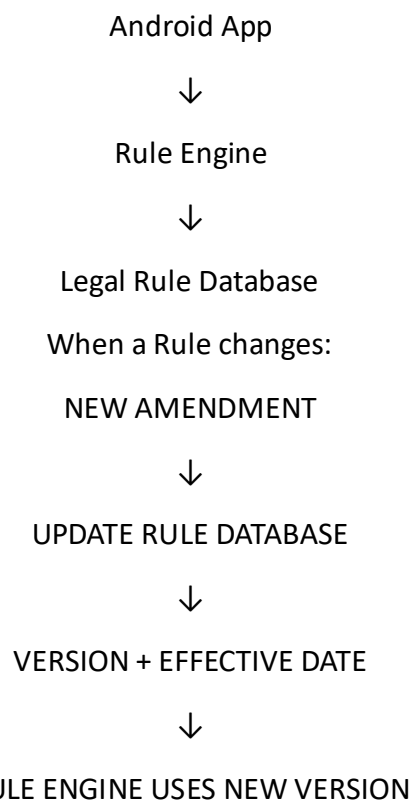
39.12 Rule Database vs Application Code

A major architectural decision is to keep legal rules **separate from the core application code**.

Poor Design



Better Design



This makes the system easier to maintain.

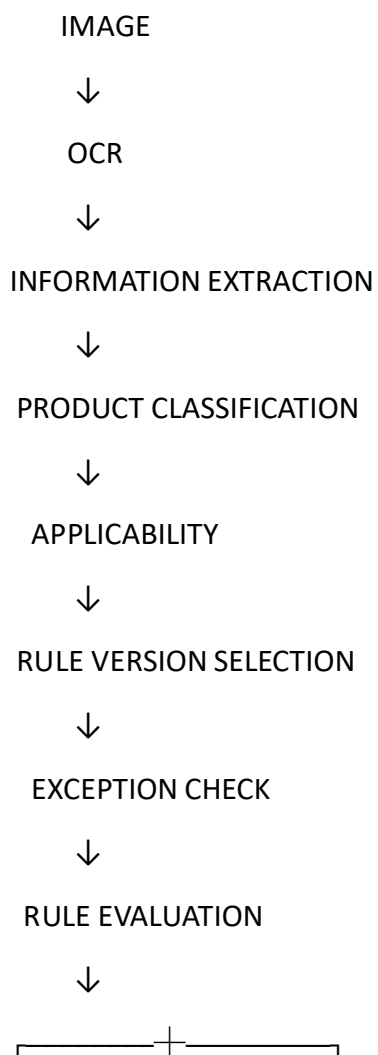
39.13 Proposed Rule Object

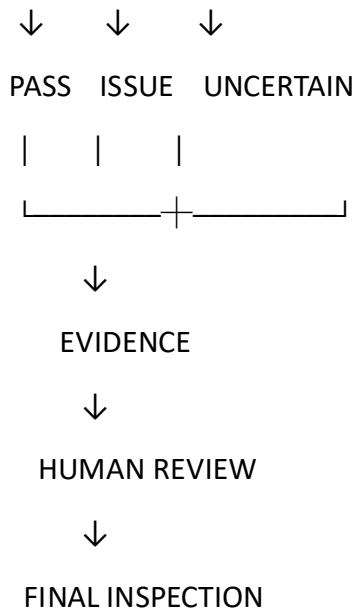
A simplified software representation could be:

```
{  
  rule_id: "PCR-R6-001",  
  rule_number: "6",  
  category: "mandatory_declaration",  
  requirement: "manufacturer_details",  
  applicability: "...",  
  exceptions: [],  
  effective_from: "...",  
  effective_until: null,  
  evidence_required: ["package_image", "ocr_text"],  
  human_review: true  
}
```

This is a **software design representation**, not the legal text itself.

39.14 Complete Rule Evaluation Pipeline





39.15 Key Design Principle

The most important principle for DrishtiTathya is:

Do not allow the AI model itself to become the law.

Instead:

AI



Extract Facts



Rule Engine



Apply Structured Legal Rules



Generate Explainable Observation



Human Officer



Final Decision

The AI should identify and structure facts; the **rule engine should apply predefined legal logic.**

39.16 Research-to-Implementation Mapping

Legal Research	Software Implementation
Rule	Rule object

Sub-rule	Individual rule condition
Applicability	Applicability engine
Exception	Exception condition
Amendment	Rule version
Commencement	Effective date
Declaration	Required field
MPE	Numerical validation
Schedule	Structured schedule table
Inspection	Inspection workflow
Evidence	Evidence record
Officer decision	Human review module

39.17 Section Conclusion

Converting legislation into software requires more than copying legal text into a database. Each provision must be represented as a structured combination of **requirement, applicability, conditions, exceptions, version and evidence**.

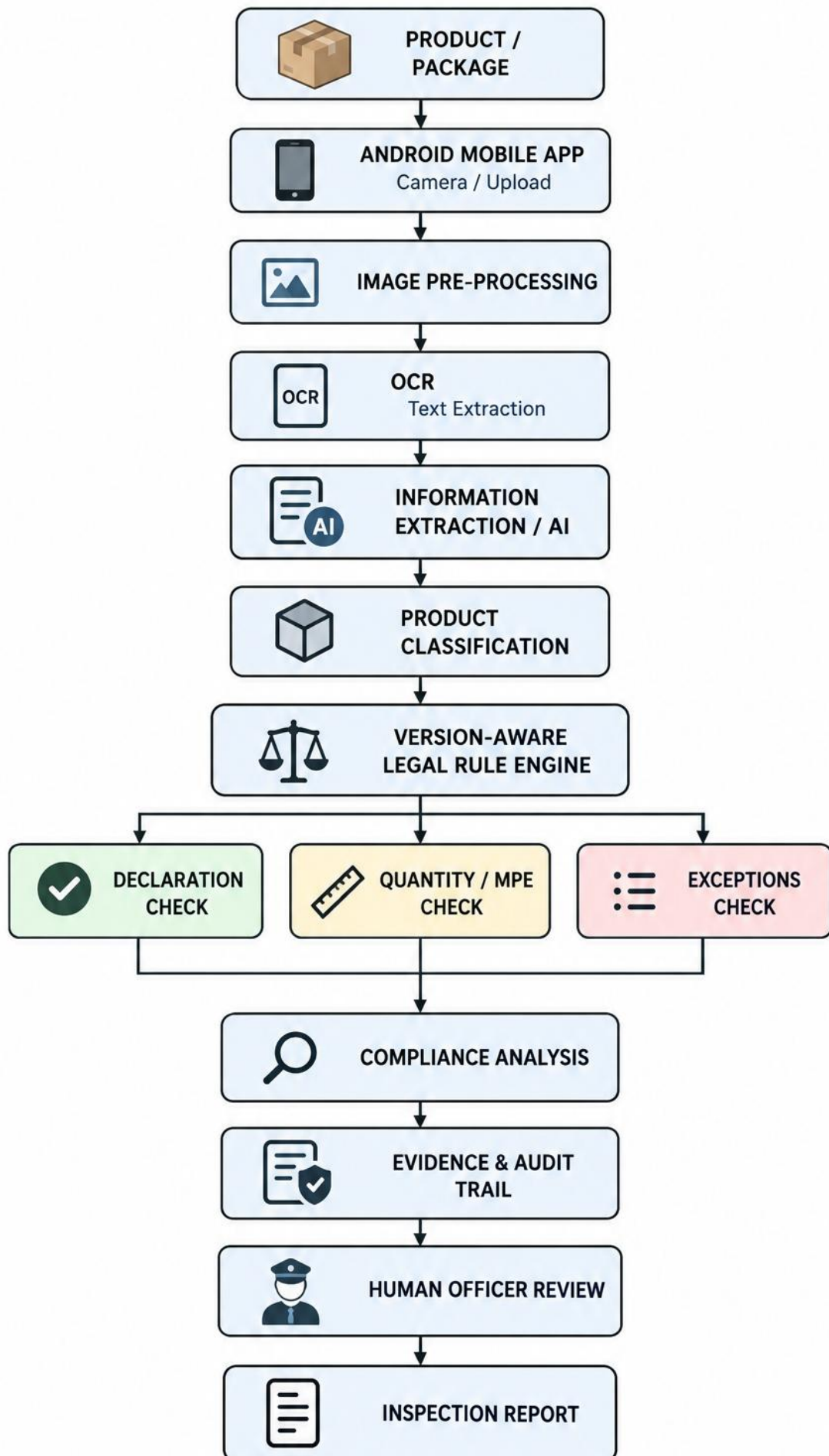
For DrishtiTathya, this creates the foundation of the **Version-Aware Legal Metrology Rule Engine**, which is one of the project's strongest technical differentiators.

40. DRISHTITATHYA — PROPOSED SYSTEM ARCHITECTURE

DrishtiTathya is proposed as an **AI-assisted Legal Metrology compliance platform** that connects product images, OCR, information extraction, legal rules, evidence and human officer verification.

The architecture is designed so that AI performs **detection and extraction**, while the legal rule engine performs **structured compliance evaluation** and the authorized officer performs the final verification.

40.1 High-Level Architecture



40.2 Layer 1 — Mobile Application

The Android application is the primary field interface.

Main functions

- Capture package photographs
- Upload existing photographs
- Scan multiple sides of a package
- Display OCR results
- Show detected declarations
- Record inspection information
- Record physical measurement results
- Display potential compliance issues
- Capture officer observations
- Submit inspection
- Generate/access reports

The mobile application should be designed for **fast field inspection**, where an officer can capture information without extensive manual typing.

40.3 Layer 2 — Image Pre-Processing

Raw package photographs may contain:

- Reflections
- Shadows
- Perspective distortion
- Low lighting
- Curved surfaces
- Small text
- Background objects

Therefore, the image can first undergo preprocessing.

RAW IMAGE



CROP



ROTATION / PERSPECTIVE CORRECTION



NOISE REDUCTION



CONTRAST / TEXT ENHANCEMENT



OCR-READY IMAGE

This layer improves the quality of subsequent OCR and AI analysis.

40.4 Layer 3 — OCR

The OCR engine converts visible package text into machine-readable text.

Example:

IMAGE



OCR



- Product Name
- MRP
- Net Quantity
- Manufacturer
- Address
- Consumer Information
- Dates
- Country of Origin
- Other Declarations

OCR should also retain **bounding-box coordinates** where possible.

For example:

"MRP ₹120"



Image Region

This allows the application to show the officer where the extracted information came from.

40.5 Layer 4 — AI Information Extraction

Raw OCR output is unstructured.

The AI extraction layer converts it into structured fields.

Example

OCR:

"MRP ₹120 incl. all taxes

Net Qty 500 g

Manufactured by ABC Foods Pvt Ltd"



Structured Data:MRP = ₹120

Net Quantity = 500 g

Manufacturer = ABC Foods Pvt Ltd

This structured information is then passed to the legal rule engine.

40.6 Layer 5 — Product Classification

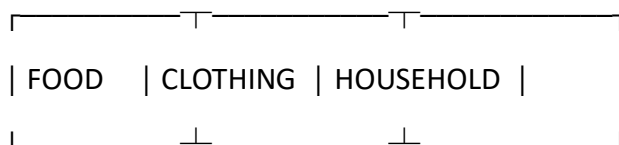
Different products may require different checks.

The system can classify the package into a relevant product/category type.

IMAGE + OCR



PRODUCT CLASSIFIER



Classification should be treated as an input to the compliance engine, not as an unquestionable fact.

If classification confidence is low:

LOW CONFIDENCE



ASK OFFICER TO CONFIRM CATEGORY

40.7 Layer 6 — Applicability Engine

Before applying individual rules, the system determines which requirements are applicable.

Inputs may include:

- Product category
- Package type
- Quantity

- Imported/domestic status
- Intended consumer category
- Inspection date
- Other relevant package characteristics

PACKAGE DATA



APPLICABILITY ENGINE



APPLICABLE RULE SET

This prevents irrelevant requirements from being applied.

40.8 Layer 7 — Version-Aware Legal Rule Engine

This is the **core intelligence layer** of DrishtiTathya.

The engine receives:

Product Facts

+

Applicability

+

Inspection Date

+

Legal Rule Database

+

Amendment History

+

Exceptions

and produces:

Applicable Requirements



Compliance Observations

The uploaded Rules demonstrate why this layer is required: different provisions govern declarations, inspection, testing, exemptions, registration and other activities.

40.9 Layer 8 — Compliance Modules

The rule engine can contain separate modules.

A. Declaration Checker

Checks whether expected declarations were detected.

Manufacturer → ✓

Net Quantity → ✓

MRP → ✓

Consumer Care → ?

B. Unit Checker

Checks the detected unit against the applicable requirements.

C. Quantity/MPE Checker

Uses declared and physically measured values where available.

D. Applicability Checker

Determines whether a requirement applies.

E. Exception Engine

Prevents incorrect application of general rules to exempt/special cases.

F. Amendment Engine

Selects the legally applicable rule version based on effective dates.

40.10 Layer 9 — Evidence Management

Every important AI finding should have supporting evidence.

The evidence record can contain:

Inspection ID



Original Image



OCR Text



Detected Field



Image Coordinates



Applicable Rule



AI Finding



Officer Decision

This creates an **audit trail**.

40.11 Layer 10 — Human Review

This layer is essential.

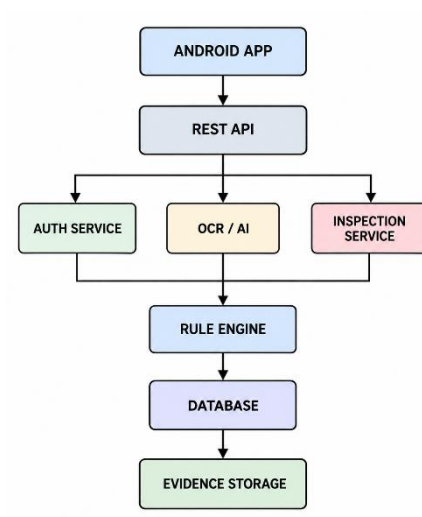
The officer sees:

POTENTIAL ISSUE	
Missing declaration detected	
Rule: Rule 6	
Evidence: Image #02	
AI Confidence: 91%	
[CONFIRM]	[REJECT]
Officer Comment: _____	

The officer can therefore verify or reject the AI observation.

40.12 Backend Architecture

A proposed backend can use a modular architecture:



Possible technologies include:

- **Android + Jetpack Compose** — mobile interface
- **CameraX** — camera integration
- **OCR engine** — text extraction
- **Python/FastAPI** — AI and processing APIs
- **Spring Boot** — enterprise backend services
- **PostgreSQL** — structured data
- **Object storage** — package images/evidence

The final technology choice can be adjusted based on the team's implementation skills and SIH prototype constraints.

40.13 Database Design

A simplified database could contain:

USERS

└— OFFICERS

PRODUCTS

└— PACKAGES

INSPECTIONS

└— SAMPLES

└— EVIDENCE

└— RESULTS

LEGAL_RULES

└— RULE_VERSIONS

└— EXCEPTIONS

└— SCHEDULES

40.14 Important Database Entities

Entity	Purpose
User	Officer/account information
Product	Product identity

Entity	Purpose
Package	Package-specific information
Inspection	Inspection event
Sample	Individual sample
Evidence	Images and supporting material
OCR Result	Extracted text
Declaration	Structured package declaration
Legal Rule	Rule definition
Rule Version	Amendment-aware version
Exception	Special applicability condition
Schedule	Schedule-based information
Compliance Result	AI/rule-engine output
Officer Review	Human decision
Report	Final inspection document

40.15 Security and Audit Trail

Since inspection information may become important official evidence, the system should maintain an audit trail.

Possible records include:

Who captured the evidence?

When was it captured?

Which image was analyzed?

What text was extracted?

Which rule was applied?

Which rule version was used?

What did the AI identify?

Who reviewed it?

What was the final decision?

This creates accountability and makes the system more suitable for government workflows.

40.16 Offline Inspection Capability

A valuable field feature is **offline-first operation**.

NO INTERNET



Capture Image



OCR / Local Processing



Save Inspection



NETWORK AVAILABLE

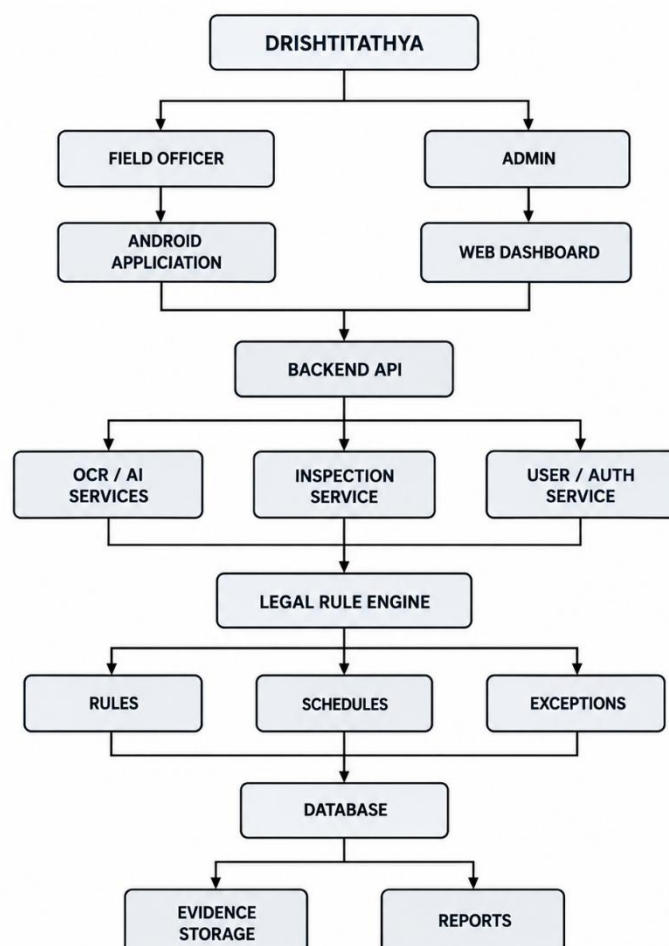


Synchronize With Server

This is useful when an officer is inspecting products in locations with unreliable connectivity.

The system should clearly distinguish between checks that can be completed offline and checks requiring current server-side legal data.

40.17 Complete DrishtiTathya Architecture



40.18 Architecture's Main Principle

The architecture follows a strict separation:

AI Layer

“What can I see?”

Rule Engine

“What requirement applies?”

Evidence Layer

“What supports this observation?”

Human Officer

“Is this actually a violation?”

This separation makes the system more explainable, maintainable and legally safer.

40.19 Section Conclusion

DrishtiTathya should therefore be designed as a **multi-layer compliance platform**, rather than a single AI model.

The complete system connects:

Camera → OCR → AI Extraction → Classification → Applicability → Version-Aware Rules → Exceptions → Compliance Analysis → Evidence → Human Review → Report

This architecture directly supports the project's objective of transforming packaged-commodity inspection into a faster and more structured digital workflow.

41. FUNCTIONAL REQUIREMENTS OF DRISHTITATHYA

The functional requirements describe **what the DrishtiTathya system should actually do**. Based on the proposed architecture, the platform can be divided into four major functional areas:

1. **Consumer Module**
2. **Officer Inspection Module**
3. **Legal Rule Engine**
4. **Administration & Analytics Module**

The system should be designed so that AI assists users in identifying and understanding package information, while the legal engine applies structured requirements and the officer remains responsible for final verification.

41.1 Consumer Module

The consumer module provides a simple way for consumers to scan a packaged commodity and understand its declarations.

Main Functions

- Scan product/package
- Capture multiple package sides
- Extract visible label information
- Display MRP
- Display net quantity
- Display manufacturer/importer information
- Display consumer-care information
- Identify potentially missing declarations
- Show important package information in simple language
- Submit a complaint or report
- Save previously scanned products

Consumer Flow

OPEN APP



SCAN PRODUCT



CAPTURE PACKAGE IMAGES



OCR



INFORMATION EXTRACTION



LEGAL CHECK



SHOW RESULT



OPTIONAL COMPLAINT / REPORT

41.2 Officer Inspection Module

The officer module is the primary enforcement-support component.

Main Functions

- Officer authentication
- Create inspection
- Capture package photographs
- Scan multiple sides
- Enter inspection location/details
- Identify product and lot
- Extract declarations using OCR
- Review AI-extracted information
- Record physical measurements
- Record sample information
- Check applicable requirements
- View potential violations
- Attach evidence
- Add officer comments
- Confirm/reject AI findings
- Generate inspection report
- View previous inspections

41.3 Inspection Creation

An officer can start a new inspection:

NEW INSPECTION



Inspection ID Generated



Date / Time



Location



Product Details



Package Images



Analysis

The system should automatically generate a unique inspection identifier.

Example:

INS-2026-000124

41.4 Multi-Side Package Scanning

A package should not necessarily be treated as a single photograph.

The application can allow:

FRONT



BACK



SIDE 1



SIDE 2



TOP / BOTTOM



COMBINE PACKAGE INFORMATION

This is important because declarations may appear on different parts of the package.

41.5 Declaration Extraction

The system converts OCR output into structured information.

Example

RAW OCR



"Net Qty 500 g"

"MRP ₹120"

"Manufactured by ABC Pvt Ltd"



STRUCTURED DATA

Field	Extracted Value
Net Quantity	500 g
MRP	₹120
Manufacturer	ABC Pvt Ltd
Address	Detected address
Consumer Care	Detected information
Country of Origin	Detected information

The officer should be able to edit incorrectly extracted information.

41.6 Potential Violation Detection

The application compares extracted information with applicable requirements.

Example:

REQUIRED DECLARATION



SEARCH OCR + IMAGE



FOUND?



YES NO



PASS POTENTIAL

ISSUE

The result should explain **what was detected and why it was flagged**.

41.7 Evidence Highlighting

When the system detects a possible issue, it should show the relevant image region.

Example:

PACKAGE IMAGE



HIGHLIGHTED REGION



POTENTIAL ISSUE



APPLICABLE RULE

This allows the officer to quickly verify the AI observation.

41.8 Human Review

Every important automated finding should be reviewable.

Possible actions:

[CONFIRM]

[REJECT]

[NEED MORE INFORMATION]

The officer can also enter:

Officer Observation: _____

Final Remarks: _____

This ensures that AI output does not automatically become a legal conclusion.

41.9 Legal Rule Engine Functions

The Legal Rule Engine should be capable of:

- Selecting applicable Rules
 - Checking applicability
 - Checking required declarations
 - Applying exceptions
 - Checking units
 - Checking quantity requirements
 - Applying MPE where relevant
 - Selecting the correct Rule version
 - Checking effective dates
 - Maintaining amendment history
 - Linking findings to legal sources
 - Returning an explainable result
-

41.10 Rule Engine Input and Output

Input

PRODUCT FACTS

+

PACKAGE DATA

+

INSPECTION DATE

+

RULE DATABASE

+

AMENDMENTS

+

EXCEPTIONS

Output

APPLICABLE REQUIREMENTS

+

DETECTED ISSUES

+

EVIDENCE

+

CONFIDENCE

+

REVIEW STATUS

41.11 Legal Rule Administration

Authorized administrators should be able to add or update rules without modifying the entire application.

Example:

NEW AMENDMENT



ADMIN ENTERS RULE



EFFECTIVE DATE



APPLICABILITY



EXCEPTIONS



SOURCE



RULE VERSION PUBLISHED

This supports the version-aware architecture described in Section 36.

41.12 Inspection History

Every completed inspection should be stored.

An officer can search by:

- Inspection ID
- Product
- Manufacturer
- Date
- Location
- Status
- Potential issue
- Officer

Example:

INSPECTION HISTORY

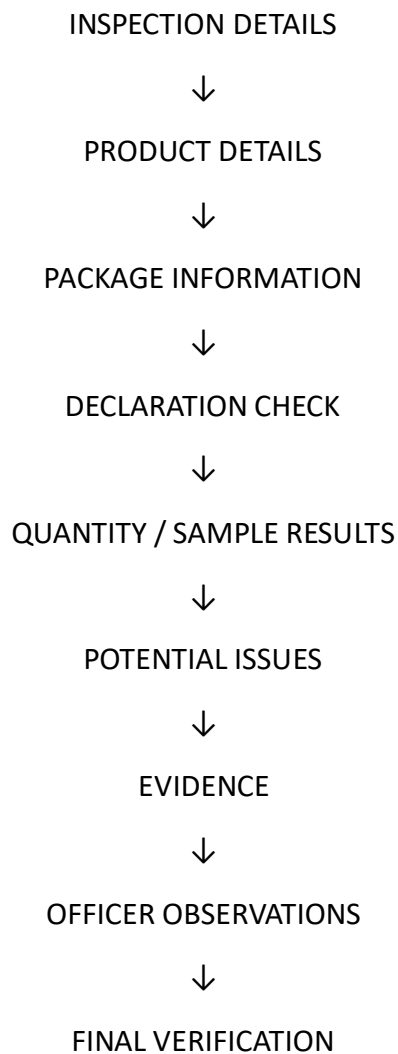
- INS-001 Product A ✓
- INS-002 Product B ⚠
- INS-003 Product C ✓
- INS-004 Product D ⚠

This can later support repeat-violation and risk analysis.

41.13 Report Generation

The system should automatically generate a structured inspection report.

Report structure



This can significantly reduce repetitive documentation work.

41.14 Admin Dashboard

The administration dashboard can provide an overview of inspections.

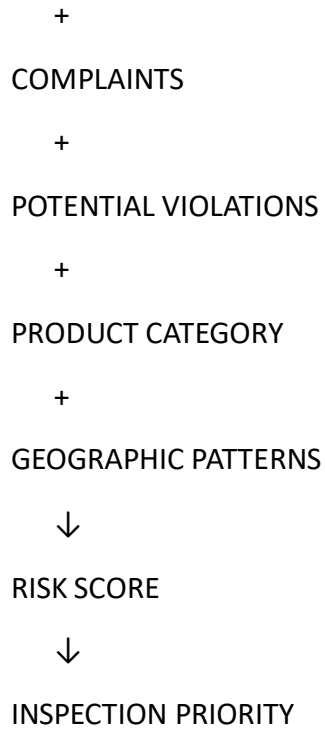
Possible indicators:

Indicator	Purpose
Total inspections	Overall workload
Completed inspections	Progress
Pending reviews	Officer workload
Potential issues	Compliance signals
Product categories	Distribution
Repeat manufacturers	Risk analysis
Geographic distribution	Inspection planning
Rule usage	Legal-rule analytics

41.15 Risk Prioritization

A future version can assign inspection priority using multiple signals.

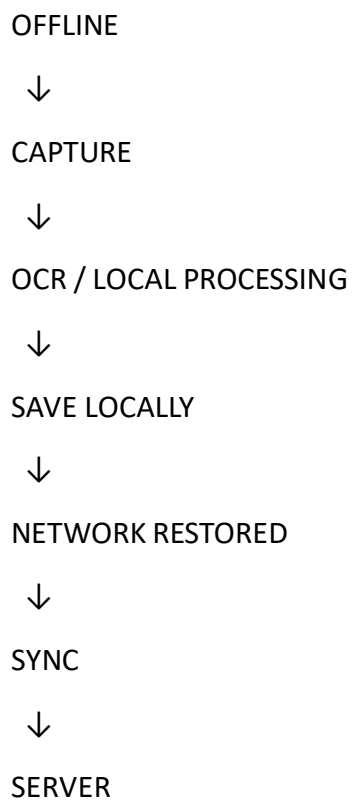
PAST INSPECTIONS



The score should be treated as a **decision-support indicator**, not proof of wrongdoing.

41.16 Offline Functionality

The officer application should support essential field operations when connectivity is poor.



This is particularly useful for field inspections where continuous internet access cannot be guaranteed.

41.17 Notifications and Alerts

The system can notify officers about:

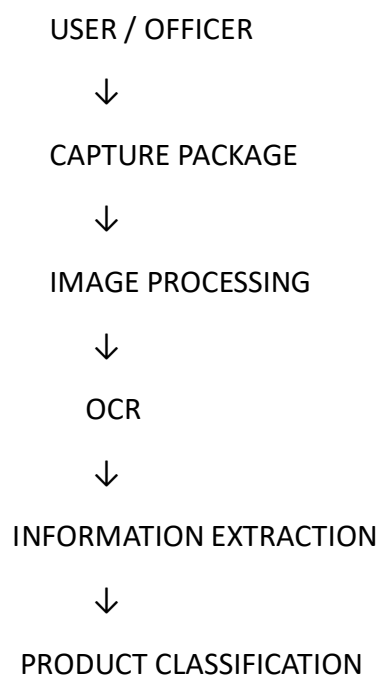
- Pending reviews
- Unsubmitted inspections
- Synchronization failures
- Updated legal rules
- Important inspection assignments
- Required follow-up actions

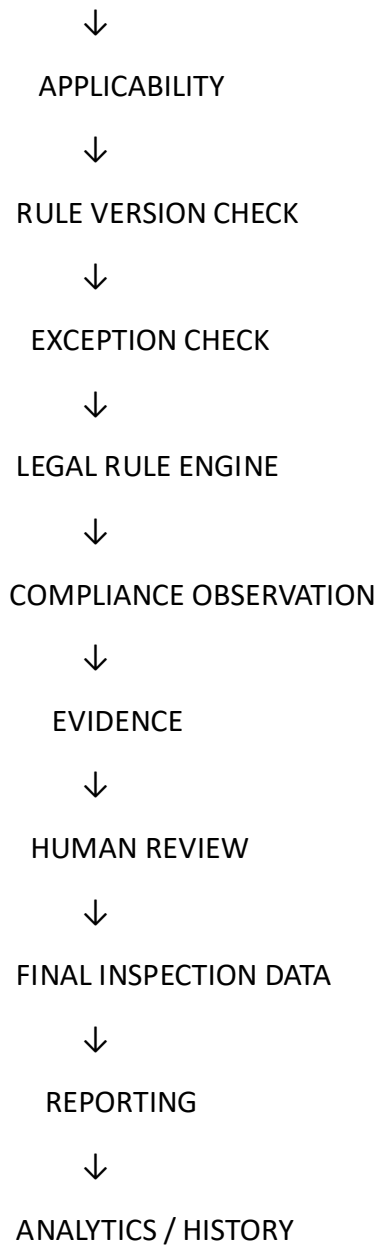
Alerts should be limited to meaningful events so that officers are not overwhelmed with notifications.

41.18 Functional Requirement Summary

Module	Key Functions
Consumer	Scan, understand, report
Officer	Inspect, verify, record
OCR	Extract package text
AI	Structure and classify information
Rule Engine	Apply legal requirements
Exception Engine	Handle special cases
Version Engine	Apply date-specific rules
Evidence	Store supporting material
Inspection	Samples and measurements
Reports	Generate inspection records
Dashboard	Monitor activities
Analytics	Identify patterns and risks
Admin	Manage rules/users/system

41.19 End-to-End Functional Flow





41.20 Minimum Viable Product for SIH

For the SIH prototype, it is not necessary to implement every advanced feature.

The **MVP** can focus on:

Must Have

1. Android package scanner
2. Image preprocessing
3. OCR
4. Declaration extraction
5. Product classification
6. Rule 6 declaration checking
7. Applicability checking

8. Basic Rule 9 readability checks
9. Version-aware rule database
10. Potential violation display
11. Evidence capture
12. Human officer review
13. Compliance report

Advanced Demonstration Features

- Multi-side scanning
- MRP extraction
- Net quantity extraction
- MPE calculator
- Offline mode
- E-commerce listing comparison
- Risk scoring
- Inspection history
- Admin dashboard

This gives the team a realistic implementation target while leaving room for innovation.

41.21 Section Conclusion

The functional requirements translate the research and architecture into actual system capabilities.

The key principle remains:

DrishtiTathya should automate repetitive detection and legal-rule matching while keeping evidence, transparency and human officer verification at the center of the inspection process.

42. NON-FUNCTIONAL REQUIREMENTS OF DRISHTITATHYA

Non-functional requirements define **how well the DrishtiTathya system should operate**, rather than what functions it performs.

Since the system is intended for field inspection and legal-compliance support, requirements such as **accuracy, reliability, security, explainability, usability and performance** are particularly important.

42.1 Accuracy

The system should provide reliable OCR, information extraction and rule evaluation.

Accuracy should be considered separately for:

- OCR accuracy
- Declaration extraction accuracy
- Product classification accuracy
- Rule applicability accuracy
- Rule-engine calculation accuracy

The system should not treat AI confidence as equivalent to legal certainty.

Proposed approach

IMAGE



AI DETECTION



CONFIDENCE SCORE



LOW CONFIDENCE?



YES NO



HUMAN SHOW RESULT

REVIEW

42.2 Explainability

Every potential compliance issue should be understandable to the officer.

The system should show:

- Detected information
- Expected information
- Applicable Rule
- Rule version
- Evidence image
- AI confidence
- Reason for flagging
- Officer review status

Example:  POTENTIAL ISSUE

- Detected:No consumer-care information found
 - Applicable Rule:Rule 6
 - Evidence:Package Image #02
 - AI Confidence:89%
 - Status:Officer Verification Required
-

42.3 Reliability

The system should continue functioning reliably during field inspections.

Important requirements include:

- Automatic saving
- Error recovery
- Local temporary storage
- Synchronization retry
- Database backup
- Prevention of duplicate inspection records

A failed network connection should not cause captured inspection data to disappear.

42.4 Performance

The application should provide quick responses during normal inspection activities.

Important performance areas include:

Operation	Requirement
Camera opening	Fast
Image capture	Immediate
OCR processing	Acceptable field delay
Declaration extraction	Fast enough for interactive use
Rule evaluation	Near real-time
Report generation	Short processing time
Data synchronization	Background operation

For the prototype, actual response times should be measured experimentally rather than claimed without testing.

42.5 Usability

The application should be simple enough for field officers who may not have advanced technical knowledge.

Interface principles

- Simple navigation
- Large buttons

- Clear status indicators
- Minimal typing
- Guided scanning
- Easy evidence review
- Simple language
- Clear error messages

A typical workflow should require only a few major actions:

SCAN



REVIEW



VERIFY



SUBMIT

42.6 Offline Capability

Field inspections may occur in locations with limited connectivity.

Therefore, essential functions should support offline operation where technically feasible.

NO INTERNET



CAPTURE DATA



STORE LOCALLY



CONTINUE INSPECTION



INTERNET AVAILABLE



SYNC WITH SERVER

The application should clearly indicate whether the current rule database is up to date when working offline.

42.7 Security

The system may contain sensitive inspection records and evidence.

Security requirements should include:

- Secure authentication
- Role-based access
- Encrypted communication
- Secure storage
- Session management
- Access logging
- Protection against unauthorized modification
- Secure backup

Role-based access example

ADMIN



Rules + Users + Analytics

OFFICER



Inspections + Evidence + Reports

CONSUMER



Scan + Information + Complaint

42.8 Evidence Integrity

Images and inspection information should maintain their association with the inspection record.

The system should preserve:

Inspection ID

+

Timestamp

+

Image

+

OCR Result

+

Rule Applied

+

Officer Review

Any subsequent modification to important inspection information should be recorded in the audit trail.

42.9 Auditability

The system should make it possible to determine:

Who did what, when, and based on which information?

An audit record may contain:

Event	Recorded Information
Image captured	Time + inspection
OCR performed	Processing event
Rule evaluated	Rule/version
Issue generated	Finding
Finding reviewed	Officer
Finding changed	Previous/new value
Report generated	Time + user

This supports accountability.

42.10 Scalability

The architecture should be capable of expanding from a prototype to a larger deployment.

The system should be designed to support:

10 INSPECTIONS

↓

100 INSPECTIONS

↓

10,000 INSPECTIONS

↓

LARGE-SCALE DEPLOYMENT

This requires modular backend services, efficient database indexing and appropriate evidence storage.

42.11 Maintainability

Legal requirements change over time.

Therefore, legal rules should not be tightly embedded into application code.

Instead:

APPLICATION



RULE ENGINE



VERSIONED RULE DATABASE

A new amendment should ideally be added as a new rule version rather than requiring major changes to the mobile application.

42.12 Legal Traceability

Every compliance observation should be traceable to its legal basis.

The system should maintain:

Finding



Requirement



Rule



Rule Version



Effective Date



Legal Source

This is especially important because the uploaded Rules contain multiple interconnected requirements and inspection procedures. Rule 19, for example, links inspection with sample selection, testing and reporting requirements.

42.13 Privacy

The system should follow data-minimization principles.

Only information necessary for the intended workflow should be collected.

For example:

- Package images
- Inspection information
- Relevant product details
- Officer information
- Evidence
- Review records

Consumer-related information should be handled carefully and should not be collected unnecessarily.

42.14 Multilingual Support

Since packaged commodities and field users may operate across different regions, the user interface can support multiple Indian languages.

The architecture should separate:

LEGAL RULE DATA

≠

USER INTERFACE LANGUAGE

Changing the interface language should not change the underlying legal rule.

42.15 Interoperability

The system should be capable of integrating with other government or enterprise systems in the future.

Possible integration points include:

- Product databases
- Manufacturer registration records
- Complaint systems
- E-commerce information
- Government dashboards
- Document/report systems

APIs should be used so that external systems can be connected without redesigning the complete application.

42.16 Extensibility

The system should be designed so additional compliance checks can be added later.

For example:

CURRENT

Declaration Checking



FUTURE

Quantity Checking



E-commerce Checking



Risk Analysis



Advanced Package Analysis

This allows the SIH prototype to start with a focused feature set while maintaining a path toward a larger platform.

42.17 Fault Tolerance

Possible failures include:

- Camera failure
- Poor image quality
- OCR failure
- Server unavailable
- Internet disconnection
- Database error
- AI service failure

The application should respond gracefully.

Example:

OCR FAILED



SHOW ERROR



RETAKE IMAGE

OR

ENTER DATA MANUALLY

The user should never lose the entire inspection simply because one AI component failed.

42.18 Human-in-the-Loop Requirement

This is a particularly important non-functional requirement.

The system should ensure that uncertain AI results can be reviewed by a human.

AI RESULT



CONFIDENCE



UNCERTAIN?



YES NO



REVIEW CONTINUE



OFFICER DECISION

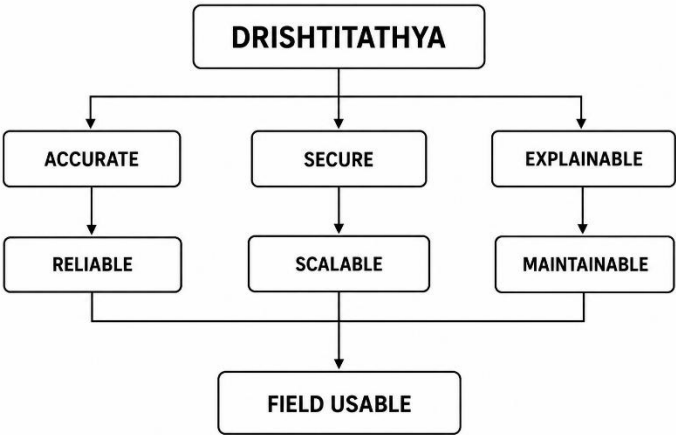
The final enforcement decision should remain with the authorized authority.

42.19 Non-Functional Requirement Summary

Requirement	DrishtiTathya Goal
Accuracy	Reliable extraction and rule evaluation
Explainability	Explain every potential issue
Reliability	Prevent data loss
Performance	Fast field operation
Usability	Simple officer workflow
Offline capability	Continue essential work without internet
Security	Protect users and inspection data
Evidence integrity	Preserve evidence association
Auditability	Record important actions
Scalability	Support future expansion
Maintainability	Easy rule/application updates
Legal traceability	Link findings to rules
Privacy	Minimize unnecessary data
Multilingual support	Improve accessibility
Interoperability	Support future integrations
Extensibility	Add future compliance modules
Fault tolerance	Recover from component failures
Human review	Keep officer in the decision loop

42.20 Quality Attributes of the Proposed System

The overall system can therefore be represented as:



42.21 Section Conclusion

The non-functional requirements ensure that DrishtiTathya is not merely a technically working prototype, but a system designed for **reliable, explainable and accountable compliance assistance**.

The most important qualities for the project are:

Accuracy + Explainability + Security + Reliability + Legal Traceability + Human Review

These qualities should be demonstrated during the SIH prototype evaluation wherever possible through measurable testing rather than only through claims.

43. TECHNOLOGY STACK AND IMPLEMENTATION PLAN

The technology stack of DrishtiTathya should support four major requirements:

- 1. **Fast package image scanning**
- 2. **Accurate information extraction**
- 3. **Rule-based legal compliance checking**
- 4. **Secure storage and officer review**

The system should use AI for visual and text processing, while the legal logic should remain in a structured and version-controlled rule engine.

43.1 Proposed Technology Stack

Layer	Proposed Technology	Purpose
Mobile App	Android + Kotlin	Field inspection application
UI	Jetpack Compose	Modern Android interface
Camera	CameraX	Package image capture
Image Processing	OpenCV	Image enhancement
OCR	Tesseract OCR / suitable OCR engine	Text extraction

AI/ML	Python-based models	Information extraction & classification
AI API	FastAPI	AI processing services
Backend	Spring Boot / FastAPI	Application APIs
Database	PostgreSQL	Structured application data
Rule Engine	Custom rule engine	Legal compliance evaluation
Evidence Storage	Object/File Storage	Images and inspection evidence
Dashboard	Web application	Administration & analytics
Authentication	Secure token-based authentication	User access control

43.2 Android Application

The Android application will act as the primary field interface.

Responsibilities

- Camera access
- Image capture
- Image upload
- Inspection creation
- OCR result display
- AI result display
- Officer review
- Manual data entry
- Evidence capture
- Offline storage
- Report access

Proposed Flow

ANDROID APP



CAMERA



CAPTURE PACKAGE



PREPROCESS IMAGE



SEND TO OCR / AI



DISPLAY RESULTS



OFFICER REVIEW

43.3 Jetpack Compose

Jetpack Compose can be used to build the Android interface.

It is suitable for creating screens such as:

LOGIN



DASHBOARD



NEW INSPECTION



SCAN PACKAGE



OCR RESULTS



COMPLIANCE RESULTS



EVIDENCE



REPORT

The interface should prioritize large controls and minimal typing because the application is intended for field use.

43.4 CameraX

CameraX can be used for package image capture.

The camera module should support:

- Focus control
- Image capture
- Preview
- Multiple photographs
- Orientation handling
- Image quality checks

A useful feature is **guided scanning**.

FRONT ✓

BACK ✓

SIDE 1 ✓

SIDE 2 ?



"Capture remaining side"

This helps reduce incomplete package scans.

43.5 OpenCV

OpenCV can be used for image preprocessing.

Potential operations include:

- Perspective correction
- Cropping
- Rotation
- Noise reduction
- Thresholding
- Contrast enhancement
- Text-region detection

RAW IMAGE



PERSPECTIVE CORRECTION



NOISE REDUCTION



TEXT ENHANCEMENT



OCR

The goal is to provide the OCR engine with a cleaner image.

43.6 OCR Layer

The OCR system extracts text from the package.

Example:

PACKAGE IMAGE



OCR



"NET QTY 500 g"

"MRP ₹120"

"Manufactured by ABC Foods"

The OCR output should ideally contain both:

- Extracted text
- Location of the text in the image

The second component is useful for evidence highlighting.

43.7 AI Information Extraction

OCR output is not yet structured legal information.

The AI layer converts text into fields.

Example

OCR TEXT



AI INFORMATION EXTRACTION



MRP	₹120	
Net Quantity	500 g	
Manufacturer	ABC Foods	
Country	India	

The system should allow officers to correct extracted information.

43.8 Product Classification Model

A classification model can help determine the likely commodity category.

Possible output:

Food Product

Confidence: 94%

or:

Household Product

Confidence: 81%

If confidence is below a predefined threshold, the application should request officer confirmation.

43.9 Backend API

The backend connects the mobile application with the AI services, database and legal rule engine.

A simplified API architecture is:

ANDROID

|



BACKEND API

|

├— Authentication

├— Inspection

├— Product

├— OCR/AI

├— Rule Engine

├— Evidence

└— Reports

The API should use authentication and authorization for protected operations.

43.10 Rule Engine

The Rule Engine is the most important non-AI software component.

It receives structured facts:

- Product
- Package
- Quantity
- Declarations
- Inspection Date
- Import Status
- Consumer Category

and evaluates them against:

LEGAL RULES

+

AMENDMENTS

+

EFFECTIVE DATES

+

EXCEPTIONS

+

SCHEDULES

The output is:

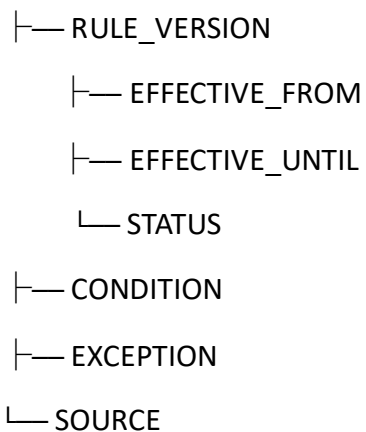
- PASS
- POTENTIAL ISSUE
- UNCERTAIN

43.11 Rule Database

Legal requirements should be stored separately from the application logic.

A simplified database design:

LEGAL_RULE



This makes future amendments easier to incorporate.

43.12 PostgreSQL Database

PostgreSQL can store structured information such as:

- Users
- Officers
- Products

- Inspections
- Samples
- Declarations
- Rule versions
- Compliance results
- Officer decisions
- Audit records

Large image files should preferably be stored separately in evidence/object storage, with their references maintained in the database.

43.13 Evidence Storage

Each inspection may contain multiple photographs.

The system can maintain:

INSPECTION ID



IMAGE 01

IMAGE 02

IMAGE 03



OCR RESULTS



DETECTED FIELDS



RULE FINDINGS



OFFICER REVIEW

The evidence record should maintain the relationship between the image and the finding derived from it.

43.14 Officer Dashboard

The web dashboard can provide:

- Inspection overview
- Pending reviews

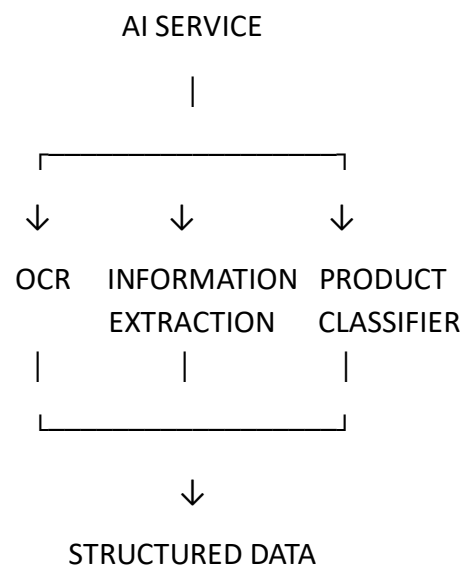
- Potential issues
- Product categories
- Inspection history
- Evidence review
- Reports
- Analytics
- Rule update management for authorized administrators

Example:

OFFICER DASHBOARD		
Today's Inspections	18	
Pending Reviews	5	
Potential Issues	7	
Completed	13	
[NEW INSPECTION] [HISTOR		

43.15 AI Service Architecture

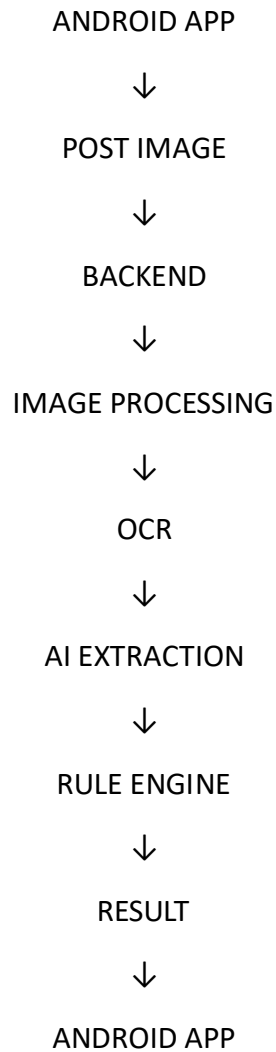
The AI components can be separated into independent services.



This makes individual AI components replaceable without redesigning the entire system.

43.16 API Communication

A typical inspection request can follow:



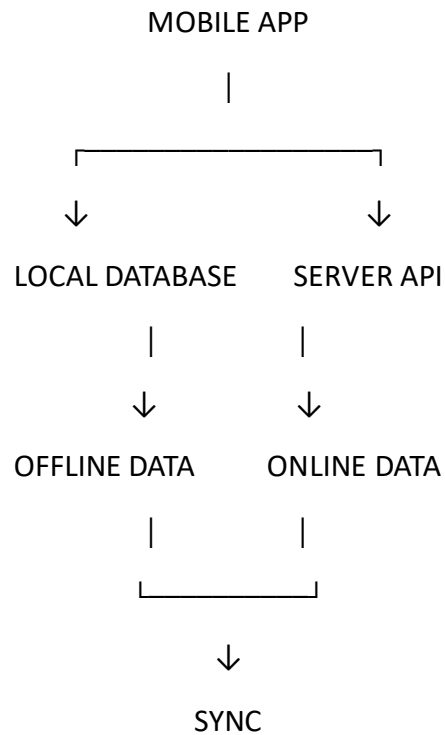
The response can contain:

```
{  
  detected_fields,  
  applicable_rules,  
  potential_issues,  
  confidence_scores,  
  evidence_references,  
  review_required  
}
```

This is a conceptual API structure for implementation.

43.17 Offline Architecture

For offline operation, the Android application can maintain a local database/cache.



The synchronization layer should handle:

- New inspections
- Evidence uploads
- Updated records
- Failed requests
- Duplicate prevention

43.18 Security Architecture

USER



AUTHENTICATION



AUTHORIZATION



API



DATABASE



AUDIT LOG

Different users should receive only the permissions appropriate to their role.

43.19 Implementation Phases

The project can be implemented incrementally.

Phase 1 — Basic Prototype

Android

+

Camera

+

OCR

Phase 2 — Intelligent Extraction

OCR

↓

Declaration Extraction

↓

Product Classification

Phase 3 — Legal Intelligence

Structured Facts

↓

Applicability

↓

Rule Engine

↓

Potential Issues

Phase 4 — Evidence & Review

Evidence

↓

Officer Review

↓

Inspection Report

Phase 5 — Advanced Features

Inspection History

+

Risk Analysis

+

E-commerce Checks

+

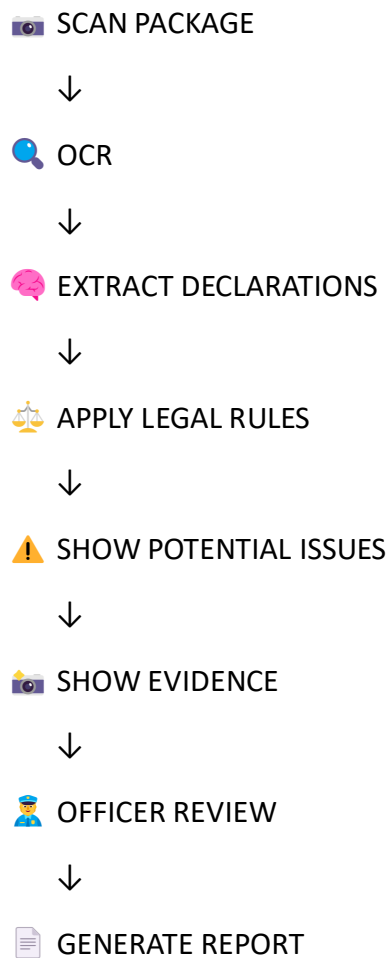
Offline Sync

+

Analytics

43.20 SIH Prototype Priority

For the SIH demonstration, the most important working pipeline should be:



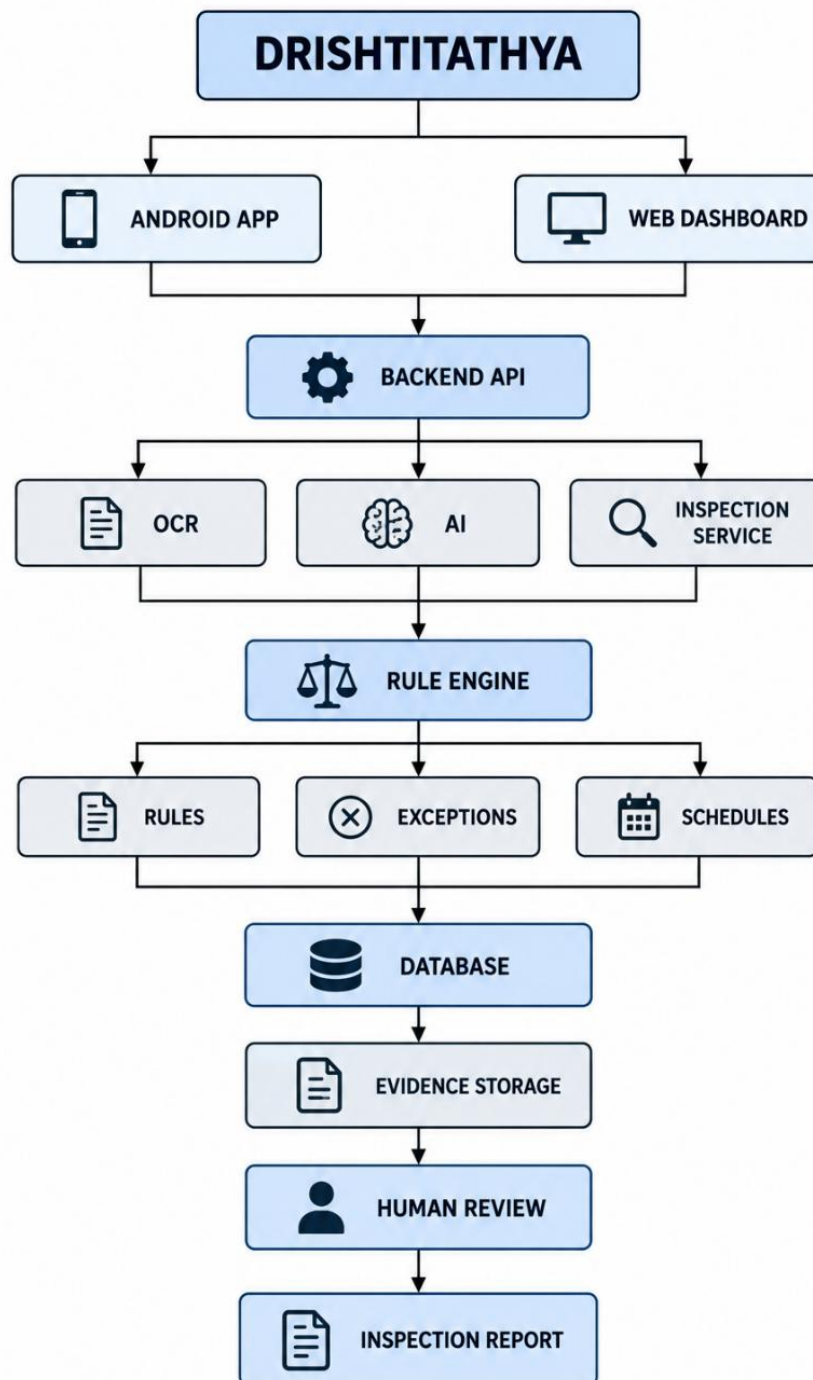
If this pipeline works reliably, additional features can be demonstrated as future extensions.

43.21 Technology-to-Function Mapping

Technology	DrishtiTathya Function
Kotlin	Android application
Jetpack Compose	User interface
CameraX	Package scanning
OpenCV	Image preprocessing

OCR	Text extraction
Python	AI/ML processing
FastAPI	AI service APIs
Spring Boot	Backend services
PostgreSQL	Structured data
Object storage	Evidence images
Rule Engine	Legal compliance
Web Dashboard	Officer/Admin interface

43.22 Final Architecture



43.23 Section Conclusion

The proposed technology stack separates the system into independent layers:

Mobile → Image Processing → OCR → AI → Rule Engine → Database → Evidence → Human Review

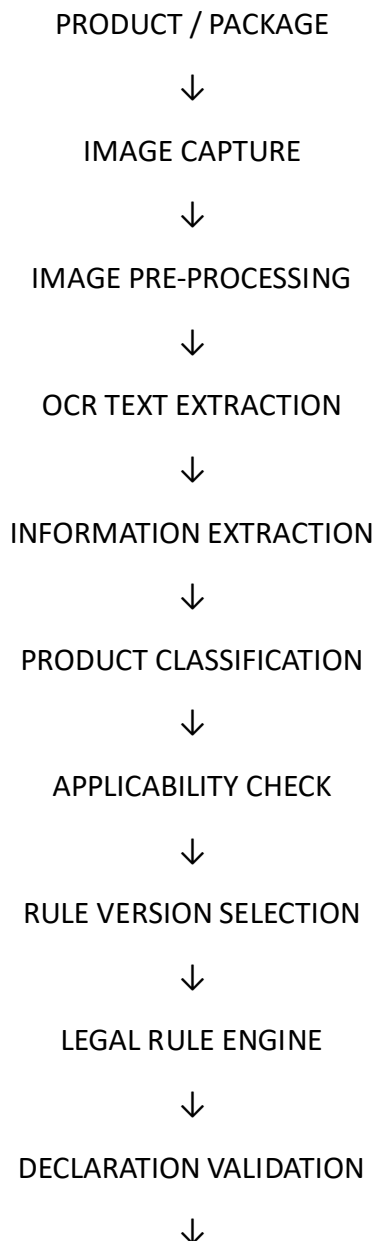
This separation is important because AI models can evolve independently from the legal rule database, while amendments to the Rules can be incorporated without redesigning the entire application.

The resulting system is therefore designed not merely as an **AI label scanner**, but as an **AI-assisted Legal Metrology inspection and compliance platform**.

44. SYSTEM WORKFLOW / DATA FLOW DIAGRAM

The DrishtiTathya system follows a structured workflow in which the package image is converted into machine-readable information, checked against applicable Legal Metrology requirements, and finally presented to a human officer for verification.

The complete workflow is:





44.1 Step 1 — Product Image Capture

The officer captures photographs of the packaged commodity using the Android application.

The system should support multiple images because mandatory declarations may appear on different parts of a package.

FRONT



BACK



SIDE



TOP / BOTTOM

The application can also indicate whether important package areas have been captured.

44.2 Step 2 — Image Pre-Processing

The captured image is processed before OCR.

Possible operations include:

- Cropping
- Rotation correction
- Perspective correction
- Noise reduction
- Contrast enhancement

- Image sharpening
- Text-region detection

RAW IMAGE



QUALITY CHECK



CROP



ROTATE / PERSPECTIVE CORRECTION



ENHANCE



OCR-READY IMAGE

This stage improves the quality of subsequent text extraction.

44.3 Step 3 — OCR Text Extraction

The OCR engine extracts textual information from the package.

For example:

INPUT IMAGE



OCR



"ABC FOODS PVT LTD"

"NET QTY 500 g"

"MRP ₹120.00"

"CONSUMER CARE: 1800..."

The system should retain the relationship between extracted text and its location in the image wherever possible.

44.4 Step 4 — Information Extraction

Raw OCR text is converted into structured declarations.

Example

Extracted Text	Structured Field
----------------	------------------

ABC Foods Pvt. Ltd.	Manufacturer
500 g	Net Quantity
₹120.00	MRP
India	Country of Origin
1800-XXXX-XXXX	Consumer Care

The AI should not simply return text. It should identify **what each piece of text represents**.

44.5 Step 5 — Product Classification

The system determines the likely category and characteristics of the package.

Possible information includes:

- Product category
- Imported/domestic status
- Quantity type
- Package type
- Special category
- Industrial/institutional applicability
- Relevant declaration requirements

Example:

Product: Packaged Food

Quantity: 500 g

Origin: India

Package Type: Retail Package

The classification is used as an input to the applicability and rule-selection stages.

44.6 Step 6 — Applicability Check

Before checking every rule, the system determines whether the relevant provisions apply to the package.

This is important because the Rules contain specific applicability conditions and exemptions.

Conceptually:

PACKAGE

↓

IS IT A RETAIL PACKAGE?

↓

YES —————→ CONTINUE

|

NO

↓

CHECK APPLICABILITY / EXEMPTION

The system should record **why** a rule was considered applicable or not applicable.

44.7 Step 7 — Rule Version Selection

Legal requirements may change through amendments.

Therefore, the system should identify the applicable rule version using information such as:

- Inspection date
- Effective date of amendment
- Product category
- Package characteristics
- Applicable provisions

INSPECTION DATE

+

PRODUCT FACTS

↓

RULE VERSION

↓

APPLICABLE REQUIREMENTS

This prevents the system from treating the Rules as a static document.

44.8 Step 8 — Legal Rule Engine

The structured product information is passed to the Legal Metrology Rule Engine.

Example:

Net Quantity = 500 g

MRP = ₹120

Manufacturer = Present

Consumer Care = Present

Country of Origin = India

The rule engine evaluates these facts against the applicable legal requirements.

44.9 Step 9 — Declaration Validation

The system checks whether the required declarations are:

- Present
- Detectable
- Legible
- In the expected format
- Associated with the correct package information

Possible checks include:

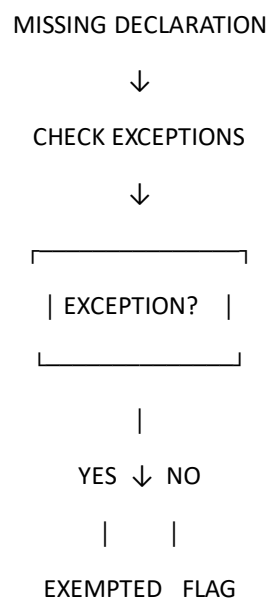
- ✓ Manufacturer / Packer / Importer
- ✓ Common / Generic Name
- ✓ Net Quantity
- ✓ MRP
- ✓ Consumer Care
- ✓ Country of Origin where applicable
- ✓ Date-related declaration where applicable
- ✓ Unit Sale Price where applicable

The exact checks should depend on the applicable product category and legal provisions.

44.10 Step 10 — Exception Check

The system should not immediately mark a missing declaration as a violation.

It should first evaluate relevant exceptions.



This is one of the important differences between a simple OCR application and a legal compliance system.

44.11 Step 11 — Compliance Analysis

After evaluating the rules and exceptions, the system generates a preliminary result.

A three-level result can be used:

COMPLIANT

No potential issue detected for the evaluated requirements.

POTENTIAL NON-COMPLIANCE

A requirement appears to be missing, incorrect or inconsistent.

REVIEW REQUIRED

The system does not have sufficient confidence to make a reliable determination.

Example:

COMPLIANCE ANALYSIS		
Net Quantity	✓	
MRP	✓	
Manufacturer	✓	
Consumer Care	✓	
Country of Origin	✓	
Result: COMPLIANT		

44.12 Step 12 — Evidence Mapping

Every potential issue should be connected to supporting evidence.

For example:

POTENTIAL ISSUE

"Net quantity declaration not detected"



IMAGE



LOCATION OF RELEVANT AREA



OCR RESULT



RULE REFERENCE

This allows an officer to understand **why the system generated the flag**.

44.13 Step 13 — AI Confidence Check

AI outputs should include confidence information.

Example:

Net Quantity: 500 g

Confidence: 98%

MRP: ₹120

Confidence: 96%

Manufacturer:

Confidence: 91%

If confidence is low:

LOW CONFIDENCE



REVIEW REQUIRED



OFFICER VERIFICATION

The AI therefore acts as an assistant rather than the final legal authority.

44.14 Step 14 — Human / Officer Review

The officer reviews:

- Package images
- OCR text
- Extracted declarations
- Applicable rules
- Potential issues
- Evidence

- AI confidence

The officer can:

CONFIRM

OR

REJECT FLAG

OR

EDIT INFORMATION

OR

REQUEST ADDITIONAL EVIDENCE

This human-in-the-loop mechanism is important for practical field deployment.

44.15 Step 15 — Final Inspection Report

After officer review, the system generates an inspection report.

The report can contain:

Inspection Details

- Inspection ID
- Date and time
- Officer
- Location
- Product information

Package Information

- Manufacturer/packer/importer
- Product name
- Net quantity
- MRP
- Country of origin
- Other extracted declarations

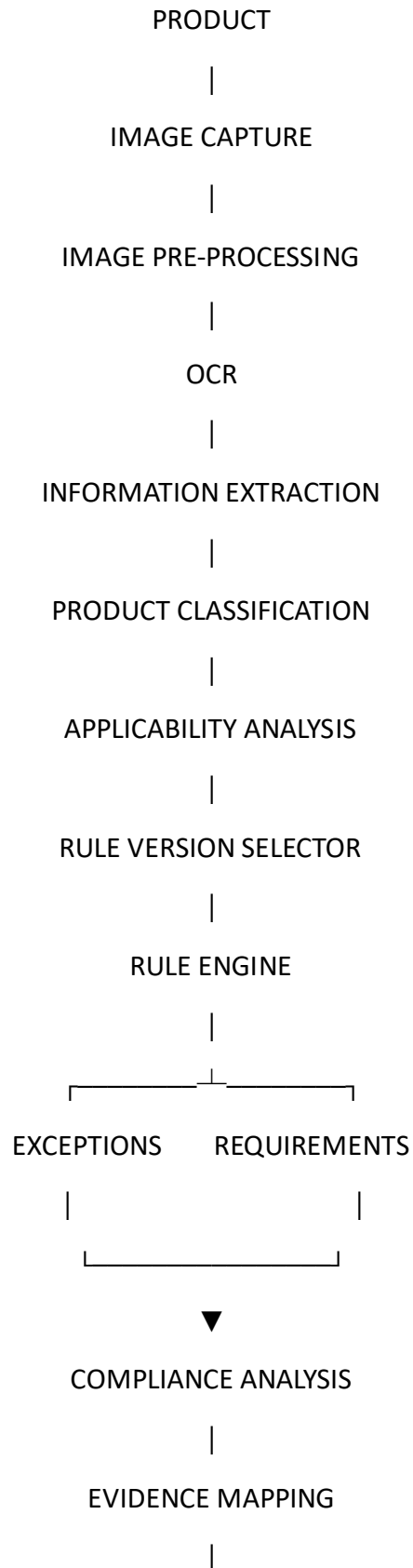
Compliance Findings

- Requirement checked
- Result
- Potential issue
- Applicable rule

- Evidence reference
 - Officer decision
-

44.16 Complete Data Flow

The complete movement of information can be represented as:



CONFIDENCE ANALYSIS

|

OFFICER REVIEW

|

FINAL DECISION

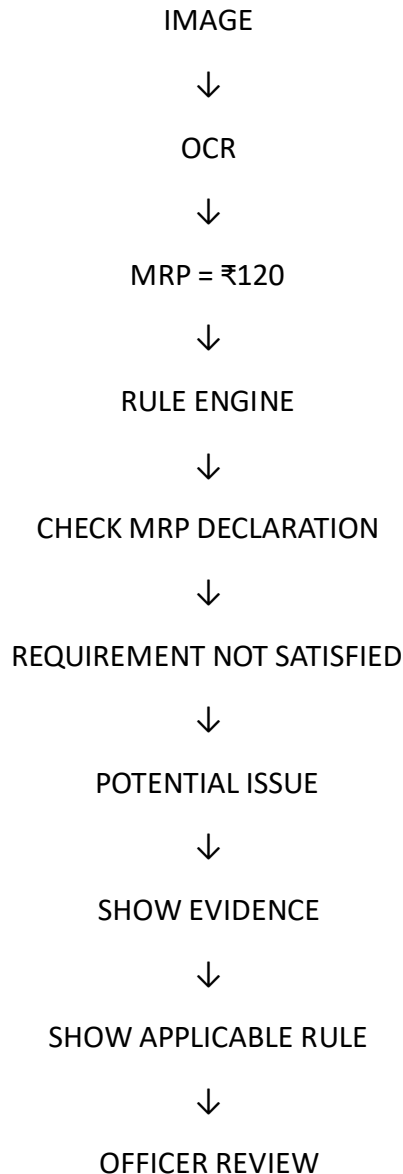
|

INSPECTION REPORT

44.17 Data Flow for a Potential Violation

A major feature of DrishtiTathya is that it should explain a potential violation rather than simply display **“FAIL.”**

Example:



The officer can therefore trace the result back to the package evidence.

44.18 Example End-to-End Inspection

Suppose an officer scans a packaged product.

Input

Product Image

OCR

ABC FOODS

NET QTY 500 g

MRP ₹120

Information Extraction

Manufacturer → ABC Foods

Net Quantity → 500 g

MRP → ₹120

Rule Engine

Required declarations



Compare with extracted information



Missing / unclear declaration detected

Output ⚠ POTENTIAL ISSUE

- Requirement:[Relevant declaration]
- Status:Not detected / unclear
- Evidence:[Package image]
- Confidence:89%
- Action:Officer Review Required

This provides a much more useful result than simply saying “**Non-Compliant.**”

44.19 Core Design Principle

The entire workflow should follow this principle:

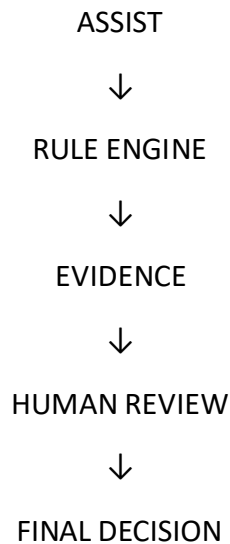
AI finds and interprets information; the rule engine evaluates requirements; evidence supports the finding; the human officer makes the final decision.

Therefore:

AI ≠ FINAL LEGAL AUTHORITY

AI





This architecture makes DrishtiTathya more suitable for an actual government inspection workflow.

44.20 Section Conclusion

DrishtiTathya transforms a physical package into a structured, evidence-backed compliance assessment through a sequence of:

Capture → Extract → Classify → Determine Applicability → Select Rule Version → Validate → Explain → Review → Report

The key innovation is that the system does not stop at **OCR**. It connects the extracted information with **applicable legal requirements, amendments, exceptions and evidence**, while keeping the final decision under human review.

45. DATABASE DESIGN AND ER DIAGRAM

The database of DrishtiTathya is responsible for storing inspection information, product details, extracted declarations, legal rules, compliance findings, evidence and officer decisions.

The database should be designed so that every compliance result can be traced back to:

Product → Image → Extracted Information → Rule → Finding → Evidence → Officer Decision

45.1 Main Database Entities

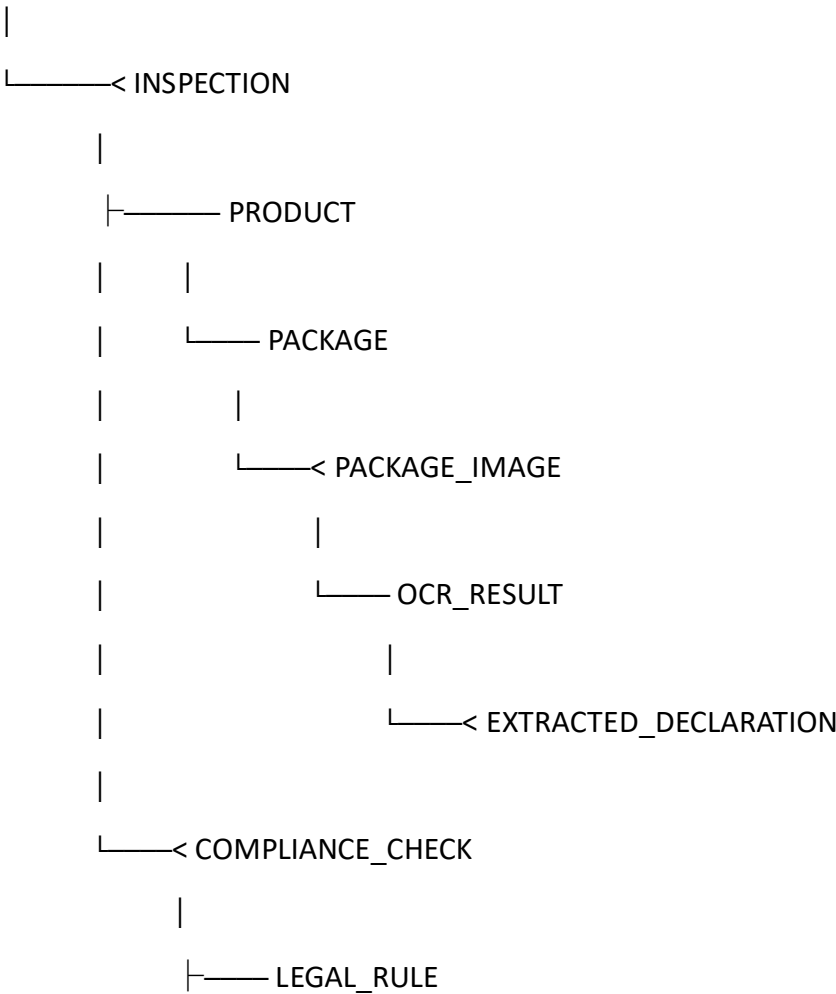
The proposed database contains the following major entities:

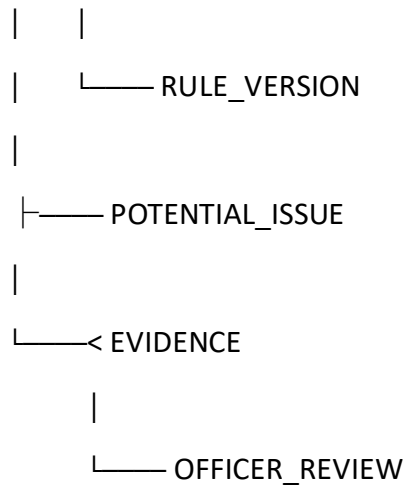
1. **User**
2. **Officer**
3. **Inspection**
4. **Product**
5. **Package**
6. **Package Image**

- 7. OCR Result
- 8. Extracted Declaration
- 9. Product Classification
- 10. Legal Rule
- 11. Rule Version
- 12. Applicability Condition
- 13. Exception
- 14. Compliance Check
- 15. Violation / Potential Issue
- 16. Evidence
- 17. Officer Review
- 18. Inspection Report
- 19. Audit Log

45.2 Entity Relationship Overview

USER





45.3 USER Table

The USER entity stores information about people using the system.

Field	Description
user_id	Unique user identifier
name	User name
email	Login email
password_hash	Encrypted/hashed password
role	User role
status	Active/inactive
created_at	Account creation date

Possible roles:

- ADMIN
- OFFICER
- MANUFACTURER
- REVIEWER

Role-based access ensures that users can access only the functions permitted to them.

45.4 INSPECTION Table

The inspection entity represents one inspection activity.

Field	Description
inspection_id	Unique inspection ID
officer_id	Officer conducting inspection
inspection_date	Date of inspection
location	Inspection location
status	Current inspection status
final_result	Final inspection result
created_at	Record creation time

Example:

- INS-2026-00125
- Officer: O102
- Date: 13-09-2026

- Status: Under Review

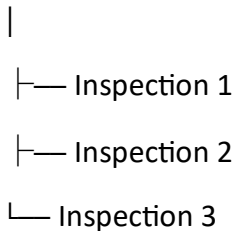
45.5 PRODUCT Table

The product table stores information about the commodity being inspected.

Field	Description
product_id	Unique product ID
product_name	Product name
brand_name	Brand
category	Product category
manufacturer	Manufacturer
importer	Importer where applicable
country_of_origin	Country of origin
barcode	Barcode/GTIN if available

A product can appear in multiple inspections.

PRODUCT



This enables repeat-violation and inspection-history analysis.

45.6 PACKAGE Table

A package represents the particular packaged commodity being evaluated.

Field	Description
package_id	Unique package ID
product_id	Associated product
package_type	Type of package
net_quantity	Declared quantity
quantity_unit	g/kg/ml/L/etc.
mrp	Retail sale price
manufacturing_date	Date information
best_before	Best-before information
batch_number	Batch/lot information

45.7 PACKAGE_IMAGE Table

This table stores references to images captured during inspection.

Field	Description
image_id	Unique image ID
package_id	Associated package
image_path	Evidence storage location

image_type	Front/back/side/etc.
capture_time	Time captured
image_hash	Integrity verification value

Example:

PACKAGE

|

|— Front Image

|— Back Image

|— Left Side

└— Right Side

Multiple images are important because declarations may occur on different package panels.

45.8 OCR_RESULT Table

The OCR table stores the text extracted from an image.

Field	Description
ocr_id	OCR record ID
image_id	Source image
extracted_text	OCR text
language	Detected language
confidence	OCR confidence
processing_time	Processing time

Example:

IMAGE

↓

OCR

↓

"NET QTY 500 g"

"MRP ₹120"

45.9 EXTRACTED_DECLARATION Table

OCR text is converted into structured declarations.

Field	Description
declaration_id	Unique ID
ocr_id	Source OCR result
declaration_type	MRP / quantity / manufacturer etc.
declaration_value	Extracted value
confidence	AI confidence
location_reference	Position in image

verification_status	Verified/unverified
---------------------	---------------------

Example:

- declaration_type = NET_QUANTITY
- declaration_value = 500 g
- confidence = 0.98

45.10 LEGAL_RULE Table

This is one of the most important tables.

It stores the logical representation of a legal requirement.

Field	Description
rule_id	Unique rule ID
rule_number	Rule reference
title	Rule title
requirement	Requirement description
source_reference	Legal source
status	Active/inactive

Example:

- Rule ID: LM-R6-001
- Rule: 6
- Requirement: Mandatory declaration
- Status: Active

45.11 RULE_VERSION Table

Because Legal Metrology requirements can be amended, each rule should have version information.

Field	Description
version_id	Unique version
rule_id	Associated rule
effective_from	Start date
effective_until	End date
amendment_reference	Amendment source
rule_logic	Machine-readable logic
status	Active/expired

Example:

RULE 6

Version 1

Effective: 2011



Version 2

Effective: 2018



Version 3

Effective: 2022



Current Version

The system selects the appropriate version based on the inspection date and applicable conditions.

45.12 APPLICABILITY_CONDITION Table

This table stores conditions that determine whether a particular requirement applies.

Possible fields:

Field	Description
condition_id	Unique ID
rule_id	Related rule
product_category	Product category
package_type	Package type
quantity_condition	Quantity-related condition
consumer_type	Consumer classification
import_status	Imported/domestic
condition_logic	Machine-readable condition

This prevents the system from applying every rule blindly to every package.

45.13 EXCEPTION Table

Special exemptions or exceptions should be stored separately.

Field	Description
exception_id	Unique ID
rule_id	Related rule
description	Exception description
conditions	Conditions for exception
effective_from	Start date
effective_until	End date
source_reference	Legal source

The rule engine can evaluate exceptions before generating a potential violation.

45.14 COMPLIANCE_CHECK Table

This table records each individual compliance check.

Field	Description
check_id	Unique check ID
inspection_id	Inspection
rule_version_id	Rule used
requirement	Requirement checked
detected_value	Extracted value
expected_condition	Expected condition

result	Pass/fail/review
confidence	AI/system confidence

Example:

- Check: Net Quantity
- Detected: 500 g
- Requirement: Applicable quantity declaration
- Result: PASS

45.15 POTENTIAL_ISSUE Table

The system should record potential issues separately from the final legal decision.

Field	Description
issue_id	Unique issue ID
check_id	Related compliance check
issue_type	Missing/incorrect/unclear
description	Explanation
severity	Low/Medium/High
confidence	System confidence
status	Pending/confirmed/rejected

Example:

Issue:Consumer-care declaration not detected

Status:Pending Officer Review

This supports the human-in-the-loop approach.

45.16 EVIDENCE Table

Evidence links a finding to the supporting material.

Field	Description
evidence_id	Unique ID
issue_id	Related issue
image_id	Supporting image
evidence_type	Image/OCR/manual
highlighted_region	Relevant image area
evidence_hash	Integrity value

Therefore:

Potential Issue



Evidence



Package Image



45.17 OFFICER_REVIEW Table

This stores the final human review.

Field	Description
review_id	Review ID
issue_id	Issue reviewed
officer_id	Reviewing officer
decision	Confirm/reject/edit
remarks	Officer comments
review_time	Time of review

Possible decisions:

- CONFIRMED
- REJECTED
- CORRECTED
- MORE_EVIDENCE_REQUIRED

45.18 INSPECTION_REPORT Table

The final report generated by the system can be stored using:

Field	Description
report_id	Report ID
inspection_id	Inspection
report_number	Report number
generated_at	Generation time
report_path	Stored report
final_status	Final status
approved_by	Reviewing officer

The report should contain the complete evidence trail.

45.19 AUDIT_LOG Table

The audit log records important system actions.

Field	Description
log_id	Log ID
user_id	User performing action
action	Action performed
entity_type	Affected entity
entity_id	Affected record
timestamp	Action time
previous_value	Previous value where applicable
new_value	New value where applicable

Example:

Officer O102



Edited Net Quantity



500 g → 550 g

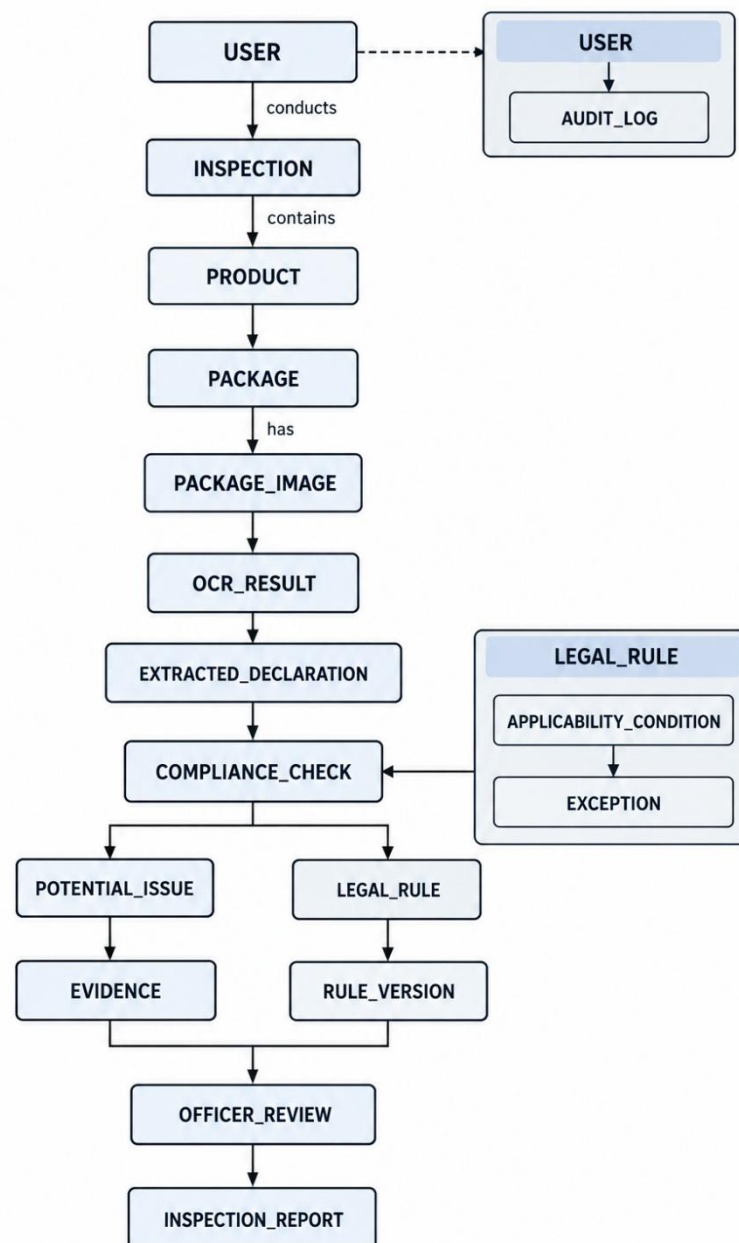


Timestamp recorded

This improves accountability and traceability.

45.20 Complete ER Diagram

The proposed conceptual ER structure is:



45.21 Database Relationships

The major relationships are:

Relationship	Type
User → Inspection	One-to-Many
Product → Inspection	One-to-Many
Product → Package	One-to-Many
Package → Package Image	One-to-Many
Image → OCR Result	One-to-Many
OCR Result → Declaration	One-to-Many
Inspection → Compliance Check	One-to-Many
Legal Rule → Rule Version	One-to-Many
Rule Version → Compliance Check	One-to-Many
Compliance Check → Potential Issue	One-to-Many
Potential Issue → Evidence	One-to-Many
Potential Issue → Officer Review	One-to-Many
Inspection → Inspection Report	One-to-One
User → Audit Log	One-to-Many

45.22 Why This Database Design Is Important

This database design provides **traceability**.

For example, if an officer asks:

“Why did the system flag this package?”

The system can trace:

ISSUE



COMPLIANCE CHECK



RULE VERSION



LEGAL REQUIREMENT



EXTRACTED DECLARATION



OCR RESULT



PACKAGE IMAGE



EVIDENCE



OFFICER REVIEW

Therefore, DrishtiTathya does not produce an unexplained AI prediction. It produces an **evidence-backed compliance finding**.

45.23 Section Conclusion

The database should be designed around the principle of:

Every compliance finding must be traceable to the product, evidence, applicable legal rule and human decision.

This structure also allows DrishtiTathya to support future features such as **repeat-violation detection, manufacturer history, risk scoring, geographic analysis, amendment tracking and e-commerce compliance monitoring**.

46. LEGAL RULE ENGINE DESIGN

The **Legal Rule Engine** is the core decision-making component of DrishtiTathya. Its purpose is to convert the legal requirements of the **Legal Metrology (Packaged Commodities) Rules, 2011** into structured, machine-readable conditions that can be evaluated against information extracted from a package.

The rule engine should not replace the Legal Metrology Officer. It should provide an **AI-assisted preliminary assessment**, explain the reason for each flag, and allow the officer to verify the final result.

46.1 Objective of the Rule Engine

The Rule Engine should answer five major questions:

1. **Does the rule apply to this package?**
2. **Which version of the rule is applicable?**
3. **What declarations or conditions are required?**
4. **Are there any exceptions?**
5. **Does the package satisfy the applicable requirement?**

The basic logic is:

PACKAGE FACTS



APPLICABILITY



RULE VERSION



REQUIREMENTS



EXCEPTIONS



COMPLIANCE CHECK



EVIDENCE



OFFICER REVIEW

46.2 Inputs to the Rule Engine

The rule engine receives structured information rather than raw images.

Example:

- Product Category = Packaged Food
- Package Type = Retail Package
- Net Quantity = 500 g
- MRP = ₹120
- Manufacturer = Detected
- Importer = Not Applicable
- Country of Origin = India
- Inspection Date = 13-09-2026
- Consumer Type = Retail Consumer

These values are obtained from OCR, AI extraction, manual entry and package metadata.

46.3 Rule Engine Processing

The processing can be divided into six stages.

1. INPUT VALIDATION



2. APPLICABILITY CHECK



3. RULE VERSION SELECTION



4. REQUIREMENT EVALUATION



5. EXCEPTION EVALUATION



6. RESULT + EXPLANATION

This modular structure makes the system easier to test and maintain.

46.4 Stage 1 — Input Validation

Before applying legal rules, the system checks whether sufficient information is available.

For example:

Product Category = Unknown

Net Quantity = Unknown

Inspection Date = Available

The system should not blindly execute every rule.

Instead:

INSUFFICIENT INFORMATION



REQUEST MANUAL INPUT

OR



REVIEW REQUIRED

This reduces incorrect conclusions caused by poor OCR or incomplete images.

46.5 Stage 2 — Applicability Engine

The applicability engine determines whether a particular provision should be evaluated.

For example, Rule 3 contains applicability exclusions for certain categories of packages. Therefore, the system should first evaluate package characteristics before applying Chapter II requirements.

Conceptually:

IF package characteristics satisfy exclusion



EXCLUDED



Do not apply corresponding Chapter II check

ELSE



APPLICABLE



Continue compliance evaluation

The system should record the reason for the applicability decision.

46.6 Stage 3 — Rule Version Selection

The rule engine must consider amendments.

A legal requirement should therefore not simply be stored as:

Rule 6 = Requirement X

Instead:

Rule 6

|

|— Version A

|

|— Version B

|

|— Version C

|

|— Current Version

The system selects the appropriate version using the relevant effective date and conditions.

INSPECTION DATE



FIND APPLICABLE VERSION



LOAD LEGAL LOGIC

This is a major requirement for a reliable legal-compliance system.

46.7 Stage 4 — Requirement Evaluation

After determining the applicable rule, the engine compares extracted package information with the requirement.

Example:

REQUIREMENT

Net quantity must be declared



EXTRACTED INFORMATION

500 g



CHECK

Declaration detected?



YES



PASS

Another example:

REQUIREMENT

Consumer-care declaration



OCR / AI

No corresponding declaration detected



CHECK EXCEPTIONS



No applicable exception identified



POTENTIAL ISSUE

46.8 Stage 5 — Exception Evaluation

Exceptions should be evaluated before generating a final compliance flag.

RULE REQUIREMENT



NOT SATISFIED?



CHECK EXCEPTIONS



└─ Exception applies → EXEMPT / NOT APPLICABLE

|

└─ No exception → POTENTIAL ISSUE

This prevents the system from incorrectly flagging products that fall within a lawful exemption.

46.9 Stage 6 — Result Generation

The engine should generate a structured result.

Example:

```
{  
  Requirement: "Net Quantity",  
  Detected_Value: "500 g",  
  Status: "PASS",  
  Rule: "Applicable Rule",  
  Confidence: "98%",  
  Evidence: "IMG_002"  
}
```

For a potential issue:

```
{  
  Requirement: "Required Declaration",  
  Detected_Value: "Not detected",  
  Status: "POTENTIAL_ISSUE",  
  Rule: "Applicable Rule",  
  Confidence: "87%",  
  Evidence: "IMG_003",  
  Review_Required: true  
}
```

46.10 Compliance Result Categories

DrishtiTathya should avoid treating every uncertain AI output as a confirmed legal violation.

The recommended categories are:

Result	Meaning
COMPLIANT	Requirement appears satisfied
POTENTIAL ISSUE	Requirement appears not satisfied

NOT APPLICABLE	Rule does not apply
EXEMPTED	An applicable exception has been identified
REVIEW REQUIRED	Insufficient confidence/evidence
NOT CHECKED	Required information unavailable

This terminology is safer and more appropriate for an AI-assisted inspection system.

46.11 Explainable Rule Execution

Every result should contain an explanation.

Instead of:

 FAILED

the application should show:  POTENTIAL ISSUE

- Requirement:[Relevant legal requirement]
- Observed:Declaration not detected
- Reason:Required declaration could not be identified from the captured package information.
- Evidence:Back-side package image
- Confidence:86%
- Action:Officer verification required

This makes the system explainable.

46.12 Rule Representation

A legal rule can be represented internally using structured fields.

Example:

- RULE_ID
- RULE_NUMBER
- REQUIREMENT
- APPLICABILITY
- CONDITION
- EXCEPTION
- EFFECTIVE_FROM
- EFFECTIVE_UNTIL
- SOURCE_REFERENCE
- SEVERITY

A simplified logical representation can be:

IF

package_is_applicable = TRUE

AND

required_declaration = TRUE

AND

declaration_detected = FALSE

AND

exception_applies = FALSE

THEN

result = POTENTIAL_ISSUE

46.13 Example — Net Quantity Check

Suppose the package contains:

Net Quantity = 500 g

The rule engine can process:

PACKAGE

↓

Quantity declaration detected?

↓

YES

↓

Unit valid?

↓

YES

↓

Quantity format acceptable?

↓

YES

↓

RESULT = PASS

The exact legal validation should be based on the applicable provisions and schedules rather than a generic AI assumption.

46.14 Example — Unit Validation

The Rule Engine can also validate the declared unit against the applicable unit requirements.

Conceptually:

INPUT

1.5 kg



UNIT CHECK



Valid unit?

└— YES → Continue

└— NO → Potential Issue

Rules concerning units should be encoded from the relevant provisions, including the requirements concerning statements of units.

46.15 Example — MRP Check

The engine can extract:

MRP = ₹120

and determine whether the required retail sale price declaration is detected and appropriately associated with the package.

The system should distinguish between:

MRP NOT DETECTED and: MRP DETECTED BUT NEEDS HUMAN VERIFICATION

because OCR failure does not necessarily mean that the physical declaration is absent.

46.16 Example — Manufacturer / Importer Check

The engine can identify whether relevant manufacturer, packer or importer information has been detected.

PACKAGE



DOMESTIC / IMPORTED?



SELECT APPLICABLE REQUIREMENT



CHECK RELEVANT DECLARATION



RESULT

This demonstrates why **product classification and applicability analysis must happen before compliance checking.**

46.17 Rule 19 Inspection Logic

The Rule Engine can also support inspection-related workflows.

The Rules provide a process involving examination/testing, sampling and recording of results.

Therefore, DrishtiTathya can represent the process as:

INSPECTION



SELECT SAMPLE



RECORD SAMPLE DETAILS



QUANTITY / DECLARATION CHECK



COMPARE WITH APPLICABLE REQUIREMENT



RECORD RESULT



EVIDENCE

The system should distinguish between **AI-based package declaration analysis** and **physical quantity testing**, since they are different inspection activities.

46.18 Schedule-Based Rules

Some requirements depend on schedules rather than only the main rule text.

Therefore, the rule engine architecture should support:

RULE



SCHEDULE



TABLE / THRESHOLD



CALCULATION



RESULT

For example, where maximum permissible error or sample-selection requirements are relevant, the engine should retrieve the appropriate schedule information instead of using a hard-coded generic threshold.

46.19 Rule Priority

When multiple legal conditions apply, the engine should evaluate them in a defined order.

Recommended sequence:

1. Applicability



2. Exemption



3. Effective Rule Version



4. Mandatory Requirement



5. Format / Unit Requirement



6. Special Category Requirement



7. Schedule / Threshold



8. Final Assessment

This avoids applying a general rule before checking whether a specific condition changes its applicability.

46.20 Conflict Handling

If two rule conditions appear to conflict, the system should not automatically choose one without a traceable legal basis.

Instead:

RULE CONFLICT / UNCERTAINTY



REVIEW REQUIRED



LEGAL SOURCE SHOWN



OFFICER DECISION

This is particularly important when amendments or special-category provisions are involved.

46.21 Legal Source Traceability

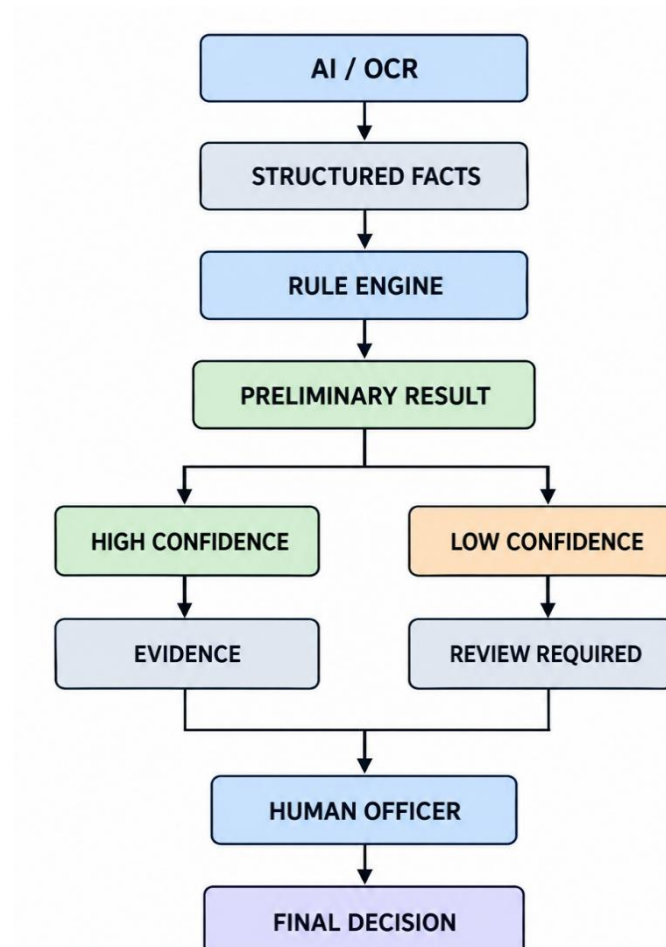
Every rule stored in the engine should have a source reference.

- Example:Rule ID: LM-R6-V3Source:
- Packaged Commodities Rules, 2011
- Provision:Rule 6
- Version:V3
- Effective From:[Date]
- Amendment:[Reference]
- Status:Active

Thus, an officer can identify **which legal provision produced the result**.

46.22 Human-in-the-Loop Architecture

The final architecture should be:



The system therefore assists the officer without presenting an AI prediction as an automatic legal determination.

46.23 Rule Engine Pseudocode

A simplified implementation can be represented as:

- START
- Receive package data
- Validate input
- Determine package applicability

IF exempt:

- return EXEMPTED
- Identify applicable rule version
- Load applicable requirements

FOR each requirement:

- Extract relevant package information
- Check declaration
- Check format / value
- Check applicable exception

IF requirement satisfied: PASS

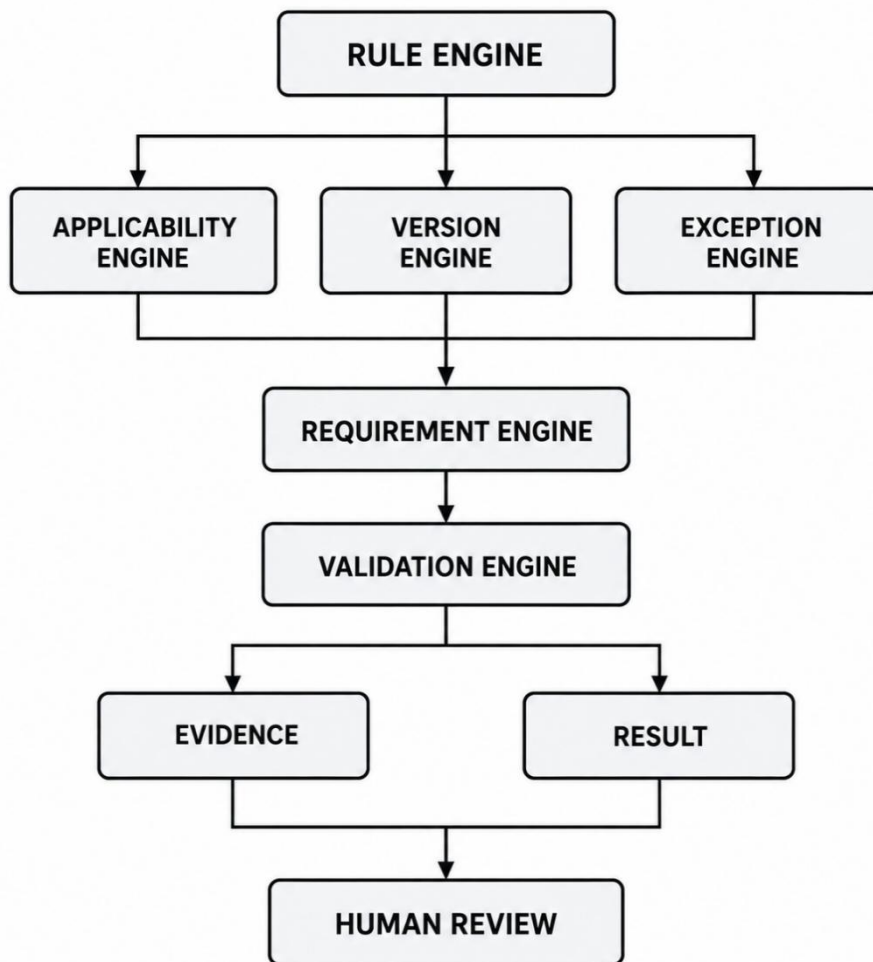
ELSE IF evidence is insufficient: REVIEW REQUIRED

ELSE:

- POTENTIAL ISSUE
- Store evidence
- Generate explanation
- Send result to officer
- Officer reviews
- Store final decision
- Generate inspection report

END

46.24 Rule Engine Architecture



46.25 Key Innovation — Version-Aware Legal Intelligence

A major differentiating feature of DrishtiTathya is the concept of a **Version-Aware Legal Rule Engine**.

Traditional systems may work like:

IMAGE → OCR → CHECKLIST

DrishtiTathya should work like:

IMAGE

↓

OCR

↓

STRUCTURED FACTS

↓

APPLICABILITY

↓

LEGAL VERSION

↓

RULE + EXCEPTION + SCHEDULE



EVIDENCE-BACKED RESULT



HUMAN REVIEW

This makes the project more than an OCR application. It becomes a **legal-compliance reasoning system**.

46.26 Section Conclusion

The Legal Rule Engine forms the bridge between **AI-extracted package information** and **Legal Metrology requirements**.

Its fundamental design principle is:

Do not directly convert AI output into a legal conclusion. Convert AI output into structured facts, evaluate those facts against the applicable version of the law, check exceptions, attach evidence, and place the result before a human officer for final review.

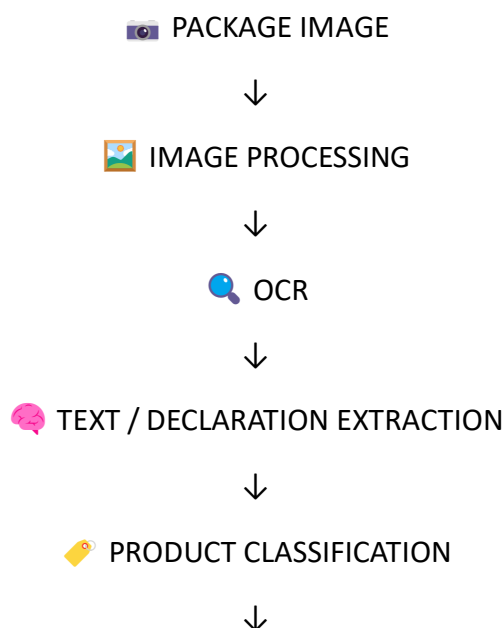
This architecture provides DrishtiTathya with **explainability, legal traceability, amendment support and human oversight**.

47. AI/ML MODULE DESIGN

The **AI/ML module** is responsible for understanding the visual and textual information present on packaged commodities. It converts package images into structured information that can later be evaluated by the Legal Rule Engine.

The AI layer should **assist the compliance process rather than make the final legal decision**.

The overall AI pipeline is:



 CONFIDENCE SCORING



 STRUCTURED PACKAGE DATA



 LEGAL RULE ENGINE

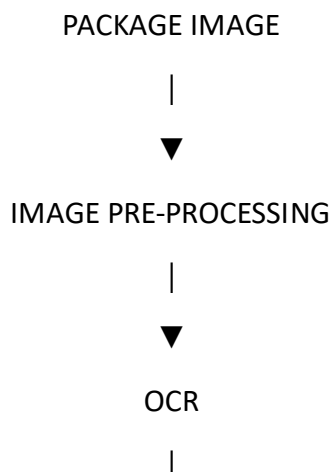
47.1 Objectives of the AI Module

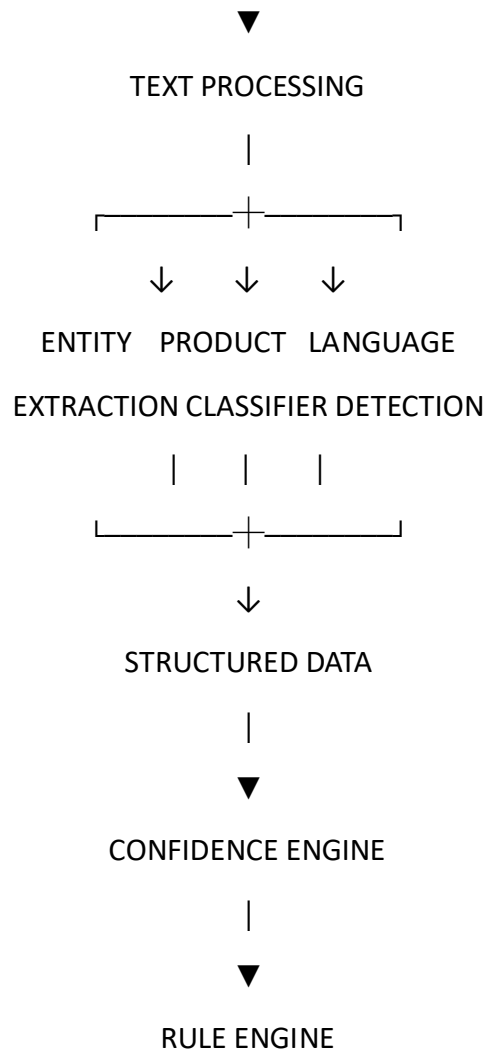
The AI module should be able to:

- Detect text on package images
- Extract readable text
- Identify important declarations
- Recognize quantities and units
- Detect MRP information
- Identify manufacturer/packer/importer information
- Identify country of origin where applicable
- Identify consumer-care information
- Classify the product/package
- Handle multiple package images
- Provide confidence scores
- Highlight evidence locations
- Identify uncertain information for human review

47.2 AI Module Architecture

The proposed architecture is:





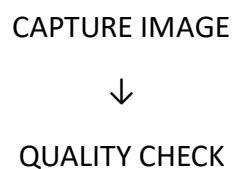
47.3 Image Quality Detection

Before performing OCR, the system should determine whether the captured image is usable.

Possible quality checks:

- Blur detection
- Low-light detection
- Excessive glare
- Incorrect orientation
- Poor framing
- Insufficient resolution
- Text too small to read

Example:



The system can identify text regions such as:

	PRODUCT NAME	
	[TEXT REGION]	
	NET QTY: 500 g	
	MRP: ₹120	
	Manufactured by ABC Foods	

The coordinates of the detected region can be stored as evidence.

47.6 Declaration Extraction

The next AI component converts OCR text into meaningful declarations.

For example:

OCR TEXT

"MRP ₹120"



ENTITY EXTRACTION



TYPE = MRP

VALUE = ₹120

Another example:

"NET QTY 500 g"



TYPE = NET_QUANTITY

VALUE = 500

UNIT = g

47.7 Named Entity Recognition

A Natural Language Processing model can identify entities from extracted text.

Possible entity types:

- PRODUCT_NAME
- BRAND
- MANUFACTURER

- PACKER
- IMPORTER
- COUNTRY
- NET_QUANTITY
- UNIT
- MRP
- DATE
- BATCH_NUMBER
- CONSUMER_CARE

This transforms unstructured text into structured package information.

47.8 Quantity Extraction

Quantity extraction requires both the **numeric value and its unit**.

Example:

"500 g"

↓

Value = 500

Unit = gram

Possible recognized units include:

- g
- kg
- mg
- mL
- L
- cm
- m
- number
- piece
- pair

The extracted value is then passed to the Legal Rule Engine for legal validation.

47.9 MRP Extraction

The AI module should identify common representations of retail sale price.

Examples:

- MRP ₹120
- MRP: Rs. 120
- Maximum Retail Price ₹120
- ₹120 incl. all taxes

The AI should normalize these into a common internal format.

RAW TEXT



NORMALIZATION



MRP = 120 INR

The legal rule engine then performs the applicable compliance evaluation.

47.10 Manufacturer / Packer / Importer Extraction

The AI should identify relevant business information from package text.

Example:

"Manufactured by:

ABC Foods Pvt. Ltd.

Pune, Maharashtra"



Manufacturer:ABC Foods Pvt. Ltd.

Address:Pune, Maharashtra

For imported products, the system can separately identify relevant importer/origin information.

47.11 Country of Origin Detection

The AI can detect phrases such as:

"Made in India"

"Country of Origin: India"

"Product of India"

and convert them to:

Country of Origin = India

For imported products, this information can become an input to the applicability and compliance logic.

47.12 Consumer-Care Information Detection

The AI can identify customer-care information such as:

- Consumer Care
- Customer Care
- Customer Service
- Helpline

- Toll Free

and extract associated contact information.

Consumer Care:1800-XXXX-XXXX

The system should distinguish between actual contact information and unrelated numbers such as batch numbers or barcodes.

47.13 Product Classification

A classification model can predict the likely product category.

Example:

IMAGE + OCR



PRODUCT CLASSIFIER



Packaged Food

Confidence: 94%

Possible categories can include:

- Food
- Household goods
- Personal-care products
- Textiles
- Consumer goods
- Other packaged commodities

The exact category list can be expanded according to project requirements.

47.14 Multi-Image Intelligence

A package may contain information on several sides.

Therefore, DrishtiTathya should combine information from multiple images.

FRONT IMAGE



|—— Product Name



BACK IMAGE



|—— Manufacturer

|—— Quantity

└—— MRP

|

SIDE IMAGE

|

└—— Consumer Care

↓

COMBINED PACKAGE DATA

This is better than treating each photograph as a completely independent inspection.

47.15 Duplicate Information Resolution

The same declaration may appear in multiple images.

Example:

Image 1 → MRP ₹120

Image 2 → MRP ₹120

Image 3 → MRP ₹120

The system should combine these observations rather than creating three different MRP values.

If conflicting values are found:

Image 1 → ₹120

Image 2 → ₹150

↓

CONFLICT DETECTED

↓

REVIEW REQUIRED

This can become an important future feature for detecting inconsistent package information.

47.16 Confidence Scoring

Every AI extraction should ideally have a confidence score.

Example:

Field	Extracted Value	Confidence
Product Name	ABC Biscuits	96%
Net Quantity	500 g	98%

MRP	₹120	97%
Manufacturer	ABC Foods	91%
Consumer Care	Detected	88%

A threshold can determine whether human verification is required.

Confidence $\geq$ Threshold



Continue

Confidence < Threshold



Review Required

The threshold should be configurable and validated experimentally rather than arbitrarily treated as legally sufficient.

47.17 Human Correction Loop

The officer should be able to correct AI results.

Example:

AI:Net Quantity = 500 g

Officer:Correct → 550 g



Corrected Value



Stored in inspection record

The system should preserve the fact that the value was manually corrected.

47.18 AI Feedback Loop

If the project later collects sufficient reviewed examples, corrected results can be used to improve models.

AI PREDICTION



OFFICER REVIEW



CORRECTION



VALIDATED DATASET



MODEL IMPROVEMENT

Any such training pipeline should be controlled and versioned so that changes to model behavior can be audited.

47.19 Multilingual AI

Packaged commodities may contain information in multiple languages.

The AI architecture should therefore be capable of handling multilingual text.

IMAGE



LANGUAGE DETECTION



OCR



TEXT NORMALIZATION



DECLARATION EXTRACTION

For example, the system could recognize equivalent concepts expressed in English and Indian languages.

The legal evaluation should still operate on standardized internal fields.

47.20 AI vs Rule Engine

A clear separation should be maintained.

AI/ML	Legal Rule Engine
Reads package image	Applies legal requirements
Extracts text	Determines applicable rule
Identifies declarations	Checks legal conditions
Classifies product	Checks exceptions
Estimates confidence	Selects rule version
Finds evidence regions	Produces compliance assessment
Handles language variation	Maintains legal traceability

Therefore:

AI = "WHAT IS ON THE PACKAGE?"

+

RULE ENGINE = "WHAT DOES THE LAW REQUIRE?"



COMPLIANCE ANALYSIS

This distinction should be clearly explained during the SIH presentation.

47.21 AI Failure Handling

AI systems can fail because of:

- Blurred images
- Damaged packaging
- Curved surfaces
- Decorative fonts
- Low contrast
- Multiple languages
- Reflections
- Occlusion
- Unusual layouts

The system should therefore provide:

AI FAILURE



RETRY / CAPTURE BETTER IMAGE

OR

MANUAL ENTRY

OR

OFFICER REVIEW

It should never silently convert an OCR failure into a legal violation.

47.22 Proposed AI Technology

A practical prototype can use:

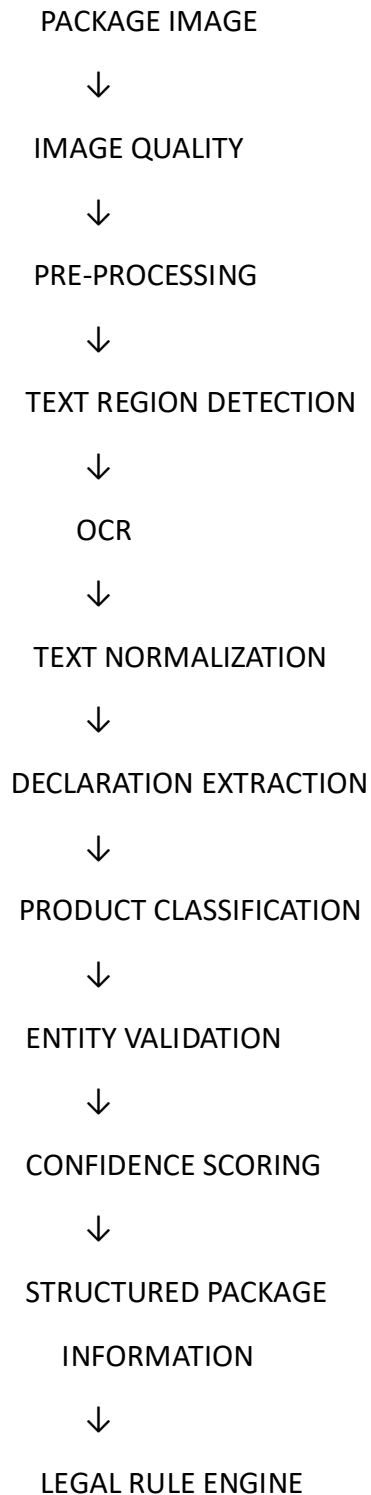
- **Image Processing**
- **OpenCV**
- **OCR**
- **Tesseract OCR** or another suitable OCR engine
- **AI/ML**
- **Python-based NLP / computer-vision models**
- **AI API**
- **FastAPI**
- **Mobile Integration**

- **Android + Kotlin**

The exact models can be selected after evaluating the quality of package images and available training data.

47.23 AI Processing Pipeline

The complete pipeline can be represented as:



47.24 AI Evidence Linking

A particularly useful feature is connecting each extracted field to its image location.

Example:

Net Quantity = 500 g



Source Image = Back.jpg



Bounding Box = (x1,y1,x2,y2)

When the officer taps **“Net Quantity”**, the application can display the corresponding region of the package.

This improves transparency and makes manual verification faster.

47.25 AI Confidence + Legal Confidence

The project should distinguish between two different concepts:

AI Confidence

“How confident is the model that it correctly read the package?”

Legal Evaluation

“Based on the extracted information and applicable law, what requirement appears to be satisfied or unsatisfied?”

Therefore:

AI CONFIDENCE



INFORMATION RELIABILITY



LEGAL RULE EVALUATION



OFFICER REVIEW

A high AI confidence score should **not** automatically mean that the legal conclusion is certain.

47.26 Section Conclusion

The AI/ML module transforms DrishtiTathya from a simple image-scanning application into an intelligent package-information extraction system.

Its primary role is:

See → Read → Understand → Structure → Provide Evidence

The Legal Rule Engine then performs:

Apply → Compare → Explain → Flag

Finally:

Human Officer → Verify → Decide

This separation provides a strong foundation for an **explainable, evidence-backed and human-supervised AI compliance platform**.

48. COMPLIANCE DETECTION AND VIOLATION ANALYSIS MODULE

The **Compliance Detection and Violation Analysis Module** converts the output of the AI/ML module into a structured preliminary compliance assessment.

Its purpose is not simply to identify whether text is present. It evaluates whether the **required information appears to be present and consistent with the applicable legal requirements**, and provides evidence for officer verification.

The overall process is:

EXTRACTED PACKAGE DATA



APPLICABILITY CHECK



REQUIRED DECLARATIONS



DECLARATION COMPARISON



EXCEPTION CHECK



COMPLIANCE ANALYSIS



POTENTIAL ISSUE



EVIDENCE + RULE REFERENCE



OFFICER REVIEW

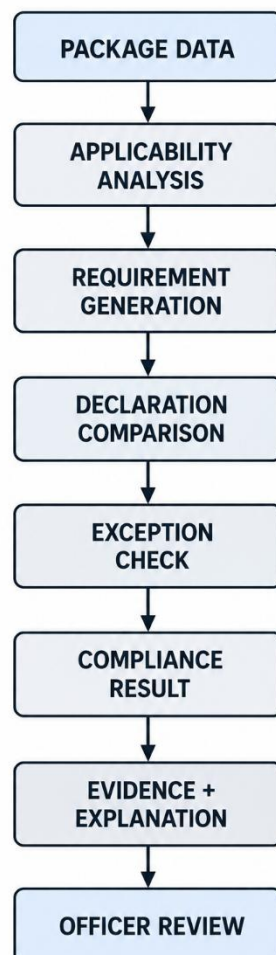
48.1 Objectives

The module should identify:

- Missing declarations

- Unclear declarations
 - Incorrect or inconsistent extracted values
 - Invalid or unexpected units
 - MRP-related issues
 - Net-quantity-related issues
 - Manufacturer/packer/importer information issues
 - Country-of-origin issues where applicable
 - Date-related declaration issues where applicable
 - Consumer-care information issues
 - Unit-sale-price issues where applicable
 - Special-category declaration issues
 - Package-to-package inconsistencies
 - Potentially deceptive or misleading quantity presentation
 - Issues requiring manual verification
-

48.2 Compliance Detection Architecture



48.3 Requirement Checklist Generation

The system should first create a checklist based on the package characteristics.

Example:

PRODUCT

Packaged Commodity



APPLICABILITY



GENERATE CHECKLIST



✓ Product Name

✓ Manufacturer / Packer / Importer

✓ Net Quantity

✓ MRP

✓ Consumer Care

✓ Country of Origin where applicable

✓ Date-related declaration where applicable

✓ Unit Sale Price where applicable

The checklist should be dynamic rather than hard-coded for every product.

48.4 Missing Declaration Detection

The simplest compliance check is whether a required declaration was detected.

Example:

REQUIRED:Consumer Care Information

OCR:Not detected

Exception:Not identified



⚠ POTENTIAL ISSUE

The result should be described as **“not detected”** rather than automatically claiming that the declaration physically does not exist.

48.5 Unclear Declaration Detection

Sometimes information exists but cannot be confidently interpreted.

Example:

IMAGE



OCR



"NET QTY 5?0 g"



LOW CONFIDENCE



REVIEW REQUIRED

This is different from a missing declaration.

The application should clearly distinguish:

NOT DETECTED

vs.

DETECTED BUT UNCLEAR

48.6 Incorrect Value Detection

Where the extracted value can be evaluated against a relevant legal condition, the system can flag a potential issue.

Example:

Detected Quantity

500 g



UNIT / FORMAT CHECK



LEGAL REQUIREMENT



COMPARISON



RESULT

The rule engine should determine the actual legal condition rather than relying on a generic AI judgment.

48.7 Unit Validation

The module should separately evaluate:

- Numeric value
- Unit
- Quantity format
- Relationship between value and unit

Example:

500 +g

↓

500 g

↓

UNIT VALIDATION

↓

RULE ENGINE

If the unit cannot be interpreted confidently:

Quantity = 500 ?

↓

REVIEW REQUIRED

48.8 MRP Analysis

The system can extract the declared MRP and associate it with evidence.

Example:

MRP

↓

₹120

↓

IMAGE LOCATION

↓

RULE CHECK

↓

RESULT

The application can also identify possible conflicts when different package images contain different price values.

Front → ₹120

Back → ₹150



CONFLICT



REVIEW REQUIRED

48.9 Manufacturer / Packer / Importer Analysis

The system should identify the relevant business declaration and classify it correctly.

Example:

"Manufactured by ABC Foods"



ENTITY TYPE= MANUFACTURER



ADDRESS= Extracted



COMPLIANCE CHECK

If the AI cannot determine whether a company is a manufacturer, packer or importer, the result should be sent for review.

48.10 Country-of-Origin Analysis

For products where country-of-origin information is applicable, the system can detect it from the package.

Example:

"Made in Japan"



Country of Origin= Japan



Applicability Check



Compliance Evaluation

This information can also support e-commerce and imported-product compliance workflows in future versions.

48.11 Date Declaration Analysis

Where a relevant date-related declaration is applicable, the system can identify date expressions.

Examples:

- MFD
- Manufactured
- Best Before
- Use By
- Expiry

The AI extracts the date, while the legal rule engine determines the applicable requirement.

48.12 Consumer-Care Analysis

The system can search for customer-care information.

- Consumer Care
- Customer Care
- Helpline

Toll Free



CONTACT INFORMATION



STRUCTURED DATA



COMPLIANCE CHECK

The system should avoid confusing customer-care numbers with:

- Batch numbers
 - Product codes
 - Barcodes
 - Manufacturing codes
-

48.13 Unit Sale Price Analysis

Where applicable, the system can identify unit-sale-price information.

For example:

PACKAGE



NET QUANTITY



UNIT SALE PRICE



COMPARE FORMAT



RULE ENGINE

The 2021 amendment introduced specified unit-sale-price formats for different quantity ranges. Therefore, this check should be implemented as a **version-aware rule**, rather than as a permanently hard-coded format.

48.14 Font and Readability Analysis

The system can provide an **AI-assisted visual screening** of declaration visibility.

Possible checks:

- Text visibility
- Contrast
- Approximate text size
- Text obstruction
- Orientation
- Legibility

Example:

DECLARATION



VISUAL ANALYSIS



[
| Clearly visible |
]

However, automated image analysis should be treated as a screening tool. Final legal assessment should remain with the officer.

48.15 Deceptive Package Screening

The Rules contain provisions relating to deceptive packages.

DrishtiTathya can provide an AI-assisted **screening indicator** based on package appearance and quantity information.

PACKAGE DESIGN

+

DECLARED QUANTITY

+

VISUAL ANALYSIS

↓

POSSIBLE MISLEADING IMPRESSION

↓

OFFICER REVIEW

The system should not independently declare that a package is legally deceptive.

48.16 Multi-Image Consistency Check

This can be a strong feature of DrishtiTathya.

The system compares extracted information from multiple package images.

Example:

Field	Image 1	Image 2	Result
MRP	₹120	₹120	✓ Consistent
Net Quantity	500 g	500 g	✓ Consistent
Manufacturer	ABC Ltd.	ABC Ltd.	✓ Consistent
Batch	A102	A102	✓ Consistent

If information differs:

Field Image 1 Image 2 Result

MRP ₹120 ₹150 ⚠ Conflict

This can trigger manual verification.

48.17 Evidence-Based Violation Card

Every potential issue should be displayed as an evidence-backed card.

Example:

 POTENTIAL ISSUE	
Requirement:	Consumer-care declaration
Observed:	Not detected
Confidence:	87%
Rule Reference:	Applicable provision
Evidence:	 [View Package Image]
✓ CONFIRM ✗ REJECT ↻ REVIEW	

This should be the main interface through which the officer interacts with system findings.

48.18 Severity Classification

Potential issues can be categorized for prioritization.

Severity Meaning

Low Minor/uncertain issue requiring review

Medium Significant potential non-compliance

High Serious potential issue requiring priority review

Severity should be assigned according to the project's defined risk framework and should not be confused with the legal penalty for an offence.

48.19 Compliance Score

For dashboard purposes, the system can optionally calculate an internal compliance score.

Example:

- Total Checks = 10
- Passed = 8
- Review Required = 1
- Potential Issues = 1
- Internal Score = 80%

This score should be clearly labeled as a **system-generated assessment**, not as a statutory legal score.

A better presentation is:

Compliance Checks

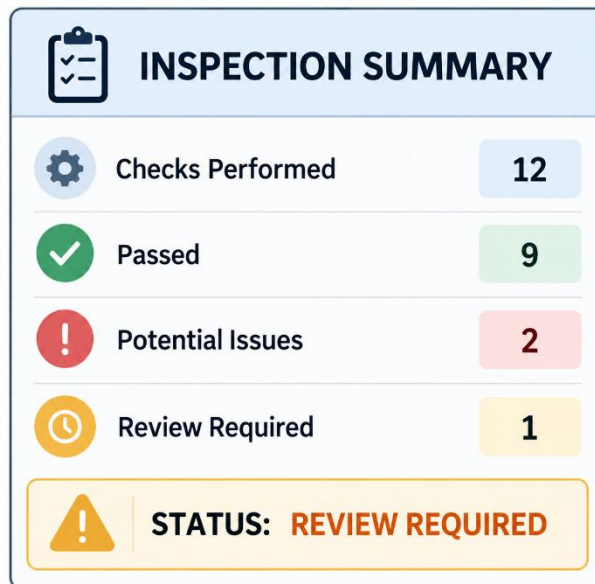
- 8 Passed

- 1 Review
- 1 Potential Issue

rather than presenting a percentage as if it were an official legal certification.

48.20 Final Compliance Summary

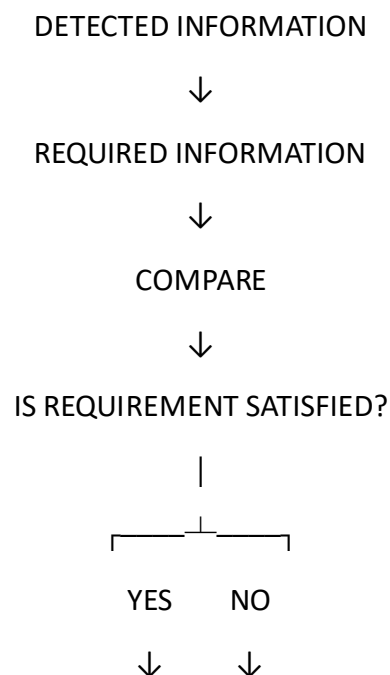
After all checks:

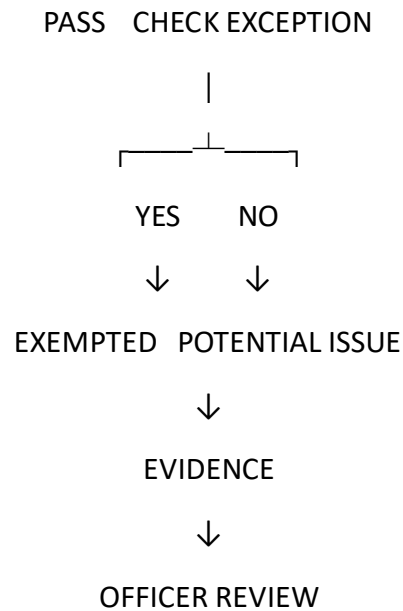


The officer can then examine each issue individually.

48.21 Violation Analysis Workflow

The complete violation-analysis process is:





48.22 False Positive Reduction

False positives are a major challenge in AI-based compliance systems.

DrishtiTathya should reduce them using:

1. Multi-image verification

Check other package panels.

2. Confidence threshold

Send uncertain extraction for review.

3. Exception evaluation

Check applicable exemptions before flagging.

4. Rule versioning

Use the correct legal version.

5. Context-aware extraction

Distinguish MRP, batch number, phone number and other numeric values.

6. Human review

Allow officers to confirm or reject findings.

48.23 False Negative Reduction

The system should also reduce the possibility of missing a potential issue.

Possible methods:

MULTIPLE IMAGES

+

OCR

+

VISUAL TEXT DETECTION

+

STRUCTURED EXTRACTION

+

RULE CHECK

+

OFFICER REVIEW

No single AI model should be responsible for the entire compliance decision.

48.24 Detection Result Model

A standardized result object can contain:

- CHECK_ID
- REQUIREMENT
- OBSERVED_VALUE
- EXPECTED_CONDITION
- STATUS
- CONFIDENCE
- RULE_ID
- RULE_VERSION
- EVIDENCE_ID
- EXCEPTION_STATUS
- OFFICER_REVIEW_STATUS

This makes the output usable by both the mobile application and officer dashboard.

48.25 Example Complete Analysis

Consider a hypothetical package:

- Product:ABC Biscuits
- Net Quantity:500 g
- MRP:₹120
- Manufacturer:Detected
- Consumer Care:Not detected

The system processes:

PACKAGE

↓

OCR



STRUCTURED DATA



APPLICABILITY



REQUIRED DECLARATIONS



CONSUMER CARE CHECK



NOT DETECTED



EXCEPTION CHECK



NO EXCEPTION IDENTIFIED



POTENTIAL ISSUE

Officer screen:[⚠](#) Potential Issue

Declaration:Consumer Care

Status:Not detected

Confidence:89%

Evidence:Back-side image

Rule:[Applicable provision]

Final Decision:Pending Officer Review

48.26 Important Legal-Safety Principle

DrishtiTathya should consistently use language such as:

- **Potential issue**
- **Not detected**
- **Review required**
- **Possible non-compliance**
- **AI-assisted finding**

It should avoid automatically displaying:

- “Company is guilty”
- “Legal offence confirmed”
- “Penalty imposed”
- “Violation conclusively established”

unless such a determination is made through the authorized legal/inspection process.

48.27 Section Conclusion

The Compliance Detection and Violation Analysis Module provides the connection between **extracted package information** and **actionable inspection findings**.

Its core principle is:

Detect → Compare → Check Exceptions → Explain → Provide Evidence → Review

This approach makes the system useful for government officers while reducing the risk of treating OCR or AI errors as legal conclusions.

The module also creates the foundation for advanced features such as **risk scoring, repeat-violation detection, manufacturer compliance history, geographic analysis and automated inspection prioritization**.

49. EVIDENCE MANAGEMENT AND AUDIT TRAIL

The **Evidence Management and Audit Trail Module** ensures that every compliance finding generated by DrishtiTathya can be traced back to the original package image, extracted information, applicable legal rule and officer review.

This module is essential because a compliance system should not only provide a result; it should also provide **evidence explaining how that result was obtained**.

The complete evidence chain is:

PACKAGE



IMAGE



OCR TEXT



EXTRACTED INFORMATION



LEGAL RULE



COMPLIANCE CHECK

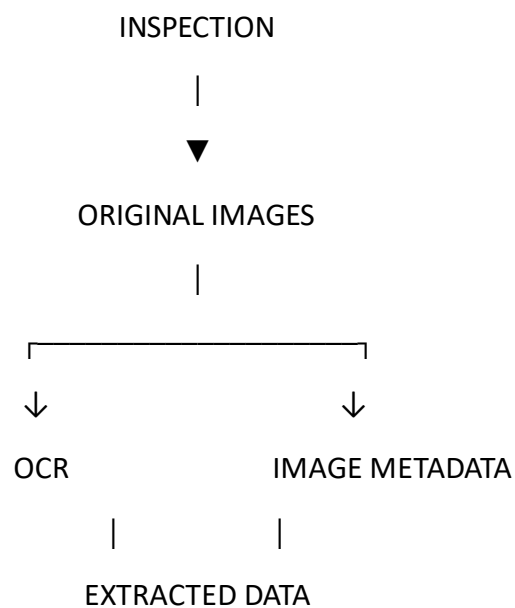
↓
POTENTIAL ISSUE
↓
OFFICER REVIEW
↓
FINAL REPORT

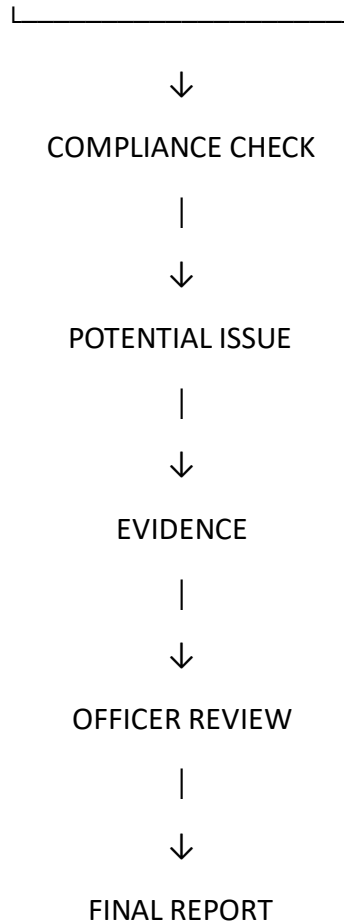
49.1 Objectives

The Evidence Management Module should:

- Store inspection photographs
 - Preserve original images
 - Store OCR results
 - Link extracted declarations to image locations
 - Link findings to applicable rules
 - Record officer decisions
 - Maintain timestamps
 - Maintain an audit trail
 - Detect accidental or unauthorized modification
 - Support report generation
 - Allow evidence to be retrieved during later review
-

49.2 Evidence Architecture





49.3 Types of Evidence

DrishtiTathya can maintain different categories of evidence.

1. Image Evidence

Photographs of the actual package.

2. OCR Evidence

Text extracted from the package image.

3. Structured Evidence

Information extracted from OCR.

Example:

MRP = ₹120

Net Quantity = 500 g

4. Rule Evidence

The legal provision used for the compliance check.

5. Review Evidence

Officer comments, corrections and decisions.

49.4 Original Image Preservation

The original captured image should be preserved separately from processed versions.

ORIGINAL IMAGE

|

--> STORAGE

|

└--> PROCESSING COPY

↓

OCR / AI

This is important because image enhancement or cropping should not overwrite the original inspection evidence.

49.5 Image Metadata

Each image should have associated metadata.

Field	Example
Image ID	IMG-00025
Inspection ID	INS-00125
Image Type	Back Panel
Capture Time	13-09-2026 15:32
File Format	JPEG
Image Hash	SHA-256 value
Processing Status	OCR Completed

The metadata helps identify the source of evidence.

49.6 Image Hashing

A cryptographic hash can be generated for each original image.

Conceptually:

ORIGINAL IMAGE

↓

HASH FUNCTION

↓

SHA-256 HASH

↓

STORE HASH

If the file is later modified:

IMAGE



NEW HASH



COMPARE WITH ORIGINAL HASH



MISMATCH



INTEGRITY WARNING

The hash does not prove the legal truth of the image; it helps detect whether the stored file has changed.

49.7 OCR Evidence

OCR output should remain connected to its source image.

Example:

Image: IMG-0025

OCR:

"NET QTY 500 g"

"MRP ₹120"

The system should ideally also store the coordinates of the text.

TEXT



Bounding Box



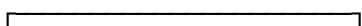
Image Location

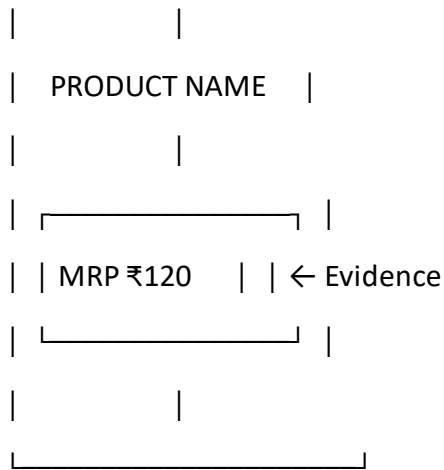
49.8 Highlighted Evidence

When an officer selects a finding, the application can highlight the relevant area.

Example:

PACKAGE IMAGE





This allows the officer to quickly verify the AI extraction.

49.9 Evidence-to-Rule Linking

Each compliance finding should be linked to both:

EVIDENCE

+

LEGAL RULE

↓

COMPLIANCE FINDING

For example:

- Finding:Required declaration not detected
- Evidence:Back-side image
- Rule:Applicable provision
- Review:Pending

This makes the system explainable.

49.10 Evidence Chain

The system should maintain a complete chain:

IMAGE

↓

OCR

↓

DECLARATION

↓

COMPLIANCE CHECK



ISSUE



EVIDENCE



OFFICER REVIEW



FINAL DECISION

Every stage should contain a unique identifier.

49.11 Inspection Evidence ID

Each inspection can receive a unique identifier.

Example:

Inspection ID:

LM-2026-000125

All related records can then be associated with this ID.

LM-2026-000125

|

|— IMG-001

|— IMG-002

|— OCR-001

|— CHECK-001

|— ISSUE-001

|— REVIEW-001

|— REPORT-001

49.12 Audit Trail

The audit trail records significant actions performed on an inspection.

Examples:

- 15:32 Image captured
- 15:33 OCR completed
- 15:34 AI extraction completed
- 15:34 Potential issue generated

- 15:36 Officer opened finding
- 15:38 Officer corrected extracted value
- 15:40 Finding confirmed
- 15:42 Report generated

This creates a chronological history of the inspection.

49.13 Audit Log Structure

Field	Description
Audit ID	Unique audit record
User ID	Person performing action
Action	Action performed
Entity	Affected object
Entity ID	Object identifier
Timestamp	Date and time
Previous Value	Previous value if changed
New Value	Updated value
Device/Session	Relevant session information

49.14 Officer Correction Tracking

If an officer changes an AI-generated value, the system should preserve both versions.

Example:

AI VALUE

500 g



OFFICER CORRECTION

550 g



AUDIT LOG

The system should not simply overwrite the original AI result without recording the correction.

49.15 Human Review Record

Each potential issue can contain:

AI Finding



Officer Review



- Confirmed

- Rejected
- Corrected
- More Evidence Required

The officer can also add remarks.

49.16 Evidence Integrity

The system should protect evidence against accidental modification.

Recommended controls include:

- Original-file preservation
- Hash generation
- Access control
- Timestamping
- Audit logs
- Version tracking
- Controlled deletion
- Backup

The objective is to make the evidence history **traceable and tamper-evident**.

49.17 Role-Based Evidence Access

Not every user should have the same permissions.

Example:

Role	View	Upload	Edit	Delete	Review
Officer	✓	✓	✓	Limited	✓
Reviewer	✓	✓	Limited	No	✓
Admin	✓	✓	✓	Controlled	✓
Manufacturer	Limited	✓	Own data	No	No

The exact permissions can be configured according to the intended deployment model.

49.18 Evidence Versioning

When evidence-related information changes, the system should maintain versions.

Example:

Declaration V1

MRP = ₹120



Officer correction



Declaration V2

MRP = ₹125

The system can retain the original value and the corrected value.

49.19 Offline Evidence Handling

Since field inspections may occur without continuous internet access, evidence should initially be stored securely on the device.

CAPTURE



LOCAL ENCRYPTED STORAGE



INSPECTION CONTINUES



NETWORK AVAILABLE



SECURE SYNC



SERVER STORAGE

If synchronization fails:

UPLOAD FAILED



KEEP LOCAL COPY



RETRY LATER

The system should prevent duplicate uploads during synchronization.

49.20 Evidence Synchronization

A synchronization mechanism can use unique IDs and upload status.

IMAGE



LOCAL ID

↓

UPLOAD

↓

SERVER ID

↓

SYNC COMPLETE

Possible statuses:

- PENDING
- UPLOADING
- SYNCED
- FAILED
- RETRY

49.21 Evidence Storage Structure

A logical storage structure can be:

INSPECTION/

|

└─ INS-00125/

|

└─ ORIGINAL/

|

└─ IMG-001.jpg

|

└─ IMG-002.jpg

|

└─ PROCESSED/

|

└─ IMG-001-P.jpg

|

└─ IMG-002-P.jpg

|

└─ OCR/

|

└─ OCR-001.json

|

└─ OCR-002.json

|

└─ REPORT/

└─ REPORT-001.pdf

This is a conceptual storage structure; the actual implementation can use object storage or another secure storage mechanism.

49.22 Evidence Retrieval

An officer should be able to retrieve evidence quickly.

Example:

SEARCH

Inspection ID



INSPECTION



FINDING



EVIDENCE



ORIGINAL IMAGE

The officer should not have to manually search through unrelated files.

49.23 Evidence for Each Finding

Each finding should contain:

Finding ID



Requirement



Observed Value



Rule Version



Image ID



OCR Text



Evidence Region



AI Confidence



Officer Decision

This creates an end-to-end evidence record.

49.24 Tamper-Evident Inspection Record

A stronger future architecture can create a chained integrity record.

Conceptually:

RECORD 1



HASH



RECORD 2



HASH



RECORD 3



HASH

If an earlier record is modified, subsequent integrity checks can detect a mismatch.

This can be implemented without requiring blockchain; a conventional cryptographic audit mechanism may be sufficient for a prototype.

49.25 Privacy and Data Protection

The system should collect only information necessary for the inspection workflow.

Important controls include:

- Authentication
- Authorization
- Encryption in transit
- Secure storage
- Limited access
- Audit logging

- Controlled retention
- Secure deletion according to organizational policy

Sensitive information should not be exposed unnecessarily through reports or dashboards.

49.26 Evidence in Final Report

The final inspection report can contain:

Inspection Information

Inspection ID

Date

Officer

Location

Product Information

Product

Brand

Manufacturer

Package

Quantity

MRP

Findings

Requirement

Observed Information

Result

Rule Reference

Evidence

Image

Highlighted Region

OCR Result

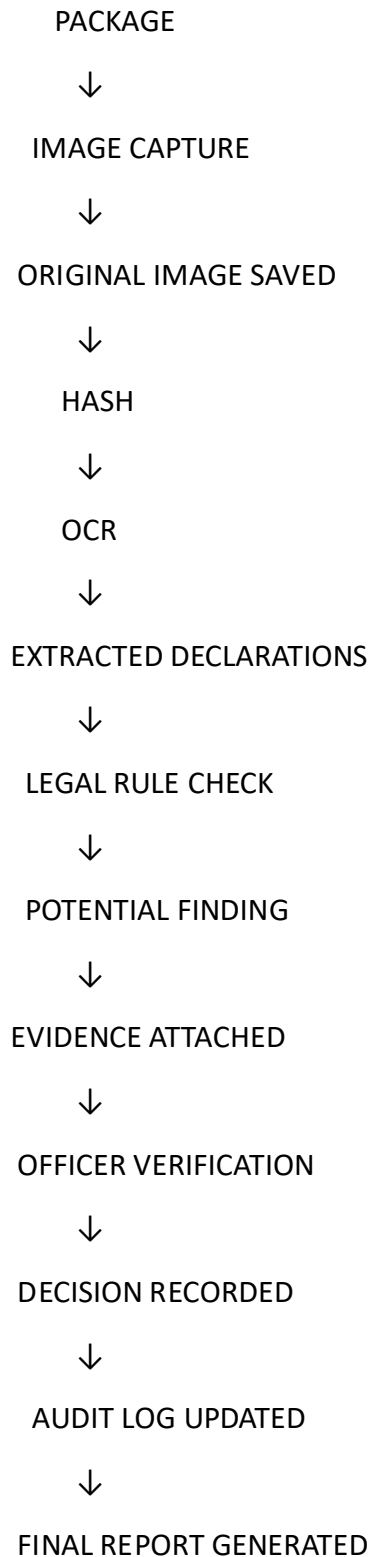
Review

Officer Decision

Remarks

Timestamp

49.27 Complete Evidence Workflow



49.28 Why Evidence Management Is Important

A conventional AI scanner may produce:

✗ NON-COMPLIANT

DrishtiTathya should instead provide: **⚠** POTENTIAL ISSUE

- Requirement:[Relevant requirement]

- Observed:Declaration not detected
- Evidence:Back-side image
- OCR:[Relevant OCR result]
- Rule:[Applicable rule/version]
- Confidence:87%
- Officer Decision:Pending

This makes the result **explainable, reviewable and traceable**.

49.29 Section Conclusion

Evidence Management transforms DrishtiTathya from an AI prediction system into an **evidence-backed inspection platform**.

The fundamental chain is:

Image → OCR → Extracted Fact → Rule → Finding → Evidence → Officer Review → Final Record

The audit trail additionally ensures that important actions and corrections are recorded rather than silently changing the inspection history.

This architecture provides the foundation for **accountability, transparency, reproducibility and future legal/administrative review**.

50. OFFICER DASHBOARD AND USER INTERFACE DESIGN

The **Officer Dashboard and User Interface** is the primary interaction layer of DrishtiTathya. It should allow a Legal Metrology Officer to conduct an inspection with minimum manual effort while still providing complete visibility into the AI findings, applicable rules and supporting evidence.

The interface should follow the principle:

Simple for field operation, detailed for verification.

The officer should be able to move from **inspection → scanning → analysis → review → report** without navigating through unnecessary screens.

50.1 UI Design Objectives

The interface should provide:

- Simple package scanning
- Fast inspection creation
- Clear compliance results
- Evidence-based findings
- AI confidence visibility
- Manual correction capability
- Rule reference access

- Inspection history
 - Offline operation
 - Report generation
 - Role-based access
-

50.2 Overall Application Flow

LOGIN



OFFICER DASHBOARD



NEW INSPECTION



PRODUCT DETAILS



SCAN PACKAGE



CAPTURE IMAGES



OCR + AI ANALYSIS



COMPLIANCE RESULTS



REVIEW FINDINGS



OFFICER DECISION



GENERATE REPORT

50.3 Login Screen

The login screen provides secure access to the application.

The application should authenticate the user before allowing access to inspection data.

50.4 Officer Dashboard

The dashboard provides an overview of current inspection activity.

The dashboard should prioritize actions that require immediate attention.

50.5 New Inspection Screen

When the officer starts an inspection, basic information is recorded.

The system should automatically generate the inspection ID.

50.6 Package Scanning Screen

This is one of the most important screens.

The system can guide the officer to capture different package panels.

50.7 Image Quality Warning

If the captured image is poor, the application should notify the officer.

Example:

This reduces avoidable OCR errors.

50.8 OCR Processing Screen

After image capture:

The processing status helps the officer understand that the system is working.

50.9 Extracted Information Screen

The extracted information should be displayed in an editable format.

The officer can correct an AI extraction before compliance evaluation or during review.

50.10 Confidence Indicators

Each AI-extracted field can display its confidence.

Example:

Net Quantity

500 g



MRP

₹120



Manufacturer

ABC Foods



Low-confidence fields can be highlighted for manual verification.

50.11 Compliance Result Screen

After the rule engine completes its analysis:

The interface should avoid overwhelming the officer with raw technical information.

50.12 Potential Issue Screen

When the system detects a potential issue:

This is the central review interface.

50.13 Evidence Viewer

When the officer selects **View Image**, the application should display the relevant package image.

The relevant text region can be highlighted.

50.14 Rule Reference Screen

The officer should be able to see the legal basis for a finding.

This makes the system explainable to the officer.

50.15 Officer Decision Screen

After reviewing the evidence:

The decision and remarks should be stored in the audit trail.

50.16 Inspection Summary Screen

After all findings have been reviewed:

50.17 Report Generation

The system can generate a structured inspection report.

INSPECTION REPORT

- |
- |— Inspection Details
- |— Product Details
- |— Package Images
- |— Extracted Declarations
- |— Compliance Checks
- |— Potential Issues
- |— Evidence
- |— Rule References
- └— Officer Decisions

The report should clearly distinguish AI-generated information from officer-confirmed findings.

50.18 Inspection History

Officers should be able to search previous inspections.

Filters can include:

- Date
- Product
- Brand
- Category
- Status
- Potential issue
- Officer

50.19 Officer Dashboard — Analytics

The dashboard can later provide higher-level analytics.

Example:

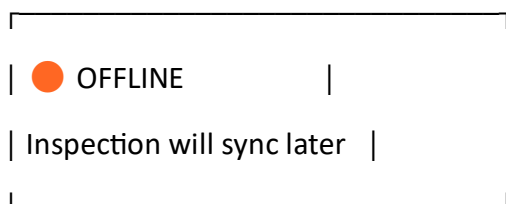
This can help authorities identify areas requiring greater attention.

50.20 Offline Indicator

Because field connectivity may be unreliable, the application should clearly indicate its current state.



or:



The officer should still be able to continue supported inspection activities offline.

50.21 Sync Status

After connectivity is restored:

OFFLINE INSPECTION



LOCAL STORAGE



NETWORK AVAILABLE



SYNCING...



✓ SYNCHRONIZED

A sync indicator should make the status visible to the officer.

50.22 User Experience Principles

The application should follow these principles:

1. Minimum Typing

Use OCR and AI to reduce manual entry.

2. Clear Results

Use simple statuses such as:

PASS / POTENTIAL ISSUE / REVIEW REQUIRED

3. Evidence First

Every finding should have an accessible evidence source.

4. Explainability

The officer should be able to see why a finding was generated.

5. Human Control

The officer must be able to correct or reject AI findings.

6. Field Usability

Buttons and text should be large enough for practical mobile use.

50.23 Recommended Screen Architecture

The complete Android navigation can be:

LOGIN

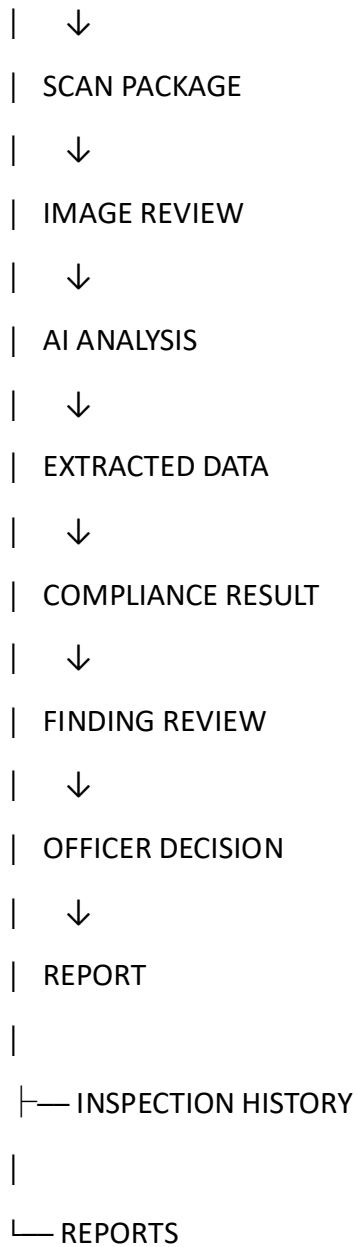


DASHBOARD

| — NEW INSPECTION



| PRODUCT DETAILS



50.24 Consumer Mode — Future Extension

Although the primary interface is designed for officers, a simplified consumer mode can be added later.

SCAN PRODUCT



READ LABEL



KEY INFORMATION



POTENTIAL ISSUES



CONSUMER EXPLANATION

The consumer interface should provide simplified information rather than exposing internal enforcement workflows.

50.25 Manufacturer Mode — Future Extension

A manufacturer/self-audit interface can use the same underlying rule engine.

UPLOAD LABEL



AI EXTRACTION



LEGAL RULE CHECK



POTENTIAL ISSUES

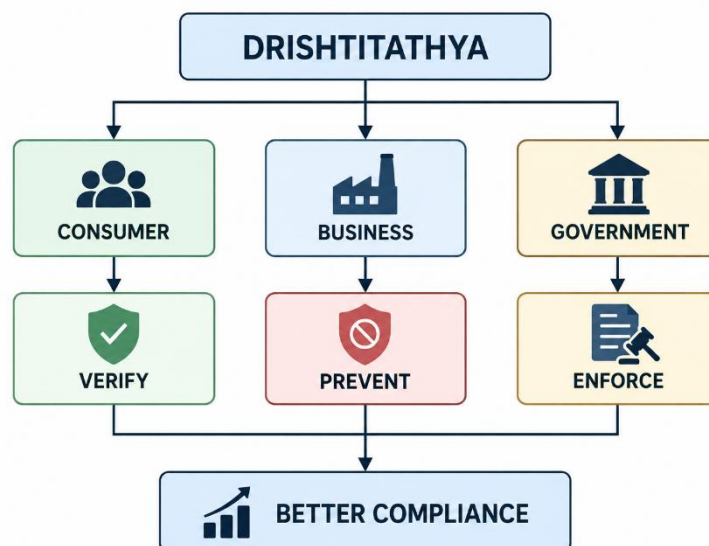


COMPLIANT LABEL SUGGESTIONS

This creates a prevention-oriented component in addition to government enforcement.

50.26 Three-User Ecosystem

A future complete platform can therefore contain:



This gives the project a broader ecosystem vision while keeping the **government inspection workflow** as its core.

50.27 Accessibility

The UI should support:

- Large text
- High readability
- Simple icons
- Voice guidance where feasible
- Multiple Indian languages
- Clear error messages
- Minimal technical terminology

For example, instead of displaying only:

OCR_CONFIDENCE = 0.72

the officer-facing interface should display:

⚠ Needs Verification

Technical information can remain available through an advanced-details option.

50.28 Section Conclusion

The DrishtiTathya interface should make a complex legal-compliance workflow feel simple:

Open → Scan → Analyse → Review → Decide → Report

The officer should not need to understand the internal AI architecture or database structure. At the same time, every important result should remain **explainable, evidence-backed and legally traceable**.

The recommended UI therefore combines:

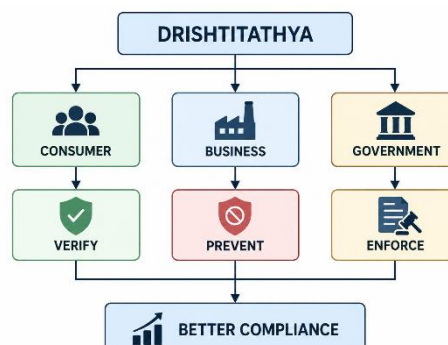
Field Simplicity + AI Assistance + Legal Transparency + Human Control

51. CONSUMER, MANUFACTURER AND GOVERNMENT ECOSYSTEM

DrishtiTathya can be designed as a broader compliance ecosystem connecting **consumers, manufacturers/businesses and government enforcement authorities**.

The core government inspection workflow remains the primary function, while consumer and manufacturer modules can extend the system into a continuous compliance cycle.

The proposed ecosystem is:



51.1 Consumer Module

The Consumer Module allows consumers to scan a packaged commodity and understand important information present on the label.

The objective is **consumer awareness**, not legal enforcement.

Consumer Flow

SCAN PRODUCT



OCR



INFORMATION EXTRACTION



KEY DECLARATIONS



POTENTIAL ISSUES



CONSUMER-FRIENDLY EXPLANATION

51.2 Consumer Scan

A consumer can photograph the package using the mobile application.

The system can extract information such as:

- Product name
- Brand
- Net quantity
- MRP
- Manufacturer/packer/importer
- Country of origin where applicable
- Consumer-care information
- Relevant date information

Example:

┌
| PRODUCT SUMMARY |

Product: ABC Biscuits	
Net Quantity: 500 g	
MRP: ₹120	
Manufacturer: ABC Foods	
Origin: India	

51.3 Consumer-Friendly Compliance Result

Legal rules can be difficult for ordinary consumers to understand.

Therefore, instead of displaying complex legal terminology, the application can provide simple explanations.

Example:

- ✓ MRP detected
- ✓ Net quantity detected
- ✓ Manufacturer information detected

⚠ Consumer-care information

could not be detected.

Please verify the package label.

The system should clearly indicate that this is an **AI-assisted screening result**, not an official enforcement determination.

51.4 Consumer Complaint Support

A future version can allow consumers to submit a potential packaging complaint.

SCAN



POTENTIAL ISSUE



[REPORT ISSUE]



ATTACH IMAGE



ADD REMARK



SUBMIT

The complaint can be assigned a unique reference number.

Complaint ID:

DT-C-2026-00125

51.5 Consumer Evidence

A consumer-submitted report can contain:

PACKAGE IMAGE

+

OCR TEXT

+

DETECTED INFORMATION

+

USER COMMENT

+

TIMESTAMP

This information can potentially become an input for government inspection prioritization, subject to appropriate validation and administrative controls.

51.6 Manufacturer / Business Module

The manufacturer module changes the system from purely reactive enforcement to **preventive compliance**.

A manufacturer can upload a proposed package label before printing.

LABEL DESIGN



UPLOAD



OCR / AI



LEGAL RULE ENGINE



POTENTIAL ISSUES



CORRECTION



RE-SCAN



READY FOR REVIEW

51.7 Pre-Print Label Audit

One of the strongest future features is a **Pre-Print Compliance Audit**.

Before thousands of labels are printed:

PROPOSED LABEL



DRISHTITATHYA CHECK



	✓ Product information	
	✓ Quantity	
	✓ MRP	
	⚠ Declaration issue	
	✓ Consumer care	

This can help businesses identify potential issues before products reach the market.

51.8 Manufacturer Compliance Dashboard

A manufacturer can have a dashboard showing:

This enables businesses to monitor their own compliance activities.

51.9 Label Version Management

Manufacturers frequently modify packaging.

The system can maintain:

PRODUCT

|

|— LABEL V1

|— LABEL V2

|— LABEL V3

└— CURRENT LABEL

Each version can be associated with:

- Upload date
- Changes
- Compliance findings
- Review status

This can help identify whether a potential issue belongs to an older or current label version.

51.10 Government Enforcement Module

The government side remains the central enforcement component.

It can provide:

- Officer inspections
- Inspection history
- Potential issue review
- Evidence management
- Reports
- Analytics
- Risk prioritization
- Rule management
- Amendment updates

FIELD INSPECTION

↓

DRISHTITATHYA

↓

AI ANALYSIS

↓

LEGAL RULE ENGINE



EVIDENCE



OFFICER REVIEW



OFFICIAL WORKFLOW

51.11 Complaint-to-Inspection Workflow

Consumer signals can potentially support targeted inspection.

CONSUMER REPORT



VALIDATION / SCREENING



RISK ANALYSIS



PRIORITY SCORE



OFFICER DASHBOARD



TARGETED INSPECTION



EVIDENCE



OFFICER REVIEW

This creates a feedback loop between consumers and enforcement.

51.12 Risk-Based Inspection

Instead of treating every product equally, the system can calculate an internal inspection-priority indicator based on signals such as:

- Previous confirmed findings
- Repeated potential issues
- Consumer complaints

- Product category
- Geographic concentration
- Historical inspection outcomes
- Package inconsistencies

Conceptually:

MULTIPLE SIGNALS



RISK ANALYSIS



LOW / MEDIUM / HIGH PRIORITY



OFFICER DECISION

The score should be used to **prioritize inspection**, not to automatically accuse a business.

51.13 Geographic Compliance Intelligence

If authorized location information is available for inspections, the government dashboard can visualize patterns.

INSPECTION DATA



GEOGRAPHIC ANALYSIS



AREAS WITH HIGHER SIGNALS



INSPECTION PRIORITIZATION

Example:

REGION A → Low signals

REGION B → Medium signals

REGION C → Higher signals

The dashboard can help authorities allocate inspection resources more effectively.

51.14 Repeat Issue Detection

The system can compare historical inspection records.

INSPECTION 1

Potential Issue



INSPECTION 2

Similar Issue



INSPECTION 3

Similar Issue



REPEAT PATTERN



OFFICER REVIEW

This can help identify recurring compliance concerns.

51.15 Package-to-E-Commerce Comparison

A future extension can compare physical package information with an online product listing.

PHYSICAL PACKAGE

+

ONLINE LISTING



INFORMATION COMPARISON



MRP	Match ✓
Quantity	Match ✓
Origin	Conflict



REVIEW

This is particularly relevant as packaged commodities are increasingly sold through e-commerce channels.

51.16 E-Commerce Compliance

The platform can eventually support e-commerce listing analysis.

Possible workflow:

ONLINE PRODUCT LISTING



TEXT / IMAGE EXTRACTION



STRUCTURED INFORMATION



LEGAL RULE ENGINE

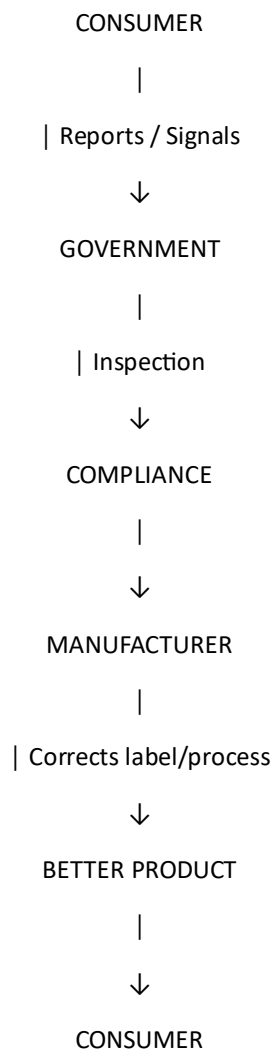


POTENTIAL ISSUE

The system can also support checking requirements that specifically apply to e-commerce entities where applicable under the current rule version.

51.17 Three-Sided Feedback Loop

The complete ecosystem becomes:



This creates a continuous improvement cycle.

51.18 Consumer → Government

Consumers can provide signals about potentially problematic packages.

Consumer



Scan / Report



Evidence



Government Review

Importantly, consumer reports should be treated as **inputs for review**, not automatically as established violations.

51.19 Government → Manufacturer

Government inspection findings can identify areas requiring corrective action through the appropriate official process.

Inspection



Officer Review



Finding



Official Action / Communication

DrishtiTathya can maintain the digital record supporting the workflow.

51.20 Manufacturer → Consumer

Manufacturers can use the system to improve their packaging before products reach consumers.

LABEL DESIGN



SELF-AUDIT



CORRECTION



FINAL LABEL

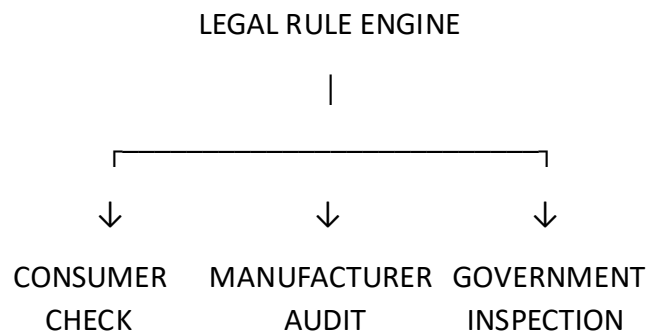


CONSUMER

This creates a preventive layer.

51.21 Shared Legal Rule Engine

All three modules can use the same underlying rule engine.



This avoids maintaining separate interpretations of the same legal requirements.

51.22 Different Interfaces, Same Core

Although the three users have different needs:

User	Primary Goal
Consumer	Understand and verify
Manufacturer	Detect and prevent
Government Officer	Inspect and enforce

They can share:

- OCR
- AI extraction
- Product database
- Legal rule engine
- Evidence framework

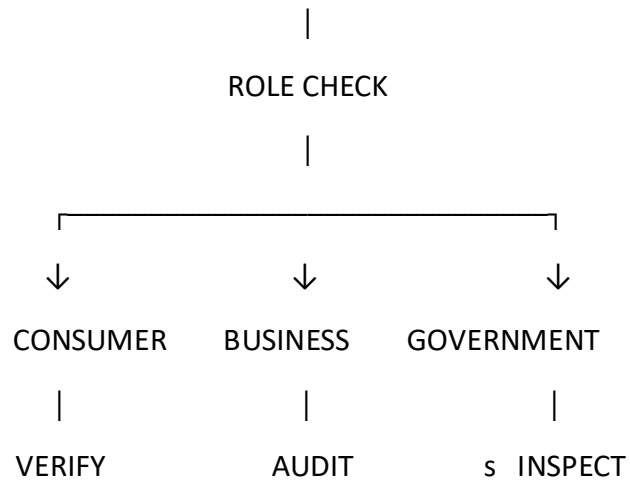
Only the interface and permissions change.

51.23 Role-Based Architecture

DRISHTITATHYA



AUTHENTICATION



This prevents users from accessing functions outside their role.

51.24 Potential Benefits

For Consumers

- Better label awareness
- Easier understanding of package information
- Potential issue reporting
- Greater transparency

For Manufacturers

- Early detection of packaging issues
- Reduced manual checking
- Pre-print label audit
- Version management
- Internal compliance monitoring

For Government

- Faster inspections
 - Evidence-backed findings
 - Digital inspection records
 - Risk-based prioritization
 - Historical analysis
 - Better use of officer resources
-

51.25 Strategic Value of the Ecosystem

A traditional compliance application may focus only on:

OFFICER → INSPECTION → REPORT

DrishtiTathya can evolve into:



This makes the project more scalable and socially impactful.

51.26 Important Governance Principle

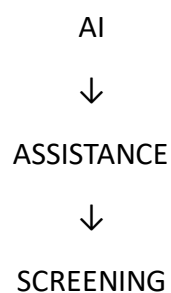
The ecosystem must maintain clear separation between:

AI-generated information

and

official legal decisions.

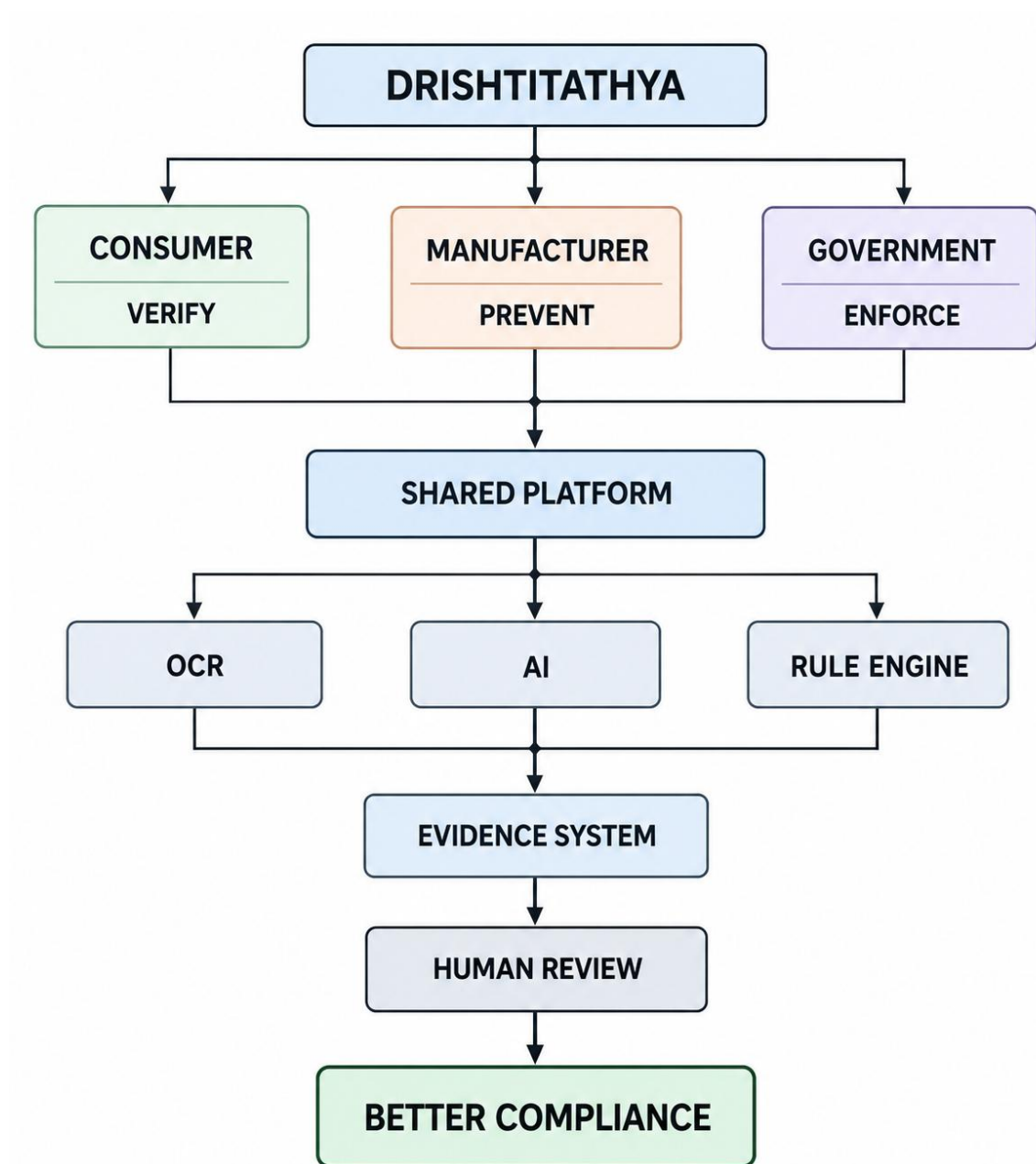
Therefore:





Consumer complaints and AI risk scores should similarly be treated as **signals**, not automatic proof of an offence.

51.27 Final Ecosystem Architecture



51.28 Section Conclusion

The long-term vision of DrishtiTathya is not limited to scanning a package.

It can become a **three-sided compliance ecosystem**:

Consumer → Verify

Manufacturer → Prevent

Government → Enforce

All three can operate on a shared foundation of **AI-assisted information extraction, version-aware legal rules and evidence management**.

This creates a continuous cycle:

Detect → Verify → Prevent → Enforce → Improve

52. ADVANCED AND UNIQUE FEATURES OF DRISHTITATHYA

DrishtiTathya should go beyond a conventional **OCR + checklist application**. The advanced features proposed in this section are designed to make the system more useful for field officers, manufacturers, consumers and enforcement authorities.

The central idea is:

Do not only detect what is written on a package. Understand the package, connect it with the law, compare it with available evidence, identify patterns and help the officer decide what requires attention.

52.1 Feature Classification

The advanced features can be divided into five groups:

Category	Purpose
AI Intelligence	Better understanding of package information
Legal Intelligence	Version-aware and exception-aware checking
Evidence Intelligence	Explainable and traceable findings
Inspection Intelligence	Identify where inspection attention is needed
Preventive Intelligence	Help businesses prevent potential issues

52.2 Version-Aware Legal Rule Engine

This is one of the strongest differentiating features.

Instead of treating the Rules as a fixed checklist, DrishtiTathya stores:

RULE



AMENDMENTS



EFFECTIVE DATES



APPLICABILITY



EXCEPTIONS



CURRENT REQUIREMENT

When an inspection is performed, the system identifies the relevant rule version.

Benefit

The same package information can potentially produce different rule evaluations depending on the applicable legal version.

This makes the system **legally maintainable** as the Rules evolve.

52.3 Explainable Compliance

Most AI systems provide a prediction.

DrishtiTathya should provide an explanation.

Instead of:

✗ NON-COMPLIANT

it should show:⚠ POTENTIAL ISSUE

- Requirement:[Applicable requirement]
- Observed:[Detected information]
- Evidence:[Package image]
- Rule:[Applicable rule/version]
- Reason:[Explanation]
- Confidence:87%
- Action:Officer Review Required

This is particularly useful in a government inspection environment.

52.4 Package-to-Online Listing Comparison

A highly useful advanced feature is comparison between the **physical package** and an **online product listing**.

PHYSICAL PACKAGE



ONLINE LISTING



EXTRACT INFORMATION



COMPARE



Product	✓ Match	
Quantity	✓ Match	
MRP	⚠ Differ	
Origin	⚠ Differ	



REVIEW

This can help identify inconsistencies between physical packaging and online representations.

52.5 Multi-Image Package Intelligence

Instead of processing photographs independently, the system creates a combined package representation.

FRONT

|
└─ Product Name
|

BACK

|
└─ Quantity
└─ MRP
|

SIDE

|
└─ Manufacturer
└─ Consumer Care



COMBINED PACKAGE PROFILE

This reduces incomplete analysis when declarations are distributed across different package panels.

52.6 Contradiction Detection

The system can identify conflicting information.

Example:

Image 1:

MRP = ₹120

Image 2:

MRP = ₹150



CONTRADICTION DETECTED



⚠ REVIEW REQUIRED

Similar comparisons can be performed for:

- Net quantity
- Manufacturer
- Country of origin
- Dates
- Product name
- Batch information

52.7 Fake / Duplicate Label Detection

A future version can compare package designs and label information against previously inspected products.

NEW PACKAGE



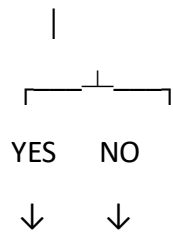
IMAGE + TEXT FEATURES



DATABASE COMPARISON



SIMILAR EXISTING LABEL?



REVIEW NORMAL

This should be presented as a **similarity or anomaly signal**, not an automatic declaration that a product is counterfeit.

52.8 Compliance History

The system can maintain a historical profile for products or businesses.

Example:

PRODUCT / BUSINESS



Inspection 1



Inspection 2



Inspection 3



Historical Pattern

The dashboard could show:

Previous Inspections: 8

Potential Issues: 3

Confirmed Findings: [recorded through official workflow]

Last Inspection: [date]

This helps officers understand inspection history before conducting a new inspection.

52.9 Repeat-Issue Detection

The system can identify recurring types of potential issues.

Inspection 1 → Quantity issue

Inspection 2 → Quantity issue

Inspection 3 → Quantity issue



REPEATED PATTERN



OFFICER ATTENTION

This can support targeted inspection planning.

52.10 Risk-Based Inspection Prioritization

DrishtiTathya can generate an internal **inspection-priority score** using multiple signals.

Possible inputs:

Previous Inspection History

Consumer Signals

Repeated Potential Issues

Product Category

Geographic Signals

Package Inconsistencies



RISK ANALYSIS



PRIORITY

Example:

- LOW PRIORITY
- MEDIUM PRIORITY
- HIGHER PRIORITY

This score should support resource allocation and **should not itself establish legal wrongdoing**.

52.11 Geographic Anomaly Detection

Where authorized inspection-location data is available, the system can identify clusters.

INSPECTION DATA



GEOGRAPHIC ANALYSIS



CLUSTER DETECTION



UNUSUAL PATTERN



INSPECTION PLANNING

For example, authorities could identify areas where similar potential issues repeatedly appear.

52.12 Smart Inspection Planning

Instead of randomly selecting inspection targets, future versions could help officers prioritize available resources.

AVAILABLE RESOURCES

+

INSPECTION PRIORITY

+

LOCATION



INSPECTION PLAN

This can eventually support route planning and workload distribution.

52.13 Pre-Print Compliance Audit

Manufacturers can upload a proposed package design before printing.

LABEL DESIGN



OCR + AI



RULE ENGINE



POTENTIAL ISSUES



CORRECT LABEL



RECHECK

This turns DrishtiTathya into a **preventive compliance tool**, rather than only an enforcement tool.

52.14 Auto-Generated Compliance Checklist

Instead of showing the officer the same checklist for every product, the system generates a dynamic checklist.

PACKAGE FACTS



APPLICABILITY



RULE ENGINE



DYNAMIC CHECKLIST

Example:

✓ Product information

✓ Quantity

✓ MRP

✓ Manufacturer information

✓ Consumer care

⚠ Special requirement

— Not applicable requirement

This reduces unnecessary checks.

52.15 Compliance Heatmap

The government dashboard can visualize inspection patterns.

INSPECTION DATA



CATEGORY



LOCATION



TIME



COMPLIANCE HEATMAP

The purpose is to identify **patterns requiring investigation**, rather than automatically labeling an area or business as non-compliant.

52.16 Consumer Trust / Observed Compliance Score

A future consumer-facing feature can summarize observed information.

For example: LABEL INFORMATION

✓ MRP detected

✓ Quantity detected

✓ Manufacturer detected

✓ Consumer care detected

Observed label checks: 4 / 4

A more advanced score can be calculated, but it should clearly be labeled as a **system-generated indicator**, not a government certification.

52.17 Shelf-to-Receipt Comparison

A future consumer feature can compare:

SHELF / PACKAGE

↓

MRP

↓

RECEIPT

↓

PAID PRICE

↓

COMPARISON

Example:

Package MRP: ₹120

Receipt Price: ₹120

Status: ✓

A discrepancy could be presented as a consumer alert requiring verification.

52.18 Unit-Price Intelligence

DrishtiTathya can help consumers compare products using normalized unit prices.

Example:

Product A

₹100 / 500 g



₹20 / 100 g

Product B

₹180 / 1 kg



₹18 / 100 g

This helps consumers understand actual price differences rather than comparing package prices alone.

52.19 Label Translation Consistency

For multilingual packages, the system can compare declarations across languages.

ENGLISH

Net Quantity: 500 g



HINDI

[Corresponding declaration]



CONSISTENCY CHECK

Potential inconsistencies can be highlighted for human review.

52.20 Voice-Assisted Inspection

A future version can support voice commands for field officers.

Example:

Officer:

"Show potential issues"



APPLICATION

"Two potential issues require review."

This can reduce interaction with the phone during physical inspections.

52.21 Smart Capture Guidance

The application can intelligently identify missing package views.

Example:

Front ✓

Back ✓

Left ✓

Right X



"Right side may contain additional
information. Capture it?"

This improves evidence completeness.

52.22 Automatic Evidence Selection

When a potential issue is generated, the system can automatically identify the most relevant image.

POTENTIAL ISSUE



SEARCH OCR REGIONS



FIND RELEVANT IMAGE



HIGHLIGHT REGION



SHOW TO OFFICER

This reduces manual evidence searching.

52.23 Smart Report Generation

The system can automatically generate a structured inspection report.

INSPECTION DATA

+

FINDINGS

+

EVIDENCE

+

RULE REFERENCES

+

OFFICER DECISIONS

↓

FINAL REPORT

The report should distinguish clearly between:

AI-generated observations and **officer-confirmed findings**.

52.24 Bulk Multi-SKU Audit

For manufacturers or enforcement teams, multiple products can be processed together.

100 PRODUCTS

↓

BATCH PROCESSING

↓

OCR + AI

↓

RULE ENGINE

↓

PRIORITIZE ISSUES

Example:

100 Labels Scanned

↓

82 No Potential Issues

11 Review Required

7 Potential Issues

This can dramatically reduce repetitive manual checking.

52.25 What-If Policy Simulator

A future administrative feature can simulate the impact of a rule change.

CURRENT RULE

↓

CHANGE CONDITION



SIMULATE



ESTIMATE AFFECTED PRODUCTS



ANALYSE

For example, administrators could examine how a proposed change might affect different categories of packages.

This would be a **decision-support tool**, not a mechanism for changing the law.

52.26 Compliance Cost Calculator

For businesses, the system can estimate the operational impact of correcting labels.

Possible inputs:

- Number of SKUs
- Number of labels
- Printing quantity
- Rework requirement
- Packaging replacement

POTENTIAL ISSUE



AFFECTED SKUs



ESTIMATED REWORK



COST ESTIMATE

This can encourage early correction.

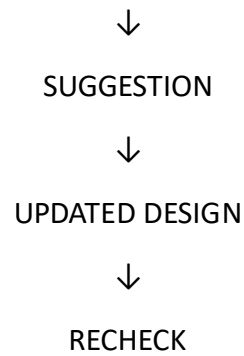
52.27 Packaging Redesign Assistant

A manufacturer could upload a label and receive suggestions such as:

CURRENT LABEL



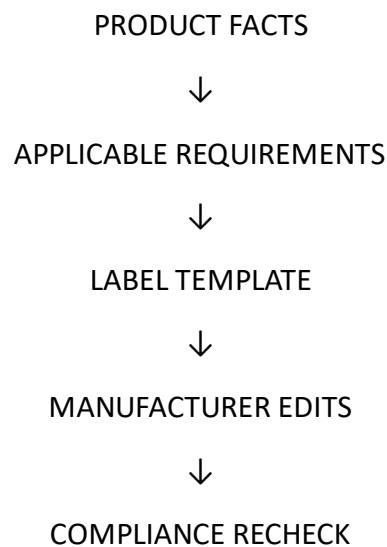
DETECT ISSUE



The system should provide suggestions based on configured legal requirements and not present AI-generated design output as an official approval.

52.28 Auto-Generated Label Template

A future feature can generate a structured template containing the declarations relevant to a particular package.



This could significantly reduce the effort involved in preparing new package labels.

52.29 Compliance Certificate Workflow

A controlled future workflow could allow businesses to request a formal review of a label.

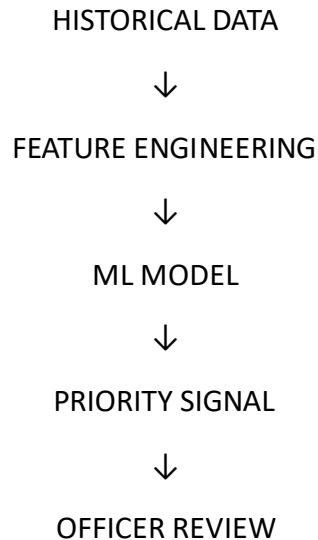


DECISION

Any formal certificate or approval must remain subject to the authority's actual statutory process; the AI system should not independently issue legal approval.

52.30 Predictive Inspection Intelligence

Once sufficient historical data is available, machine-learning models can identify patterns associated with higher inspection priority.



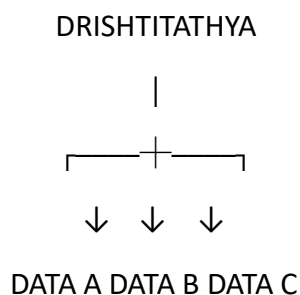
Possible signals could include:

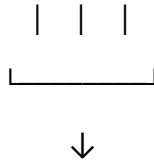
- Repeated potential issues
- Product category
- Inspection history
- Complaint patterns
- Geographic concentration
- Package inconsistencies

The model should remain a **decision-support mechanism**.

52.31 Cross-Department Intelligence

A future version could potentially integrate authorized data sources from relevant government systems.





COMBINED INTELLIGENCE

This could help identify broader patterns while respecting access controls and data-governance requirements.

52.32 Advanced Feature Priority

For the SIH prototype, it is not necessary to implement every advanced feature.

A practical priority order is:

Priority	Feature
5	Version-aware Legal Rule Engine
5	Explainable potential violations
5	Evidence-linked findings
4	Multi-image package analysis
4	Pre-print label audit
4	Package vs online listing comparison
4	Inspection history
3	Risk-based inspection prioritization
3	Repeat-issue detection
3	Geographic intelligence
2	Consumer mode
2	Manufacturer dashboard
2	Advanced predictive ML

52.33 What Makes DrishtiTathya Different?

A conventional solution might be:

IMAGE



OCR

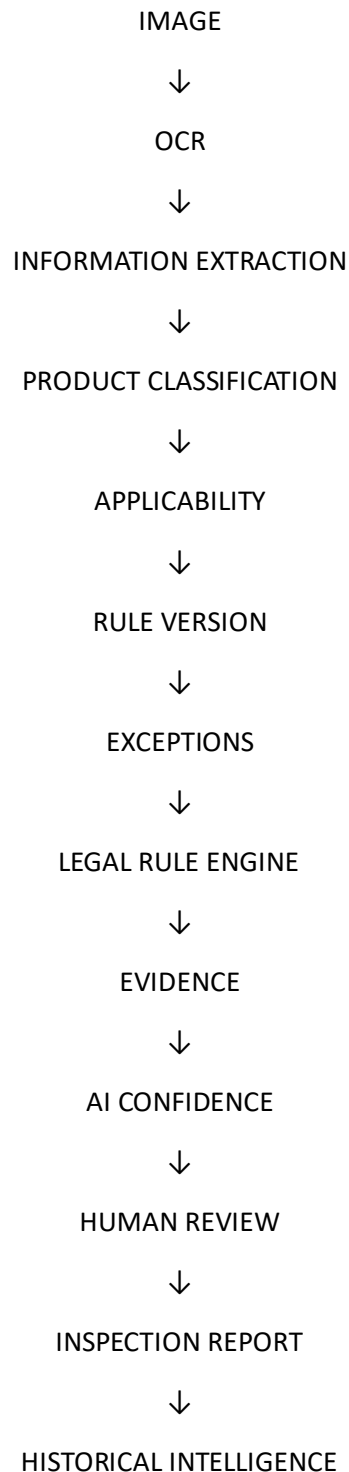


CHECKLIST



RESULT

DrishtiTathya proposes:



This is the key distinction to emphasize during SIH.

52.34 Core Innovation Statement

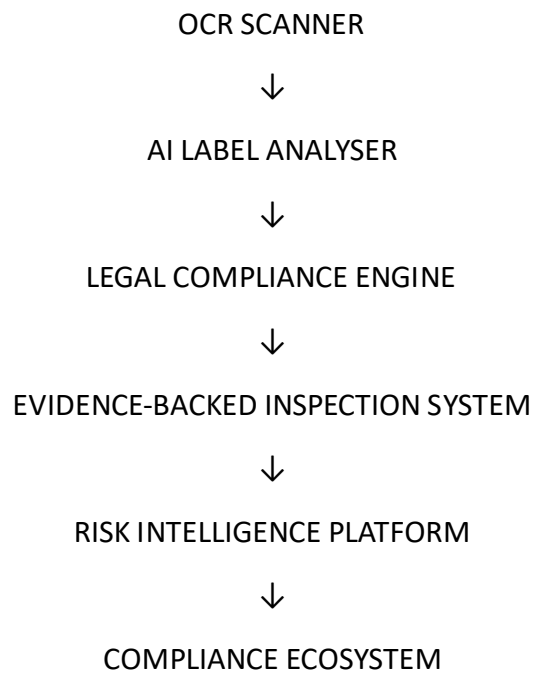
The strongest way to describe the innovation is:

DrishtiTathya is not merely an OCR-based label scanner. It is an AI-assisted, version-aware, evidence-backed Legal Metrology compliance platform that converts package images into structured facts, evaluates them against applicable legal requirements and presents explainable findings for human verification.

52.35 Section Conclusion

The advanced features provide DrishtiTathya with a path from a basic prototype to a complete compliance ecosystem.

The evolution can be represented as:



For the SIH prototype, the strongest combination is:

AI Extraction + Version-Aware Rules + Explainable Findings + Evidence + Human Review

while features such as **predictive inspection, e-commerce comparison, pre-print auditing and risk intelligence** demonstrate the system's scalability and future potential.

53. UNIQUE INNOVATION / USP OF DRISHTITATHYA

The primary innovation of DrishtiTathya is that it does not treat packaged-commodity compliance as a simple **OCR problem**.

Instead, it combines **computer vision, OCR, AI-based information extraction, a version-aware legal rule engine, evidence management and human review** into one integrated system.

The central USP is:

“From seeing the label to understanding the law.”

53.1 Conventional Approach vs DrishtiTathya

A conventional label-scanning system may follow:

PACKAGE IMAGE



OCR



TEXT



CHECKLIST



RESULT

The major limitation is that OCR only answers:

“What text is visible?”

DrishtiTathya goes further:

PACKAGE IMAGE



OCR



INFORMATION EXTRACTION



PACKAGE CLASSIFICATION



APPLICABILITY



RULE VERSION



EXCEPTIONS



LEGAL VALIDATION



EVIDENCE



HUMAN REVIEW

It therefore attempts to answer:

“What information is present, which legal requirements apply, what potential issue was detected, and what evidence supports it?”

53.2 USP 1 — Version-Aware Legal Intelligence

One of the strongest innovations is the **version-aware legal rule engine**.

Legal requirements can change through amendments. Therefore, storing only one static checklist can result in outdated assessments.

DrishtiTathya maintains:

LEGAL RULE



AMENDMENT



EFFECTIVE DATE



RULE VERSION



APPLICABILITY



CURRENT REQUIREMENT

The system can therefore select the relevant rule version based on the inspection context.

Why it is innovative

The project treats law as **versioned software logic**, rather than as a static PDF.

53.3 USP 2 — AI + Rule Engine Separation

A major architectural innovation is separating AI interpretation from legal interpretation.

AI



"What is written on the package?"



STRUCTURED FACTS



RULE ENGINE



"What does the applicable law require?"

This prevents an AI model from being directly responsible for making a legal conclusion.

Result

AI = INFORMATION INTERPRETATION

RULE ENGINE = LEGAL EVALUATION

OFFICER = FINAL REVIEW

This is a much safer architecture for a government-oriented application.

53.4 USP 3 — Explainable Compliance

Instead of providing only:

 FAILED

DrishtiTathya can provide:

 POTENTIAL ISSUE

Requirement:

[Requirement]

Observed:

[Extracted information]

Rule:

[Applicable rule/version]

Evidence:

[Package image]

Confidence:

[AI confidence]

Action:

Officer Review Required

The officer can understand **why** the system produced the finding.

53.5 USP 4 — Evidence-Backed AI

Every AI finding can be connected to its source.

FINDING



EVIDENCE



IMAGE



TEXT REGION



OCR



EXTRACTED FACT

This creates a traceable chain between the physical package and the digital result.

The system therefore moves from:

AI prediction

to:

AI-assisted, evidence-backed finding.

53.6 USP 5 — Human-in-the-Loop Legal Review

DrishtiTathya does not assume that AI is always correct.

The workflow is:

AI



PRELIMINARY FINDING



EVIDENCE



OFFICER REVIEW

└— Confirm

└— Reject

└— Correct

└— Request More Evidence

The officer remains in control of the final inspection decision.

This is particularly important because image quality, OCR errors and unusual package designs can produce uncertain AI outputs.

53.7 USP 6 — Multi-Image Package Understanding

A package may contain declarations on different sides.

DrishtiTathya can combine information from:

FRONT

+

BACK

+

SIDE

+

TOP / BOTTOM

↓

COMPLETE PACKAGE PROFILE

This makes the system more practical than scanning a single photograph.

53.8 USP 7 — Contradiction Detection

DrishtiTathya can compare information extracted from different package images.

Example:

FRONT

MRP = ₹120

BACK

MRP = ₹150

↓

CONFLICT DETECTED

↓

REVIEW REQUIRED

The same mechanism can potentially identify inconsistent:

- Quantity

- MRP
 - Product name
 - Origin
 - Manufacturer information
 - Dates
-

53.9 USP 8 — Dynamic Legal Checklist

The application does not need to show every possible requirement for every product.

Instead:

PACKAGE FACTS



APPLICABILITY



LEGAL RULE ENGINE



DYNAMIC CHECKLIST

The officer sees only the checks relevant to the inspection context.

This reduces unnecessary manual work.

53.10 USP 9 — Pre-Print Compliance

DrishtiTathya can extend from **enforcement** to **prevention**.

A manufacturer can upload a proposed label:

PROPOSED LABEL



AI ANALYSIS



LEGAL RULE ENGINE



POTENTIAL ISSUES



CORRECTION



RECHECK

This allows potential problems to be identified before large quantities of labels are printed.

53.11 USP 10 — Physical Package vs Online Listing

Another advanced feature is comparing physical package information with an online listing.

PHYSICAL PACKAGE

+

ONLINE LISTING

↓

INFORMATION EXTRACTION

↓

COMPARISON

↓

MATCH / CONFLICT

↓

REVIEW

For example:

Net Quantity

Physical: 500 g

Online: 500 g

✓ Match

Country of Origin

Physical: India

Online: Japan

⚠ Conflict

This provides a broader compliance perspective across offline and online commerce.

53.12 USP 11 — Compliance History

The system can build an inspection history.

PRODUCT

↓

INSPECTION 1



INSPECTION 2



INSPECTION 3



HISTORICAL PATTERN

This allows officers to see previous inspection records and recurring potential issues.

53.13 USP 12 — Risk-Based Inspection Intelligence

DrishtiTathya can eventually use historical and operational signals to help prioritize inspections.

HISTORICAL DATA

+

COMPLAINT SIGNALS

+

REPEATED ISSUES

+

OTHER AUTHORIZED DATA



RISK / PRIORITY ANALYSIS



INSPECTION PRIORITY

The important distinction is:

Risk score = inspection prioritization, not proof of violation.

53.14 USP 13 — Offline Field Inspection

Field officers may not always have reliable connectivity.

DrishtiTathya can support:

SCAN



LOCAL STORAGE



AI / SUPPORTED OFFLINE PROCESSING



INSPECTION



NETWORK AVAILABLE



SECURE SYNCHRONIZATION

This makes the system more suitable for real-world field conditions.

53.15 USP 14 — Automatic Evidence Mapping

The system can automatically associate a detected declaration with its location in the package image.

Example:

Net Quantity = 500 g



Image 02



Text Region



Highlighted Evidence

The officer can immediately inspect the relevant portion instead of searching the entire photograph.

53.16 USP 15 — AI Confidence + Review Priority

Not all AI outputs should be treated equally.

Example:

MRP

98% confidence



Normal verification

Manufacturer

61% confidence



REVIEW REQUIRED

The system can prioritize uncertain information for human verification.

This provides a practical connection between **AI uncertainty and officer workload**.

53.17 USP 16 — Consumer → Government Feedback Loop

The long-term platform can connect consumer signals with government inspection.

CONSUMER



POTENTIAL ISSUE REPORT



SCREENING



OFFICER DASHBOARD



TARGETED INSPECTION



EVIDENCE



OFFICIAL REVIEW

This creates a feedback mechanism between market observations and enforcement.

53.18 USP 17 — Manufacturer → Government Prevention Loop

Manufacturers can use the same legal engine before products reach the market.

MANUFACTURER



LABEL SELF-AUDIT



POTENTIAL ISSUE



CORRECTION



COMPLIANT LABEL



MARKET

Thus, the system can support both:

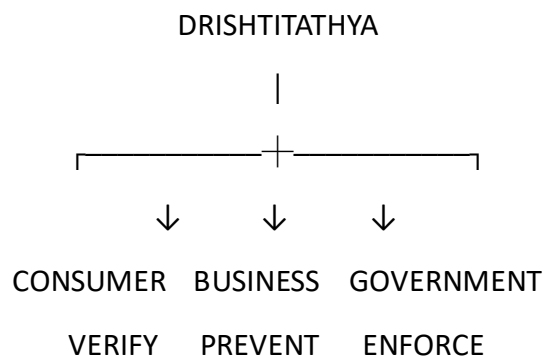
Preventive compliance

and

Enforcement compliance.

53.19 USP 18 — One Platform, Three Users

The same core technology can support:



The underlying:

- OCR
- AI extraction
- Legal Rule Engine
- Evidence system

can remain common.

Only the permissions and interface change.

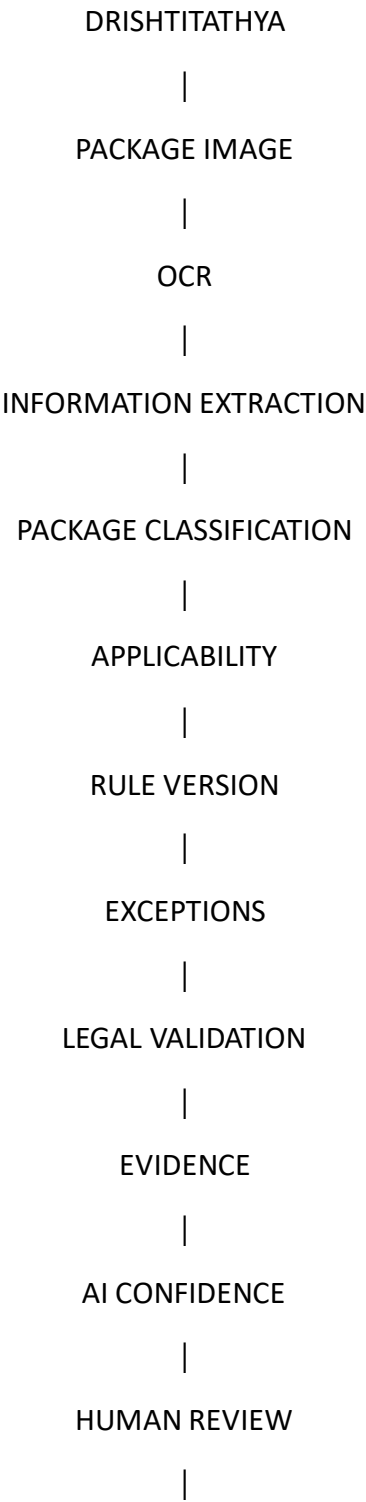
53.20 USP Comparison

Capability	Basic OCR System	DrishtiTathya
Package image scanning	✓	✓
OCR	✓	✓
Declaration extraction	Limited	✓
Product classification	Limited	✓
Applicability analysis	X	✓
Legal rule engine	Limited	✓
Rule versioning	X	✓
Exception handling	X	✓
Evidence mapping	Limited	✓
AI confidence	Limited	✓
Human review	Limited	✓

Multi-image analysis	Limited	✓
Contradiction detection	X	✓
Pre-print audit	X	✓
Online listing comparison	X	✓
Inspection history	Limited	✓
Risk prioritization	X	✓
Offline inspection	Limited	✓

53.21 Core USP Diagram

For the SIH presentation, the following diagram can summarize the innovation:



53.22 Five Core Innovations for SIH

If judges ask:

“What is unique about your project?”

the team can answer with these five points:

1. Version-Aware Legal Rule Engine

The system understands that legal requirements change with amendments and effective dates.

2. Explainable AI

Every potential issue is accompanied by the observed information, legal requirement and supporting evidence.

3. AI + Legal Rules + Human Review

AI extracts information; the rule engine evaluates it; the officer makes the final decision.

4. Evidence-Linked Inspection

Every finding can be traced back to the actual package image and extracted text.

5. From Enforcement to Prevention

The same platform can support government inspection, manufacturer self-audit and consumer verification.

53.23 One-Line USP

For the presentation:

“DrishtiTathya converts a package image into an evidence-backed, version-aware Legal Metrology compliance assessment with human verification.”

53.24 Short SIH Pitch

A 30-second explanation can be:

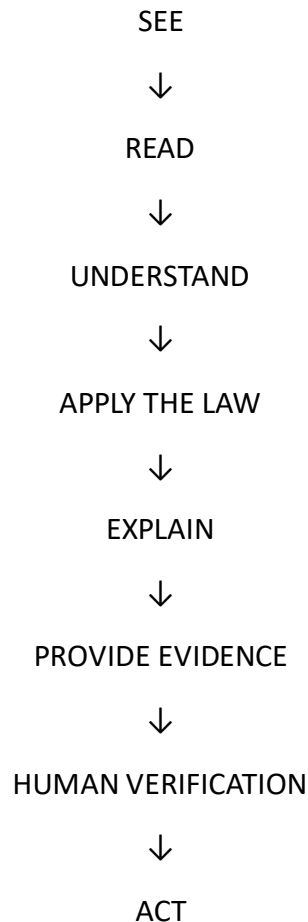
“DrishtiTathya is an AI-assisted Legal Metrology compliance system. Instead of simply performing OCR, it scans the entire package, extracts declarations, classifies the product, determines applicability, selects the relevant version of the law, checks exceptions and evaluates compliance through a legal rule engine. Every potential issue is linked to visual evidence and AI confidence, and the final decision remains with the authorized officer. The platform can further support consumer verification, manufacturer self-audit and risk-based government inspection.”

53.25 Section Conclusion

The central innovation of DrishtiTathya is the combination of:

Computer Vision + OCR + AI + Legal Rule Engine + Version Control + Evidence + Human Review

Its evolution can be summarized as:



Therefore, DrishtiTathya is positioned not as another **OCR scanner**, but as an **AI-assisted Legal Metrology compliance and inspection intelligence platform**.

54. SYSTEM SECURITY, PRIVACY AND DATA GOVERNANCE

Security is an important requirement for DrishtiTathya because the system may handle **product photographs, inspection records, officer information, manufacturer details, complaints and compliance evidence**.

The system should therefore follow a **secure-by-design architecture** in which only authorized users can access or modify sensitive information.

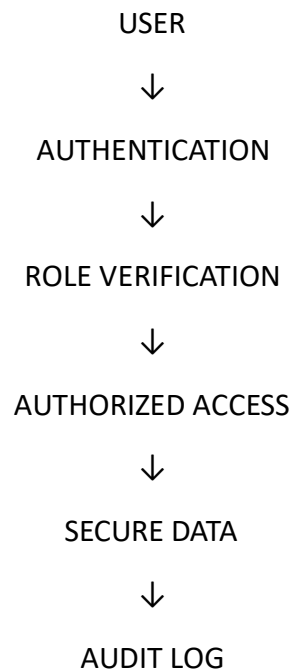
54.1 Security Objectives

The security architecture of DrishtiTathya should provide:

- **Confidentiality** — prevent unauthorized access to data.
- **Integrity** — prevent unauthorized modification of evidence or reports.
- **Availability** — ensure the application remains usable during inspections.
- **Authentication** — verify the identity of users.
- **Authorization** — control what each user is allowed to access.

- **Accountability** — maintain an audit trail of important actions.
- **Evidence protection** — preserve the integrity of inspection photographs and records.

The basic security model is:



54.2 Role-Based Access Control

Different users should have different permissions.

Proposed user roles

Role	Main Permissions
Consumer	Scan package, view general results, submit complaint
Manufacturer	Perform self-audit, manage own products
Inspector / Officer	Conduct inspections, review evidence, create reports
Supervisor	Review cases, approve actions, monitor officers
Administrator	Manage users, rules, system configuration
Legal/Policy Officer	Review and manage legal rule versions

For example, a manufacturer should **not** be able to modify an official inspection report created by an authorized officer.

54.3 Authentication

The application should use secure authentication mechanisms.

Possible implementation:

LOGIN



IDENTITY VERIFICATION



SESSION / TOKEN



ROLE CHECK



APPLICATION ACCESS

For government users, stronger authentication such as **multi-factor authentication (MFA)** can be considered.

54.4 Data Encryption

Sensitive information should be encrypted both:

During transmission

MOBILE APP



ENCRYPTED CONNECTION



SERVER

During storage

DATABASE



ENCRYPTED DATA

Inspection photographs and evidence should receive appropriate protection because modification of evidence could affect the credibility of an inspection.

54.5 Evidence Integrity

Evidence should not simply be stored as an ordinary image.

A stronger approach is:

IMAGE



METADATA



TIMESTAMP



INSPECTION ID



HASH



SECURE STORAGE

A cryptographic hash can be generated for an evidence file.

If the file is later changed:

ORIGINAL HASH $\neq$ CURRENT HASH

The system can flag the evidence for review.

Important principle

The system should preserve evidence integrity rather than allowing AI or users to silently modify the original evidence.

54.6 Original Evidence vs Processed Evidence

The system should distinguish between:

Original evidence

The actual photograph captured by the officer.

Processed evidence

Examples:

- Cropped image
- OCR image
- Highlighted image
- Enhanced image
- Annotated image

The architecture can maintain:

ORIGINAL IMAGE

|

|— OCR VERSION

|— CROPPED VERSION

└— ANNOTATED VERSION

The original image should remain preserved.

54.7 Audit Trail

Important actions should be recorded.

Example:

Officer A



Created Inspection



Captured Image



AI Analysis



Reviewed Finding



Modified Finding



Submitted Report

The system can store:

- User ID
- Action performed
- Date and time
- Inspection ID
- Record affected
- Previous value where appropriate
- New value where appropriate
- Device/session information where appropriate

This creates an **audit trail** for important system actions.

54.8 AI Decision Logging

AI results should also be traceable.

For each AI-assisted finding, the system can store:

AI MODEL



MODEL VERSION



INPUT



EXTRACTED VALUE



CONFIDENCE



RULE APPLIED



RESULT

For example:

Field: MRP

Extracted: ₹120

Confidence: 96%

Applicable Rule:

Rule 6

Result:

Potential issue detected

This makes the AI process more explainable and auditable.

54.9 Legal Rule Security

The legal rule database is a critical component.

Unauthorized modification of a rule could result in incorrect compliance decisions.

Therefore:

LEGAL RULE DATABASE



VERSION CONTROL



AUTHORIZED UPDATE



VALIDATION



PUBLISHED RULE

Each rule version should contain:

- Rule number
- Requirement
- Effective date
- Amendment reference
- Applicability
- Exceptions
- Validation status
- Source document reference

Only authorized administrators or legal/policy users should be able to publish a new legal rule version.

54.10 Rule Version History

Old rule versions should not simply be deleted.

Instead:

RULE V1



AMENDMENT



RULE V2



AMENDMENT



RULE V3

This allows the system to maintain historical traceability.

This is especially important for inspections where the relevant legal requirement depends on the applicable date.

54.11 Privacy by Design

DrishtiTathya should follow the principle of **data minimization**.

The system should collect only information necessary for the intended function.

For example, a package-compliance scan should not unnecessarily collect unrelated personal information.

Proposed principles:

- Collect minimum necessary data.
 - Restrict access according to role.
 - Avoid unnecessary personal information.
 - Define retention periods.
 - Secure stored information.
 - Maintain access logs.
 - Provide appropriate deletion/retention controls according to applicable government requirements.
-

54.12 Secure Offline Mode

Since field inspection may occur in locations with poor connectivity, the application can temporarily store inspection information locally.

OFFLINE



CAPTURE



ENCRYPTED LOCAL STORAGE



NETWORK AVAILABLE



SECURE SYNCHRONIZATION

The system should prevent unauthorized access to locally stored inspection data.

54.13 Secure Synchronization

When an offline device reconnects:

LOCAL RECORD



SYNC VALIDATION



SERVER AUTHENTICATION



DATA TRANSFER



SERVER VERIFICATION



SYNC COMPLETE

Duplicate records and conflicting updates should be detected rather than silently overwritten.

54.14 API Security

The mobile application should communicate with backend services through authenticated APIs.

Example:

ANDROID APP



API GATEWAY



AUTHENTICATION



AUTHORIZATION



APPLICATION SERVER



DATABASE

APIs should include appropriate protections against:

- Unauthorized access
 - Invalid requests
 - Excessive requests
 - Injection attacks
 - Token misuse
 - Data leakage
-

54.15 Database Security

The database should use:

- Role-based database access
- Secure credentials
- Encryption where required
- Input validation
- Backup mechanisms
- Access logging
- Database-level permissions

A user should not directly access the database.

USER



APPLICATION



SECURE API



DATABASE

54.16 Secure Backup and Recovery

Inspection records should be backed up according to the required organizational policy.

Proposed architecture:

PRIMARY DATABASE



SECURE BACKUP



RECOVERY SYSTEM

The system should also periodically test whether backups can actually be restored.

54.17 Privacy of Consumer Complaints

If a consumer complaint feature is implemented, the system should separate:

COMPLAINT INFORMATION

+

PRODUCT INFORMATION

+

OPTIONAL USER INFORMATION

Only information necessary for complaint processing should be exposed to authorized personnel.

Consumer identity should not automatically be exposed to every officer or user.

54.18 Secure Report Generation

Once an inspection report is finalized:

INSPECTION



OFFICER REVIEW



FINAL REPORT



REPORT ID



TIMESTAMP



SECURE STORAGE

If digitally signing reports is supported, it can provide an additional layer of authenticity and integrity.

54.19 Security Monitoring

The backend can monitor unusual activities such as:

- Repeated failed logins
- Unusual access patterns
- Large-scale data downloads
- Repeated rule modifications
- Unauthorized API requests
- Attempts to modify finalized evidence
- Suspicious account activity

Conceptually:

SYSTEM EVENTS



SECURITY MONITORING



ANOMALY DETECTION



ALERT



ADMIN / SECURITY TEAM

54.20 Data Lifecycle

DrishtiTathya should define the complete lifecycle of information:

CAPTURE



PROCESS



STORE



USE



ARCHIVE



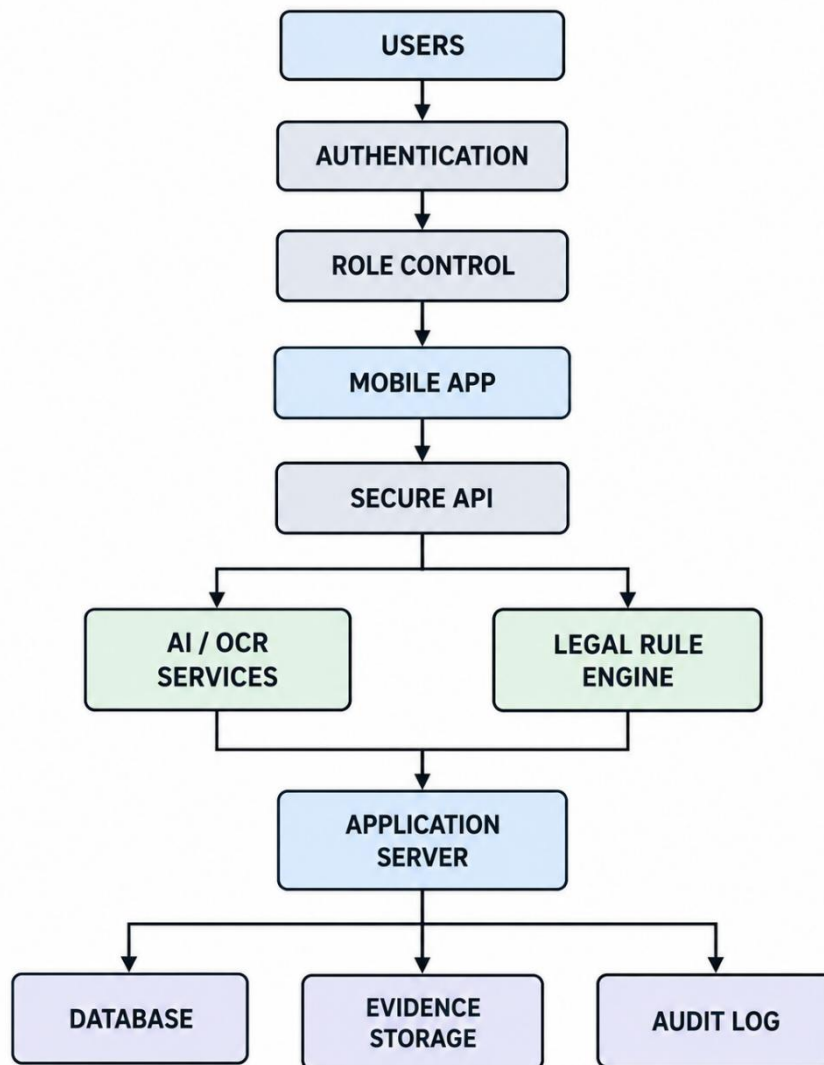
RETENTION / DISPOSAL

Different categories of information may require different retention periods.

The final retention policy should be determined according to the requirements of the implementing authority.

54.21 Security Architecture

The overall security architecture can be represented as:



54.22 Security Principles

The following principles should guide implementation:

Principle	DrishtiTathya Implementation
Least privilege	Users receive only required permissions
Defense in depth	Multiple security layers
Data minimization	Collect only required information
Evidence integrity	Preserve original inspection evidence
Traceability	Maintain audit logs
Secure communication	Protected API communication
Version control	Track legal rule changes
Human oversight	AI findings reviewed by authorized users
Secure storage	Protect inspection and user data
Recovery	Backup and restoration mechanisms

54.23 Security Is Not the Same as Legal Validation

An important distinction should be maintained:

Security protects the system and evidence.

The Legal Rule Engine evaluates applicable requirements.

The authorized officer makes the final enforcement decision.

Therefore:

SECURITY



PROTECTS DATA

AI



EXTRACTS INFORMATION

RULE ENGINE



EVALUATES REQUIREMENTS

OFFICER



FINAL REVIEW / DECISION

This separation improves the reliability and accountability of the overall system.

54.24 Section Conclusion

DrishtiTathya should be designed as a **secure, auditable and privacy-conscious government technology platform**, rather than simply as an image-processing application.

Its security architecture should protect:

Users → Data → Evidence → Legal Rules → Reports → Audit Trail

The core principle is:

“Every action should be authorized, every important change should be traceable, and original evidence should remain protected.”

55. TESTING AND VALIDATION STRATEGY

Testing is essential for DrishtiTathya because the system combines **computer vision, OCR, AI-based information extraction, legal-rule evaluation and human review**.

The objective is not only to check whether the application works technically, but also to determine whether it produces **reliable, explainable and legally traceable compliance results**.

55.1 Testing Objectives

The testing process should verify:

- Accuracy of product image processing.
 - Accuracy of OCR text extraction.
 - Correct extraction of mandatory declarations.
 - Correct identification of package type.
 - Correct selection of applicable rules.
 - Correct handling of exceptions.
 - Detection of missing or potentially incorrect declarations.
 - Accuracy of evidence mapping.
 - Reliability of offline functionality.
 - Security and access control.
 - Performance under realistic workloads.
 - Usability for consumers, manufacturers and officers.
-

55.2 End-to-End Testing Flow

The complete system should be tested as one pipeline:

PRODUCT IMAGE



IMAGE PRE-PROCESSING



OCR



DECLARATION EXTRACTION



PACKAGE CLASSIFICATION



APPLICABILITY CHECK



RULE VERSION SELECTION



LEGAL RULE ENGINE



COMPLIANCE ANALYSIS



EVIDENCE MAPPING



HUMAN REVIEW



FINAL REPORT

Testing each individual module alone is not sufficient. The **complete pipeline must also be validated**.

55.3 Test Dataset Creation

A representative dataset should be created for testing.

It can contain:

- Food packages
- Household products
- Personal-care products
- Imported products
- Different package sizes
- Different label designs
- Multi-language labels
- Damaged labels
- Low-light photographs
- Blurred photographs
- Curved packaging
- Partially visible declarations
- Promotional packages
- Packages with missing declarations
- Packages with potentially incorrect declarations

The dataset should contain both **compliant and intentionally non-compliant examples**.

55.4 Image Processing Testing

The image-processing module should be tested under different conditions.

Test Condition	Expected Result
Clear image	Text region detected
Low light	Enhancement attempted
Slight blur	OCR remains usable where possible
Rotated package	Orientation corrected
Curved label	Important text detected where possible
Partial package image	System requests additional image
Multiple package images	Images combined for analysis

The objective is to ensure that poor image quality does not automatically result in a false compliance decision.

55.5 OCR Testing

OCR performance should be evaluated by comparing extracted text with manually verified ground truth.

Important test cases include:

- English text
- Hindi text
- Mixed-language labels
- Numbers
- ₹ symbol
- Decimal values
- Units such as g, kg, mL and L
- Manufacturer addresses
- Dates
- MRP declarations
- Small fonts
- Text on curved surfaces

A useful metric is:

Character Error Rate (CER)

CER = $\frac{\text{Insertions} + \text{Deletions} + \text{Substitutions}}{\text{Total Characters}}$

Total Characters

A lower CER indicates better OCR performance.

55.6 Declaration Extraction Testing

After OCR, the system must identify important fields.

Example:

OCR TEXT



FIELD EXTRACTION



Product Name	
Manufacturer	
Address	
Net Quantity	
MRP	
Country of Origin	
Consumer Care	
Dates	

Each extracted field should be compared with manually verified values.

55.7 Extraction Accuracy

The following metrics can be used:

Precision

Measures how many detected fields are actually correct.

Recall

Measures how many required fields were successfully detected.

F1 Score

Combines precision and recall.

$F1 = 2 \times \text{Precision} \times \text{Recall}$

Precision + Recall

These metrics can be calculated separately for:

- MRP
- Net quantity
- Manufacturer

- Importer
 - Country of origin
 - Consumer-care information
 - Dates
 - Other declarations
-

55.8 Package Classification Testing

The system should determine which rules apply to the package.

Example classifications:

PACKAGE



Retail Package?	
Imported?	
Industrial/Institutional?	
Special Commodity?	
Container-Type?	

Incorrect classification can lead to incorrect legal evaluation, so this module requires separate validation.

55.9 Legal Rule Engine Testing

The Legal Rule Engine should be tested using **rule-based test cases**.

Example:

INPUT:

Net Quantity = 500 g

MRP = ₹100

Manufacturer = Present

Consumer Care = Missing

RULE ENGINE



Required declaration missing



Potential Non-Compliance

The expected result should be predefined before running the test.

55.10 Rule Version Testing

Because DrishtiTathya is designed to be version-aware, the same package may need to be evaluated against different legal versions.

Test structure:

PACKAGE DATA



EFFECTIVE DATE



RULE VERSION



APPLICABLE REQUIREMENTS



RESULT

Testing should verify that an amendment does not unintentionally alter the result for cases where the amendment is not applicable.

55.11 Exception Testing

Every important exception should have dedicated test cases.

For example:

GENERAL RULE



EXCEPTION CHECK



Applicable?



YES / NO

This is important because a system that only checks general requirements may produce large numbers of false violations.

55.12 Compliance Detection Testing

The system should be tested against different conditions:

Scenario	Expected Output
All required declarations present	Compliant / no issue detected
MRP missing	Potential violation
Net quantity missing	Potential violation
Manufacturer information missing	Potential violation
Country of origin missing where applicable	Potential violation
Declaration unclear	Review required
Image insufficient	Recapture required
Rule exception applicable	Exception applied
AI confidence low	Human review

The system should avoid presenting uncertain AI results as confirmed legal violations.

55.13 False Positive and False Negative Testing

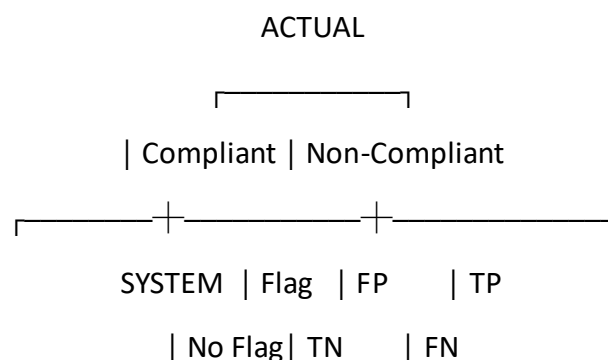
Two important errors must be measured.

False Positive

The system flags a package as potentially non-compliant when it is actually compliant.

False Negative

The system fails to flag a genuine compliance issue.



Both should be monitored.

For legal compliance software, **false negatives are particularly important** because a missed issue may prevent further inspection.

55.14 Confidence-Based Testing

AI should provide confidence values.

Example:

MRP: ₹120

Confidence: 97%

Another example:

- Consumer Care
- Not detected
- Confidence: 54%

→ HUMAN REVIEW REQUIRED

A confidence threshold can determine when manual verification is required.

The threshold should be validated experimentally rather than selected arbitrarily.

55.15 Evidence Validation

Every important finding should be linked to supporting evidence.

Example:

VIOLATION FLAG



RULE 6



EXTRACTED FIELD



IMAGE REGION



ORIGINAL PHOTOGRAPH

Testing should verify that the highlighted region actually supports the generated finding.

55.16 Human-in-the-Loop Testing

The officer review interface should be tested separately.

The officer should be able to:

- View original image.
- View extracted information.
- View applicable rule.
- View highlighted evidence.
- Accept a finding.

- Reject a finding.
- Correct extracted information.
- Request another image.
- Add remarks.
- Finalize the inspection.

The system should record important changes made during review.

55.17 Offline Testing

Offline inspection is an important field capability.

Test cases should include:



Testing should verify that data is not lost when connectivity is interrupted.

55.18 Performance Testing

The system should be tested for:

- Image-processing time
- OCR processing time
- Rule-engine response time
- Report-generation time
- API response time

- Database query performance
- Concurrent users
- Large inspection datasets

Example target metrics can be defined during implementation:

Metric	Target
Image upload	Fast and reliable
OCR processing	Acceptable field response time
Rule evaluation	Near real-time
Report generation	Few seconds
Offline storage	Reliable
Synchronization	No data loss

Exact numerical targets should be finalized after prototype benchmarking.

55.19 Security Testing

Security testing should cover:

- Login security
- Role-based access
- API authentication
- Unauthorized access attempts
- Data encryption
- Evidence modification attempts
- Session management
- Input validation
- Audit-log integrity
- Backup recovery

Example:

MANUFACTURER USER



TRIES TO ACCESS

OFFICER INSPECTION



ACCESS DENIED



SECURITY LOG

55.20 Usability Testing

The application should be tested with representative users.

Consumer

Can the user scan a product and understand the result?

Manufacturer

Can the manufacturer identify what needs to be corrected?

Officer

Can an officer complete an inspection quickly and confidently?

Important usability factors:

- Number of steps
- Clarity of warnings
- Readability
- Navigation
- Language support
- Error messages
- Image-capture guidance
- Report generation

55.21 Field Testing

Laboratory testing alone is insufficient.

A field pilot should test the application under realistic conditions such as:

- Retail stores
- Supermarkets
- Warehouses
- Different lighting conditions
- Different mobile devices
- Poor network connectivity
- Multiple package types
- Multiple label languages

The purpose is to identify problems that may not appear in controlled testing.

55.22 Stress Testing

The backend should be tested under high workload.

Example:

100 USERS



500 SCANS



MANY OCR REQUESTS



RULE EVALUATION



REPORT GENERATION

The system should remain stable and should fail gracefully if resources become unavailable.

55.23 Regression Testing

Whenever a legal rule, AI model or software component changes, previously tested cases should be executed again.

NEW VERSION



REGRESSION TESTS



COMPARE RESULTS



PASS / FAIL

This is particularly important for the Legal Rule Engine.

A new amendment or software update should not accidentally break existing compliance logic.

55.24 Validation Dashboard

A testing dashboard can be created to monitor system performance.

Example:

DRISHTITATHYA VALIDATION DASHBOARD

OCR Accuracy



Declaration Extraction



Rule Evaluation	
Evidence Mapping	
System Reliability	
Offline Sync	

The actual percentages should come from measured testing results and should not be fabricated for the SIH presentation.

55.25 Acceptance Criteria

Before deployment, the system should satisfy predefined acceptance criteria.

Example:

1. Required package fields can be extracted reliably.
2. Applicable rules are selected correctly.
3. Legal-rule versions are handled correctly.
4. Exceptions are applied correctly.
5. Evidence is linked to findings.
6. AI uncertainty is identified.
7. Officers can review and correct AI findings.
8. Offline inspections do not lose data.
9. Unauthorized users cannot access restricted information.
10. Reports maintain traceability.

55.26 Testing Strategy Summary

The complete validation strategy can be represented as:

DATASET TESTING



IMAGE TESTING



OCR TESTING



AI EXTRACTION TESTING



PACKAGE CLASSIFICATION



LEGAL RULE ENGINE



COMPLIANCE DETECTION



EVIDENCE VALIDATION



HUMAN REVIEW



SECURITY TESTING



PERFORMANCE TESTING



FIELD TESTING



FINAL VALIDATION

55.27 Key Principle

The most important testing principle for DrishtiTathya is:

AI accuracy alone does not prove legal accuracy.

The system must validate the complete chain:

Image → Information → Applicable Rule → Compliance Result → Evidence → Human Review

This approach makes the proposed system more suitable for a **government inspection and compliance environment** than a simple OCR-based application.

55.28 Section Conclusion

DrishtiTathya should use a **multi-layer testing strategy** covering AI, OCR, legal rules, security, performance, usability and real-world field conditions.

The final goal is not to claim that the AI is always correct. Instead, the system should ensure that:

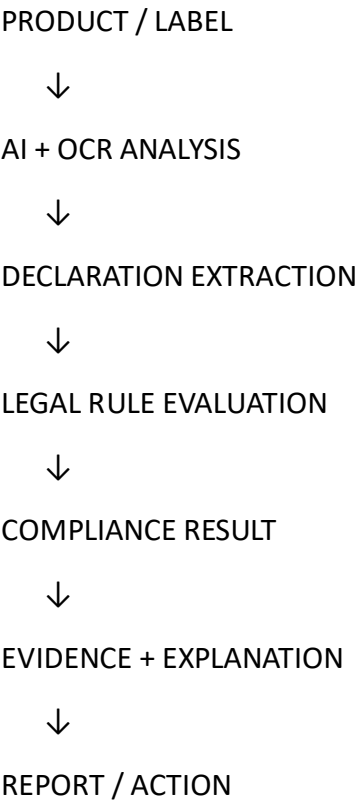
Every automated finding is measurable, explainable, evidence-backed and available for human verification.

56. EXPECTED OUTPUTS AND SYSTEM DELIVERABLES

DrishtiTathya is designed to convert a product image into **structured compliance information, potential violations, supporting evidence and actionable reports**.

The system should provide different outputs depending on whether the user is a **consumer, manufacturer or government officer**.

56.1 Overall Output Flow



The main output categories are:

- 1. Product information
- 2. Declaration checklist
- 3. Compliance result
- 4. Potential violation report
- 5. Evidence report
- 6. Inspection report
- 7. Manufacturer self-audit report
- 8. Consumer report
- 9. Officer dashboard
- 10. Analytics and risk intelligence

56.2 Product Information Output

After scanning a package, the system should organize the extracted information into structured fields.
Example:

Field	Extracted Information
-------	-----------------------

Product Name	XYZ Biscuits
Manufacturer	ABC Foods Pvt. Ltd.
Address	Extracted address
Net Quantity	200 g
MRP	₹80
Country of Origin	India
Consumer Care	Detected
Manufacturing Date	Detected
Best Before	Detected

This converts unstructured package text into machine-readable information.

56.3 Declaration Checklist

The system should display which required declarations were detected.

Example:

DECLARATION CHECKLIST

✓ Manufacturer / Packer

✓ Net Quantity

✓ MRP

✓ Consumer Care

✓ Date Information

✓ Country of Origin

⚠ Review Required: Declaration Readability

This allows users to understand the result without reading the entire OCR output.

56.4 Compliance Result

The application should provide a simple overall result.

Example:

COMPLIANCE STATUS

⚠ REVIEW REQUIRED

Potential issues detected: 2

Fields requiring verification: 1

Evidence available: 4

Possible system statuses:

- **No issue detected**
- **Potential non-compliance**
- **Review required**
- **Insufficient image**
- **Applicable exception**
- **Analysis completed**

The wording should avoid presenting an AI-generated result as a final legal determination.

56.5 Potential Violation Report

For every detected issue, the system should explain:

1. What was detected.
2. Which requirement was considered.
3. What evidence supports the finding.
4. Why the system flagged it.
5. Whether human review is required.

Example: POTENTIAL ISSUE

Field: Consumer Care Information

Observation: Required information was not confidently detected.

Applicable Requirement: Relevant Legal Metrology requirement

Evidence: Image 2 → Label Region B

Confidence: 78%

Action: Officer verification recommended

56.6 Explainable Compliance Output

Instead of simply displaying:

“NON-COMPLIANT”

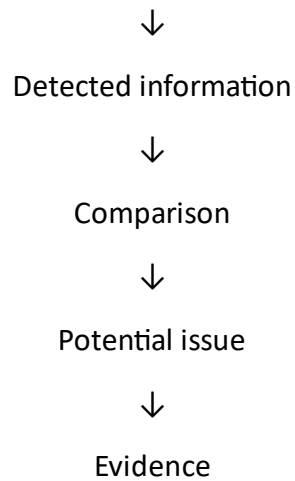
DrishtiTathya should provide an explanation.

WHY WAS THIS FLAGGED?

Requirement



Expected declaration



This is one of the important differences between DrishtiTathya and a basic OCR application.

56.7 Evidence Report

The evidence module should connect findings with the original photographs.

Example:

Finding #01



Applicable Rule



Extracted Field



Highlighted Image Region



Original Photograph



Timestamp / Inspection ID

The officer can therefore verify the finding directly from the package image.

56.8 Multi-Image Evidence

A package often has important information on different sides.

For example:

IMAGE 1 → FRONT

IMAGE 2 → BACK

IMAGE 3 → SIDE

IMAGE 4 → BOTTOM

The system should combine these observations.

Example output:

PACKAGE ANALYSIS

Front:

✓ Product identity

✓ MRP

Back:

✓ Manufacturer

✓ Consumer care

Side:

✓ Net quantity

✓ Date information

Overall:

Package information compiled from 3 images.

56.9 Consumer Output

The consumer interface should be simple and understandable.

Example:

SCAN COMPLETE

Product: XYZ

✓ MRP detected

✓ Net quantity detected

✓ Manufacturer detected

⚠ One declaration needs verification.

[\[View Details\]](#)

[\[Report Issue\]](#)

Consumers should not be required to understand complex legal provisions.

56.10 Consumer Complaint Output

If a complaint facility is implemented:

SCAN PRODUCT



IDENTIFY ISSUE



CAPTURE EVIDENCE



SUBMIT COMPLAINT



COMPLAINT ID



STATUS TRACKING

The consumer can receive a complaint/reference number.

56.11 Manufacturer Self-Audit Report

Manufacturers can use DrishtiTathya before releasing products.

Example:

PRODUCT SELF-AUDIT

Product: XYZ

SKU: 250 g

Declarations:

✓ Manufacturer

✓ Net Quantity

✓ MRP

✓ Consumer Care

Potential Issues:

⚠ Font/readability review

⚠ Country-of-origin verification

Overall:

PRE-PRINT REVIEW REQUIRED

This shifts the system from only **detecting violations** to also **preventing violations**.

56.12 Officer Inspection Report

The government officer interface should generate a structured inspection report.

Suggested structure:

A. Inspection Details

- Inspection ID
- Date and time
- Location
- Officer
- Establishment
- Product information

B. Package Information

- Product name
- Manufacturer/packer/importer
- Net quantity
- MRP
- Country of origin
- Other extracted declarations

C. Compliance Analysis

- Applicable rules
- Detected declarations
- Missing/uncertain declarations
- Potential violations

D. Evidence

- Original photographs
- Highlighted regions
- OCR information
- Evidence references

E. Officer Review

- Accepted findings
- Rejected findings
- Corrections
- Remarks

F. Final Report

- Final inspection status
- Officer comments
- Report ID
- Timestamp

56.13 Inspection History

Every completed inspection can be stored.

INSPECTION HISTORY

ID	PRODUCT	DATE	STATUS

I001	Product A	12/09/26	Review
I002	Product B	12/09/26	Clear
I003	Product C	13/09/26	Flagged

This allows authorized users to review previous inspections.

56.14 Repeat Violation Output

The system can identify repeated issues associated with the same product, manufacturer or establishment.

Example:

REPEAT ISSUE DETECTED

Previous Inspection:

Potential MRP issue

Current Inspection:

Potential MRP issue

Recommendation:

Prior inspection history should be reviewed.

This can help authorities prioritize further inspection.

56.15 Risk Score Output

For government users, the system can generate a risk indicator based on configured factors.

Possible factors:

- Previous inspection findings
- Repeated potential violations
- Consumer complaints
- Product category
- Evidence confidence
- Package/online listing mismatch
- Geographic patterns

Example:

INSPECTION PRIORITY

Risk Level: HIGH

Reasons:

- Repeated issues
- Multiple complaints
- Previous inspection finding

The score should be treated as **decision-support information**, not as a final enforcement decision.

56.16 Geographic Intelligence Output

If location information is legitimately available to the implementing authority, the system can visualize inspection information geographically.

Example:

REGION



INSPECTIONS



POTENTIAL ISSUES



PATTERN ANALYSIS



INSPECTION PRIORITY

A dashboard could show areas with higher concentrations of recorded issues.

56.17 Package vs Online Listing Report

For products sold online, DrishtiTathya can compare package information with the corresponding online listing where such data is available.

Example:

Field	Physical Package	Online Listing
Net Quantity	500 g	500 g
MRP	₹250	₹250
Country of Origin	India	Not shown
Product Name	XYZ	XYZ

The system can flag discrepancies for review.

56.18 Compliance Certificate / Clearance Workflow

For manufacturer self-audits, a workflow can be designed:

SELF-AUDIT



ISSUES CORRECTED



RE-SCAN



NO ISSUE DETECTED



AUTHORIZED REVIEW



CERTIFICATE / RECORD

Any formal certificate should only be issued according to the authority's approved process.

56.19 Automated Report Generation

Instead of manually compiling information, the system can generate a structured report.

SCAN



ANALYSE



REVIEW



GENERATE REPORT



STORE

The report can contain:

- Product details
 - Extracted declarations
 - Applicable rules
 - Potential issues
 - Evidence
 - AI confidence
 - Officer remarks
 - Final review status
-

56.20 Administrator Dashboard

The administrator dashboard can provide system-level information.

Example:

DRISHTITATHYA ADMIN DASHBOARD

- Total Scans
- Total Inspections
- Potential Issues

- Pending Reviews
- Completed Reviews
- Active Users
- Rule Updates
- System Alerts

This helps monitor system usage and operational status.

56.21 Legal Rule Update Output

When a new amendment is incorporated:

NEW AMENDMENT



RULE VERSION CREATED



EFFECTIVE DATE



APPLICABILITY



RULE ENGINE UPDATED



TEST CASES



PUBLISHED VERSION

The system should maintain the previous version rather than silently replacing it.

56.22 Analytics Dashboard

The system can provide aggregated analytics such as:

- Most frequently detected potential issues
- Product categories with more findings
- Common missing declarations
- Repeat findings
- Inspection trends
- Consumer complaint trends
- Manufacturer self-audit trends
- Regional patterns

Example:

COMPLIANCE ANALYTICS

Most Common Potential Issues

- 1. Declaration missing
- 2. Readability concern
- 3. MRP-related issue
- 4. Net quantity issue
- 5. Country-of-origin issue

Actual statistics must be generated from real system data.

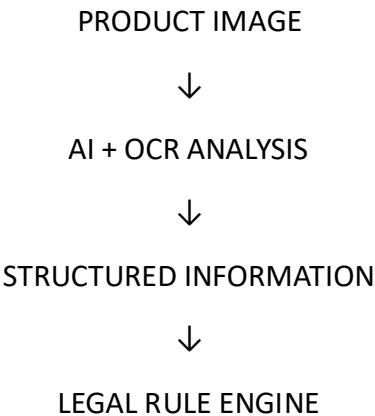
56.23 Expected System Deliverables

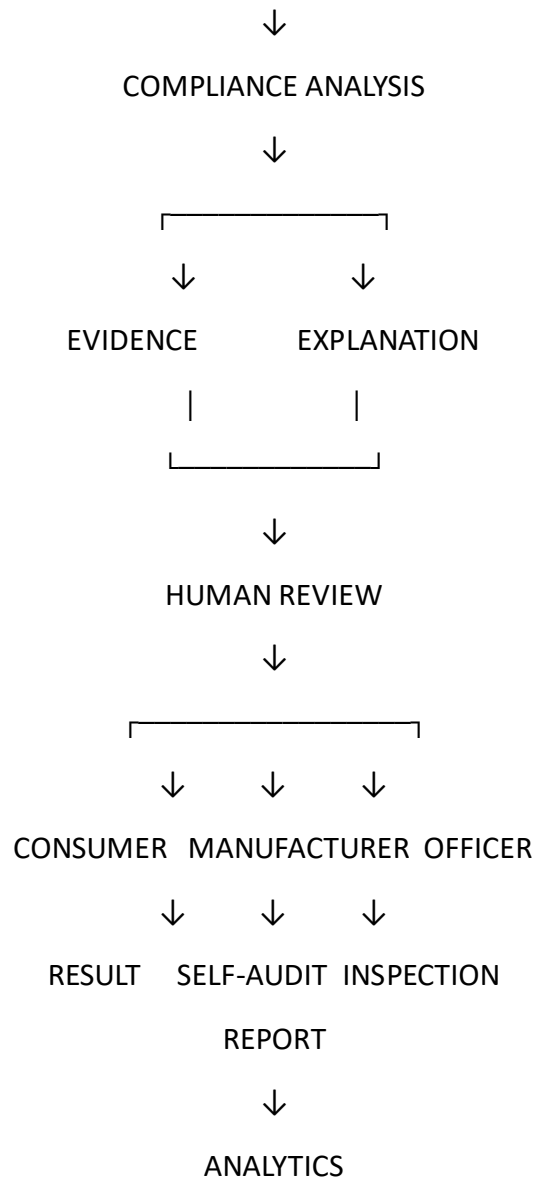
The proposed project can deliver the following major components:

Deliverable	Purpose
Android Application	Product scanning and field inspection
AI/OCR Pipeline	Extract information from package images
Legal Rule Engine	Evaluate applicable requirements
Evidence Module	Preserve and connect supporting evidence
Compliance Report	Summarize findings
Officer Dashboard	Manage inspections
Consumer Interface	Simple package verification
Manufacturer Portal	Self-audit
Admin Panel	Manage users and rules
Analytics Dashboard	Compliance intelligence
Rule Version Database	Maintain amendment history
Inspection Database	Maintain historical records

56.24 Final Output Architecture

The complete output architecture can be represented as:





56.25 Key Value of the Outputs

The important transformation provided by DrishtiTathya is:

Image → Text → Information → Legal Interpretation → Evidence → Action

A basic OCR system stops at:

Image → Text

DrishtiTathya continues beyond OCR and produces **structured, explainable and reviewable compliance information**.

56.26 Section Conclusion

The expected outputs of DrishtiTathya are not limited to a simple **“compliant/non-compliant”** result.

The system is designed to generate:

Structured product information + applicable legal requirements + potential issues + supporting evidence + human review + actionable reports.

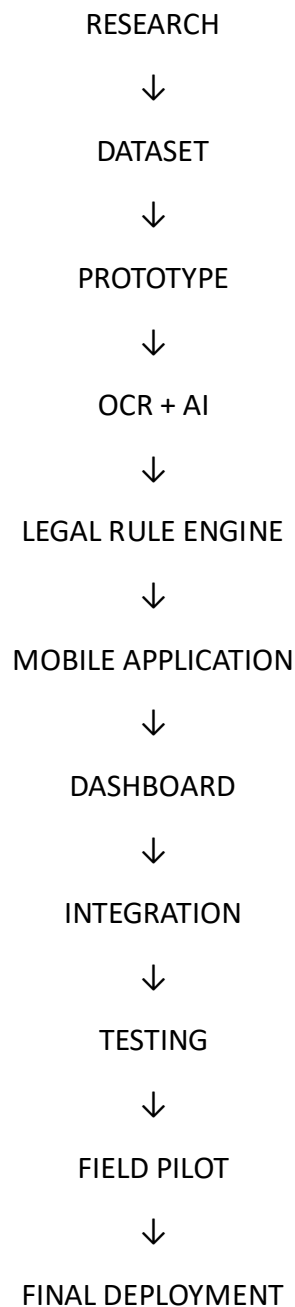
This makes the system useful across the complete ecosystem:

- **Consumer → Verify**
 - **Manufacturer → Prevent**
 - **Government → Inspect & Enforce**
 - **System → Learn & Improve**
-

57. PROJECT IMPLEMENTATION ROADMAP

The implementation of **DrishtiTathya** should be carried out in multiple phases so that the team first develops a reliable core compliance system and then adds advanced AI and intelligence features.

The roadmap follows a practical progression:



57.1 Phase 1 — Legal and Domain Research

The first phase is to understand the Legal Metrology (Packaged Commodities) Rules and convert important legal requirements into structured software requirements.

Activities

- Study the Legal Metrology Act, 2009.
- Study the Packaged Commodities Rules, 2011.
- Study relevant amendments.
- Identify mandatory declarations.
- Identify applicability conditions.
- Identify exemptions.
- Identify special commodity requirements.
- Identify quantity and unit requirements.
- Identify inspection-related requirements.
- Record effective dates of amendments.

Output

Legal Requirement Specification

Rule



Requirement



Condition



Exception



Effective Date



Software Rule

57.2 Phase 2 — Rule Structuring

Legal requirements should be converted into structured rule records.

Example:

Rule ID	Requirement	Condition	Exception	Effective From
R6-X	Required declaration	Retail package	Applicable exceptions	Relevant date

R12-X	Quantity declaration	Applicable package	Schedule/commodity conditions	Relevant date
-------	-------------------------	-----------------------	----------------------------------	---------------

The exact legal interpretation should be validated against the applicable official rule version before implementation.

Output

Version-Aware Legal Rule Database

57.3 Phase 3 — Dataset Preparation

A dataset of package photographs should be prepared.

The dataset should include:

- Different product categories
- Different package sizes
- Different label layouts
- Different languages
- Clear and unclear images
- Front/back/side photographs
- Compliant examples
- Examples containing deliberately introduced issues for testing

Each image should be associated with manually verified information.

IMAGE



GROUND TRUTH



DECLARATIONS



PACKAGE TYPE



EXPECTED RESULT

57.4 Phase 4 — OCR Prototype

The first technical prototype should focus on extracting text from package images.

IMAGE



PRE-PROCESSING



OCR



TEXT

The team should test:

- Rotation correction
- Noise removal
- Contrast enhancement
- Cropping
- Text-region detection
- Numerical recognition
- Multilingual OCR where required

Output

Working Package OCR Prototype

57.5 Phase 5 — AI Declaration Extraction

Raw OCR text is not enough.

The next phase converts OCR output into structured declarations.

OCR TEXT



AI / NLP



STRUCTURED FIELDS

Example:

"Net Qty: 500 g"



Field = Net Quantity

Value = 500

Unit = g

The same process can be applied to MRP, manufacturer information, dates, country of origin and other relevant declarations.

57.6 Phase 6 — Package Classification

The system should determine the type and characteristics of the package before applying legal checks.

Possible classifications include:

RETAIL PACKAGE

IMPORTED PACKAGE

SPECIAL CATEGORY

CONTAINER-TYPE

MULTI-PACK

OTHER / REVIEW REQUIRED

This prevents the system from applying every rule to every package.

57.7 Phase 7 — Legal Rule Engine

This is one of the most important implementation phases.

The extracted package information is supplied to the Legal Rule Engine.

PACKAGE DATA



APPLICABILITY



RULE VERSION



REQUIREMENTS



EXCEPTIONS



COMPLIANCE RESULT

The engine should be **deterministic and explainable** wherever possible.

AI should assist in extracting information, while the rule engine evaluates structured information against configured legal requirements.

57.8 Phase 8 — Compliance Analysis Module

The compliance module combines:

- Extracted declarations
- Package classification
- Applicable legal rules
- Rule version
- Exceptions
- AI confidence
- Evidence

Example:

MRP



Detected



Applicable requirement



Rule evaluation



PASS / FLAG / REVIEW

The system should distinguish between:

Detected issue → Potential violation → Human verification → Final decision

57.9 Phase 9 — Evidence Management

The evidence module should be integrated before finalizing the compliance workflow.

It should store:

- Original photographs
- Processed images
- Relevant image regions
- OCR output
- Inspection ID
- Timestamp
- Finding ID

- Rule reference

Example:

FINDING



RULE



FIELD



IMAGE REGION



ORIGINAL IMAGE

This provides traceability for every important finding.

57.10 Phase 10 — Android Application

The core mobile application can then be developed.

Main screens

LOGIN



HOME



SCAN PRODUCT



CAPTURE IMAGES



ANALYSIS



RESULT



EVIDENCE



REPORT

The interface should prioritize **simple field operation**, especially for inspectors who may need to inspect multiple products quickly.

57.11 Phase 11 — Officer Inspection Workflow

The officer workflow should support:

1. Start inspection.
2. Capture package images.
3. Run OCR/AI analysis.
4. Review extracted declarations.
5. View applicable rules.
6. Examine potential issues.
7. Verify evidence.
8. Add remarks.
9. Finalize inspection.
10. Generate report.

CAPTURE



ANALYSE



REVIEW



VERIFY



REPORT

57.12 Phase 12 — Consumer Interface

A simplified consumer mode can be developed separately.

The consumer should see only the information necessary to understand the package.

Example:

SCAN



PRODUCT INFORMATION



DECLARATION CHECK



SIMPLE RESULT



REPORT ISSUE

The consumer interface should avoid overwhelming users with complex legal terminology.

57.13 Phase 13 — Manufacturer Self-Audit

A manufacturer portal can allow businesses to test packages before production or distribution.

UPLOAD LABEL



ANALYSE



CHECK REQUIREMENTS



IDENTIFY POTENTIAL ISSUES



CORRECT LABEL



RE-SCAN

This creates a **preventive compliance workflow**.

57.14 Phase 14 — Government Dashboard

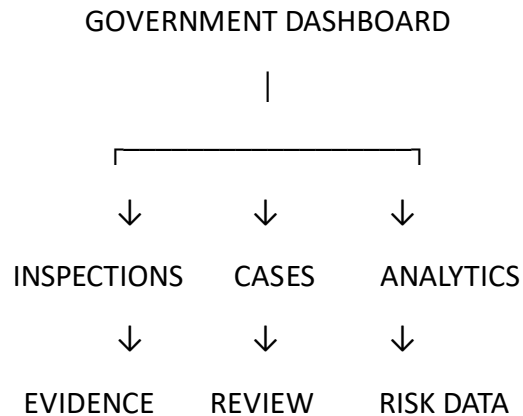
The dashboard should provide authorized officers and administrators with a centralized view.

Main modules

- Active inspections
- Pending reviews
- Completed inspections
- Potential issues
- Inspection history
- Risk indicators
- Complaints
- Rule updates

- Analytics

Example:



57.15 Phase 15 — Offline Inspection

Offline functionality should be integrated into the mobile workflow.

NO INTERNET



CAPTURE



LOCAL SECURE STORAGE



INSPECTION COMPLETED



NETWORK RESTORED



SECURE SYNC

This feature is particularly useful for field environments with unreliable connectivity.

57.16 Phase 16 — Advanced Features

After the core system becomes stable, advanced features can be implemented.

Priority features

1. Multi-image package intelligence

Combine front, back and side images.

2. Contradiction detection

Identify conflicting information across package surfaces.

3. Package vs online listing comparison

Compare physical and digital product information.

4. Risk-based inspection prioritization

Use historical information and configured risk factors.

5. Repeat violation detection

Identify recurring issues.

6. Pre-print compliance audit

Help manufacturers detect issues before printing packaging.

7. Bulk SKU audit

Analyze multiple product labels together.

57.17 Phase 17 — Testing

Testing should occur continuously rather than only at the end.

MODULE



UNIT TEST



INTEGRATION TEST



SYSTEM TEST



USER TEST



FIELD TEST

Important evaluation areas:

- OCR accuracy
- Declaration extraction
- Package classification
- Rule-engine correctness
- Exception handling
- Evidence mapping
- Security

- Performance
 - Offline synchronization
 - Usability
-

57.18 Phase 18 — Pilot Deployment

A controlled pilot should be conducted before large-scale deployment.

The pilot can involve:

- Selected product categories
- Selected package types
- Controlled inspection scenarios
- Representative users
- Different mobile devices
- Different lighting conditions
- Online and offline operation

The pilot should collect feedback on:

- Accuracy
 - Speed
 - Usability
 - False positives
 - False negatives
 - Missing legal rules
 - Evidence quality
 - Officer workflow
-

57.19 Phase 19 — Feedback and Improvement

Pilot results should be used to improve the system.

PILOT



FEEDBACK



ERROR ANALYSIS



MODEL / RULE IMPROVEMENT



RETEST



NEW VERSION

AI model updates and legal rule updates should be handled separately so that a change in one does not unintentionally alter the other.

57.20 Phase 20 — SIH Prototype Finalization

For the SIH prototype, the team should prioritize a **working end-to-end demonstration** instead of trying to implement every advanced feature.

Recommended SIH MVP

IMAGE CAPTURE



OCR



DECLARATION EXTRACTION



PACKAGE CLASSIFICATION



LEGAL RULE ENGINE



COMPLIANCE CHECK



HIGHLIGHTED EVIDENCE



OFFICER REVIEW



REPORT

This demonstrates the core innovation clearly.

57.21 Suggested Development Priority

Priority	Module
----------	--------

P0	Legal Rule Database
P0	OCR
P0	Declaration Extraction
P0	Rule Engine
P0	Compliance Analysis
P0	Evidence Management
P0	Android Scanning
P1	Officer Dashboard
P1	Offline Mode
P1	Multi-image Analysis
P1	Explainable Results
P2	Manufacturer Self-Audit
P2	Consumer Mode
P2	Online Listing Comparison
P2	Risk Intelligence
P3	Predictive Analytics
P3	Policy Simulator
P3	Advanced Intelligence Features

57.22 Team Division

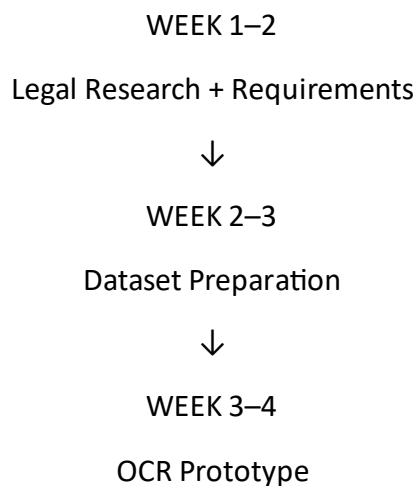
A multidisciplinary student team can divide the implementation into modules.

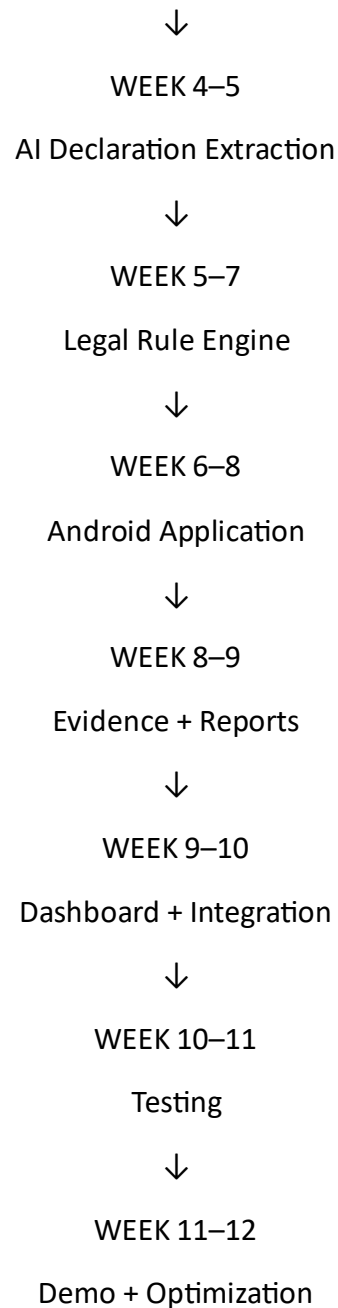
Team Member / Team	Responsibility
Legal Research Team	Rules, amendments, applicability
AI/ML Team	OCR, extraction, classification
Backend Team	APIs, database, rule engine
Android Team	Mobile application
Frontend Team	Dashboard and portals
Security Team	Authentication, access control, evidence security
Testing Team	Dataset, validation and performance
Integration Team	Complete system integration

One person can handle multiple roles depending on team size.

57.23 Implementation Timeline

A practical development sequence can be represented as:



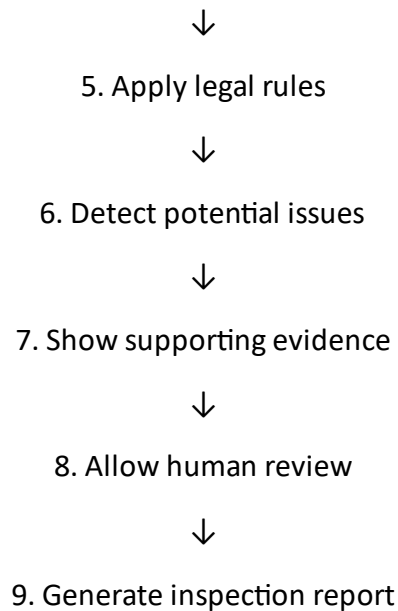


This is a **proposed development schedule**, not a mandatory timeline.

57.24 Minimum Viable Product

The minimum viable DrishtiTathya should successfully demonstrate:

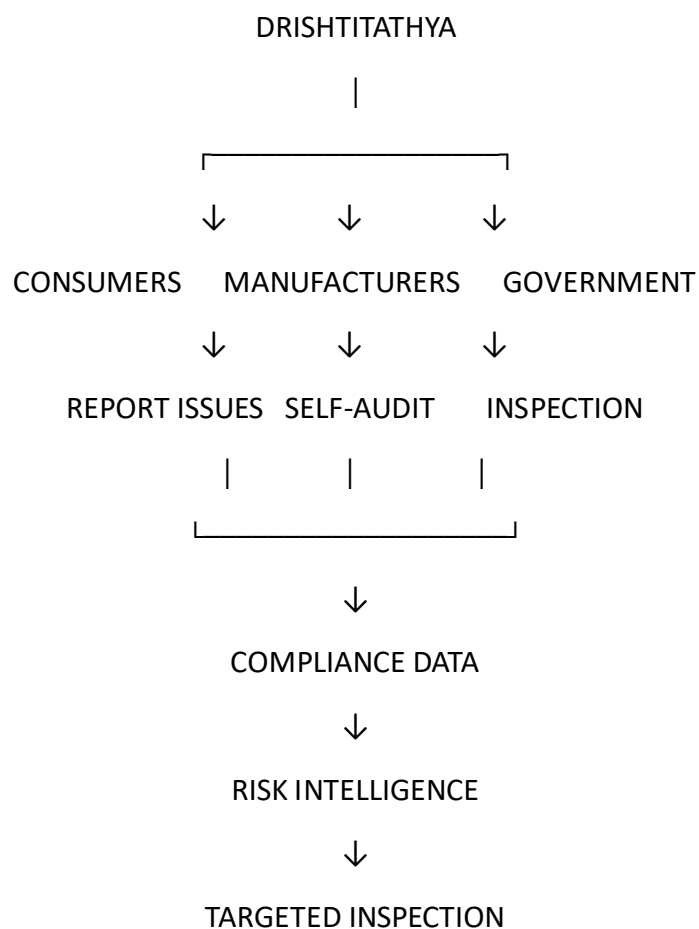
1. Capture product image
- ↓
2. Extract package text
- ↓
3. Identify declarations
- ↓
4. Determine applicable checks



If this complete flow works reliably, the team has demonstrated the **central innovation of the project**.

57.25 Future Expansion

After the core system is validated, DrishtiTathya can be expanded into a larger compliance ecosystem:



This creates a continuous **detect → prevent → inspect → learn** ecosystem.

57.26 Section Conclusion

The implementation of DrishtiTathya should follow a **core-first approach**.

The team should first make the fundamental pipeline reliable:

Image → OCR → Information Extraction → Legal Rule Engine → Evidence → Human Review → Report

Only after this pipeline is stable should advanced features such as predictive risk analysis, online-listing comparison and policy simulation be added.

This approach makes the project **technically achievable, demonstrable during SIH, and scalable for future government deployment**.

58. COST, RESOURCE AND INFRASTRUCTURE REQUIREMENTS

DrishtiTathya is designed as a software-first solution. Therefore, the initial prototype can be developed using **existing student hardware, open-source technologies and limited cloud infrastructure**.

The major resources required are:

- Development computers
- Android smartphones
- Product-label image dataset
- OCR and AI services/models
- Backend server
- Database
- Secure file storage
- Development and testing tools
- Internet connectivity for development and synchronization

58.1 Hardware Requirements

Development Hardware

Hardware	Suggested Requirement
Developer Laptop/Desktop	8 GB RAM minimum, 16 GB recommended
Processor	Modern multi-core CPU
Storage	256 GB minimum, 512 GB recommended
Smartphone	Android device with camera
Internet	Stable broadband/mobile internet
Optional GPU	Useful for local AI model training

A dedicated GPU is **not mandatory for the initial prototype** if pretrained OCR/AI models or cloud inference are used.

58.2 Mobile Device Requirements

The Android application should support commonly available smartphones.

Minimum requirements can include:

- Camera
- Android operating system supported by the selected development framework
- Adequate storage
- Internet connectivity for online synchronization
- Local storage for offline inspection
- GPS/location capability where required and authorized

The application should also work under different camera qualities because field officers may use different devices.

58.3 Server Requirements

The backend can initially be hosted on a cloud server.

The server should support:

- REST APIs
- Authentication
- AI/OCR processing
- Legal Rule Engine
- Database connectivity
- Evidence storage
- Report generation
- Monitoring

A scalable architecture can be used:

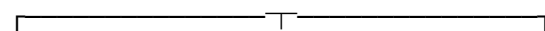
MOBILE APP



API SERVER



APPLICATION SERVICES



DATABASE AI/OCR FILE STORAGE

58.4 Database Requirements

The database stores structured information such as:

- Users
- Products
- Manufacturers
- Inspections
- Extracted declarations
- Legal rules
- Rule versions
- Findings
- Evidence references
- Complaints
- Audit logs

A relational database can be used for structured transactional data.

Example:

USERS



INSPECTIONS



PRODUCTS



DECLARATIONS



FINDINGS



EVIDENCE

58.5 Evidence Storage

Product photographs can require considerably more storage than textual data.

Therefore, images should preferably be stored in dedicated object/file storage while the database stores their metadata and references.

IMAGE



SECURE FILE STORAGE



IMAGE ID / LOCATION



DATABASE REFERENCE

This avoids unnecessarily increasing the size of the main transactional database.

58.6 AI/OCR Resources

The AI pipeline may require:

OCR

Used for extracting text from package images.

NLP / Information Extraction

Used to identify fields such as:

- MRP
- Net quantity
- Manufacturer
- Importer
- Consumer-care information
- Dates
- Country of origin

Computer Vision

Used for:

- Package/label region detection
- Text-region localization
- Image quality assessment
- Evidence highlighting
- Package classification

The initial prototype can use **pretrained models** rather than training every model from scratch.

58.7 Dataset Requirements

The dataset is one of the most important resources.

A useful dataset should contain:

PRODUCT IMAGES



OCR TEXT



GROUND-TRUTH DECLARATIONS



PACKAGE CLASS



APPLICABLE RULES



EXPECTED RESULT

The dataset should include both:

- Correct/compliant packages
- Packages with deliberately introduced or known issues for testing

The dataset should be collected and used in accordance with applicable permissions, licensing and privacy requirements.

58.8 Legal Rule Database

A separate legal-rule database should contain structured requirements.

Example:

Attribute	Example
Rule ID	R6
Requirement	Mandatory declaration
Field	Net Quantity
Package Type	Applicable package category
Condition	Configured condition
Exception	Configured exception
Effective From	Date
Source	Official notification
Status	Valid / historical

This database forms the foundation of the **version-aware Legal Rule Engine**.

58.9 Software Requirements

Possible technology choices include:

Layer	Possible Technology
Android	Kotlin / Android SDK
Frontend Dashboard	React / similar framework

Backend	Python / FastAPI or Node.js
Database	PostgreSQL
OCR	Tesseract / PaddleOCR / cloud OCR
AI/NLP	Python ML/NLP stack
Computer Vision	OpenCV
API	REST
Authentication	OAuth/JWT or government-approved identity system
Deployment	Cloud infrastructure
Version Control	Git

These are **proposed implementation choices** and can be changed according to the team's expertise and deployment requirements.

58.10 Development Tools

The development team may require:

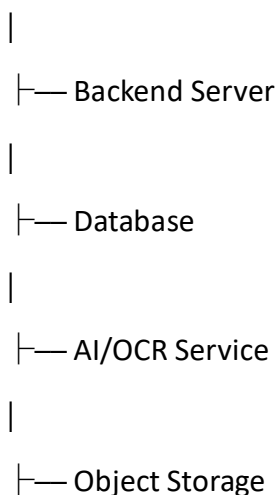
- Android Studio
- Visual Studio Code or equivalent IDE
- Git
- GitHub/GitLab or authorized repository
- Python environment
- Database management tools
- API testing tools
- Image annotation tools
- Model evaluation tools

Most of these can be used through free or open-source options during prototype development.

58.11 Cloud Infrastructure

For the prototype, a basic cloud deployment can contain:

CLOUD



|

└─ Monitoring / Logs

The infrastructure can later be scaled based on actual user volume.

58.12 Prototype-Level Cost

The initial SIH prototype can potentially be developed with relatively low direct infrastructure expenditure if the team uses:

- Existing laptops
- Existing smartphones
- Open-source software
- Free development tools
- Limited cloud resources
- Pretrained AI/OCR models
- A small testing dataset

A realistic budget should therefore distinguish between **prototype cost** and **production deployment cost**.

58.13 Prototype Cost Categories

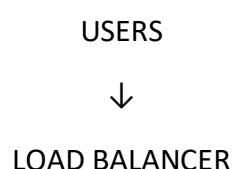
Resource	Prototype Requirement
Developer systems	Existing team hardware
Android phones	Existing/test devices
Software tools	Mostly open-source/free
OCR	Open-source or limited API usage
AI models	Pretrained/open-source
Database	Local or low-cost cloud
Server	Small cloud instance
Image storage	Limited prototype storage
Domain/hosting	Optional
Testing	Student/team resources

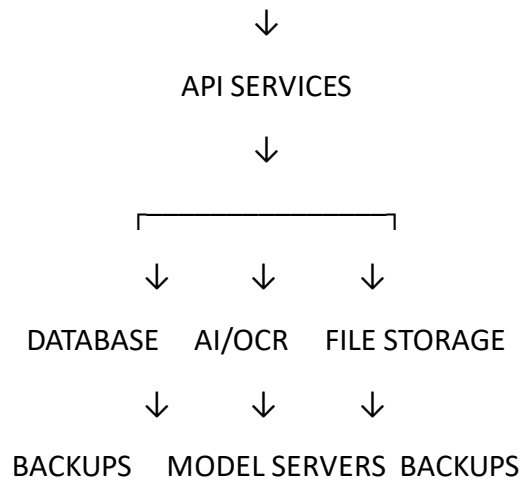
Actual cloud/API costs depend on the selected providers and usage volume.

58.14 Production Infrastructure

A government-scale deployment would require significantly stronger infrastructure.

Possible architecture:





Production infrastructure would also require appropriate:

- Availability
- Monitoring
- Disaster recovery
- Security controls
- Access management
- Backup
- Scalability
- Operational support

These requirements should be finalized with the implementing government authority.

58.15 Human Resources

A successful implementation requires more than programmers.

Recommended team

1. Legal Research Team

Converts legislation and amendments into structured requirements.

2. AI/ML Team

Develops OCR, extraction and computer-vision components.

3. Backend Team

Develops APIs, databases and rule-engine services.

4. Android Team

Develops the field inspection application.

5. UI/UX Team

Designs consumer, manufacturer and officer interfaces.

6. Testing Team

Creates datasets and validates system accuracy.

7. Security/DevOps Team

Handles deployment, security, monitoring and backups.

58.16 Resource Allocation

A simple allocation can be:

LEGAL RESEARCH



AI / OCR



BACKEND + RULE ENGINE



ANDROID APP



DASHBOARD



TESTING + SECURITY



INTEGRATION

The exact team size can be adjusted according to the available SIH team members.

58.17 Cost Optimization Strategy

The prototype cost can be controlled through:

1. Open-source OCR

Use open-source OCR where accuracy is sufficient.

2. Pretrained AI models

Avoid unnecessary model training from scratch.

3. Local processing

Perform lightweight image processing on the device where practical.

4. Cloud only when required

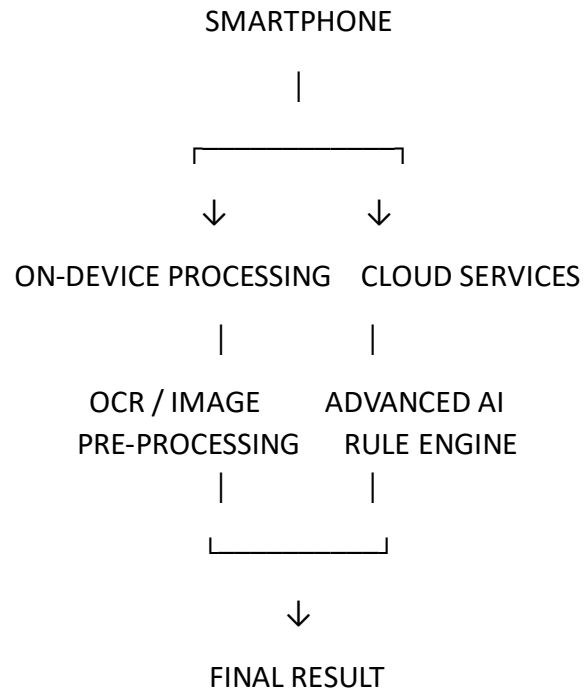
Send computationally expensive tasks to the backend when necessary.

5. Tiered storage

Keep frequently accessed data readily available and archive older data according to policy.

58.18 Edge + Cloud Architecture

A particularly useful architecture for DrishtiTathya is a hybrid approach.

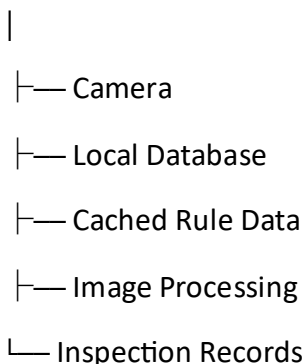


This can reduce network dependency while retaining access to more powerful backend services.

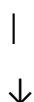
58.19 Offline-First Resource Model

For field inspections:

DEVICE



NETWORK AVAILABLE



CLOUD SYNC

Only the data required for offline operation should be cached on the device.

58.20 Scalability

The system should be designed so that infrastructure can grow with usage.

PROTOTYPE



PILOT



REGIONAL DEPLOYMENT



STATE LEVEL



NATIONAL SCALE

The architecture should avoid designs that work only for a few hundred scans.

58.21 Resource Requirements by Development Stage

Stage	Main Resources
Research	Legal documents, laptop
Dataset	Product images, annotation tools
Prototype	Laptop, Android phone
AI Development	Python, OCR/ML libraries
Backend	Server, database
Mobile	Android development environment
Testing	Dataset + multiple devices
Pilot	Field devices + server
Deployment	Scalable infrastructure + operations

58.22 Key Infrastructure Principle

The project should avoid unnecessary infrastructure complexity during SIH.

The recommended approach is:

Build a lightweight, functional prototype first and design the architecture so that it can scale later.

For example:

SIH PROTOTYPE



SMALL-SCALE CLOUD



VALIDATION



OPTIMIZATION



PRODUCTION-SCALE INFRASTRUCTURE

58.23 Section Conclusion

DrishtiTathya can be developed as a **cost-efficient software prototype** using existing hardware, open-source technologies and pretrained AI/OCR components.

The most important investment during the prototype stage is not expensive hardware. It is:

High-quality labeled data + correctly structured legal rules + reliable testing.

The long-term production system can then scale its:

- AI infrastructure
- Database
- Storage
- Security
- Monitoring
- User capacity

according to actual government deployment requirements.

Prototype first. Validate accuracy. Then scale infrastructure.

59. PROJECT FEASIBILITY AND VIABILITY ANALYSIS

The feasibility analysis determines whether **DrishtiTathya** can be practically developed, deployed and scaled as a software system for packaged commodity compliance.

The project is evaluated across:

1. Technical feasibility
2. Operational feasibility
3. Economic feasibility
4. Legal and regulatory feasibility
5. User feasibility
6. Scalability
7. Risk feasibility
8. Long-term viability

59.1 Technical Feasibility

DrishtiTathya is technically feasible because its major components can be implemented using existing technologies.

The system combines:

- Computer vision
- OCR
- Natural language processing
- Rule-based decision systems
- Mobile application development
- Cloud/backend services
- Database systems
- Evidence management

The overall architecture is:

PRODUCT IMAGE



COMPUTER VISION



OCR



AI INFORMATION EXTRACTION



LEGAL RULE ENGINE



COMPLIANCE ANALYSIS



EVIDENCE



HUMAN REVIEW

The project does not require the invention of a completely new AI model. Existing technologies can be adapted to the specific packaged-commodity domain.

Feasibility: High

59.2 AI Feasibility

OCR and computer-vision technology can already process text-containing images.

However, packaged labels create challenges such as:

- Small text
- Curved surfaces
- Multiple languages
- Decorative backgrounds
- Different fonts
- Poor lighting
- Blurred photographs
- Multiple package surfaces

Therefore, the project should not depend on a single AI model.

A layered approach is preferable:

IMAGE QUALITY CHECK



IMAGE PRE-PROCESSING



OCR



FIELD EXTRACTION



CONFIDENCE CHECK



HUMAN REVIEW

Feasibility: High for prototype; requires continuous improvement for production

59.3 Legal Rule Engine Feasibility

The Legal Rule Engine is technically feasible because legal requirements can be represented as structured conditions.

For example:

IF

package_type = applicable retail package

AND

required_declaration = missing

AND

no_exception_applies

THEN

flag_for_review

A more complete rule structure can include:

RULE

└─ Rule Number

└─ Requirement

└─ Applicability

└─ Conditions

└─ Exceptions

└─ Effective Date

└─ Amendment

└─ Source

This is more reliable than asking an AI model to independently interpret legislation for every scan.

Feasibility: High

59.4 Version-Aware Legal Feasibility

A major challenge is that legal requirements can change through amendments.

Therefore, the system should maintain:

RULE VERSION 1

↓

AMENDMENT

↓

RULE VERSION 2

↓

AMENDMENT

↓

RULE VERSION 3

Each version should have an effective date.

The system can then determine which version should be considered for a particular inspection.

Feasibility: High, provided legal rules are maintained by authorized personnel

59.5 Operational Feasibility

DrishtiTathya is intended to fit into existing inspection activities rather than completely replace them.

A conventional workflow may involve:

INSPECT



OBSERVE



RECORD



VERIFY



REPORT

DrishtiTathya can digitize and assist these activities:

CAPTURE



AI ASSISTANCE



RULE CHECK



EVIDENCE



OFFICER REVIEW



REPORT

The officer remains responsible for verification and final action.

Feasibility: High

59.6 Field Feasibility

The application can be used through an Android smartphone with a camera.

This is advantageous because the field workflow becomes:

TAKE PHOTO



ANALYSE



REVIEW



SAVE

The offline-first architecture further improves feasibility in locations with unreliable connectivity.

Feasibility: High

59.7 Economic Feasibility

The initial prototype can be developed with relatively limited infrastructure.

Major prototype resources include:

- Existing laptops
- Android smartphones
- Open-source software
- Pretrained AI/OCR models
- Small cloud infrastructure
- Development tools

Therefore, the initial development cost can be kept comparatively low.

The major long-term costs would arise from:

- Cloud infrastructure
- AI inference
- Storage
- Security
- Maintenance
- Support
- Model improvement
- Legal-rule maintenance

Feasibility: High for prototype; scalable cost model required for production

59.8 User Feasibility

The system has three major user groups.

Consumer

Needs:

- Simple scanning
- Easy-to-understand result
- Basic explanation
- Complaint option

Manufacturer

Needs:

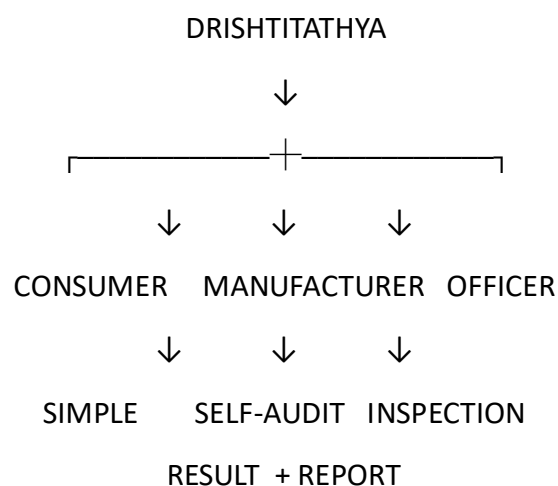
- Label self-audit
- Missing declaration detection
- Pre-print checking
- Compliance history

Government Officer

Needs:

- Detailed inspection workflow
- Evidence
- Applicable rules
- Review functionality
- Reports
- Inspection history

Therefore, the interface should be role-specific.



Feasibility: High

59.9 Legal and Regulatory Feasibility

The system is designed as a **decision-support and inspection-assistance platform**, not as an autonomous authority.

This distinction is important.

The software can:

- Extract information.
- Identify potentially relevant requirements.
- Flag possible issues.
- Show evidence.
- Recommend review.

The authorized officer can:

- Verify the package.
- Interpret the applicable circumstances.
- Confirm findings.
- Take appropriate action.

Therefore:

AI assists legal compliance assessment; it does not replace statutory authority or human legal judgment.

Feasibility: High, subject to validation and approval by the implementing authority

59.10 Scalability Feasibility

The architecture can scale from a prototype to larger deployments.

PROTOTYPE



PILOT



DISTRICT



STATE



NATIONAL

Cloud-based backend services can be scaled independently.

For example:

MOBILE USERS



LOAD BALANCER



API SERVICES



AI SERVICES



DATABASE + STORAGE

Feasibility: High

59.11 Security Feasibility

The system can incorporate:

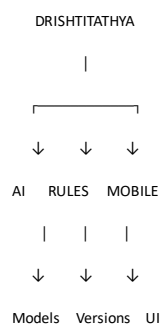
- Authentication
- Role-based access
- Encryption
- Secure APIs
- Evidence integrity
- Audit logs
- Backup and recovery
- Secure synchronization

The architecture therefore supports the security requirements of an inspection-oriented application.

Feasibility: High

59.12 Maintainability Feasibility

The system should separate its major components.



This means a legal-rule update does not necessarily require retraining the AI model.
Similarly, an OCR improvement does not require rewriting the entire legal-rule engine.

Feasibility: High

59.13 Major Technical Challenges

Despite being feasible, several challenges must be considered.

Challenge 1 — OCR Errors

Small or unclear text may be incorrectly recognized.

Mitigation:

- Image-quality detection
 - Multiple images
 - OCR confidence
 - Human review
-

Challenge 2 — Legal Interpretation

Some legal requirements depend on conditions and exceptions.

Mitigation:

- Structured rule engine
 - Applicability layer
 - Exception handling
 - Legal expert validation
-

Challenge 3 — Changing Regulations

Rules may be amended.

Mitigation:

- Version-controlled rule database
 - Effective dates
 - Amendment records
 - Authorized rule updates
-

Challenge 4 — False Positives

The system may flag a compliant package.

Mitigation:

- Confidence scores
 - Evidence display
 - Human verification
-

Challenge 5 — False Negatives

The system may fail to identify an issue.

Mitigation:

- Multiple inspection checks
 - Strong testing dataset
 - Human review
 - Continuous model evaluation
-

Challenge 6 — Poor Network Connectivity

Field locations may have weak connectivity.

Mitigation:

- Offline-first design
 - Local encrypted storage
 - Secure synchronization
-

59.14 Risk and Mitigation Matrix

Risk	Impact	Mitigation
OCR error	High	Pre-processing + confidence + review
Incorrect rule selection	Very High	Rule engine + legal validation
Outdated legal rule	Very High	Version control + update workflow
False positive	Medium	Evidence + human review
False negative	High	Testing + human review
Poor image quality	Medium	Smart capture guidance
Network failure	Medium	Offline mode
Unauthorized access	High	RBAC + authentication
Evidence modification	High	Integrity protection + audit trail
Large data volume	Medium	Scalable storage
AI model drift	Medium	Continuous evaluation
User resistance	Medium	Simple workflow + training

59.15 Feasibility Matrix

Area	Feasibility
------	-------------

Mobile Application	High
OCR	High
AI Extraction	High
Computer Vision	High
Rule Engine	High
Version Management	High
Evidence Management	High
Offline Operation	High
Dashboard	High
Security	High
Large-Scale Deployment	High with proper infrastructure
Fully Autonomous Legal Decision	Not recommended

59.16 Social Viability

DrishtiTathya can provide benefits to different stakeholders.

Consumers

- Better access to package information
- Easier identification of potential issues
- Ability to report concerns

Manufacturers

- Early identification of packaging problems
- Reduced compliance errors
- Self-audit capability

Government

- Faster inspection
- Structured evidence
- Better inspection records
- Data-driven prioritization

The overall impact can be represented as:

CONSUMER AWARENESS



MORE REPORTS / SIGNALS



BETTER INTELLIGENCE



TARGETED INSPECTION



IMPROVED COMPLIANCE



CONSUMER PROTECTION

59.17 Environmental Viability

Digitizing inspection records can reduce dependence on paper-based workflows.

Instead of:

PAPER



MANUAL RECORD



PHYSICAL STORAGE

the system can provide:

DIGITAL EVIDENCE



SECURE DATABASE



DIGITAL REPORT

This can also improve retrieval and historical analysis.

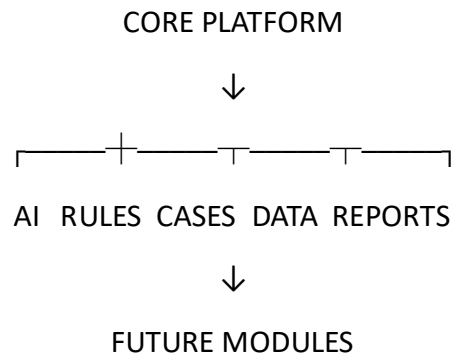
59.18 Long-Term Viability

DrishtiTathya can evolve beyond the initial packaged-commodity scanning system.

Possible future expansion includes:

- Additional product categories
- More advanced AI models
- More languages
- E-commerce compliance
- Manufacturer compliance portals
- Consumer complaint integration
- Predictive inspection intelligence
- Cross-department data integration

The architecture should therefore be modular.



59.19 Overall Feasibility

The overall feasibility can be summarized as:

TECHNICAL → HIGH

OPERATIONAL → HIGH

ECONOMIC → HIGH FOR PROTOTYPE

LEGAL → HIGH WITH HUMAN VALIDATION

SECURITY → HIGH

SCALABILITY → HIGH

USER ACCEPTANCE → HIGH POTENTIAL

The most important dependency is **accurate legal-rule modelling and validation**, not simply AI accuracy.

59.20 Go / No-Go Assessment

Technical

GO — Required technologies are available.

Operational

GO — The system can support existing inspection workflows.

Economic

GO — An SIH-level prototype can be developed using limited resources.

Legal

GO WITH HUMAN OVERSIGHT — Automated results should remain advisory until verified by authorized personnel.

Scalability

GO — Modular backend and cloud architecture can support future expansion.

59.21 Section Conclusion

The feasibility analysis indicates that **DrishtiTathya is technically and operationally viable as an SIH prototype**, provided that the project maintains a clear separation between:

AI extraction → Legal rule evaluation → Human verification.

The strongest feasibility advantage is that the project does not depend on creating an entirely new technology. Instead, it integrates existing technologies into a **domain-specific legal compliance workflow**.

DrishtiTathya is feasible because it combines proven technologies in a new, purpose-built workflow for packaged commodity compliance.

The key condition for successful deployment is continuous validation of **legal rules, AI accuracy, evidence quality and human-review workflows**.

60. RISK ANALYSIS AND MITIGATION STRATEGY

DrishtiTathya combines **AI, OCR, computer vision, legal rules, evidence management and field inspection**. Therefore, identifying risks at the design stage is essential.

The objective of risk management is not to eliminate every possible failure, but to ensure that when a failure occurs, the system can **detect it, communicate it clearly and provide a safe fallback**.

60.1 Risk Management Framework

The overall approach is:

IDENTIFY RISK



ASSESS IMPACT



DEFINE MITIGATION



IMPLEMENT CONTROL



TEST



MONITOR



IMPROVE

Each risk can be classified according to:

- Probability
- Impact

- Severity
 - Mitigation
 - Contingency
-

60.2 Risk Classification

The major risks can be divided into:

1. AI/ML risks
 2. OCR risks
 3. Legal-rule risks
 4. Data risks
 5. Security risks
 6. Operational risks
 7. Infrastructure risks
 8. User-related risks
 9. Integration risks
 10. Deployment risks
-

60.3 AI Accuracy Risk

Risk

The AI model may incorrectly extract information from a package.

For example, it may interpret:

₹120

as:

₹180

because of poor image quality.

Impact

High

Incorrect extraction can result in an incorrect compliance assessment.

Mitigation

- Image-quality assessment
- OCR confidence scores
- Multiple-image capture

- Structured validation
- Human review
- Continuous testing

Contingency

If confidence is below the configured threshold:

LOW CONFIDENCE



DO NOT FINALIZE



REQUEST REVIEW

60.4 OCR Risk

Risk

OCR may fail because of:

- Small fonts
- Curved labels
- Low lighting
- Blur
- Reflections
- Decorative backgrounds
- Multiple languages

Impact

High

Important declarations may not be detected.

Mitigation

IMAGE



QUALITY CHECK



PRE-PROCESSING



MULTIPLE OCR STRATEGIES



CONFIDENCE

The application can also guide the user to capture a clearer image.

60.5 False Positive Risk

Risk

A compliant package may be flagged as potentially non-compliant.

Impact

Medium to High

Repeated false positives could reduce user trust.

Mitigation

- Confidence thresholds
- Evidence display
- Rule explanation
- Exception checking
- Human verification
- Continuous evaluation

The interface should use terms such as:

Potential issue detected

rather than automatically declaring:

Legal violation confirmed

60.6 False Negative Risk

Risk

A genuine issue may not be detected.

Impact

Very High

A missed issue could result in an inspection problem being overlooked.

Mitigation

- Multiple checks
- Strong validation dataset
- Multiple package images

- Rule-engine testing
 - Human review
 - Periodic model evaluation
 - Random manual verification
-

60.7 Legal Rule Interpretation Risk

Risk

A legal provision may be complex or dependent on specific conditions and exceptions.

Impact

Very High

An incorrect legal interpretation can produce an incorrect compliance result.

Mitigation

The system should use:

LEGAL SOURCE



STRUCTURED REQUIREMENT



CONDITION



EXCEPTION



SOFTWARE RULE



VALIDATION

Rules should be reviewed by appropriately qualified legal/domain experts before being used operationally.

60.8 Outdated Rule Risk

Risk

The system may use an old version of a rule after an amendment becomes applicable.

Impact

Very High

Mitigation

Use a version-aware rule database:

RULE V1



AMENDMENT



RULE V2



EFFECTIVE DATE



ACTIVE VERSION

The system should store:

- Amendment date
- Effective date
- Previous version
- New version
- Source reference
- Approval status

60.9 Incorrect Applicability Risk

Risk

The system may apply a rule to a package for which it does not apply.

For example, a requirement may depend on:

- Package category
- Commodity type
- Quantity
- Intended consumer
- Import status
- Specific exemption

Impact

Very High

Mitigation

Use a separate applicability layer:

PACKAGE



CLASSIFICATION



APPLICABILITY



EXCEPTIONS



RULES

This should occur **before compliance evaluation**.

60.10 Exception Handling Risk

Risk

The system may detect a general requirement but fail to recognize an applicable exception.

Impact

High

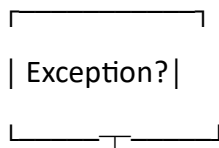
Mitigation

Each important exception should be represented explicitly in the rule engine.

GENERAL RULE



EXCEPTION CHECK



YES / NO

Exceptions should be separately tested.

60.11 Data Quality Risk

Risk

The AI model may be trained or evaluated using an unrepresentative dataset.

Impact

High

Mitigation

The dataset should include variation in:

- Product categories
- Package sizes
- Languages
- Label designs
- Lighting
- Image quality
- Package surfaces
- Compliant and problematic examples

Ground-truth information should be manually verified.

60.12 Dataset Bias Risk

Risk

If the dataset contains mostly one type of packaging, the system may perform poorly on other packaging styles.

Example:

TRAINING DATA

Mostly flat labels



MODEL



Poor performance on curved bottles

Mitigation

Use a diverse dataset and evaluate performance separately across package types.

60.13 Multilingual Risk

Risk

Indian packages may contain multiple languages.

OCR or information extraction may fail for certain scripts.

Impact

Medium to High

Mitigation

- Multilingual OCR
- Language detection
- Script-specific preprocessing
- Human review
- Testing across representative languages

The system should clearly indicate when a language or script could not be reliably interpreted.

60.14 Image Quality Risk

Risk

Users may capture:

- Blurred images
- Dark images
- Obstructed labels
- Incomplete package surfaces

Mitigation

Implement **Smart Capture Guidance**.

Example:

CAPTURE



QUALITY CHECK



Poor Image?



YES



"Move closer / improve lighting"

The application should not proceed as though the package has been fully analyzed when important information is not visible.

60.15 Security Risk

Risk

Unauthorized users could access inspection information.

Impact

High

Mitigation

- Authentication
 - Role-based access control
 - Secure APIs
 - Encryption
 - Audit logs
 - Session management
 - Security monitoring
-

60.16 Evidence Tampering Risk

Risk

Inspection photographs or reports could be modified after capture.

Impact

Very High

Mitigation

ORIGINAL IMAGE



HASH



TIMESTAMP



SECURE STORAGE



AUDIT TRAIL

Processed images should not replace the original evidence.

60.17 Offline Data Loss Risk

Risk

An officer may conduct an inspection while offline and the device may fail before synchronization.

Impact

High

Mitigation

- Encrypted local storage
- Automatic local saving
- Transaction checkpoints
- Retry mechanism
- Sync status
- Recovery mechanism

Example:

CAPTURE



SAVE LOCALLY



CONFIRM SAVED



CONTINUE

60.18 Synchronization Conflict Risk

Risk

The same record may be modified from multiple locations.

Mitigation

Use:

- Inspection IDs
- Record versions
- Timestamps
- Conflict detection
- Controlled update policies

The system should avoid silently overwriting information.

60.19 Network Dependency Risk

Risk

Cloud-based AI services may become unavailable.

Impact

Medium to High

Mitigation

Use a hybrid architecture:

DEVICE



LOCAL PROCESSING



IF ONLINE → CLOUD



IF OFFLINE → LOCAL QUEUE

The system can continue essential field operations and synchronize later.

60.20 Server Failure Risk

Risk

The backend may become temporarily unavailable.

Mitigation

- Monitoring
- Backups
- Redundant infrastructure where appropriate
- Health checks
- Retry mechanisms
- Disaster recovery procedures

60.21 Storage Growth Risk

Risk

Large numbers of inspection photographs can consume substantial storage.

Mitigation

Use:

- Object/file storage
- Image compression where appropriate

- Metadata-based database references
 - Retention policies
 - Archiving according to authorized policy
-

60.22 User Adoption Risk

Risk

Officers or manufacturers may find the system difficult to use.

Mitigation

Design the workflow around the user's existing task.

Instead of:

10+ MANUAL FORMS

provide:

CAPTURE

↓

ANALYSE

↓

REVIEW

↓

REPORT

The interface should be simple and require minimal unnecessary input.

60.23 Human Over-Reliance on AI

Risk

Users may assume that an AI result is automatically legally correct.

Impact

Very High

Mitigation

Clearly display:

- AI confidence
- Evidence
- Applicable rule
- Review status

- Human verification requirement

Example:

AI FINDING



REVIEW REQUIRED



OFFICER VERIFICATION



FINAL DECISION

60.24 Model Drift Risk

Risk

AI performance may decrease as packaging styles change over time.

Mitigation

PRODUCTION DATA



PERFORMANCE MONITORING



ERROR ANALYSIS



NEW TRAINING DATA



MODEL UPDATE



VALIDATION

A new model should not automatically replace the production model without testing.

60.25 Legal Rule Update Failure

Risk

A new amendment may be incorrectly entered into the system.

Mitigation

Use a controlled workflow:

NEW AMENDMENT



LEGAL REVIEW



RULE STRUCTURING



SOFTWARE IMPLEMENTATION



TEST CASES



APPROVAL



PUBLICATION

This should be separated from ordinary AI model updates.

60.26 Integration Risk

Risk

OCR, AI, rule engine, database and mobile application may produce inconsistent data.

Example:

OCR → 500 g

AI → 50 g

RULE ENGINE → 500 g

Mitigation

Use a standardized intermediate data format.

OCR



NORMALIZED DATA MODEL



AI



NORMALIZED DATA MODEL



RULE ENGINE

Every module should communicate through clearly defined APIs and schemas.

60.27 Performance Risk

Risk

Analysis may take too long for practical field use.

Mitigation

- Lightweight preprocessing
- Efficient OCR
- Caching
- Asynchronous processing
- Model optimization
- Edge processing where appropriate

The application should clearly show processing status rather than appearing frozen.

60.28 Privacy Risk

Risk

The system may collect unnecessary personal or location information.

Mitigation

Follow data-minimization principles:

REQUIRED DATA



COLLECT



SECURE



USE FOR INTENDED PURPOSE

Only information necessary for the authorized workflow should be collected.

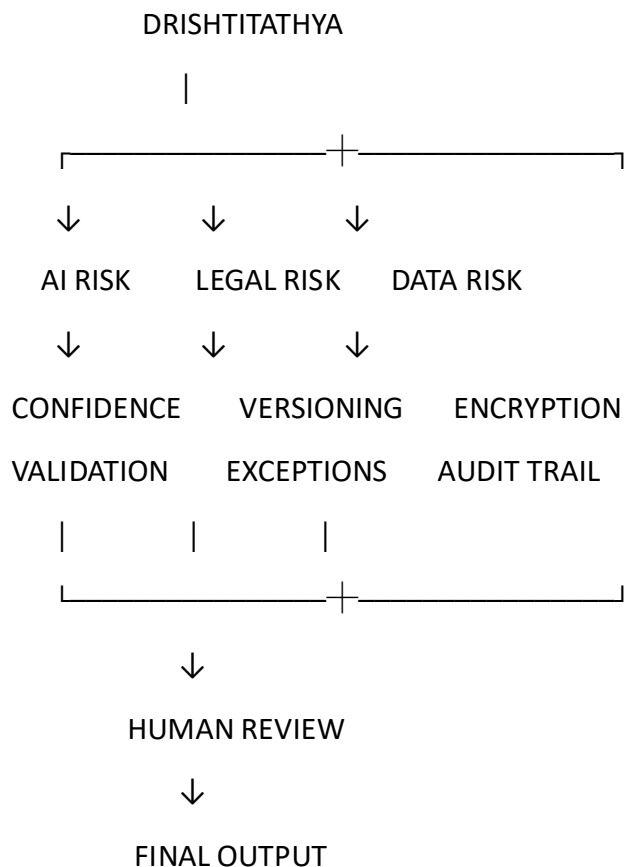
60.29 Risk Priority Matrix

Risk	Probability	Impact	Priority
OCR error	High	High	Critical

Risk	Probability	Impact	Priority
Incorrect legal rule	Medium	Very High	Critical
Outdated rule	Medium	Very High	Critical
False negative	Medium	Very High	Critical
False positive	High	Medium	High
Poor image quality	High	Medium	High
Evidence tampering	Low/Medium	Very High	Critical
Network failure	Medium	Medium	High
Data loss	Low/Medium	High	High
Security breach	Low/Medium	Very High	Critical
Model drift	Medium	Medium	Medium
User resistance	Medium	Medium	Medium
Storage growth	Medium	Medium	Medium

60.30 Risk-Control Architecture

The complete risk-control approach can be represented as:



60.31 Contingency Strategy

If a major component fails, the system should degrade safely.

AI failure

AI UNAVAILABLE



MANUAL REVIEW

OCR failure

OCR FAILURE



RECAPTURE IMAGE

Network failure

NETWORK FAILURE



OFFLINE MODE

Rule uncertainty

RULE UNCERTAIN



LEGAL / OFFICER REVIEW

Low-confidence result

LOW CONFIDENCE



HUMAN VERIFICATION

This is called **graceful degradation**.

60.32 Most Important Risk-Control Principle

The most important design principle is:

When the system is uncertain, it should ask for verification rather than confidently produce an incorrect legal conclusion.

Therefore:

HIGH CONFIDENCE



AUTOMATED ASSISTANCE

LOW CONFIDENCE



HUMAN REVIEW

This principle should apply throughout the AI pipeline.

60.33 Risk Monitoring After Deployment

Risk management should continue after deployment.

The system should monitor:

- OCR performance
- AI confidence
- Review/rejection rates
- Common extraction errors
- Rule-engine errors
- System failures
- Synchronization failures
- Security alerts
- User feedback

This creates a continuous improvement loop:

USE



MONITOR



IDENTIFY PROBLEM



CORRECT



TEST



RELEASE

60.34 Section Conclusion

DrishtiTathya involves several high-impact risks, particularly **AI errors, incorrect rule selection, outdated legal information, false negatives and evidence integrity**.

These risks can be controlled through:

Version-aware legal rules + confidence-based AI + strong testing + evidence protection + offline capability + human-in-the-loop review.

The central safety principle is:

“Automate the detection, explain the reasoning, preserve the evidence, and keep the final legal decision under authorized human review.”

This makes DrishtiTathya not only an AI-based solution, but a **controlled and accountable compliance-support system**.

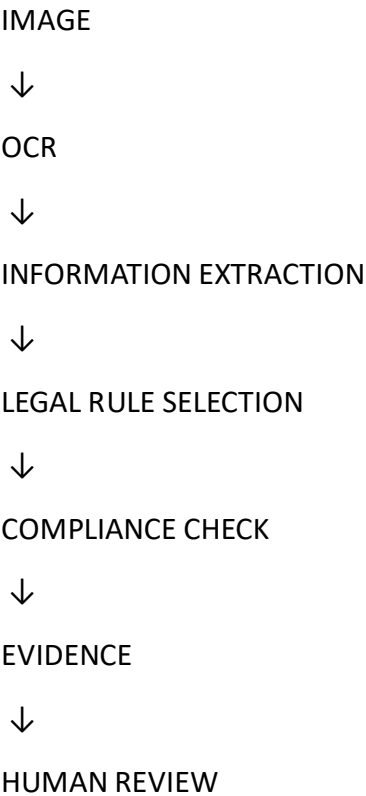
61. SUCCESS METRICS AND KPIs — SHORT VERSION

DrishtiTathya should be evaluated as a **complete compliance system**, not just an OCR application.

Key KPIs

KPI	Purpose
OCR Accuracy	Measures text extraction quality
Declaration F1 Score	Measures accuracy of MRP, quantity, manufacturer, etc.
Rule Selection Accuracy	Checks whether the correct legal rule is selected
Compliance Precision/Recall	Measures correct detection of potential issues
Evidence Coverage	Ensures findings are linked to supporting images
Inspection Time	Measures reduction in manual inspection effort
Offline Sync Success	Measures reliability during field inspections
Human Review Rate	Measures how often AI results need verification

Testing Flow





REPORT

Success Formula

Accuracy + Legal Correctness + Evidence Quality + Efficiency + Reliability + Usability

“Measure every stage from image capture to verified compliance.”

For SIH, focus mainly on **8 core KPIs**: OCR accuracy, extraction F1, rule accuracy, compliance precision/recall, evidence coverage, inspection time, offline sync, and human review rate.

62. EXPECTED SOCIAL, ECONOMIC AND GOVERNMENT IMPACT

DrishtiTathya can create value for **consumers, manufacturers and government authorities** by making packaged-commodity compliance faster, more transparent and easier to verify.

62.1 Consumer Impact

- Easy scanning of product labels.
- Better awareness of mandatory declarations.
- Simple explanation of potential issues.
- Easy evidence-based complaint submission.
- Increased consumer confidence.

Impact:

SCAN → UNDERSTAND → VERIFY → REPORT

62.2 Manufacturer Impact

- Early detection of packaging errors.
- Pre-print label self-audit.
- Reduced compliance-related mistakes.
- Faster verification of multiple SKUs.
- Better understanding of recurring issues.

Impact:

CHECK → CORRECT → RE-SCAN → COMPLY

62.3 Government Impact

- Faster inspection workflow.
- Automated extraction of package information.
- Evidence-backed findings.

- Digital inspection records.
- Reduced repetitive documentation.
- Better identification of repeat issues.
- Data-driven inspection prioritization.

Impact:

CAPTURE → ANALYSE → VERIFY → REPORT → MONITOR

62.4 Economic Impact

DrishtiTathya can potentially reduce:

- Manual documentation effort.
- Time spent checking repetitive declarations.
- Packaging correction costs through early detection.
- Administrative effort in maintaining inspection records.

It can also support manufacturers in identifying issues **before products reach the market.**

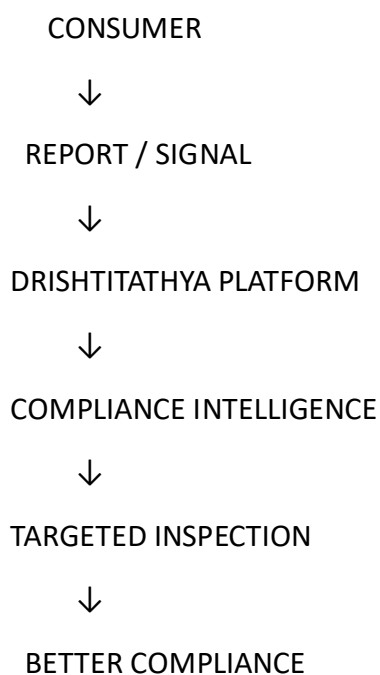
62.5 Transparency and Accountability

Every important finding can be connected to:

Rule → Extracted Information → Image Evidence → Human Review → Report

This creates a more transparent inspection process.

62.6 Overall Ecosystem Impact





CONSUMER PROTECTION

62.7 Key Impact Statement

DrishtiTathya transforms packaged-commodity compliance from a primarily manual checking process into an AI-assisted, evidence-based and digitally traceable workflow.

Core Impact

Consumer → Verify

Manufacturer → Prevent

Government → Inspect

DrishtiTathya → Connects all three

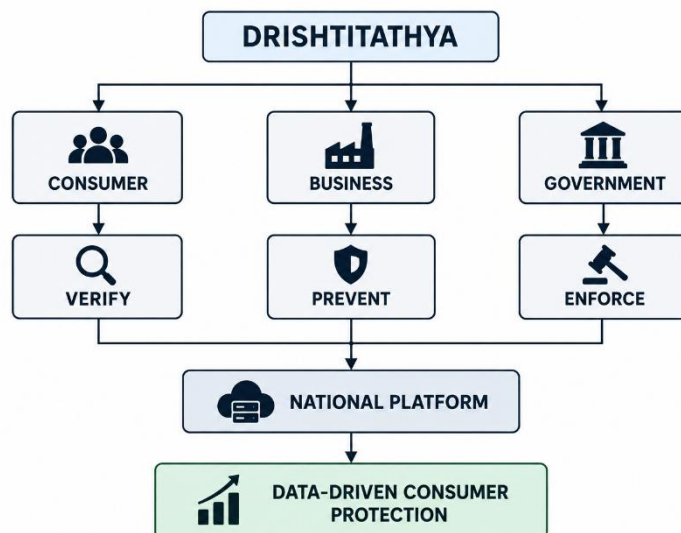
63. FUTURE SCOPE AND EXPANSION

DrishtiTathya can be expanded from a prototype into a **national-level packaged-commodity compliance platform**.

63.1 Future Enhancements

- **National-scale deployment** across states and districts.
- **Multilingual AI** for regional-language labels.
- **E-commerce monitoring** of online product listings.
- **Advanced AI models** for better label and declaration recognition.
- **Predictive inspection** to identify high-risk products and manufacturers.
- **Bulk SKU scanning** for large manufacturers and inspectors.
- **Government system integration** for digital inspection and case management.
- **Real-time compliance dashboards** for authorities.
- **Consumer complaint integration** for collecting field-level compliance signals.
- **Automated compliant-label generation** to help manufacturers correct violations.
- **Historical compliance tracking** to identify repeated violations.
- **Offline-first inspection** for areas with limited internet connectivity.

63.2 Future Expansion



63.3 Long-Term Vision

The long-term goal is to develop DrishtiTathya into an **AI-assisted regulatory intelligence platform** that can continuously support product verification, manufacturer self-audit, field inspection, evidence management and compliance monitoring.

Future Vision Statement

“From scanning a package to building a smarter, evidence-driven compliance ecosystem.”

Core principle: AI should assist detection and analysis, while **final legal validation remains under authorized human review.**

64. CONCLUSION

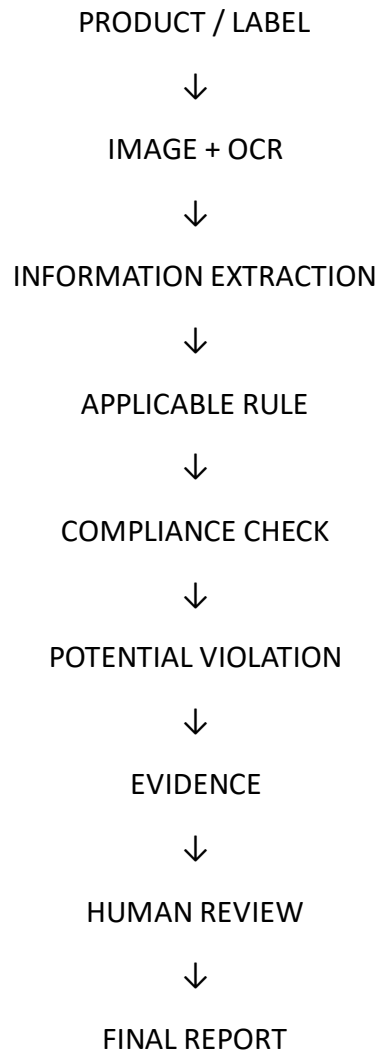
DrishtiTathya proposes an **AI-assisted software system for checking packaged-commodity compliance** under the Legal Metrology (Packaged Commodities) Rules, 2011.

The system combines:

- **Image scanning and OCR**
- **Declaration extraction**
- **Legal rule matching**
- **Compliance checking**
- **Violation highlighting**
- **Evidence management**
- **Human legal review**
- **Digital inspection reports**

Its main strength is that it does not treat compliance checking as only an OCR problem. It connects the **product image, extracted information, applicable legal rule, evidence and verification process** into one workflow.

Overall System



DrishtiTathya can therefore act as a **decision-support system** for consumers, manufacturers and Legal Metrology authorities while keeping final legal decisions with authorized human officers.

Final Vision

“दृष्टि देखे, तथ्य परखे।”

“See the Label. Verify the Facts.”

DrishtiTathya aims to make packaged-commodity compliance faster, evidence-based, transparent and scalable.

65. REFERENCES

The following sources were used as the primary references for the research and design of DrishtiTathya:

1. **Legal Metrology Act, 2009** — Department of Consumer Affairs, Government of India.
2. **Legal Metrology (Packaged Commodities) Rules, 2011** — Department of Consumer Affairs, Government of India.
3. **Amendments to the Packaged Commodities Rules, 2011** — Government of India notifications issued from time to time.
4. **Department of Consumer Affairs — Legal Metrology** — Official Government portal.

5. **Legal Metrology inspection and enforcement provisions** — Department of Consumer Affairs.
6. **Official Gazette notifications** relating to amendments of the Packaged Commodities Rules.
7. **Uploaded reference document:** *The Legal Metrology (Packaged Commodities) Rules, 2011*.
8. Research on **OCR, computer vision and document information extraction** for product-label analysis.
9. Research on **AI-assisted compliance and rule-based decision systems**.
10. Research on **digital evidence management, audit trails and inspection systems**.

Reference Principle

“Every compliance decision should be traceable to a rule and supported by evidence.”